

ĐẠI HỌC QUỐC GIA TP. HCM
TRƯỜNG ĐẠI HỌC BÁCH KHOA

ĐỖ ĐỨC MINH

ỨNG DỤNG DEEP LEARNING TRONG PHÁT HIỆN
KHUYẾT TẬT BỀ MẶT

Chuyên ngành : KỸ THUẬT ĐIỆN TỬ
Mã số: 8520203

LUẬN VĂN THẠC SĨ

TP. HỒ CHÍ MINH, tháng 5 năm 2025

**CÔNG TRÌNH ĐƯỢC HOÀN THÀNH TẠI
TRƯỜNG ĐẠI HỌC BÁCH KHOA –ĐHQG -HCM**

Cán bộ hướng dẫn khoa học :
(Ghi rõ họ, tên, học hàm, học vị và chữ ký)

Cán bộ chấm nhận xét 1 :
(Ghi rõ họ, tên, học hàm, học vị và chữ ký)

Cán bộ chấm nhận xét 2 :
(Ghi rõ họ, tên, học hàm, học vị và chữ ký)

Luận văn thạc sĩ được bảo vệ tại Trường Đại học Bách Khoa, ĐHQG
Tp. HCM ngày tháng năm

Thành phần Hội đồng đánh giá luận văn thạc sĩ gồm:
(Ghi rõ họ, tên, học hàm, học vị của Hội đồng chấm bảo vệ luận văn
thạc sĩ)

1.
2.
3.
4.
5.

Xác nhận của Chủ tịch Hội đồng đánh giá LV và Trưởng Khoa quản lý
chuyên ngành sau khi luận văn đã được sửa chữa (nếu có).

CHỦ TỊCH HỘI ĐỒNG

TRƯỞNG KHOA.....

- **Trang 3:** Tờ nhiệm vụ luận văn thạc sĩ
- **Trang 4:** Lời cảm ơn
- **Trang 5:** Tóm tắt luận văn thạc sĩ (Tiếng Việt và Tiếng Anh).
- **Trang 6:** Lời cam đoan của tác giả LV
- **Mục lục**
- **Toàn bộ nội dung luận văn (thực hiện theo đề cương đã bảo vệ)**
- **Tài liệu tham khảo**
- **Phụ lục (nếu có)**

PHẦN LÝ LỊCH TRÍCH NGANG

Họ và tên: **ĐỖ ĐỨC MINH**

Ngày, tháng, năm sinh: 12/02/1999 Nơi sinh: TP. HCM

Địa chỉ liên lạc: 329/12A Bùi Hữu Nghĩa, phường 1, Bình Thạnh

QUÁ TRÌNH ĐÀO TẠO

(Bắt đầu từ Đại học đến nay)

QUÁ TRÌNH CÔNG TÁC

(Bắt đầu từ khi đi làm đến nay)

NHIỆM VỤ LUẬN VĂN THẠC SĨ

Họ tên học viên: ĐỖ ĐỨC MINH

MSHV: 2171006

Ngày, tháng, năm sinh: 12/02/1999

Nơi sinh: TP.HCM

Chuyên ngành: KỸ THUẬT ĐIỆN TỬ

Mã số : 8520203

I. TÊN ĐỀ TÀI: Ứng dụng Deep Learning trong phát hiện khuyết tật bề mặt.

II. NHIỆM VỤ VÀ NỘI DUNG:

- Tìm hiểu lý thuyết về các mô hình Deep learning trong phát hiện khuyết tật bề mặt cho các sản phẩm trong công nghiệp. Lựa chọn mô hình và giải thuật phù hợp.
- Phân tích lý thuyết mô hình giải thuật đã chọn.
- Thiết kế hệ thống phần cứng để áp dụng mô hình thuật toán đã chọn.
- Thiết kế phần mềm, chương trình cho hệ thống.
- Thực hiện huấn luyện mô hình trên tập dữ liệu, thực hiện phát hiện khuyết tật bề mặt trên tập kiểm thử.
- Đánh giá các thông số huấn luyện và hiệu suất phát hiện khuyết tật bề mặt.

III. NGÀY GIAO NHIỆM VỤ : 13/01/2025

IV. NGÀY HOÀN THÀNH NHIỆM VỤ: 12/05/2025

V. CÁN BỘ HƯỚNG DẪN : PGS. TS. Trương Quang Vinh

Tp. HCM, ngày 31 tháng 5 năm 2025

CÁN BỘ HƯỚNG DẪN
(Họ tên và chữ ký)

CHỦ NHIỆM BỘ MÔN ĐÀO TẠO
(Họ tên và chữ ký)

TRƯỞNG KHOA.....
(Họ tên và chữ ký)

LỜI CẢM ƠN

Trong suốt quá trình thực hiện luận văn thạc sĩ, em đã nhận được sự quan tâm, giúp đỡ quý báu từ các thầy cô, gia đình và bạn bè. Em xin trân trọng gửi lời cảm ơn sâu sắc đến tất cả những người đã đồng hành, hỗ trợ và tạo điều kiện thuận lợi để em hoàn thành luận văn này.

Trước hết, em xin bày tỏ lòng biết ơn chân thành đến Thầy Trương Quang Vinh, giảng viên Bộ môn Điện tử – Trường Đại học Bách Khoa, Đại học Quốc gia TP. Hồ Chí Minh, người đã tận tình hướng dẫn, định hướng và hỗ trợ em trong suốt quá trình thực hiện luận văn. Những ý kiến đóng góp quý báu và sự chỉ dẫn tận tâm của Thầy là nền tảng quan trọng giúp em hoàn thành tốt đề tài nghiên cứu này.

Em cũng xin chân thành cảm ơn quý thầy cô trong Trường Đại học Bách Khoa TP. Hồ Chí Minh, những người đã truyền đạt cho em những kiến thức chuyên môn cũng như tư duy khoa học trong suốt quá trình học tập, tạo nền tảng vững chắc để em thực hiện đề tài một cách hiệu quả.

Bên cạnh đó, em xin gửi lời cảm ơn sâu sắc đến gia đình và bạn bè – những người luôn bên cạnh, động viên, khích lệ tinh thần em trong suốt thời gian học tập và nghiên cứu.

Với điều kiện thời gian cũng như kinh nghiệm còn hạn chế của một học viên, luận văn này không thể tránh được những thiếu sót. Em rất mong nhận được sự chỉ bảo, đóng góp ý kiến của các thầy cô để tôi có điều kiện bổ sung, nâng cao kiến thức của mình, phục vụ tốt hơn công tác thực tế sau này.

Trân trọng cảm ơn.

Tp. Hồ Chí Minh, ngày 31 tháng 5 năm 2025

Học viên

TÓM TẮT LUẬN VĂN

Trong bối cảnh hiện nay, việc phát hiện và phân đoạn các khuyết tật bề mặt của các sản phẩm trong sản xuất công nghiệp là một nhiệm vụ quan trọng để đảm bảo chất lượng và tính đồng nhất của các sản phẩm. Với sự phát triển mạnh mẽ của Học sâu (Deep learning), có rất nhiều phương pháp học sâu hiện đại đã được giới thiệu, các phương pháp này cho phép mô hình có thể nhận diện và phân loại khuyết tật bề mặt một cách chính xác và hiệu quả hơn bao giờ hết.

Trong luận văn này, mô hình MemSeg, một phương pháp học bán giám sát sẽ được tìm hiểu, phân tích, triển khai và đánh giá chi tiết trên một hệ thống phần cứng hoàn chỉnh. Điểm đặc trưng của mô hình này là nó sẽ sử dụng cả ảnh bình thường và ảnh bất thường giả tạo trong quá trình huấn luyện để phân loại và phát hiện trực tiếp các vùng khuyết tật mà không cần tái tạo toàn bộ ảnh đầu vào. Quá trình huấn luyện và suy luận được thực hiện trên bộ dữ liệu tiêu chuẩn MVTec AD[2] với các chỉ số đánh giá AUROC và AUPRO.

Đồng thời, luận văn này cũng nghiên cứu để xây dựng một hệ thống phần cứng để đánh giá thực tế, kết hợp giữa thiết bị nhúng là Kit NVIDIA Jetson Nano và CSI Camera. Hệ thống này cho phép đánh giá thuật toán, khả năng phát hiện cũng như hiệu suất về thời gian của mô hình trong một hệ thống nhúng.

ABSTRACT

In the current industrial context, the detection and segmentation of surface defects in manufactured products play a crucial role in ensuring product quality and consistency. With the rapid advancement of deep learning, many modern approaches have been introduced, enabling models to recognize and classify surface defects more accurately and efficiently than ever before.

In this thesis, MemSeg, a semi-supervised learning model is studied, analyzed, implemented, and thoroughly evaluated on a complete hardware system. The key feature of this model is its use of both normal and artificially generated abnormal images during training, allowing it to classify and localize defects directly, without the need to reconstruct the entire input image. The model is trained and inferred on the benchmark MVTec AD dataset, with these evaluation metrics: AUROC and AUPRO.

In addition, this thesis also involves the design and construction of a practical hardware evaluation system, integrating an NVIDIA Jetson Nano embedded kit and a CSI camera. This system enables evaluation of the algorithm's inference capability and real-time performance within an embedded environment.

Lời cam đoan

Tôi xin cam đoan rằng luận văn này là kết quả của quá trình nghiên cứu và thực hiện nghiêm túc của bản thân. Trong luận văn, tôi có tham khảo mô hình, thuật toán và ý tưởng từ một số công trình nghiên cứu khoa học đã được công bố trước đó, đặc biệt là mô hình MemSeg, nhằm mục đích tìm hiểu, phân tích và đánh giá hiệu quả trong điều kiện cụ thể. Các tài liệu tham khảo đã được trích dẫn và ghi rõ nguồn đầy đủ trong phần Tài liệu tham khảo.

Tôi xin chịu hoàn toàn trách nhiệm về tính trung thực, nội dung trình bày cũng như kết quả nghiên cứu trong luận văn này.

MỤC LỤC

1. GIỚI THIỆU	1
1.1. Tổng quan	1
1.2. Tình hình nghiên cứu trong và ngoài nước	2
1.3. Nhiệm vụ luận văn	3
2. LÝ THUYẾT	4
2.1. Khái niệm	4
2.2. Phương pháp học bán giám sát (semi-supervised learning)	4
2.3. Mô hình MemSeg.....	5
2.4. Phương pháp tạo bất thường mô phỏng	7
3. THIẾT KẾ PHẦN CỨNG HỆ THỐNG.....	10
3.1. Sơ đồ khối tổng quát của hệ thống.....	10
4. THIẾT KẾ PHẦN MỀM HỆ THỐNG.....	14
4.1. Môi trường phát triển phần mềm	14
4.2. Giải thuật đề xuất	16
4.3. Lưu đồ của chương trình.....	28
5. KẾT QUẢ HUẤN LUYỆN VÀ THỰC NGHIỆM.....	38
5.1. Mô tả phần cứng hệ thống.....	38
5.2. Tập dữ liệu MVTec AD	40
5.3. Quá trình huấn luyện mô hình	47
5.4. Kết quả quá trình huấn luyện	51
5.5. Quá trình phát hiện khuyết tật trên Jetson Nano.....	85
6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....	107
6.1. Kết luận	107
6.2. Hướng phát triển	107
TÀI LIỆU THAM KHẢO	108

DANH SÁCH HÌNH MINH HỌA

Hình 2.1: Ba bước tạo bất thường mô phỏng.....	8
Hình 3.1: Sơ đồ khối tổng quát của hệ thống	10
Hình 3.2: Kit NVIDIA Jetson nano	12
Hình 3.3: Camera CSI Raspberry Pi Module V2	13
Hình 4.1: Mô hình giải thuật MemSeg	16
Hình 4.2: Sơ đồ chi tiết giải thuật của MSFF	21
Hình 4.3: Lưu đồ của chương trình huấn luyện	28
Hình 4.4: Lưu đồ của hàm training.....	31
Hình 4.5: Lưu đồ của hàm <i>evaluate</i>	33
Hình 4.6: Lưu đồ của chương trình phát hiện khuyết tật	36
Hình 5.1: Jetson Nano sau khi kết nối với CSI Camera.....	38
Hình 5.2: Hệ thống sau khi kết nối hoàn chỉnh	39
Hình 5.3: Kết nối jetson nano với PC thông qua SSH	39
Hình 5.4: Một số mẫu dữ liệu trong bộ MVTec	41
Hình 5.5: Biểu đồ hàm train loss của bottle.....	54
Hình 5.6: Biểu đồ hàm train L1 loss của bottle.....	54
Hình 5.7: Biểu đồ hàm train focal loss của bottle	55
Hình 5.8: Biểu đồ chỉ số AUROC-image của bottle.....	55
Hình 5.9: Biểu đồ AUROC-pixel của bottle	56
Hình 5.10: Biểu đồ AUPRO-pixel của bottle	56
Hình 5.11: Biểu đồ hàm train loss của capsule	57
Hình 5.12: Biểu đồ hàm train L1 loss của capsule.....	57
Hình 5.13: Biểu đồ hàm train focal loss của capsule.....	58
Hình 5.14: Biểu đồ AUROC-image của Capsule.....	58
Hình 5.15: Biểu đồ AUROC-pixel của Capsule	59

Hình 5.16: Biểu đồ AUPRO-pixel của tập Capsule.....	59
Hình 5.17: Biểu đồ hàm train loss của Pill.....	60
Hình 5.18: Biểu đồ hàm train L1 loss của Pill	60
Hình 5.19: Biểu đồ hàm train focal loss của Pill	61
Hình 5.20: Biểu đồ AUROC-image của Pill.....	61
Hình 5.21: Biểu đồ AUROC-pixel của Pill	62
Hình 5.22: Biểu đồ AUPRO-pixel của Pill	62
Hình 5.23: Biểu đồ hàm train loss của Leather	63
Hình 5.24: Biểu đồ hàm train L1 loss của tập Leather	63
Hình 5.25: Biểu đồ hàm train focal loss của Leather	64
Hình 5.26: Biểu đồ AUROC-image của Leather	64
Hình 5.27: Biểu đồ AUROC-pixel của tập Leather	65
Hình 5.28: Biểu đồ AUPRO-pixel của tập Leather	65
Hình 5.29: Biểu đồ hàm train loss của Tile.....	66
Hình 5.30: Biểu đồ hàm train L1 loss của Tile	66
Hình 5.31: Biểu đồ hàm train focal loss của Tile	67
Hình 5.32: Biểu đồ AUROC-image của Tile.....	67
Hình 5.33: Biểu đồ AUROC-pixel của Tile	68
Hình 5.34: Biểu đồ AUPRO-pixel của Tile	68
Hình 5.35: Biểu đồ hàm train loss của Grid.....	69
Hình 5.36: Biểu đồ hàm train L1 loss của Grid	69
Hình 5.37: Biểu đồ hàm train focal loss của Grid	70
Hình 5.38: Biểu đồ AUROC-image của Grid.....	70
Hình 5.39: Biểu đồ AUROC-pixel của Grid.....	71
Hình 5.40: Biểu đồ AUPRO-pixel của Grid.....	71
Hình 5.41: Biểu đồ hàm train loss của Wood.....	72

Hình 5.42: Biểu đồ hàm train L1 loss của Wood	72
Hình 5.43: Biểu đồ hàm train focal loss của Wood	73
Hình 5.44: Biểu đồ AUROC-image của Wood.....	73
Hình 5.45: Biểu đồ AUROC-pixel của Wood	74
Hình 5.46: Biểu đồ AUPRO-pixel của Wood	74
Hình 5.47: Biểu đồ hàm train loss của hazelnut.....	75
Hình 5.48: Biểu đồ hàm train L1 loss của hazelnut	75
Hình 5.49: Biểu đồ hàm train focal loss của hazelnut.....	76
Hình 5.50: Biểu đồ AUROC-image của hazelnut	76
Hình 5.51: Biểu đồ AUROC-pixel của hazelnut.....	77
Hình 5.52: Biểu đồ AUPRO-pixel của tập hazelnut	77
Hình 5.53: Biểu đồ hàm train loss của carpet	78
Hình 5.54: Biểu đồ hàm train L1 loss của carpet.....	78
Hình 5.55: Biểu đồ hàm train focal loss của carpet	79
Hình 5.56: Biểu đồ AUROC-image của carpet	79
Hình 5.57: Biểu đồ AUROC-pixel của carpet	80
Hình 5.58: Biểu đồ AUPRO-pixel của carpet	80
Hình 5.59: Biểu đồ hàm train loss của metal_nut	81
Hình 5.60: Biểu đồ hàm train L1 loss của metal_nut	81
Hình 5.61: Biểu đồ train focal loss của metal_nut	82
Hình 5.62: Biểu đồ AUROC-image của metal_nut.....	82
Hình 5.63: Biểu đồ AUROC-image của metal_nut.....	83
Hình 5.64: Biểu đồ AUPRO-pixel của metal_nut.....	83
Hình 5.65: Giao diện kết quả phát hiện của hệ thống	85
Hình 5.66: Phát hiện cho mẫu chai (bottle) bị vỡ lớn	86
Hình 5.67: Phát hiện cho mẫu chai (bottle) bị vỡ nhỏ	86

Hình 5.68: Phát hiện cho mẫu chai (bottle) bị nhiễm bẩn	86
Hình 5.69: Phát hiện cho mẫu chai (bottle) bình thường	86
Hình 5.70: Phát hiện cho mẫu viên nang (capsule) bị nứt vỏ	87
Hình 5.71: Phát hiện cho mẫu viên nang (capsule) bị thủng	87
Hình 5.72: Phát hiện cho mẫu viên nang (capsule) bị trầy xước	88
Hình 5.73: Phát hiện cho mẫu viên nang (capsule) bị móp méo	88
Hình 5.74: Phát hiện cho mẫu viên nang (capsule) bình thường	88
Hình 5.75: Phát hiện cho mẫu viên thuốc (pill) bị dính màu.....	89
Hình 5.76: Phát hiện cho mẫu viên thuốc (pill) bị dính màu.....	89
Hình 5.77: Phát hiện cho mẫu viên thuốc (pill) nhiễm bẩn bề mặt	90
Hình 5.78: Phát hiện cho mẫu viên thuốc (pill) bị nứt vỡ cạnh	90
Hình 5.79: Phát hiện cho mẫu viên thuốc (pill) bị trầy xước bề mặt	90
Hình 5.80: Phát hiện cho mẫu viên thuốc (pill) bị nứt vỡ cạnh và trầy xước	90
Hình 5.81: Phát hiện cho mẫu viên thuốc (pill) bị nứt vỡ 2 cạnh	91
Hình 5.82: Phát hiện cho mẫu viên thuốc (pill) bị in sai ký hiệu	91
Hình 5.83: Phát hiện cho mẫu viên thuốc (pill) bình thường.....	91
Hình 5.84: Phát hiện cho mẫu thảm (carpet) bị dính màu.....	92
Hình 5.85: Phát hiện cho mẫu thảm (carpet) bị rách.....	92
Hình 5.86: Phát hiện cho mẫu thảm (carpet) bị thủng lỗ	93
Hình 5.87: Phát hiện cho mẫu thảm (carpet) bị dị vật kim loại bề mặt.....	93
Hình 5.88: Phát hiện cho mẫu thảm (carpet) bị lỗi sợi chỉ.....	93
Hình 5.89: Phát hiện cho mẫu thảm (carpet) bình thường.....	93
Hình 5.90: Phát hiện cho mẫu da (leather) dính màu.....	94
Hình 5.91: Phát hiện cho mẫu da (leather) bị rách	94
Hình 5.92: Phát hiện cho mẫu da (leather) bị nhăn	95
Hình 5.93: Phát hiện cho mẫu da (leather) bị dính keo	95

Hình 5.94: Phát hiện cho mẫu da (leather) bị thủng lỗ.....	95
Hình 5.95: Phát hiện cho mẫu da (leather) bình thường.....	95
Hình 5.96: Phát hiện cho mẫu gạch lát (tile) bị nứt vỡ	96
Hình 5.97: Phát hiện cho mẫu gạch lát (tile) có vết xám bất thường	96
Hình 5.98: Phát hiện cho mẫu gạch lát (tile) bị dính dầu.....	97
Hình 5.99: Phát hiện cho mẫu gạch lát (tile) bị trầy xước	97
Hình 5.100: Phát hiện cho mẫu gạch lát (tile) bị vệt keo	97
Hình 5.101: Phát hiện cho mẫu gạch lát (tile) bình thường.....	97
Hình 5.102: Phát hiện cho mẫu gỗ (wood) bị dính màu	98
Hình 5.103: Phát hiện cho mẫu gỗ (wood) bị thủng lỗ	98
Hình 5.104: Phát hiện cho mẫu gỗ (wood) có vết loang, thấm nước	99
Hình 5.105: Phát hiện cho mẫu gỗ (wood) bị trầy xước	99
Hình 5.106: Phát hiện cho mẫu gỗ (wood) bị dính màu và thủng lỗ	99
Hình 5.107: Phát hiện cho mẫu gỗ (wood) bình thường	99
Hình 5.108: Phát hiện cho mẫu lưới (grid) bị cong	100
Hình 5.109: Phát hiện cho mẫu lưới (grid) bị gãy	100
Hình 5.110: Phát hiện cho mẫu lưới (grid) bị dính keo	101
Hình 5.111: Phát hiện cho mẫu lưới (grid) có dị vật	101
Hình 5.112: Phát hiện cho mẫu lưới (grid) bình thường.....	101
Hình 5.113: Phát hiện cho mẫu đai ốc (metal nut) dính mực.....	102
Hình 5.114: Phát hiện cho mẫu đai ốc (metal nut) bị cong	102
Hình 5.115: Phát hiện cho mẫu đai ốc (metal nut) bị trầy xước.....	102
Hình 5.116: Phát hiện cho mẫu đai ốc (metal nut) bình thường.....	103
Hình 5.117: Phát hiện cho mẫu hạt phỉ (hazelnut) bị nứt.....	104
Hình 5.118: Phát hiện cho mẫu hạt phỉ (hazelnut) bị thủng lỗ	104
Hình 5.119: Phát hiện cho mẫu hạt phỉ (hazelnut) bị lỗi in ấn.....	104

Hình 5.120: Phát hiện cho mẫu hạt phỉ (hazelnut) bị trầy xước.....	104
Hình 5.121: Phát hiện cho mẫu hạt phỉ (hazelnut) bình thường.....	105

DANH SÁCH BẢNG SỐ LIỆU

Bảng 1: Tổng hợp kết quả huấn luyện	52
--	----

1. GIỚI THIỆU

1.1. Tổng quan

Hiện nay, trong ngành công nghiệp sản xuất hiện đại, việc phát hiện và xử lý các khuyết tật bề mặt trên sản phẩm đã trở thành một trong những yêu cầu quan trọng nhất nhằm đảm bảo chất lượng đầu ra và duy trì tính đồng nhất trong dây chuyền sản xuất. Các khuyết tật như vết nứt, trầy xước, bong tróc, vết lõm hay các khiếm khuyết vi mô khác có thể ảnh hưởng trực tiếp đến chức năng, độ bền hoặc thẩm mỹ của sản phẩm, đặc biệt trong các ngành sản xuất yêu cầu độ chính xác cao như điện tử, ô tô, hàng không, và y tế. Trước đây, việc kiểm tra này thường được thực hiện thủ công hoặc bằng các phương pháp truyền thống, tuy nhiên, các phương pháp này tồn tại nhiều hạn chế như tốn thời gian, dễ bị sai sót và không thể phát hiện được tất cả các khuyết tật tinh vi. Với sự phát triển mạnh mẽ của Deep Learning, xu hướng hiện nay là xây dựng các hệ thống kiểm tra khuyết tật bề mặt thời gian thực (real-time) và tự động dựa trên các mô hình thuật toán của deep learning, các hệ thống kiểm tra tự động này đã có thể nhận diện và phân loại các khuyết tật bề mặt một cách chính xác và hiệu quả.

Deep learning, đặc biệt là các mạng nơ-ron tích chập (CNN), đã chứng minh khả năng vượt trội trong việc xử lý hình ảnh và phát hiện các bất thường. Các mô hình học sâu có thể được huấn luyện để học từ một lượng lớn dữ liệu hình ảnh, giúp chúng có khả năng nhận diện hầu hết các dạng khuyết tật trên bề mặt của sản phẩm mà không cần đến sự can thiệp của con người. Điều này không chỉ nâng cao độ chính xác mà còn tiết kiệm thời gian và chi phí so với các phương pháp truyền thống. Hơn nữa, nhờ vào tính linh hoạt và khả năng thích nghi với dữ liệu mới, các hệ thống học sâu có thể được triển khai dễ dàng cho nhiều loại sản phẩm và dây chuyền sản xuất khác nhau mà không cần thay đổi cấu trúc phần cứng.

Một số ứng dụng cụ thể của việc ứng dụng deep learning trong việc phát hiện khuyết tật bề mặt bao gồm nhận dạng và phát hiện vết nứt nhỏ trên bo mạch, nhận dạng trầy xước hoặc bong tróc trên bề mặt kim loại, gỗ, da, xác định các khuyết tật do lỗi in hoặc ép khuôn trong công nghiệp bao bì, và nhiều trường hợp tương tự

khác. Nhờ vào khả năng học từ dữ liệu, các mô hình học sâu có thể phân biệt được những khuyết tật nhỏ nhất mà con người có thể bỏ qua. Việc áp dụng deep learning trong kiểm tra chất lượng sản phẩm không chỉ giúp nâng cao năng suất mà còn cải thiện chất lượng sản phẩm, giảm thiểu sai sót và tăng cường hiệu quả của các quy trình sản xuất công nghiệp.

1.2. Tình hình nghiên cứu trong và ngoài nước

Trong lĩnh vực phát hiện khuyết tật bề mặt trong công nghiệp, nhiều mô hình học sâu hiện đại đã và đang được áp dụng để cải thiện độ chính xác và hiệu quả của quá trình kiểm tra chất lượng sản phẩm. Những mô hình này không chỉ giúp phát hiện các khuyết tật một cách tự động mà còn giảm thiểu sự can thiệp của con người, tiết kiệm thời gian và chi phí. Các phương pháp này có thể xử lý và phân tích hàng loạt hình ảnh sản phẩm trong thời gian ngắn, đồng thời nhận diện các bất thường mà mắt người khó có thể phát hiện ra.

Các phương pháp phát hiện khuyết tật hiện nay có thể được chia thành ba nhóm chính: (1) phương pháp dựa trên tái tạo (reconstruction-based), (2) phương pháp mô phỏng bất thường (anomaly simulation-based) và (3) phương pháp dựa trên đặc trưng (embedding-based).

Trong nhóm đầu tiên, các mô hình như Autoencoder (AE) [3,4] hoặc Generative Adversarial Network (GAN) [5] được huấn luyện để tái tạo lại hình ảnh bình thường, và phát hiện khuyết tật thông qua sai lệch giữa ảnh gốc và ảnh tái tạo. Tuy nhiên, do khả năng học mạnh của mạng nơ-ron, các vùng bất thường đôi khi cũng bị tái tạo đúng, khiến mô hình bỏ sót lỗi. Nhóm thứ hai tạo ra các mẫu bất thường nhân tạo bằng cách biến đổi ảnh bình thường, ví dụ như CutPaste [14], DRAEM [9] hay AnoSeg [15], giúp mô hình học được sự khác biệt giữa bình thường và bất thường mà không cần dữ liệu lỗi thật. Dù vậy, nhiều phương pháp trong nhóm này vẫn cần tái tạo ảnh hoặc chỉ hỗ trợ phân loại, không thực hiện được phân đoạn chính xác.

Nhóm thứ ba sử dụng các mô hình tiền huấn luyện (như ResNet [7], ImageNet [8]) để trích xuất đặc trưng ảnh, sau đó so sánh khoảng cách giữa ảnh test và tập

mẫu bình thường để xác định bất thường. Các đại diện tiêu biểu gồm SPADE [16] và PaDiM [17]. Tuy nhiên, các phương pháp này thường yêu cầu quá trình so khớp đặc trưng phức tạp trong giai đoạn suy luận, ảnh hưởng đến tốc độ và hiệu quả thời gian thực.

1.3. Nhiệm vụ luận văn

Nội dung 1: Tìm hiểu lý thuyết về các mô hình Deep learning trong phát hiện khuyết tật bề mặt cho các sản phẩm trong công nghiệp. Lựa chọn mô hình và giải thuật phù hợp.

Nội dung 2: Phân tích lý thuyết mô hình giải thuật đã chọn.

Nội dung 3: Thiết kế hệ thống phần cứng để áp dụng mô hình thuật toán đã chọn.

Nội dung 4: Thiết kế phần mềm, chương trình cho hệ thống.

Nội dung 5: Thực hiện huấn luyện mô hình trên tập dữ liệu, thực hiện phát hiện khuyết tật bề mặt trên bộ kiểm thử.

Nội dung 6: Đánh giá các thông số huấn luyện và hiệu suất phát hiện khuyết tật bề mặt.

2. LÝ THUYẾT

2.1. Khái niệm

Phát hiện các bất thường trên bề mặt sản phẩm là một bài toán quan trọng trong kiểm tra chất lượng công nghiệp, đặc biệt trong các dây chuyền sản xuất hiện đại, nơi mà tốc độ và độ chính xác là yếu tố then chốt. Bài toán này nhằm xác định vị trí và hình dạng của các vùng bất thường trong ảnh sản phẩm, như vết xước, trầy, nứt hoặc các khiếm khuyết về cấu trúc và kết cấu.

Tuy nhiên, trong thực tế sản xuất, các mẫu chứa khuyết tật thường rất hiếm và đa dạng, gây khó khăn cho các phương pháp học có giám sát truyền thống vốn đòi hỏi dữ liệu nhãn đầy đủ. Vì vậy, các phương pháp học bán giám sát (semi-supervised learning) đã được nghiên cứu và ứng dụng để giải quyết bài toán này. Thay vì yêu cầu lượng lớn ảnh lỗi có nhãn thực tế, các phương pháp này chỉ sử dụng ảnh bình thường và một lượng nhỏ ảnh có nhãn trong quá trình huấn luyện, và thông qua việc học các đặc trưng phổ biến (common patterns) của ảnh bình thường, mô hình có thể phát hiện các điểm khác biệt bất thường trong ảnh mới.

Gần đây, các phương pháp hiện đại như **MemSeg [1]** kết hợp giữa phương pháp tạo bất thường giả lập và mô-đun bộ nhớ (memory module) đã cho thấy hiệu quả cao trong việc xác định chính xác các vùng lỗi ở mức điểm ảnh (pixel-level), đồng thời đạt tốc độ xử lý phù hợp với yêu cầu thời gian thực trong công nghiệp. Điều này mở ra hướng tiếp cận mới cho các hệ thống kiểm tra chất lượng tự động, linh hoạt và tiết kiệm chi phí.

2.2. Phương pháp học bán giám sát (semi-supervised learning)

Học bán giám sát (Semi-Supervised Learning – SSL) là một nhánh quan trọng trong học máy. Phương pháp này kết hợp giữa học có giám sát (supervised learning) và học không giám sát (unsupervised learning). Mục tiêu của SSL là khai thác hiệu quả cả dữ liệu có nhãn và dữ liệu không nhãn, từ đó cải thiện độ chính xác của mô hình mà không đòi hỏi khối lượng lớn dữ liệu được gán nhãn thủ công, vốn rất tốn kém và khó khăn trong thực tế.

Học có giám sát là phương pháp huấn luyện mô hình từ dữ liệu đã được gán nhãn đầy đủ. Mỗi mẫu dữ liệu đầu vào đi kèm với một nhãn đầu ra tương ứng, giúp mô hình học được ánh xạ giữa đặc trưng và mục tiêu. Mục tiêu là giảm sai số dự đoán trên các mẫu chưa từng thấy. Phương pháp này thường cho kết quả cao khi có đủ dữ liệu chất lượng. Tuy nhiên, việc thu thập và gán nhãn dữ liệu có thể rất tốn kém và không khả thi trong nhiều tình huống.

Học không giám sát là phương pháp sử dụng dữ liệu chưa được gán nhãn để khám phá các cấu trúc tiềm ẩn trong dữ liệu. Mô hình học từ sự phân bố và mối quan hệ giữa các mẫu để thực hiện các nhiệm vụ như phân cụm, phát hiện bất thường, hoặc giảm chiều dữ liệu. Không cần nhãn giúp tiết kiệm chi phí và dễ áp dụng với dữ liệu lớn. Tuy nhiên, việc đánh giá chất lượng mô hình và xác định mục tiêu rõ ràng thường khó khăn hơn. Phương pháp này phù hợp với các bài toán khai phá dữ liệu ban đầu.

2.3. Mô hình MemSeg

Mô hình MemSeg là một phương pháp phát hiện và phân đoạn khuyết tật bề mặt sản phẩm công nghiệp dựa trên học sâu, được thiết kế để hoạt động hiệu quả trong môi trường sản xuất thực tế. Mô hình này được xây dựng trong khuôn khổ học bán giám sát, trong đó chỉ sử dụng các hình ảnh bình thường kết hợp với các bất thường giả lập (simulated anomalies) trong giai đoạn huấn luyện, nhằm giúp mô hình học được sự khác biệt giữa bình thường và bất thường. Điều này cho phép MemSeg đạt được hiệu quả cao mà không cần đến các mẫu lỗi thực tế đã được gán nhãn, vốn thường rất hiếm và tốn kém trong các ứng dụng công nghiệp.

MemSeg được xây dựng trên nền tảng U-Net [6], một mạng nơ-ron tích chập (CNN) được thiết kế đặc biệt cho các bài toán phân đoạn ảnh. Phần mã hóa (Encoder) trong MemSeg sử dụng ResNet18 [7], là một mạng nơ-ron sâu đã được huấn luyện trước trên ImageNet [8], điều này giúp mô hình tận dụng các đặc trưng hình ảnh cấp cao ngay từ đầu, đồng thời giảm bớt thời gian và chi phí huấn luyện. Mô hình này cho phép huấn luyện theo hướng đầu-cuối (end-to-end), tức là không

cần tách riêng giai đoạn trích đặc trưng và phân đoạn, mà toàn bộ quy trình được học đồng thời trong một pipeline duy nhất.

Tương tự như học không giám sát (SSL), MemSeg thực hiện tạo ra các bất thường giả lập trong giai đoạn huấn luyện để giúp mô hình phân biệt có ý thức giữa bình thường và không bình thường, mà không yêu cầu các bất thường này phải giống với những bất thường thực tế trong các tình huống thực tế, điều này giúp khắc phục hạn chế của SSL truyền thống đồng thời cho phép mô hình học được ranh giới phân loại vững chắc hơn. MemSeg sử dụng cả hình ảnh bình thường và hình ảnh bất thường giả lập để hoàn tất quá trình huấn luyện, đồng thời xác định trực tiếp các vùng bất thường trong ảnh đầu vào mà không cần bất kỳ tác vụ phụ trợ nào trong giai đoạn phát hiện khuyết tật bề mặt.

MemSeg sẽ trích xuất hai loại thông tin là Sự khác biệt và Sự tương đồng (Differences and Commonalities) trong các sản phẩm. Đây là một trong những điểm nổi bật của phương pháp đề xuất, điều này sẽ giúp mô hình có khả năng học tốt hơn. Nhờ vào Sự khác biệt, mô hình có thể học được những điểm khác nhau giữa các sản phẩm, và nhờ vào Sự tương đồng, mô hình sẽ nhận diện được các đặc điểm chung giữa các sản phẩm. Điều này giúp mô hình hiểu rõ hơn các mẫu bình thường và phân biệt chính xác các mẫu không bình thường.

MemSeg cũng giới thiệu một bộ nhớ (memory pool) để ghi lại các mẫu chung của các hình ảnh bình thường. Trong quá trình huấn luyện và phát hiện khuyết tật của mô hình, MemSeg so sánh sự tương đồng và khác biệt giữa các mẫu đầu vào và các mẫu trong bộ nhớ để cung cấp thông tin hiệu quả hơn cho việc định vị các vùng bất thường. Thêm vào đó, để phối hợp thông tin từ bộ nhớ và hình ảnh đầu vào một cách hiệu quả hơn, MemSeg giới thiệu một mô-đun Tổng hợp đặc trưng đa tỷ lệ (multi-scale feature fusion module-MSFF) và một Bản đồ chú ý không gian (Spatial Attention Maps). MSFF cho phép mô hình kết hợp thông tin từ nhiều cấp độ trích xuất đặc trưng khác nhau, từ các đặc trưng cục bộ đến toàn cục, giúp mô hình nhìn nhận được bất thường cả ở quy mô nhỏ và lớn. Trong khi đó, Bản đồ chú ý không gian giúp mô hình tập trung hơn vào các vùng có khả năng chứa bất thường, thay vì

dàn trải sự chú ý ra toàn bộ ảnh, từ đó tăng độ chính xác trong quá trình phân đoạn. Việc kết hợp hai thành phần này sẽ giúp cải thiện đáng kể hiệu suất của mô hình.

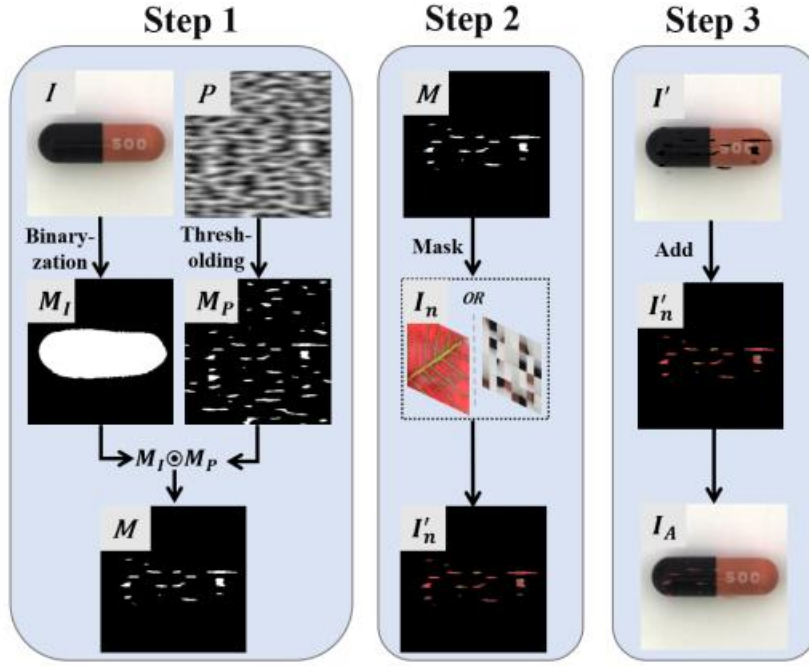
2.4. Phương pháp tạo bất thường mô phỏng

Trong các môi trường công nghiệp, bất thường có thể xuất hiện dưới nhiều hình thức khác nhau và không thể thu thập được tất cả các dạng này trong quá trình thu thập dữ liệu, điều này gây hạn chế cho việc xây dựng mô hình bằng các phương pháp học có giám sát. Tuy nhiên, trong khuôn khổ học bán giám sát, việc chỉ sử dụng các mẫu bình thường và không có sự so sánh với các mẫu không bình thường là chưa đủ để mô hình học được đâu là các mẫu chuẩn. Lấy cảm hứng từ mô hình DRAEM [9], một phương pháp tạo bất thường mô phỏng hiệu quả hơn được đề xuất và đồng thời đưa các mẫu mô phỏng này vào quá trình huấn luyện.

Bước đầu tiên, một nhiễu Perlin [10] hai chiều (P) sẽ được tạo ra, sau đó P được nhị phân hóa bằng một ngưỡng T để thu được mặt nạ M_p . Nhiễu Perlin có nhiều điểm cực trị ngẫu nhiên, và M_p được tạo ra từ nó có thể trích xuất các vùng liên tục trong ảnh.

Đồng thời, do tỷ lệ kích thước của một số thành phần công nghiệp trong ảnh thu được là nhỏ, nếu tăng cường dữ liệu được thực hiện trực tiếp mà không xử lý, sẽ dễ tạo ra nhiễu ở phần nền của ảnh. Điều này làm tăng sự khác biệt giữa các mẫu bất thường mô phỏng và các mẫu bất thường thực tế về mặt phân bố dữ liệu, khiến mô hình khó học được đặc trưng phân biệt hiệu quả.

Do đó, đề xuất áp dụng một chiến lược tăng cường tiền cảnh cho loại ảnh này. Cụ thể, ảnh đầu vào I được nhị phân hóa để thu được mặt nạ M_I , và nhiễu sinh ra trong quá trình nhị phân hóa được loại bỏ bằng các thao tác mở (open) hoặc đóng (close). Sau đó, mặt nạ cuối cùng M được tạo ra bằng cách thực hiện phép nhân từng phần tử giữa hai mặt nạ đã thu được.



Hình 2.1: Ba bước tạo bất thường mô phỏng

Bước thứ hai, ảnh mặt nạ M và ảnh nhiễu I_n được thực hiện phép nhân từng phần tử để lấy vùng quan tâm (ROI) trong I_n , được xác định bởi mặt nạ M . Tương tự như DRAEM [9], ta đưa vào một hệ số trọng số δ trong quá trình này nhằm cân bằng sự pha trộn giữa ảnh gốc và ảnh nhiễu, sao cho các mẫu bất thường mô phỏng có đặc trưng gần hơn với các bất thường thực tế. Do đó, ảnh tiền cảnh có nhiễu I'_n được tạo ra theo công thức sau:

$$I'_n = \delta(M \odot I_n) + (1 - \delta)(M \odot I) \quad (1)$$

Với ảnh nhiễu I_n , ta mong muốn độ trong suốt lớn nhất có thể nhằm tăng độ khó trong việc học của mô hình, từ đó cải thiện độ bền vững. Vì vậy, δ sẽ được lấy ngẫu nhiên trong khoảng $[0.15, 1]$.

Bước thứ ba, mặt nạ M được đảo ngược để thu được $\bar{M}$, sau đó thực hiện phép nhân từng phần tử giữa $\bar{M}$ và ảnh gốc I để thu được ảnh I' . Cuối cùng, ảnh đầu vào có tăng cường dữ liệu I_A được tính như sau:

$$I_A = \bar{M} \odot I + I'_n \quad (2)$$

Ảnh I_A , tức là ảnh bất thường mô phỏng, sẽ lấy ảnh đầu vào ban đầu I làm nền và vùng ROI trong ảnh nhiễu I_n được trích xuất bởi mặt nạ M làm tiền cảnh.

Trong đó, ảnh nhiễu I_n được tạo thành từ hai phần: một phần từ tập dữ liệu kết cấu DTD [12], nhằm mô phỏng các bất thường về kết cấu và phần còn lại từ chính ảnh đầu vào, nhằm mô phỏng các bất thường về cấu trúc.

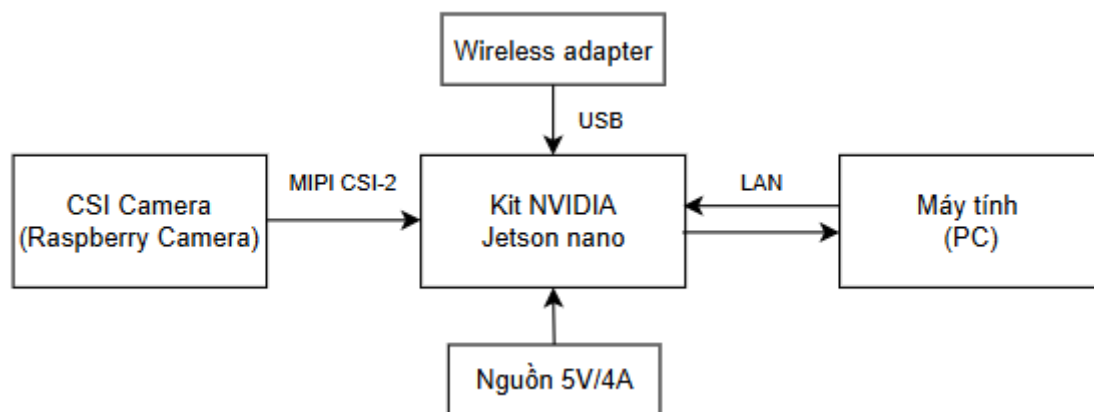
Để mô phỏng các bất thường cấu trúc, trước tiên cần thực hiện các điều chỉnh ngẫu nhiên như lấy đối xứng, xoay, thay đổi độ sáng, độ bão hòa và sắc độ trên ảnh đầu vào I . Sau đó, ảnh đã được xử lý sơ bộ sẽ được chia đều thành một lưới 4×8 và sắp xếp ngẫu nhiên để thu được ảnh I_n bị xáo trộn.

Với phương pháp tạo bất thường mô phỏng như trên, ta có thể tạo ra các mẫu bất thường mô phỏng từ cả hai góc độ kết cấu và cấu trúc. Phần lớn các vùng bất thường được sinh ra trên khu vực tiền cảnh mục tiêu, điều này giúp tối đa hóa sự tương đồng giữa các mẫu bất thường mô phỏng và các mẫu bất thường thực tế.

3. THIẾT KẾ PHẦN CỨNG HỆ THỐNG

Ở phần trên ta đã trình bày lý thuyết và giới thiệu mô hình MemSeg[1] trong phát hiện khuyết tật bề mặt. Chương này ta sẽ tập trung mô tả một hệ thống phần cứng áp dụng mô hình này vào thực tế. Hệ thống này sẽ bao gồm các thành phần chính như Kit Jetson nano, Camera và máy tính.

3.1. Sơ đồ khối tổng quát của hệ thống



Hình 3.1: Sơ đồ khối tổng quát của hệ thống

- **NVIDIA Jetson Nano Developer Kit**

Jetson Nano là một nền tảng máy tính nhúng hiệu năng cao do NVIDIA phát triển, được thiết kế đặc biệt để phục vụ cho các ứng dụng trí tuệ nhân tạo (AI) và học sâu (deep learning) trên các thiết bị nhúng có hạn chế về tài nguyên và năng lượng. Với một cấu hình mạnh mẽ, dễ triển khai và giá thành tốt, Jetson Nano là một lựa chọn lý tưởng cho các dự án như nhận dạng hình ảnh, phân tích video thời gian thực, robot tự hành, Internet of Things (IoT) và nhiều ứng dụng học sâu khác trong công nghiệp.

Kit Jetson Nano được trang bị vi xử lý ARM Cortex-A57 và GPU kiến trúc Maxwell gồm 128 nhân cho phép xử lý song song hiệu quả và đáp ứng tốt các mô hình học sâu phổ biến. Bên cạnh đó, Jetson Nano có bộ nhớ RAM 4GB, mang lại tốc độ truy xuất dữ liệu nhanh chóng và đủ khả năng xử lý nhiều tác vụ suy luận phức tạp trong thời gian thực. Nó hỗ trợ nhiều framework hữu dụng cho deep

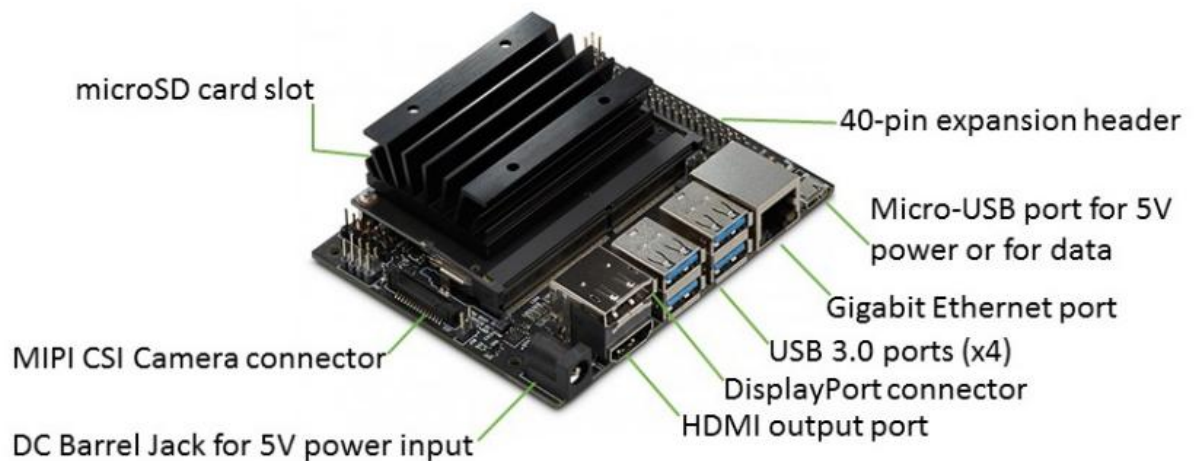
learning như TensorFlow, PyTorch, Caffe, Keras,... và đặc biệt là bộ công cụ JetPack SDK do NVIDIA cung cấp, bao gồm CUDA, cuDNN và TensorRT giúp tối ưu hóa hiệu suất suy luận (inferencing) trên thiết bị.

Vi xử lý ARM Cortex-A57 là một vi xử lý hiệu năng cao thuộc kiến trúc ARMv8A, được thiết kế để cung cấp sức mạnh tính toán vượt trội cho các thiết bị nhúng và di động. Đây là một CPU 64-bit, hỗ trợ cả hai chế độ thực thi là 32-bit (AArch32) và 64-bit (AArch64), giúp tương thích tốt với nhiều nền tảng phần mềm và tận dụng được hiệu suất của các ứng dụng hiện đại. ARM Cortex-A57 có 4 lõi (quad-core), hoạt động ở xung nhịp tối đa khoảng 1.43 GHz, cho phép thực hiện các tác vụ hệ thống và xử lý dữ liệu nhẹ song song với hoạt động của GPU. Khi kết hợp với GPU Maxwell 128 nhân, CPU này đóng vai trò xử lý chính cho các thao tác logic, điều khiển luồng dữ liệu và hỗ trợ suy luận ở tầng ứng dụng.

Một trong những điểm mạnh lớn nhất của Jetson Nano là kích thước nhỏ gọn và tiêu thụ điện năng thấp (thường chỉ 5–10W), trong khi vẫn đảm bảo khả năng xử lý mạnh mẽ. Nhờ đó, thiết bị này đặc biệt phù hợp với các hệ thống nhúng di động, không gian hạn chế hoặc các ứng dụng chạy bằng pin.

Ngoài ra, Jetson Nano còn có khả năng mở rộng cao với nhiều cổng kết nối như GPIO, I2C, UART, USB, HDMI, CSI camera..., giúp các nhà phát triển dễ dàng tích hợp vào các hệ thống robot, máy ảnh thông minh, hay các thiết bị IoT tùy chỉnh.

Trong hệ thống phát hiện khuyết tật bề mặt sử dụng mô hình MemSeg, Jetson Nano đóng vai trò là nền tảng phần cứng cho quá trình suy luận (inference). Mô hình MemSeg, sau khi được huấn luyện, sẽ cho đầu ra là các tệp chứa trọng số, các tệp này có thể được cập nhật cho Jetson nano theo thời gian thực bởi kết nối LAN với máy tính. Sau đó, jetson nano sẽ tiến hành suy luận để phát hiện các khuyết tật bề mặt dựa trên đầu vào là thư mục ảnh kiểm tra (test data) trong thư mục bộ nhớ hoặc được ghi lại từ CSI Camera được kết nối trên kit. Tất cả kết quả sẽ được gửi lên một giao diện trên trình duyệt để người dùng có thể tương tác và giám sát một cách dễ dàng thông qua máy tính (PC).



Hình 3.2: Kit NVIDIA Jetson nano

- **Camera CSI Raspberry Pi Module V2**

Camera CSI Raspberry Pi Module V2 là một camera được phát triển bởi Raspberry Pi Foundation, sử dụng cảm biến hình ảnh Sony IMX219 với độ phân giải 8 megapixel. Thiết bị này hỗ trợ chụp ảnh tĩnh với độ phân giải lên đến 3280×2464 pixel và quay video với độ phân giải Full HD (1080p) ở tốc độ 30 khung hình/giây, giúp thu được hình ảnh chất lượng cao với độ chi tiết rõ nét – một yếu tố quan trọng trong các ứng dụng phát hiện dị thường bề mặt.

Camera được kết nối với Jetson Nano thông qua giao diện CSI (Camera Serial Interface), mang lại băng thông cao và độ trễ thấp hơn so với các kết nối USB thông thường. Điều này rất có lợi cho các hệ thống yêu cầu xử lý hình ảnh theo thời gian thực. Hơn nữa, giao diện CSI giúp giảm tải cho CPU vì dữ liệu hình ảnh được truyền trực tiếp đến GPU để xử lý – một điểm cộng lớn trong các hệ thống nhúng sử dụng mô hình học sâu như MemSeg. Sau khi kết nối thành công, camera có thể được kiểm tra và sử dụng thông qua các thư viện như GStreamer, OpenCV hoặc v4l2-ctl.

Với kích thước nhỏ gọn, trọng lượng nhẹ, giá thành t và khả năng tích hợp dễ dàng, Camera CSI V2 là lựa chọn lý tưởng cho các ứng dụng kiểm tra chất lượng sản phẩm trong môi trường công nghiệp. Khi được kết hợp với Jetson Nano, hệ thống có thể hoạt động một cách linh hoạt, hiệu quả và tiết kiệm năng lượng, đảm bảo khả năng phát hiện khuyết tật bề mặt một cách nhanh chóng và chính xác.



Hình 3.3: Camera CSI Raspberry Pi Module V2

- **TP-link wireless N nano USB adapter**

TP-Link Wireless N Nano USB Adapter (model TL-WN725N) là một thiết bị thu sóng Wi-Fi nhỏ gọn, được thiết kế để mang lại khả năng kết nối không dây cho các thiết bị không có sẵn Wi-Fi tích hợp chẳng hạn như máy tính để bàn hoặc các nền tảng nhúng như Jetson Nano. Chuẩn Wi-Fi 802.11n (Wireless N) hỗ trợ tốc độ truyền tải lên đến 150 Mbps, đáp ứng đủ nhu cầu kết nối Internet và truyền dữ liệu cho nhiều ứng dụng nhúng. Kết nối và tương thích dễ dàng với Kit Jetson nano thông qua USB.

- **Máy tính (PC)**

Trong hệ thống này, PC đảm nhận việc huấn luyện mô hình phát hiện khuyết tật bề mặt sử dụng framework MemSeg. Sau khi hoàn tất quá trình huấn luyện, các trọng số mô hình được truyền trực tiếp qua mạng LAN đến thiết bị Jetson Nano để thực hiện phát hiện khuyết tật. Sau đó, thông qua địa chỉ IP nội bộ (LAN), và người dùng chỉ cần mở trình duyệt từ PC để truy cập web cục bộ chứa giao diện được chạy trên jetson nano. Qua giao diện đó, người dùng có thể chọn loại mô hình, duyệt qua

ảnh đầu vào, xem kết quả suy luận dưới dạng bản đồ bất thường, mặt nạ phân đoạn, điểm số bất thường, và phân lớp đầu ra.

4. THIẾT KẾ PHẦN MỀM HỆ THỐNG

Trong chương này, ta sẽ trình bày về giải thuật đề nghị và thực hiện thiết kế phần mềm cho hệ thống để thực hiện các chức năng như huấn luyện mô hình dựa vào thuật toán tham khảo đã đề cập ở phần lý thuyết, chương trình thực hiện việc suy luận, dự đoán trên Kit NVIDIA Jetson nano với các trọng số được cập nhật từ máy tính (PC) sau khi hoàn tất huấn luyện, chương trình này còn thực hiện tương tác CSI camera cũng như thực hiện một giao diện người dùng trên trình duyệt để tương tác với hệ thống và quan sát kết quả.

4.1. Môi trường phát triển phần mềm

Môi trường thiết kế phần mềm được xây dựng trên nền tảng ngôn ngữ lập trình Python 3 và thư viện PyTorch 1.8, đây là nền tảng rất phù hợp cho các ứng dụng thị giác máy tính và học sâu.

PyTorch là một thư viện mã nguồn mở dành cho học sâu (deep learning), được phát triển bởi nhóm nghiên cứu AI của Facebook (Meta AI). Nó cung cấp một nền tảng linh hoạt và mạnh mẽ cho việc xây dựng, huấn luyện và triển khai các mô hình mạng nơ-ron nhân tạo. Điểm mạnh nổi bật của PyTorch là kiến trúc define-by-run (định nghĩa theo thời gian thực), cho phép người phát triển dễ dàng theo dõi và kiểm soát luồng dữ liệu, thuận tiện trong quá trình gỡ lỗi và tinh chỉnh mô hình.

Bên cạnh đó, PyTorch hỗ trợ tính toán song song trên GPU thông qua CUDA, giúp tăng tốc đáng kể các tác vụ xử lý học sâu. Thư viện này cũng tích hợp tốt với hệ sinh thái Python và các công cụ phụ trợ như NumPy, OpenCV, và đặc biệt là các công cụ trực quan hóa và quản lý thí nghiệm như TensorBoard hoặc Weights & Biases. Với khả năng mở rộng cao và cộng đồng phát triển lớn mạnh, PyTorch hiện là một trong những framework học sâu phổ biến nhất được sử dụng rộng rãi trong cả nghiên cứu lẫn ứng dụng thực tiễn.

- **Các thư viện và công cụ chính được sử dụng bao gồm:**

Einops: Hỗ trợ thao tác và tái cấu trúc tensor trực quan, đơn giản hóa quy trình xử lý dữ liệu.

Timm: Thư viện chứa nhiều mô hình trích xuất đặc trưng (feature extractor) hiện đại.

wandb: Công cụ theo dõi và trực quan hóa quá trình huấn luyện, giúp phân tích hiệu suất mô hình.

omegaconf: Quản lý tệp cấu hình dạng YAML một cách rõ ràng, linh hoạt.

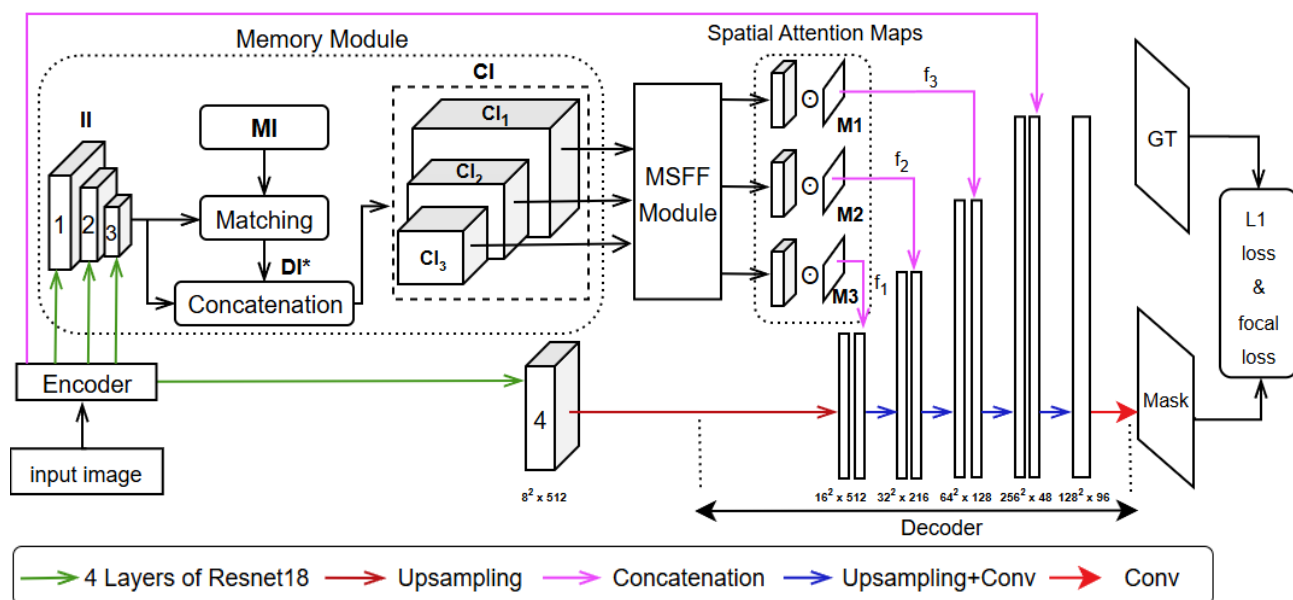
imgaug Tăng cường dữ liệu ảnh đầu vào để cải thiện khả năng tổng quát hóa của mô hình.

ipywidgets: Tạo giao diện người dùng tương tác ngay trong Jupyter Notebook.

voila: Chuyển đổi notebook sang giao diện web, giúp người dùng cuối thao tác dễ dàng qua trình duyệt.

4.2. Giải thuật đề xuất

Trong phần này ta sẽ trình bày chi tiết về giải thuật MemSeg đã được giới thiệu ở phần lý thuyết, mô hình này được miêu tả với các khối chức năng như hình 4.1 bên dưới.



Hình 4.1: Mô hình giải thuật MemSeg

- **Ảnh đầu vào (input image)**

Như đã trình bày ở phần cơ sở lý thuyết, do mô hình áp dụng phương pháp học bán giám sát, cho nên ảnh đầu vào vừa có thể là ảnh bình thường, vừa có thể là ảnh đã được chèn lỗi mô phỏng, chi tiết về cách tạo ảnh lỗi mô phỏng được trình bày ở phần 2.4.

- **Bộ mã hóa (Encoder)**

Trong kiến trúc MemSeg, khối encoder có vai trò trích xuất đặc trưng của ảnh đầu vào và được xây dựng dựa trên mạng ResNet-18. Ảnh ban đầu có kích thước 256×256 với 3 kênh màu (RGB), được đưa qua các tầng tích chập và pooling để giảm dần độ phân giải không gian đồng thời tăng chiều đặc trưng (channel), nhằm biểu diễn các thông tin ngữ nghĩa sâu hơn. Quá trình này gồm các bước như sau:

➤ **Tích chập ban đầu (Conv1) và MaxPooling:**

Ảnh đầu vào được đưa qua một lớp conv với kernel kích thước 7×7 , stride = 2 và padding = 3. Lớp này có 64 kernel, biến đổi ảnh từ kích thước $256 \times 256 \times 3$ thành $128 \times 128 \times 64$. Batch normalization và hàm kích hoạt ReLU được áp dụng sau đó. Kết quả từ Conv1 được đưa qua một lớp max pooling kernel 3×3 , với stride = 2, padding = 1, giúp tiếp tục giảm kích thước không gian xuống còn 64×64 , giữ nguyên số kênh.

$$x_1 = ReLU(BN(W_0 * x_0)), x_1 \in R^{N \times 128 \times 128 \times 64} \quad (3)$$

$$x_2 = Maxpool(x_1), x_2 \in R^{N \times 64 \times 64 \times 64} \quad (4)$$

Với:

x_0 : Là ảnh đầu vào, kích thước $N \times 256 \times 256 \times 3$.

x_1 : Là đầu ra sau lớp conv1, kích thước $N \times 128 \times 128 \times 64$.

x_2 : Là đầu ra sau max pooling, kích thước $N \times 64 \times 64 \times 64$.

W_0 : Là trọng số (weights) của lớp convolution đầu tiên.

BN : Batch Normalization – chuẩn hóa đầu ra tích chập theo từng batch.

$ReLU$: Hàm kích hoạt phi tuyến ReLU.

➤ **Tầng mã hóa 1 (ResNet18 – layer 1)**

Layer 1 bao gồm hai BasicBlocks. Mỗi BasicBlock gồm hai lớp convolution 3×3 với stride = 1 kết hợp với BatchNorm và ReLU. Do đầu vào và đầu ra của mỗi block có cùng kích thước, nên kết nối tắt (identity shortcut) được sử dụng để cộng trực tiếp đầu vào vào đầu ra của nhánh chính.. Kết thúc layer 1, kích thước đầu ra vẫn được giữ nguyên $64 \times 64 \times 64$.

$$x_3 = ReLU(W_{1,2} * ReLU(BN(W_{1,1} * x_2) + x_2)) \quad (5)$$

$$x_4 = ReLU(W_{2,2} * ReLU(BN(W_{2,1} * x_3) + x_3)) \quad (6)$$

Với:

x_2 : là đầu vào của layer 1 (đầu ra của lớp maxpooling đầu tiên).

x_3 : là đầu ra của block đầu tiên trong layer 1.

x_4 : là đầu ra của block thứ 2 trong layer 1 (cũng là đầu ra cuối của layer 1).

$W_{1,2}, W_{1,2}$: Là trọng số của 2 lớp tích chập trong block đầu tiên.

$W_{2,1}, W_{2,2}$: Là trọng số của 2 lớp tích chập trong block thứ hai.

➤ Tầng mã hóa 2 (ResNet18 – layer 2)

Layer này cũng gồm hai BasicBlocks. Block đầu sử dụng stride = 2 để giảm độ phân giải từ 64×64 xuống 32×32 , đồng thời tăng số kênh từ 64 lên 128. Do kích thước thay đổi, kết nối tắt (shortcut) trong block này không thể giữ nguyên mà phải sử dụng một lớp convolution 1×1 ($W_s^{(3)} * x_4$). Block thứ hai giữ stride = 1 vì đầu vào và đầu ra có cùng kích thước.

$$x_5 = ReLU(W_{3,2} * ReLU(BN(W_{3,1} * x_4) + W_s^{(3)} * x_4)) \quad (7)$$

$$x_6 = ReLU(W_{4,2} * ReLU(BN(W_{4,1} * x_5) + x_5)) \quad (8)$$

➤ Tầng mã hóa 3 (ResNet18 – layer 3)

Tương tự layer 2, layer này tiếp tục giảm kích thước không gian xuống 16×16 và tăng số kênh lên 256. Block đầu dùng stride = 2 và projection shortcut, block thứ hai giữ nguyên stride = 1.

$$x_7 = ReLU(W_{5,2} * ReLU(BN(W_{5,1} * x_6) + W_s^{(5)} * x_6)) \quad (9)$$

$$x_8 = ReLU(W_{6,2} * ReLU(BN(W_{6,1} * x_7) + x_7)) \quad (10)$$

➤ Tầng mã hóa 4 (ResNet18 – layer 4)

Đây là layer cuối cùng trong encoder, thường được gọi là bottleneck. Layer này giảm độ phân giải còn 8×8 và tăng số kênh lên 512. Cấu trúc gồm hai block giống các layer trước.

$$x_9 = ReLU(W_{7,2} * ReLU(BN(W_{7,1} * x_8) + W_s^{(7)} * x_8)) \quad (11)$$

$$x_{10} = ReLU(W_{8,2} * ReLU(BN(W_{8,1} * x_9) + x_9)) \quad (12)$$

Sau khi đi qua toàn bộ các tầng trong khối mã hóa (encoder), mô hình tạo ra 4 bản đồ đặc trưng (feature map) tương ứng với 4 layer chính trong mạng, được ký hiệu lần lượt là x_4 , x_6 , x_8 và x_{10} , mỗi đầu ra mang thông tin ở các mức trừu tượng khác nhau, với độ phân giải không gian giảm dần và số lượng kênh tăng dần.

• Mô-đun Bộ Nhớ (Memory Module).

Khối Memory Module trong kiến trúc MemSeg giúp mô hình phát hiện bất thường dựa trên sự khác biệt về đặc trưng. Mô hình sử dụng một cơ chế so sánh đặc trưng giữa ảnh đầu vào và các mẫu bình thường đã lưu trữ sẵn, thông qua hai thành phần chính là Input Information (II) và Memory Information (MI).

Input Information (II) là tập hợp các bản đồ đặc trưng (feature map) được trích xuất từ ba tầng đầu tiên của encoder. Các đặc trưng này lần lượt được ký hiệu lần lượt là II_1 , II_2 , và II_3 , tương ứng với các đầu ra x_4 , x_6 và x_8 của encoder ở phần trên. Các feature map đại diện cho các mức độ không gian khác nhau: 64×64 , 32×32 , và 16×16 . Ba đặc trưng này sẽ được đưa vào Memory Module để thực hiện phép so sánh (matching) với MI.

Memory Information (MI) là tập hợp các đặc trưng được xây dựng từ các ảnh bình thường trong tập huấn luyện. Cụ thể, khi bắt đầu huấn luyện, một số ảnh bình thường sẽ được chọn ngẫu nhiên và đưa vào encoder để trích xuất đặc trưng từ ba tầng (layer) đầu tiên. Các đầu ra tại ba tầng này có kích thước lần lượt là $N \times 64 \times 64 \times 64$, $N \times 128 \times 32 \times 32$ và $N \times 256 \times 16 \times 16$.

Sau đó, khoảng cách L2 (khoảng cách giữa hai vector) giữa II và toàn bộ thông tin bộ nhớ MI được tính toán theo (13), từ đó thu được N thông tin khác biệt DI giữa ảnh đầu vào và các mẫu bộ nhớ.

$$DI = \bigcup_{i=1}^N \|MI_i - II\|_2 \quad (13)$$

Trong đó, N là số lượng mẫu bộ nhớ. Với N thông tin khác biệt, ta lấy tổng nhỏ nhất của tất cả các phần tử trong từng DI làm tiêu chuẩn để chọn ra thông tin khác biệt tốt nhất DI^* giữa II và MI . Cụ thể là:

$$DI^* = \underset{DI_i \in DI}{\operatorname{argmin}} \sum_{x \in DI_i} x \quad (14)$$

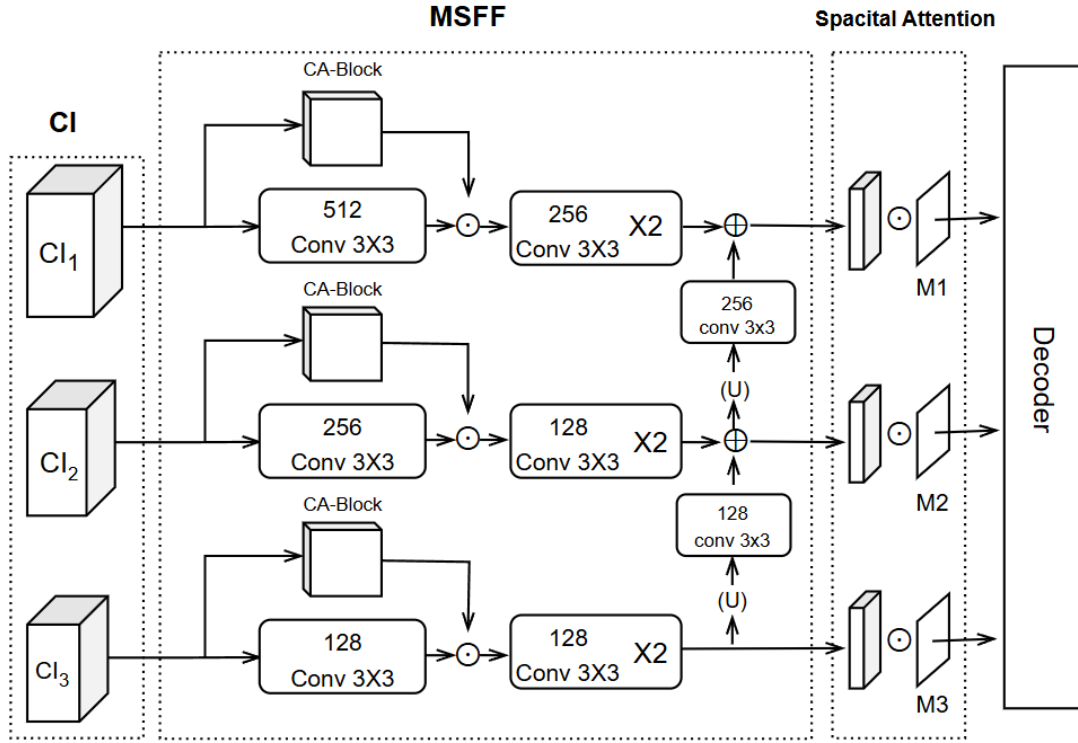
Trong đó $i \in [1, N]$. Thông tin khác biệt tốt nhất DI^* chứa sự khác biệt giữa ảnh đầu vào và mẫu bộ nhớ giống nhất của nó. Giá trị khác biệt tại một vị trí càng lớn thì xác suất khu vực tương ứng trong ảnh đầu vào là bất thường càng cao.

Sau đó, thông tin khác biệt tốt nhất DI^* được nối theo chiều kênh (concatenation) với các đặc trưng mức cao của ảnh đầu vào II theo chiều kênh để thu được thông tin nối chuỗi CI_1 , CI_2 và CI_3 . Cuối cùng, tệp thông tin nối chuỗi sẽ đi qua mô-đun tổng hợp đặc trưng đa tỉ lệ (MSFF) để thực hiện việc hợp nhất đặc trưng.

$$CI_i = \operatorname{Concat}(II_i, DI^*) \quad (15)$$

- **Mô-đun Tổng Hợp Đặc Trưng Đa Tỉ Lệ (MSFF)**

Sau khi có được ba đặc trưng CI_1 , CI_2 và CI_3 từ đầu ra của memory module, mô hình sẽ xử lý những đặc trưng này thông qua MSFF. Bởi vì việc sử dụng trực tiếp các đặc trưng này có thể dẫn đến dư thừa thông tin và làm tăng độ phức tạp của mô hình. Để khắc phục điều này, MSFF được thiết kế nhằm lọc, điều chỉnh và hợp nhất các đặc trưng này trước khi đưa vào bộ giải mã (decoder).



Hình 4.2: Sơ đồ chi tiết giải thuật của MSFF

Cụ thể, mỗi thông tin nôi chuỗi CI_n với $n = 1, 2, 3$ được xử lý lần lượt qua 2 bước chính.

➤ **Bước 1:**

Trước tiên, đầu vào CI_n được chia làm hai nhánh, một nhánh sẽ đi qua một lớp tích chập 3×3 được sử dụng để trích xuất thông tin cục bộ mà vẫn giữ nguyên độ phân giải và số kênh như công thức (16).

$$F'_n = (W_n^{(1)} * CI_n), F'_n \in R^{N \times C_n \times H_n \times W_n} \quad (16)$$

Với:

CI_n : là đầu vào CI thứ n (đầu ra của memory module).

F'_n : là đầu ra của bước conv 3×3 .

$W_n^{(1)}$: Là tập trọng số của lớp convolution.

C_n, H_n, W_n : là số kênh và độ phân giải của đặc trưng F_n .

Trong khi đó nhánh thứ hai được đưa qua khối chú ý theo kênh (Coordinate Attention – CA) để tăng cường sự nổi bật của các kênh quan trọng, tạo thành đặc trưng đã được điều chỉnh như (17).

$$F_n'' = CI_n \odot \sigma(W_n^h * Z_H) \odot (W_n^w * Z_w) \quad (17)$$

Với:

CI_n : là đầu vào CI thứ n (đầu ra của memory module).

F_n'' : là đầu ra của khối CA.

σ : là hàm *sigmoid* để chuẩn hóa về $[0,1]$.

W_n^h, W_n^z : Là tập trọng số của các lớp convolution.

Z_h, Z_w : là đặc trưng trung gian theo chiều cao và chiều rộng, trích từ CI_n

Công thức (17) có thể được viết rút gọn như công thức số (18).

$$F_n'' = CA(CI_n) \quad (18)$$

Sau đó, hai nhánh này được kết hợp bằng phép nhân phần tử để tạo ra đặc trưng F_n đã được làm nổi bật bởi cơ chế chú ý, như công thức số (19).

$$F_n = F_n' \odot F_n'' \quad (19)$$

➤ Bước 2:

Ở bước này, hai lớp tích chập bổ sung 3×3 được sử dụng để giảm một nửa số kênh, tạo thành đặc trưng đã được lọc tại mỗi mức. Đầu ra của mỗi lớp conv được mô tả như (20) và (21) .

$$G_n' = W_{n,1}^{(2)} * F_n, \quad G_n' \in R^{\frac{N \times C_n \times H_n \times W_n}{2}} \quad (20)$$

$$G_n = W_{n,2}^{(2)} * G_n', \quad G_n \in R^{\frac{N \times C_n \times H_n \times W_n}{2}} \quad (21)$$

Sau khi áp dụng công thức (21) với $n = 1, 2, 3$ ta thu được ba đặc trưng G_1, G_2 và G_3 , mô hình tiến hành hợp nhất theo hướng đa tỉ lệ. Đầu tiên, G_3 sẽ qua một bước phóng đại không gian (upsampling) và một lớp tích chập 3×3 để đưa về cùng số kênh và độ phân giải với G_2 . Sau đó thực hiện phép cộng từng phần tử theo công thức dưới đây:

$$G_{32} = G_2 + (W_3 * (\text{Upsample}(G_3))) \quad (22)$$

Tương tự, G_{32} được phóng đại lên kích thước 64×64 và cộng với G_1 theo:

$$G_{64} = G_1 + (W_4 * \text{Upsample}(G_{32})) \quad (23)$$

Sau đó, 3 đặc trưng đầu ra G_3, G_{32} và G_{64} sẽ được đưa sang Spatial Attention Maps để tiếp tục thực hiện các bước tính toán.

- **Bản đồ tập trung không gian (Spatial Attention Maps)**

Với các đặc trưng có ba kích thước khác nhau trong DI^* , giá trị trung bình được tính theo chiều kênh, tạo ba bản đồ đặc trưng có kích thước lần lượt là 16×16 , 32×32 và 64×64 . Bản đồ kích thước 16×16 được sử dụng trực tiếp làm bản đồ chú ý không gian M_3 . Sau khi nội suy (up-sample) M_3 , thực hiện phép nhân từng phần tử với bản đồ đặc trưng 32×32 để thu được M_2 , tương tự, nội suy M_2 và nhân với bản đồ đặc trưng 64×64 để thu được M_1 .

Công thức tính M_1, M_2 và M_3 được cho như sau:

$$M_3 = \frac{1}{C_3} \sum_{i=1}^{C_3} DI_{3i}^* \quad (24)$$

$$M_2 = \frac{1}{C_2} \left(\sum_{i=1}^{C_2} DI_{2i}^* \right) \odot M_3^U \quad (25)$$

$$M_1 = \frac{1}{C_1} \left(\sum_{i=1}^{C_1} DI_{1i}^* \right) \odot M_2^U \quad (26)$$

Trong đó, C_3 là số kênh của DI_3^* , DI_{3i}^* là bản đồ đặc trưng tại kênh thứ i trong DI_3^* . M_3^U và M_2^U lần lượt là các bản đồ đặc trưng thu được sau khi nội suy (up-sample) từ M_2 và M_3 .

Sau khi có được ba bản đồ chú ý không gian M_1 , M_2 và M_3 mô hình sẽ thực hiện phép nhân phần tử với các đặc trưng G_3 , G_{32} và G_{64} thu được từ MSFF.

$$f_3 = G_3 \odot M_3 \quad (27)$$

$$f_2 = G_{32} \odot M_2 \quad (28)$$

$$f_2 = G_{64} \odot M_1 \quad (29)$$

- **Bộ giải mã (Decoder)**

Bộ giải mã (decoder) trong kiến trúc MemSeg có nhiệm vụ phục hồi thông tin không gian từ các đặc trưng trườ tượng đầu ra từ Encoder kết hợp với MSFF và được hiệu chỉnh Spatial Attention Maps. Decoder được thiết kế dựa trên kiến trúc U-Net.

Đầu tiên, Decoder sẽ nhận đầu vào là bản đồ đặc trưng (feature map) ở layer 4 của Encoder z_0 (tương ứng với x_{10} ở phần encoder) với kích thước $8^2 \times 512$. Một phép nội suy không gian (upsampling) được áp dụng để tăng độ phân giải lên 16×16 trong khi vẫn giữ nguyên số lượng kênh, theo biểu thức sau:

$$z_1 = Upsample(z_0), z_1 \in R^{N \times 16 \times 16 \times 512} \quad (30)$$

Việc tăng độ phân giải giúp tạo ra đặc trưng có kích thước phù hợp để kết nối với thông tin từ tầng CI_3 (có cùng không gian 16×16). Đồng thời, điều này giúp phục hồi dần cấu trúc không gian đã bị nén trong quá trình mã hóa trước đó.

Sau khi thực hiện phép nội suy không gian, đặc trưng z_1 được kết nối theo chiều kênh với đặc trưng từ f_3 . Kết quả nối giữa z_1 và f_3 được ký hiệu là z_2 , được biểu diễn như sau:

$$z_2 = Concat(z_1, f_3) \quad (31)$$

Ở bước tiếp theo, z_2 (đã concat ở tầng trước) sẽ được upsampling và đi qua một khối conv như sau:

$$z'_3 = \text{Upsample}(z_2), z'_3 \in R^{N \times 32 \times 32 \times 512} \quad (32)$$

$$z_3 = \text{ReLU}(\text{BN}(W_2 * z'_3)), z_3 \in R^{N \times 32 \times 32 \times 256} \quad (33)$$

Trong đó, W_2 là kernel convolution kích thước 3×3 , hàm BN là chuẩn hóa batch, và ReLU là hàm kích hoạt phi tuyến.

Sau khi có được z_3 , ta sẽ kết nối theo chiều kênh với f_2 để có được z_4 như sau:

$$z_4 = \text{Concat}(z_3, f_2) \quad (34)$$

Thực hiện lặp lại các bước này và ta có được các biểu thức (35), (36) và (37).

$$z'_5 = \text{Upsample}(z_4), z'_5 \in R^{N \times 64 \times 64 \times 256} \quad (35)$$

$$z_5 = \text{ReLU}(\text{BN}(W_3 * z'_5)), z_5 \in R^{N \times 64 \times 64 \times 128} \quad (36)$$

$$z_6 = \text{Concat}(z_5, f_1) \quad (37)$$

Ở bước tiếp theo, ta tiếp thực hiện phép upsampling và conv để tăng độ phân giải lên 128×128 và đồng thời giảm số kênh từ 128 xuống 96 như (38) và (39).

$$z'_7 = \text{Upsample}(z_6), z'_7 \in R^{N \times 128 \times 128 \times 128} \quad (38)$$

$$z_7 = \text{ReLU}(\text{BN}(W_4 * z'_7)), z_7 \in R^{N \times 128 \times 128 \times 96} \quad (39)$$

Sau đó ta thực hiện phép kết nối theo chiều kênh giữa z_7 và x_4 (đặc trưng từ layer 1 của encoder). Việc này giữ lại các đặc trưng không gian gốc, quan trọng cho việc khôi phục chính xác hình dạng vật thể. Biểu thức được mô tả ở công thức (40).

$$z_8 = \text{Concat}(z_7, x_4) \quad (40)$$

Bước tiếp theo, ta thực hiện thêm 1 tầng upsampling nữa để thu được ảnh có độ phân giải 256×256 và số kênh là 48, quá trình này được mô tả qua các công thức (41) và (42).

$$z'_9 = \text{Upsample}(z_8), z'_9 \in R^{N \times 256 \times 256 \times 128} \quad (41)$$

$$z_9 = \text{ReLU}(BN(W_5 * x'_9)), z_9 \in R^{N \times 256 \times 256 \times 48} \quad (42)$$

Cuối cùng, sau khi đặc trưng z_9 đã được giải mã hoàn chỉnh, một lớp tích chập cuối cùng được áp dụng để sinh ra bản đồ phân đoạn hai lớp bất thường và bình thường. Phép toán này được biểu diễn theo công thức (43).

$$\hat{Y} = W_{out} * z_9 + b_{out}, \hat{Y} \in R^{N \times 256 \times 256 \times 2} \quad (43)$$

Trong đó W_{out} là kernel đưa 48 kênh đầu vào về 2 kênh đầu ra, và b_{out} là thành phần bias tương ứng. Để chuyển các giá trị này thành xác suất, ta tiếp tục sử dụng hàm softmax theo chiều kênh như (44).

$$P_{n,c,i,j} = \frac{\exp(\hat{Y}_{n,c,i,j})}{\sum_{c'=1}^2 \exp(\hat{Y}_{n,c',i,j})}, \quad P \in R^{N \times 256 \times 256 \times 2} \quad (44)$$

- **Hàm mất mát (loss function)**

Để đảm bảo giá trị dự đoán của MemSeg gần với giá trị thực, ở bước này ta dùng hai hàm mất mát là L1 loss và focal loss để đảm bảo sự tương đồng của tất cả các điểm ảnh trong không gian ảnh. Các ảnh phân đoạn được mô hình dự đoán dưới ràng buộc L1 giữ lại nhiều thông tin về cạnh hơn so với khi sử dụng hàm L2 loss. Trong khi đó, focal loss giúp giảm bớt vấn đề mất cân bằng diện tích giữa vùng bình thường và bất thường trong ảnh, khiến mô hình tập trung hơn vào việc phân đoạn các mẫu khó, từ đó nâng cao độ chính xác của việc phân đoạn bất thường.

Cụ thể, ta thực hiện tối thiểu hóa cả L1 loss ($\mathcal{L}_{l1}$) và focal loss ($\mathcal{L}_f$) giữa ảnh nhãn thực S của vùng bất thường trong ảnh giả lập và giá trị dự đoán $\hat{S}$ của mô hình, theo công thức (45) và (46) tương ứng.

$$\mathcal{L}_{l1} = \|S - \hat{S}\|_1 \quad (45)$$

$$\mathcal{L}_f = -\alpha_t(1 - p_t)^{\gamma} \log(p_t) \quad (46)$$

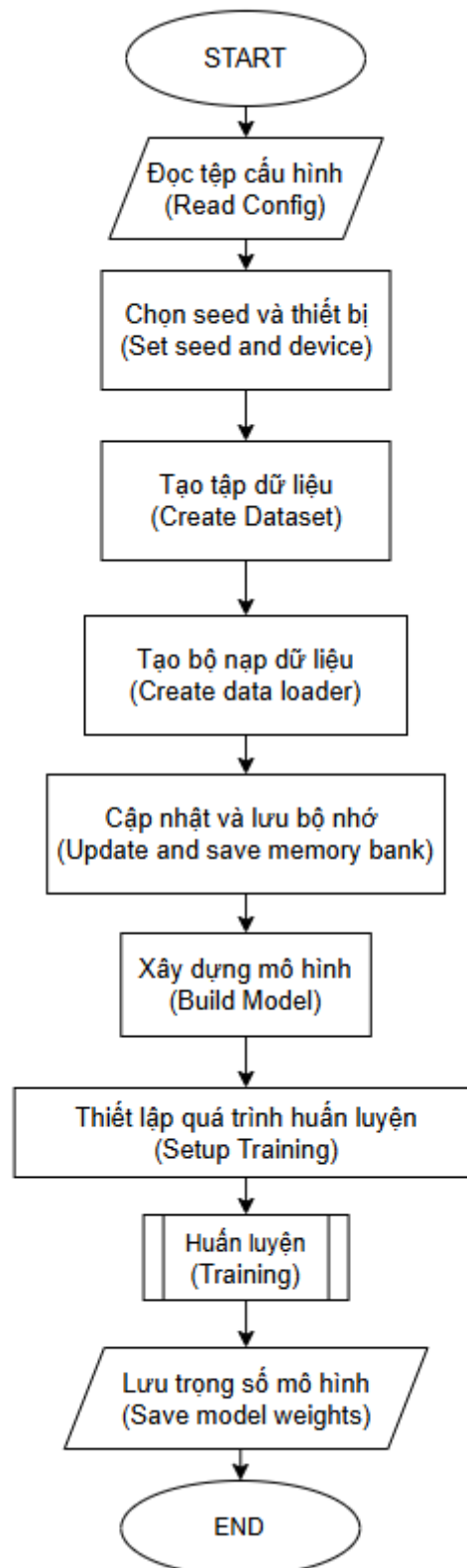
Trong đó, p_t là xác suất dự đoán p thuộc về lớp nhãn của pixel khi giá trị thực của pixel tương ứng trong ảnh S là 1 và $p_t = 1 - p$ khi giá trị thực của pixel trong S là 0. Các tham số α_t và γ là tham số để điều chỉnh mức độ trọng số.

Kết hợp cả hai hàm mất mát ta được công thức hàm mất mát chung như (47).

$$\mathcal{L}_{all} = \lambda_{l1}\mathcal{L}_{l1} + \lambda_f\mathcal{L}_f \quad (47)$$

4.3. Lưu đồ của chương trình

- Lưu đồ của chương trình huấn luyện



Hình 4.3: Lưu đồ của chương trình huấn luyện

Như lưu đồ giải thuật được mô tả trong hình trên, quá trình thực hiện việc huấn luyện mô hình MemSeg được thực hiện qua các bước chính như sau:

➤ **Bước 1: Đọc vào tệp cấu hình**

Chương trình khởi đầu bằng việc đọc vào tệp cấu hình (config.yaml) chứa các tham số và thông tin đầu vào như mạng trích xuất đặc trưng (backbone), tốc độ học (learning rate), kích thước batch (batch size), số bước huấn luyện, thông tin về dữ liệu và các thông số huấn luyện khác. Cấu hình này được xử lý bằng thư viện OmegaConf, đồng thời hỗ trợ cập nhật lại bằng các tham số dòng lệnh nếu cần.

➤ **Bước 2: Thiết lập seed và thiết bị**

Tiếp theo, chương trình thiết lập giá trị Seed ngẫu nhiên để đảm bảo kết quả huấn luyện có thể tái lập. Đồng thời, hệ thống sẽ kiểm tra khả năng sử dụng GPU và chọn thiết bị phù hợp (cuda nếu có, ngược lại dùng cpu) để phục vụ việc huấn luyện.

➤ **Bước 3: Tạo tập dữ liệu**

Ở bước này, ba tập dữ liệu chính được khởi tạo gồm: tập huấn luyện (trainset), tập dùng để cập nhật bộ nhớ đặc trưng (memoryset) và tập kiểm tra (testset). Quá trình tạo dataset bao gồm cả các bước tiền xử lý như resize, thêm nhiễu Perlin, sử dụng mask, v.v., được cấu hình bởi các thông số trong tệp cấu hình.

➤ **Bước 4: Tạo bộ nạp dữ liệu**

Sau khi đã tạo các tập dữ liệu, chương trình tiếp tục khởi tạo các Dataloader tương ứng cho tập huấn luyện và kiểm tra. DataLoader giúp chia nhỏ dữ liệu thành các batch, tối ưu quá trình nạp dữ liệu vào mô hình và cho phép xử lý song song nếu có nhiều luồng (num_workers).

➤ **Bước 5: Cập nhật và lưu bộ nhớ (memory bank)**

Bước tiếp theo là xây dựng và cập nhật bộ nhớ đặc trưng (memory bank). Dựa trên tập memoryset, chương trình trích xuất các đặc trưng từ ảnh bình thường và lưu trữ chúng để sử dụng trong quá trình phát hiện bất thường.

➤ **Bước 6: Xây dựng mô hình**

Tại bước này, mô hình MemSeg sẽ được khởi tạo với các thành phần như mô hình giải thuật đã được mô tả ở phần 4.2.

➤ **Bước 7: Thiết lập quá trình huấn luyện**

Chương trình tiến hành khởi tạo các thành phần phục vụ huấn luyện, bao gồm các hàm mất mát (L1 Loss và Focal Loss), bộ tối ưu hóa (AdamW), cùng các tham số như learning rate và weight decay. Scheduler cho learning rate cũng sẽ được sử dụng để điều chỉnh tốc độ học theo thời gian.

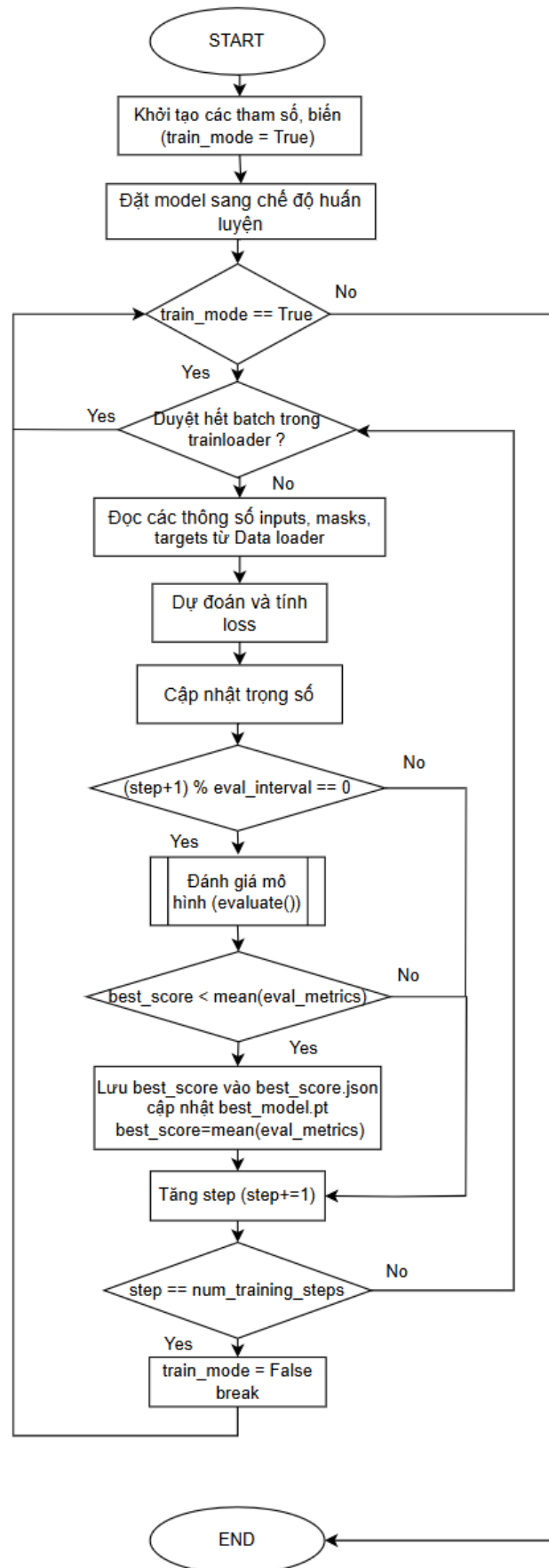
➤ **Bước 8: Huấn luyện**

Đây là giai đoạn huấn luyện chính của mô hình. Gọi hàm training, mỗi bước huấn luyện (step) gồm các thao tác: forward pass, tính loss, lan truyền ngược (backpropagation) và cập nhật trọng số. Theo chu kỳ định sẵn, mô hình sẽ được đánh giá trên tập kiểm tra bằng các chỉ số như AUROC và AUPRO. Nếu hiệu suất vượt qua điểm tốt nhất trước đó, trọng số mô hình sẽ được lưu lại trong tệp best_model.pt.

➤ **Bước 9: Lưu trọng số mô hình**

Sau khi hoàn tất toàn bộ số bước huấn luyện, chương trình kết thúc. Các tệp trọng số tốt nhất, chỉ số đánh giá và memory bank đều đã được lưu để sử dụng cho bước suy luận (inference) sau này.

- **Lưu đồ của hàm *training***



Hình 4.4: Lưu đồ của hàm training

Hình trên mô tả lưu đồ giải thuật tổng quát của hàm huấn luyện mô hình (hàm training), được triển khai trong file train.py. Quá trình huấn luyện bắt đầu với việc khởi tạo các biến cần thiết và đặt chế độ huấn luyện (train_mode = True). Mô hình sau đó được chuyển sang chế độ huấn luyện (model.train()), đảm bảo các lớp như BatchNorm và Dropout hoạt động đúng với vai trò của chúng trong quá trình học.

Vòng lặp huấn luyện được điều khiển bởi biến train_mode. Trong mỗi epoch, chương trình thực hiện duyệt qua từng batch dữ liệu từ trainloader. Với mỗi batch, các tensor đầu vào (input), mặt nạ (mask) và nhãn (target) được đưa vào thiết bị tính toán (CPU hoặc GPU) và truyền qua mô hình để tạo ra dự đoán. Dự đoán sau đó được chuẩn hóa bằng hàm softmax.

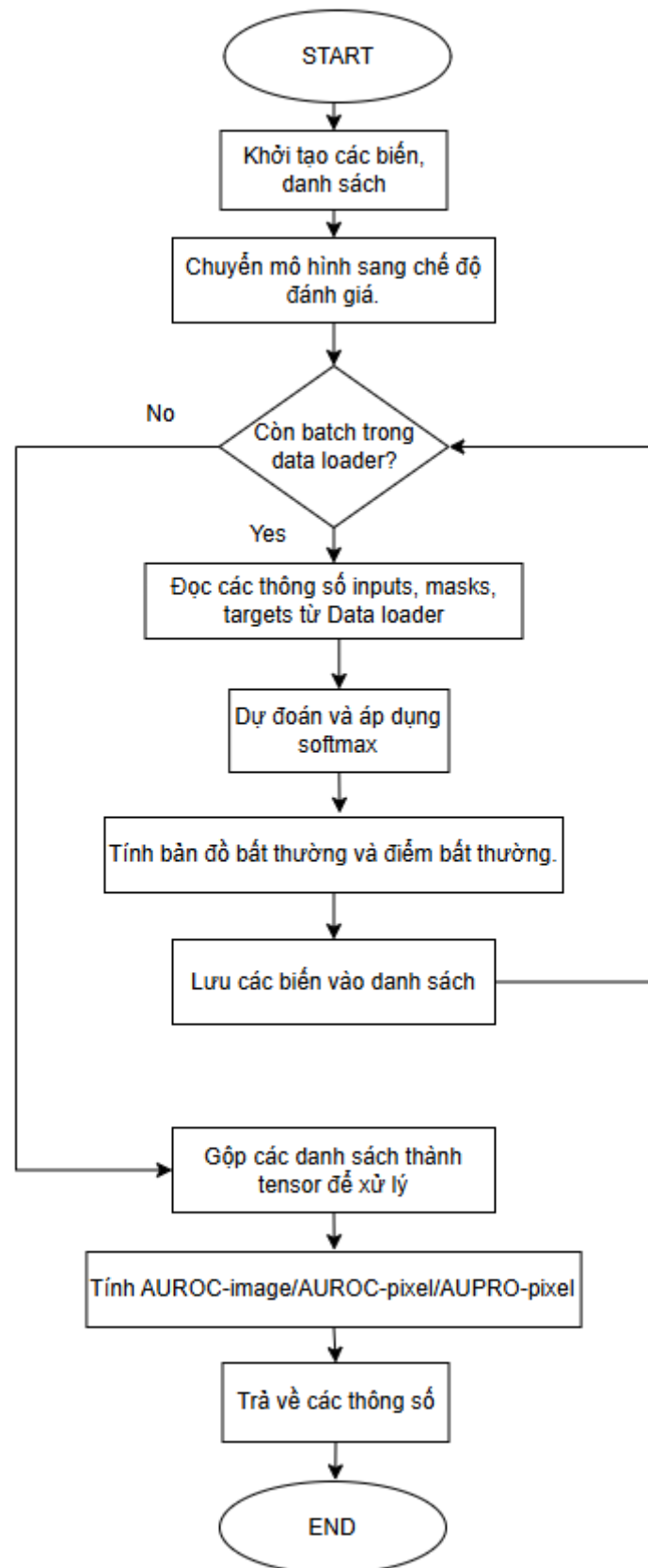
Tiếp theo, chương trình thực hiện tính toán giá trị hàm mất mát (loss) gồm hai thành phần là L1 loss và focal loss. Tổng giá trị loss được tính theo giá trị trọng số (mặc định là 0.6 và 0.4), sau đó được lan truyền ngược (backpropagation) để cập nhật trọng số mô hình thông qua bộ tối ưu hóa.

Sau mỗi số bước cố định (eval_interval), chương trình sẽ chuyển mô hình sang chế độ đánh giá để thực hiện hàm evaluate() trên tập kiểm tra, từ đó tính toán các chỉ số đánh giá hiệu suất gồm: AUROC-image, AUROC-pixel và AUPRO-pixel. Nếu giá trị trung bình của các chỉ số đánh giá hiện tại lớn hơn giá trị best_score trước đó, chương trình sẽ cập nhật lại giá trị trọng số cao nhất (best_score), lưu mô hình hiện tại dưới tên best_model.pt và ghi thông tin vào tệp best_score.json.

Sau mỗi vòng lặp, biến step sẽ được tăng lên một đơn vị. Khi số bước đạt đến số bước huấn luyện tối đa (num_training_steps) đã định trước, chương trình đặt giá trị của biến train_mode = False để kết thúc quá trình huấn luyện.

Toàn bộ quy trình này đảm bảo rằng mô hình không chỉ học được tốt trong tập huấn luyện, mà còn được theo dõi sát sao về hiệu năng trên tập kiểm tra, nhờ đó tránh được tình trạng overfitting và lựa chọn được mô hình tối ưu nhất cho suy luận.

- Lưu đồ giải thuật hàm `evaluate()`



Hình 4.5: Lưu đồ của hàm *evaluate*

Hình trên mô tả chi tiết thuật toán được thực hiện trong hàm *evaluate*, vốn thường được gọi định kỳ bởi hàm *training* trong quá trình huấn luyện mô hình để đánh giá hiệu suất của mô hình hiện tại trên tập dữ liệu kiểm tra. Quá trình đánh giá này nhằm theo dõi độ chính xác và chất lượng dự đoán của mô hình tại từng thời điểm cụ thể trong quá trình học.

Bắt đầu từ bước đầu tiên, chương trình thực hiện khởi tạo các biến và danh sách cần thiết. Đây là giai đoạn chuẩn bị các cấu trúc dữ liệu dùng để lưu trữ thông tin trung gian như kết quả dự đoán, nhãn thực tế, bản đồ bất thường và các giá trị đánh giá. Các danh sách này sẽ được sử dụng để tổng hợp kết quả sau khi quá trình duyệt qua toàn bộ dữ liệu kiểm tra hoàn tất.

Sau khi khởi tạo xong, mô hình được chuyển sang chế độ đánh giá (evaluation mode). Việc này giúp đảm bảo các lớp như BatchNorm và Dropout (nếu có trong kiến trúc) hoạt động đúng cách khi đánh giá, tức là không cập nhật thống kê và không áp dụng dropout. Đây là bước quan trọng để mô hình phản ánh đúng hiệu suất thực tế mà không bị ảnh hưởng bởi sự ngẫu nhiên trong quá trình huấn luyện.

Tiếp theo, thuật toán tiến hành duyệt tuần tự qua từng batch trong trainloader (cụ thể là tập kiểm tra). Ở mỗi vòng lặp, chương trình kiểm tra xem đã duyệt hết tất cả batch hay chưa. Nếu chưa, nó sẽ tiếp tục vòng lặp. Ngược lại, nếu đã duyệt hết thì chương trình sẽ chuyển sang bước xử lý kết quả.

Trong mỗi vòng lặp duyệt batch, chương trình đọc các thông số đầu vào, bao gồm: inputs (ảnh cần đánh giá), masks (bản đồ bất thường thực tế), và targets (nhãn hình ảnh: bình thường hay bất thường) từ dataloader. Đây là những thông tin cơ bản để mô hình thực hiện dự đoán và đối chiếu với nhãn gốc. Sau đó, mô hình sẽ tiến hành dự đoán đầu ra cho ảnh đầu vào, và thường áp dụng hàm softmax để chuẩn hóa kết quả thành xác suất. Kết quả này sẽ thể hiện mức độ tin tưởng của mô hình về mỗi pixel hoặc toàn bộ ảnh thuộc nhóm bình thường hay bất thường.

Dựa trên xác suất đầu ra, chương trình sẽ tính toán bản đồ bất thường (anomaly map) và điểm bất thường (anomaly score) tương ứng với mỗi ảnh. Bản đồ bất thường

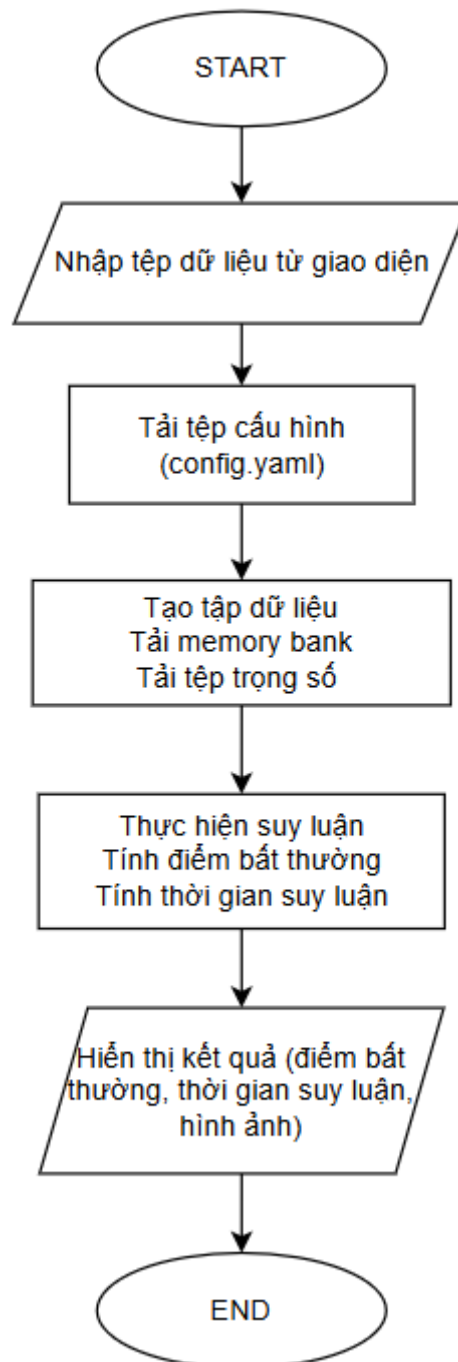
thể hiện các vùng trong ảnh bị nghi ngờ là lỗi, trong khi điểm bất thường tổng hợp là một giá trị duy nhất biểu diễn độ nghi ngờ của toàn ảnh.

Sau khi hoàn tất việc tính toán cho batch hiện tại, các kết quả như bản đồ bất thường, điểm số, nhãn thực tế và dự đoán sẽ được lưu vào các danh sách tương ứng đã khởi tạo từ đầu. Quá trình này được lặp lại cho đến khi tất cả các batch trong dataloader được xử lý.

Khi toàn bộ ảnh trong tập kiểm tra đã được xử lý, chương trình sẽ gộp các danh sách kết quả lại thành các tensor để phục vụ cho bước tính toán hiệu suất. Ở giai đoạn kế tiếp, chương trình tiến hành tính toán các chỉ số đánh giá bao gồm AUROC-image, AUROC-pixel và AUPRO-pixel.

Cuối cùng, sau khi có đầy đủ các giá trị đánh giá, chương trình trả về các thông số này để phục vụ cho việc hiển thị kết quả, ghi log hoặc cập nhật quá trình huấn luyện, rồi kết thúc hàm evaluate.

- Lưu đồ giải thuật chương trình dự đoán



Hình 4.6: Lưu đồ của chương trình phát hiện khuyết tật

Hình trên mô tả lưu đồ giải thuật của chương trình thực hiện việc phát hiện khuyết tật bề mặt, dự đoán và hiển thị kết quả cho người dùng, sử dụng mô hình đã được huấn luyện ở phần trước

Khi chương trình bắt đầu thực thi, bước đầu tiên là người dùng sẽ lựa chọn danh mục dữ liệu mong muốn trong số 10 danh mục dữ liệu của bộ dữ liệu MvTec đã được huấn luyện trước đó. Sau đó, chương trình sẽ tiến hành tải tệp cấu hình (config.yaml), trong đó lưu trữ các thông số cần thiết như tên mô hình, kích thước ảnh, đường dẫn đến dữ liệu, số lượng mẫu bộ nhớ (memory sample), cũng như các tham số liên quan đến quá trình đánh giá.

Tiếp theo, chương trình tiến hành khởi tạo tập dữ liệu đầu vào, đồng thời tải bộ nhớ trung gian (memory bank), một thành phần quan trọng giúp mô hình ghi nhớ đặc trưng của các ảnh bình thường, từ đó phát hiện được các bất thường. Mô hình chứa các trọng số đã được huấn luyện trước đó cũng sẽ được nạp lại để sẵn sàng thực hiện suy luận.

Trong bước suy luận, mô hình sẽ lần lượt xử lý từng ảnh trong tập dữ liệu kiểm tra (test dataset). Với mỗi ảnh, mô hình sẽ tính toán điểm bất thường (anomaly score) đây là giá trị đại diện cho mức độ khác biệt giữa ảnh đầu vào và ảnh bình thường cũng như ghi lại thời gian suy luận (inference time) để đánh giá hiệu suất. Ngoài ra, mô hình còn tạo ra bản đồ dự đoán bất thường (predicted mask) để trực quan hóa vị trí vùng lỗi.

Cuối cùng, chương trình hiển thị kết quả đầu ra bao gồm: điểm bất thường, thời gian suy luận, cùng với hình ảnh hiển thị vùng lỗi được đánh dấu trực tiếp trên ảnh đầu vào (overlay). Quá trình này hỗ trợ người dùng có thể đánh giá nhanh chóng tính chính xác và hiệu quả của mô hình khi áp dụng vào thực tế. Sau khi kết thúc hiển thị kết quả, chương trình hoàn tất quá trình dự đoán và kết thúc.

5. KẾT QUẢ HUẤN LUYỆN VÀ THỰC NGHIỆM

Ở các phần trước của luận văn ta đã trình bày về cơ sở lý thuyết, mô hình hệ thống cũng như các phương pháp thiết kế và xây dựng một chương trình để huấn luyện, suy luận và một hệ thống hoàn chỉnh để ứng dụng mô hình phát hiện khuyết tật bề mặt. Trong phần này ta sẽ trình bày cụ thể về chi tiết phần cứng cũng như các kết quả thu được.

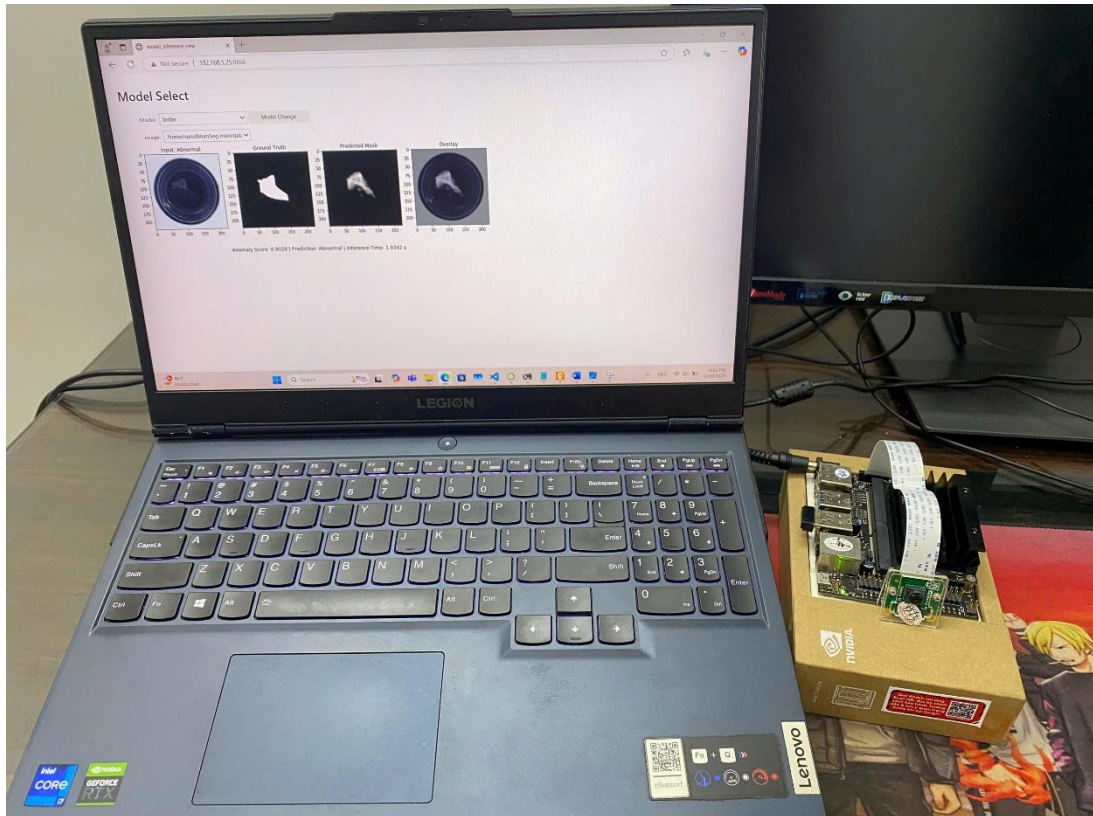
5.1. Mô tả phần cứng hệ thống

Như đã trình bày ở phần mô hình hệ thống ta sẽ xây dựng một hệ thống phát hiện khuyết tật bề mặt dựa trên ba thành phần chính là NVIDIA Jetson Nano Developer Kit, Camera CSI Raspberry Pi Module V2 và máy tính.

- **Phần cứng chi tiết của hệ thống**
 - NVIDIA Jetson Nano Developer Kit
 - Camera CSI Raspberry Pi Module V2
 - TP-link wireless N nano USB adapter
 - Bộ nguồn 5V/4A
 - Máy tính



Hình 5.1: Jetson Nano sau khi kết nối với CSI Camera



Hình 5.2: Hệ thống sau khi kết nối hoàn chỉnh

```
nano@jetson: ~  
Windows PowerShell  
Copyright (C) Microsoft Corporation. All rights reserved.  
  
Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows  
  
PS C:\Users\Admin> ssh nano@192.168.1.25  
The authenticity of host '192.168.1.25 (192.168.1.25)' can't be established.  
ED25519 key fingerprint is SHA256:l5Ftlq5H10t5D3fpwhK7AnJfd0xBkq+0ao6YVKJfIhk.  
This host key is known by the following other names/addresses:  
C:\Users\Admin/.ssh/known_hosts:1: 192.168.1.149  
C:\Users\Admin/.ssh/known_hosts:3: 192.168.1.163  
C:\Users\Admin/.ssh/known_hosts:4: 192.168.1.17  
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes  
Warning: Permanently added '192.168.1.25' (ED25519) to the list of known hosts.  
nano@192.168.1.25's password:  
Welcome to Ubuntu 18.04.6 LTS (GNU/Linux 4.9.337-tegra aarch64)  
  
* Documentation:  https://help.ubuntu.com  
* Management:    https://landscape.canonical.com  
* Support:        https://ubuntu.com/pro  
This system has been minimized by removing packages and content that are  
not required on a system that users do not log into.  
  
To restore this content, you can run the 'unminimize' command.  
Expanded Security Maintenance for Infrastructure is not enabled.  
  
0 updates can be applied immediately.  
  
215 additional security updates can be applied with ESM Infra.
```

Hình 5.3: Kết nối jetson nano với PC thông qua SSH

5.2. Tập dữ liệu MVTec AD

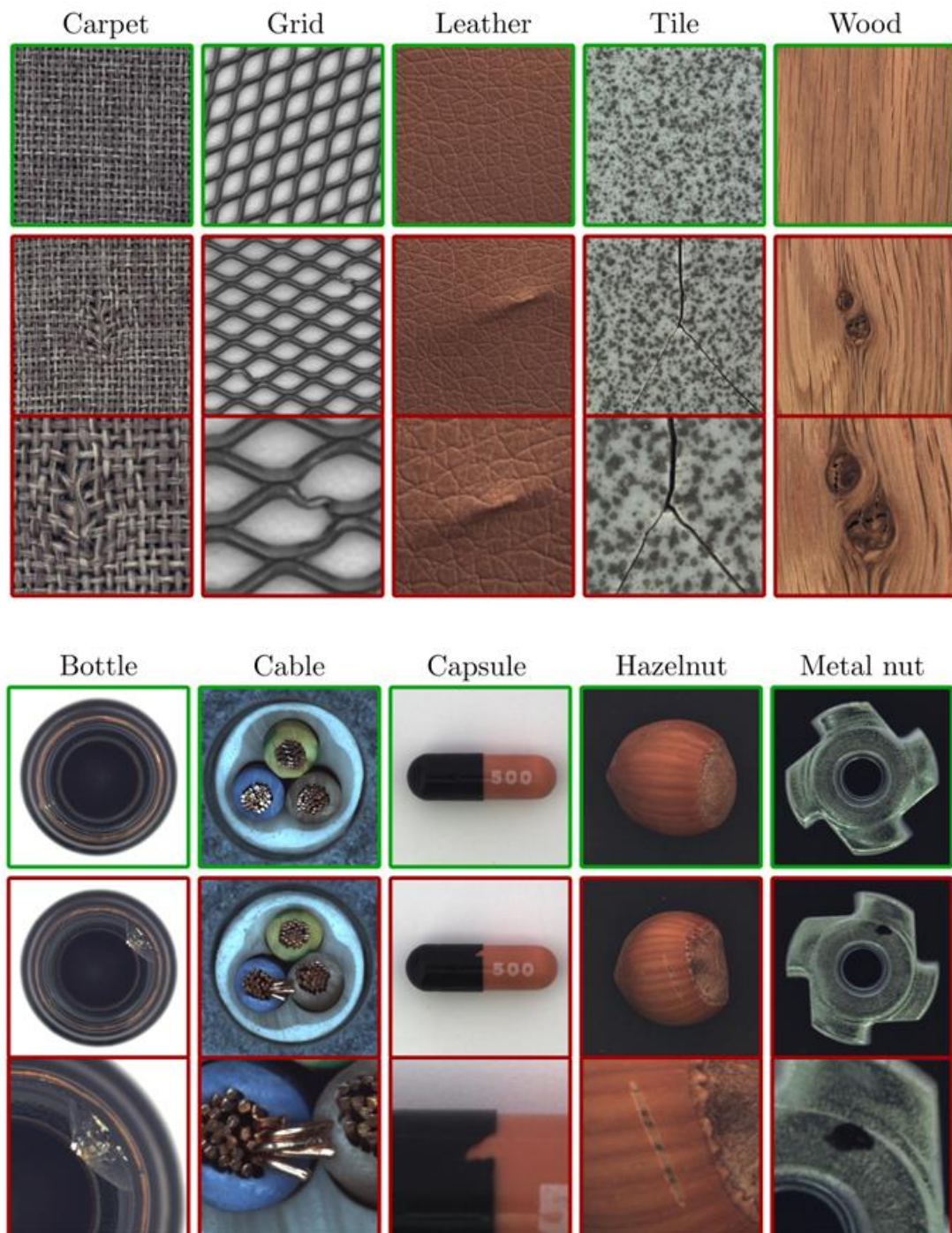
Tập dữ liệu MVTec AD[2] là một bộ dữ liệu tiêu chuẩn, được thiết kế chuyên biệt nhằm phục vụ cho bài toán phát hiện bất thường không giám sát (unsupervised anomaly detection), đặc biệt trong bối cảnh kiểm tra chất lượng bề mặt vật liệu công nghiệp. Đây là một trong những bộ dữ liệu tập trung vào ứng dụng thực tế, nơi mà việc phát hiện những sai sót nhỏ nhưng có thể gây ảnh hưởng lớn đến sản phẩm là cực kỳ quan trọng.

Tập dữ liệu này bao gồm tổng cộng 15 danh mục (categories), được chia thành hai nhóm chính: (1) nhóm bề mặt dạng kết cấu (textures) gồm 5 loại như gạch, vải, gỗ, da, thảm,... và (2) nhóm vật thể (objects) gồm 10 loại như transistor, cáp điện, chai, thuốc viên, v.v. Sự phân chia này phản ánh tính đa dạng trong các ứng dụng công nghiệp, từ vật liệu có hoa văn lặp lại đến những vật thể có hình dạng xác định.

Mỗi danh mục đều cung cấp một tập huấn luyện chỉ bao gồm ảnh bình thường (không có lỗi), với số lượng dao động từ 60 đến 400 ảnh tùy vào loại danh mục. Điều này mô phỏng đúng điều kiện thực tế, khi mà việc thu thập dữ liệu bất thường thường rất khó khăn và tốn kém. Bên cạnh đó, tập kiểm tra (test set) đi kèm sẽ bao gồm cả ảnh bình thường và bất thường, với các lỗi được mô phỏng một cách sát thực tế. Các bất thường này rất đa dạng về loại hình (như trầy xước, nứt, mòn, lỗ hổng, vết bẩn, biến dạng,...) cũng như kích thước và kết cấu, tạo điều kiện để đánh giá khả năng tổng quát hóa và phát hiện sai lệch của các mô hình học sâu trong môi trường công nghiệp phức tạp.

Một điểm mạnh khác của MVTec AD là việc cung cấp ground truth ở cấp độ pixel cho các vùng bất thường trong tập kiểm tra. Điều này giúp việc đánh giá mô hình trở nên chính xác hơn, cho phép đo lường hiệu suất không chỉ ở mức độ hình ảnh (image-level) mà còn ở mức độ điểm ảnh (pixel-level). Hơn nữa, dữ liệu được thu thập với chất lượng cao, có độ phân giải tốt, ánh sáng ổn định và không bị nhiễu, giúp đảm bảo tính tin cậy và độ chính xác trong quá trình huấn luyện và kiểm thử. Chính vì vậy, MVTec AD đã trở thành bộ dữ liệu đánh giá (benchmark) được sử

dụng phổ biến trong nhiều nghiên cứu về phát hiện bất thường trong thị giác máy tính công nghiệp.



Hình 5.4: Một số mẫu dữ liệu trong bộ MVTec

- **Các danh mục được lựa chọn để huấn luyện**

1. Chai (bottle)

Danh mục này bao gồm các chai trong suốt, thường dùng trong đóng gói thực phẩm hoặc đồ uống. Bất thường trong tập dữ liệu có thể bao gồm các vết móp méo, rạn nứt hoặc biến dạng hình dạng bề mặt chai. Đây là đối tượng hình học đơn giản nhưng đòi hỏi mô hình phải nhạy bén trong việc phát hiện những thay đổi nhỏ trên bề mặt trong suốt.

- Số lượng tập huấn luyện: 209 ảnh của sản phẩm bình thường
- Số lượng tập kiểm tra: tổng cộng 80 ảnh bao gồm:
 - ✓ **Broken large** - Vỡ lớn
 - ✓ **Broken small** - Vỡ nhỏ
 - ✓ **Contamination** - Bị nhiễm bẩn
 - ✓ **Good** - Bình thường (không có lỗi)

2. Viên nang (capsule)

Đại diện cho các viên nang dược phẩm, danh mục này chứa các hình ảnh của viên thuốc dạng tròn hoặc hình con nhộng. Bất thường có thể là vết nứt, thiếu mảnh hoặc sự thay đổi màu sắc bề mặt. Đây là loại hình vật thể nhỏ với chi tiết tinh vi, thường xuất hiện trong kiểm tra chất lượng ngành dược phẩm.

- Số lượng tập huấn luyện: 220 ảnh của sản phẩm bình thường (không có lỗi)
- Số lượng tập kiểm tra: tổng cộng 120 ảnh bao gồm:
 - ✓ **Crack** - Nứt
 - ✓ **Poke** - Bị đâm (lỗ thủng)
 - ✓ **Scratch** - Trầy xước
 - ✓ **Squeeze** - Bị bóp méo (biến dạng do lực ép)
 - ✓ **Good** - Bình thường (không có lỗi)

3. Tấm thảm (carpet)

Là một trong những danh mục thuộc nhóm bề mặt kết cấu (textures), carpet có đặc điểm là các hoa văn và cấu trúc lặp lại. Bất thường trong thảm thường xuất hiện dưới dạng vết bẩn, thớ bị lệch hoặc lỗ hỏng, khiến mô hình cần học được các mẫu hoa văn chuẩn để phát hiện sai lệch.

- Số lượng tập huấn luyện: 280 ảnh của sản phẩm bình thường (không có lỗi)
- Số lượng tập kiểm tra: tổng cộng 116 ảnh bao gồm:
 - ✓ **Color** - Sai màu (thay đổi màu sắc không mong muốn)
 - ✓ **Cut** - Bị cắt (vết cắt không đúng vị trí hoặc kích thước)
 - ✓ **Hole** - Có lỗ thủng
 - ✓ **Metal contamination** - Nhiễm bẩn kim loại (có dị vật bằng kim loại bề mặt)
 - ✓ **Thread** - Sợi chỉ thừa hoặc bị lỏng (sai lệch về cấu trúc sợi trên bề mặt thảm)
 - ✓ **Good** - Bình thường (không có lỗi)

4. Tấm lưới (grid)

Danh mục lưới (grid) thể hiện các bề mặt có họa tiết dạng lưới đều, thường xuất hiện trong công nghiệp chế tạo linh kiện điện tử hoặc lưới lọc. Bất thường thường là các đường nét sai lệch, lưới bị gãy hoặc mất cân đối hình học, yêu cầu mô hình phải nhạy với các cấu trúc hình học tuần hoàn.

- Số lượng tập huấn luyện: 264 ảnh của sản phẩm bình thường (không có lỗi)
- Số lượng tập kiểm tra: tổng cộng 76 ảnh bao gồm:
 - ✓ **Bent** - Bị cong (các thanh hoặc phần tử của lưới bị uốn cong)
 - ✓ **Broken** - Bị gãy (cấu trúc lưới bị đứt đoạn hoặc gãy)
 - ✓ **Glue** - Bị dính keo (dư keo hoặc dính tạp chất làm sai lệch hình dạng lưới)
 - ✓ **Metal contamination** - Nhiễm bẩn kim loại (dị vật kim loại dính bề mặt lưới)
 - ✓ **Thread** - Có sợi chỉ (sợi vướng vào lưới gây rối cấu trúc)
 - ✓ **Good** - Bình thường (không có lỗi)

5. Da (leather)

Bề mặt da là loại kết cấu phức tạp và không đồng nhất, thường dùng trong sản xuất ghê, giày hoặc các sản phẩm cao cấp. Bất thường thường là các vết trầy, đốm màu, hoặc kết cấu lạ. Mô hình phải có khả năng phân biệt giữa sự đa dạng tự nhiên và các lỗi sản xuất không mong muốn.

- Số lượng tệp huấn luyện: 245 ảnh của sản phẩm bình thường (không có lỗi)
- Số lượng tệp kiểm tra: tổng cộng 124 ảnh bao gồm:
 - ✓ **Color** - Sai màu (thay đổi màu sắc không mong muốn)
 - ✓ **Cut** - Vết cắt (đường cắt không mong muốn trên bề mặt da)
 - ✓ **Fold** – Nếp gấp (da bị gấp lại tạo thành nếp nhăn)
 - ✓ **Glue** - Dính keo (vết keo thừa hoặc sai vị trí)
 - ✓ **Poke** - Lỗ chích (bị đâm tạo thành lỗ nhỏ trên bề mặt da)
 - ✓ **Good** - Bình thường (không có lỗi)

6. Đai ốc kim loại (metal nut)

Đây là danh mục thuộc nhóm vật thể, gồm các đai ốc kim loại với hình dạng chuẩn và màu sắc phản chiếu. Các lỗi bất thường có thể là gỉ sét, biến dạng hình học hoặc thiếu khuyết một phần. Việc phát hiện lỗi trên các vật thể kim loại đòi hỏi mô hình xử lý tốt độ bóng và phản xạ ánh sáng.

- Số lượng tệp huấn luyện: 219 ảnh của sản phẩm bình thường (không có lỗi)
- Số lượng tệp kiểm tra: tổng cộng 115 ảnh bao gồm:
 - ✓ **Bent** - Bị cong (đai ốc bị méo mó, không giữ được hình dạng ban đầu)
 - ✓ **Color** - Sai màu (màu sắc bị lệch chuẩn hoặc không đồng nhất)
 - ✓ **Scratch** - Trầy xước (bề mặt có các vết xước do va chạm hoặc xử lý sai)
 - ✓ **Good** - Bình thường (không có lỗi)

7. Thuốc viên (pill)

Tương tự như viên nang (capsule), danh mục thuốc viên (pill) gồm các viên thuốc nhỏ có hình dạng đơn giản. Tuy nhiên, bề mặt và màu sắc có thể thay đổi tùy loại thuốc. Bất thường bao gồm mẻ cạnh, vỡ, vết bẩn hoặc biến màu. Đây là bài toán phổ biến trong kiểm định chất lượng dược phẩm.

- Số lượng tệp huấn luyện: 267 ảnh của sản phẩm bình thường (không có lỗi)
- Số lượng tệp kiểm tra: tổng cộng 120 ảnh bao gồm:
 - ✓ **Crack** - Nứt
 - ✓ **Faulty imprint** - In sai (dấu in bị lỗi)
 - ✓ **Poke** - Bị đâm (lỗ thủng)
 - ✓ **Scratch** - Trầy xước
 - ✓ **Squeeze** - Bị bóp méo (biến dạng do lực ép)
 - ✓ **Good** - Bình thường (không có lỗi)

8. Gạch lát (tile)

Tile là danh mục kết cấu với đặc trưng là các hoa văn lát gạch lặp lại. Bất thường có thể là nứt, bẩn, xước hoặc hoa văn bị lệch. Vì tính chất lặp lại của kết cấu nên mô hình cần phát hiện được những điểm phá vỡ mẫu để đánh giá chính xác.

- Số lượng tệp huấn luyện: 230 ảnh của sản phẩm bình thường (không có lỗi)
- Số lượng tệp kiểm tra: tổng cộng 115 ảnh bao gồm:
 - ✓ **Crack** - Nứt vỡ trên bề mặt gạch lát
 - ✓ **Glue strip** - Lỗi do vệt keo bề mặt
 - ✓ **Gray stroke** - Lỗi có vết xám bất thường
 - ✓ **Oil** - Lỗi dính dầu trên bề mặt
 - ✓ **Rough** - Lỗi bề mặt bị nhám hoặc trầy xước
 - ✓ **Good** - Bình thường (không có lỗi)

9. Hạt phỉ (hazelnut)

Hazelnut là danh mục thuộc nhóm vật thể, đại diện cho một loại nông sản có hình dạng đặc trưng và kích thước nhỏ. Trong môi trường công nghiệp, việc phát hiện các lỗi nhỏ trên bề mặt hạt như nứt, lỗ, vết cắt hay lỗi in ấn là rất quan trọng nhằm đảm bảo chất lượng đầu ra đồng đều. Dữ liệu trong danh mục này có độ chi tiết cao và độ tương phản rõ ràng, phù hợp để kiểm tra khả năng định vị chính xác các vùng bất thường nhỏ và phân biệt với các mẫu tốt.

- Số lượng tệp huấn luyện: 391 ảnh của sản phẩm bình thường (không có lỗi)
- Số lượng tệp kiểm tra: tổng cộng 108 ảnh bao gồm:
 - ✓ **Cut** - Lỗi do vết cắt lên vỏ hạt
 - ✓ **Crack** - Nứt vỡ trên bề mặt hạt phỉ
 - ✓ **Hole** - Lỗ hổng trên bề mặt
 - ✓ **Print** - Lỗi in ấn hoặc vết lạ trên vỏ
 - ✓ **Good** - Bình thường (không có lỗi)

10. Gỗ (wood)

Bề mặt gỗ là một kết cấu tự nhiên với nhiều biến thể, bao gồm vân gỗ, mắt gỗ,... Bất thường có thể là mẻ, nứt, vết bẩn hoặc trầy xước. Đây là một trong những thử thách thú vị cho mô hình vì phải phân biệt sự đa dạng tự nhiên và lỗi sản xuất.

- Số lượng tệp huấn luyện: 247 ảnh của sản phẩm bình thường (không có lỗi)
- Số lượng tệp kiểm tra: tổng cộng 79 ảnh bao gồm:
 - ✓ **Color** - Lỗi về màu sắc, đốm màu hoặc biến đổi tông màu trên bề mặt gỗ
 - ✓ **Hole** - Lỗ thủng hoặc lõm bất thường trên bề mặt
 - ✓ **Liquid** - Vết loang, thấm nước, hoặc rò rỉ chất lỏng
 - ✓ **Scratch** - vết xước cơ học do ma sát hoặc va chạm trên bề mặt gỗ
 - ✓ **Good** - Bình thường (không có lỗi)

5.3. Quá trình huấn luyện mô hình

Quá trình huấn luyện mô hình phát hiện bất thường trên tập dữ liệu MVTec AD được triển khai trong môi trường phát triển mạnh mẽ, linh hoạt và tối ưu hóa cho các tác vụ thị giác máy tính.

- **Môi trường phần cứng thực hiện huấn luyện**

Hệ thống được triển khai trên máy tính cá nhân có trang bị GPU NVIDIA RTX 3050Ti, đảm bảo khả năng xử lý song song hiệu quả, rút ngắn đáng kể thời gian huấn luyện so với CPU truyền thống.

- **Môi trường phần mềm thực hiện huấn luyện**

Môi trường phần mềm được xây dựng trên nền tảng Python 3 kết hợp cùng PyTorch 1.8, một trong những framework học sâu phổ biến nhất hiện nay, đặc biệt phù hợp cho việc xử lý hình ảnh và huấn luyện các mô hình mạng nơ-ron sâu.

➤ **Các thư viện và công cụ chính được sử dụng bao gồm:**

Einops: Hỗ trợ thao tác và tái cấu trúc tensor trực quan, đơn giản hóa quy trình xử lý dữ liệu.

Timm: Thư viện chứa nhiều mô hình trích xuất đặc trưng (feature extractor) hiện đại như EfficientNet, ResNet, Vision Transformer (ViT)...., giúp nâng cao chất lượng biểu diễn đặc trưng đầu vào.

wandb: Công cụ theo dõi và trực quan hóa quá trình huấn luyện, giúp phân tích hiệu suất mô hình, so sánh giữa các lần thực nghiệm, và lưu trữ lịch sử huấn luyện đầy đủ.

omegaconf: Quản lý tệp cấu hình dạng YAML một cách rõ ràng, linh hoạt, thuận tiện cho việc tinh chỉnh siêu tham số trong các thí nghiệm.

imgaug: Tăng cường dữ liệu ảnh đầu vào với các kỹ thuật như xoay, thay đổi độ sáng, cắt xén, và thêm nhiễu, từ đó cải thiện khả năng tổng quát hóa của mô hình trong các tình huống thực tế.

ipywidgets: Tạo giao diện người dùng tương tác ngay trong Jupyter Notebook, hỗ trợ người phát triển kiểm tra kết quả mô hình một cách trực quan và tiện lợi.

voila: Chuyển đổi notebook sang giao diện web, giúp người dùng cuối có thể dễ dàng thao tác với mô hình qua trình duyệt mà không cần can thiệp vào mã nguồn.

- **Chi tiết triển khai (Implementation Details)**

Mô hình MemSeg được phát triển dựa trên kiến trúc AutoEncoder với backbone là ResNet18 để trích xuất đặc trưng, phù hợp cho các bài toán phát hiện bất thường (anomaly detection) trong ảnh công nghiệp. Dưới đây là các thông số cấu hình cụ thể đã được sử dụng trong quá trình huấn luyện

- **Dataset chính:** MVTec AD

- **Dataset phụ:** Describable Textures Dataset (DTD)

- **Resize ảnh đầu vào:** 288×288

Trước khi xử lý, tất cả ảnh đều được resize về kích thước 288×288 pixel. Việc này đảm bảo kích thước đầu vào đồng nhất, phù hợp với kiến trúc mô hình, đồng thời giúp giảm thiểu độ phức tạp tính toán và tăng tốc độ huấn luyện.

- **Crop ảnh đầu vào:** 256×256

Sau khi resize, ảnh được crop (cắt) lại về kích thước 256×256 . Phần lớn nội dung chính của ảnh nằm ở trung tâm, nên việc crop này giúp tập trung vào vùng quan trọng, đồng thời loại bỏ viền rìa có thể chứa nhiều nhiễu không cần thiết.

- **Structure grid size:** 8

Tham số này chỉ định chia ảnh thành các ô lưới kích thước 8×8 để áp dụng mô phỏng lỗi dạng cấu trúc. Mỗi ô có thể bị biến dạng hoặc chèn thêm nhiễu nhằm tạo ra lỗi mang tính hình học hoặc kết cấu, phục vụ huấn luyện mô hình phát hiện lỗi đa dạng.

- **Transparency_range:** [0.15, 1.0]

Khoảng giá trị cho độ trong suốt khi chồng lỗi lên ảnh bình thường. Giá trị thấp khiến lỗi khó nhận biết (giả lập lỗi nhẹ), trong khi giá trị cao thể hiện lỗi rõ nét. Khoảng này giúp mô hình học tốt hơn với nhiều cấp độ dị thường khác nhau.

➤ **Perlin_scale: 6**

Tham số điều chỉnh cường độ và độ rộng của nhiễu Perlin. Giá trị càng lớn thì vùng nhiễu càng mượt và rộng, giúp sinh lỗi có dạng tự nhiên hơn.

➤ **Perlin noise threshold: 0.5**

Ngưỡng để quyết định vùng nào trong ảnh sẽ bị gán lỗi khi sử dụng Perlin noise. Chỉ những điểm có cường độ nhiễu vượt quá ngưỡng 0.5 mới được coi là lỗi. Điều này tránh việc lan tỏa lỗi lên toàn ảnh, giữ cho vùng lỗi có giới hạn rõ ràng.

➤ **Kích thước batch: 8** (gồm 4 ảnh bình thường và 4 ảnh bất thường mô phỏng)

Số lượng ảnh trong mỗi batch được chia đều giữa ảnh bình thường và ảnh bất thường, điều này giữ cho mô hình cân bằng khi học.

➤ **Số lượng mẫu bộ nhớ (memory bank): 30**

MemSeg sử dụng bộ nhớ để lưu trữ các đặc trưng từ ảnh bình thường giúp cải thiện khả năng phát hiện điểm bất thường trong ảnh mới.

➤ **Backbone trích xuất đặc trưng: ResNet18**

ResNet18 là một mạng hiệu quả, giúp giảm thiểu chi phí tính toán nhưng vẫn giữ lại các đặc trưng cần thiết. Đây là lựa chọn phù hợp cho mô hình này

➤ **Số bước huấn luyện: 5000 bước**

Đây là tổng số vòng lặp mà mô hình được huấn luyện. Trong mỗi bước, mô hình sẽ cập nhật trọng số dựa trên loss thu được từ một batch ảnh. Việc huấn luyện đủ số bước là cần thiết để mô hình hội tụ và học được đặc điểm dị thường chính xác.

➤ **Learning rate: 0.003**

Learning rate là tốc độ mà mô hình cập nhật trọng số trong quá trình huấn luyện. Giá trị 0.003 được lựa chọn từ quá trình thử nghiệm (grid search), đảm bảo cân bằng giữa tốc độ học và độ ổn định trong quá trình tối ưu.

➤ **Weight decay: 0.0005**

Weight decay là hệ số điều chuẩn nhằm giới hạn độ lớn của các trọng số trong mô hình, từ đó giúp tránh hiện tượng overfitting. Mức 0.0005 là lựa chọn phù hợp với mô hình ResNet18 để giữ cho mô hình không bị học quá mức vào dữ liệu huấn luyện.

➤ **Trọng số hàm mất mát**

L1_weight: 0.6

Trọng số dành cho thành phần L1 Loss, dùng để đo sai khác tuyệt đối giữa ảnh đầu ra và ảnh gốc. Hàm L1 giúp mô hình tái tạo ảnh nền tốt, từ đó phát hiện rõ các dị thường (nơi ảnh bị tái tạo kém).

Focal_weight: 0.4

Trọng số cho hàm Focal Loss, nhằm tăng cường khả năng học từ các điểm bất thường khó nhận biết. Hàm này rất hữu ích trong bài toán dị thường vì những điểm lỗi thường chiếm tỷ lệ rất nhỏ trong ảnh.

Focal_gamma: 4

Tham số gamma điều chỉnh độ nhạy của Focal Loss. Với $\gamma = 4$, mô hình sẽ "phớt lờ" các điểm dễ đoán và tập trung nhiều hơn vào những điểm lỗi khó phát hiện, từ đó cải thiện hiệu suất phân biệt.

➤ **Tần suất đánh giá mô hình (eval_interval): 100 bước**

Sau mỗi 100 bước huấn luyện, mô hình sẽ được đánh giá lại trên tập dữ liệu kiểm thử. Việc này cho phép theo dõi quá trình học của mô hình theo thời gian thực, phát hiện sớm các vấn đề như overfitting hoặc underfitting, từ đó điều chỉnh các tham số huấn luyện kịp thời nếu cần thiết.

➤ **Warm-up scheduler**

Trong giai đoạn đầu huấn luyện, learning rate được tăng dần từ giá trị nhỏ ($\text{min_lr} = 0.0001$) đến mức tối ưu ($\text{lr} = 0.003$) theo tỷ lệ warm-up ($\text{warmup_ratio} = 0.1$). Việc khởi đầu với tốc độ học thấp giúp mô hình ổn định hơn, tránh việc cập nhật trọng số quá lớn khi chưa học được đặc trưng cơ bản. Sau giai đoạn warm-up, learning rate được giữ nguyên để mô hình học sâu hơn. Scheduler warm-up đặc biệt hữu ích trong bài toán dị thường nơi dữ liệu đầu vào rất nhạy cảm với các thay đổi trọng số ban đầu.

5.4. Kết quả quá trình huấn luyện

- **Các thông số đánh giá**

- a. AUROC-image**

Area Under the ROC Curve at image level thực hiện đánh giá ở cấp độ toàn ảnh. Ở đây, mỗi ảnh được gán một nhãn đơn giản: 1 nếu là ảnh có lỗi và 0 nếu là ảnh bình thường. Mô hình sẽ đưa ra một giá trị “điểm bất thường” (anomaly score) cho toàn bộ ảnh, thay vì cho từng pixel. Sau đó, đường ROC được xây dựng bằng cách thay đổi ngưỡng phân loại và tính toán các tỷ lệ FPR và TPR trên tập ảnh. AUROC-image đo khả năng của mô hình trong việc phân biệt giữa các ảnh chứa lỗi và ảnh không lỗi. Một giá trị AUROC-image cao chứng tỏ mô hình có hiệu suất cao trong việc phân loại tổng thể và có khả năng tổng hợp tốt các dấu hiệu bất thường phân tán trên ảnh.

- b. AUROC-pixel**

Area Under the Receiver Operating Characteristic Curve at pixel level đánh giá khả năng phân biệt giữa pixel bình thường và pixel bất thường. Tại mỗi pixel, mô hình dự đoán một giá trị xác suất thể hiện mức độ bất thường. Các giá trị này sau đó được so sánh với nhãn thực tế (pixel có lỗi hay không) để xây dựng đường ROC. Diện tích dưới đường ROC (AUROC) phản ánh khả năng phân tách: giá trị càng gần 1 chứng tỏ mô hình càng giỏi phân biệt giữa pixel lỗi và pixel không lỗi. Ngược lại, giá trị gần 0.5 cho thấy mô hình dự đoán gần như ngẫu nhiên. AUROC-pixel rất hữu ích để hiểu mức độ chính xác vi mô của mô hình, đặc biệt trong các trường hợp lỗi nhỏ hoặc rải rác.

- c. AUPRO-pixel**

Area Under the Per-Region Overlap Curve lại đi sâu hơn vào việc đánh giá chất lượng phân đoạn theo vùng (region-level). Không chỉ xét riêng lẻ từng pixel, AUPRO xem xét khả năng mô hình phát hiện và khoanh vùng đầy đủ toàn bộ các khu vực bất thường. Cụ thể, với mỗi vùng lỗi thực tế, ta tính tỉ lệ intersection over

union - IoU giữa vùng dự đoán và vùng thật. Quá trình này được thực hiện với nhiều ngưỡng khác nhau để xây dựng một đường cong, sau đó tính diện tích dưới đường cong đó (AUPRO). Giá trị AUPRO cao cho thấy mô hình không chỉ phát hiện chính xác các pixel lẻ mà còn phát hiện chính xác hình dạng và ranh giới của vùng lỗi, đảm bảo tính nhất quán và đầy đủ của phân đoạn. Đây là chỉ số đặc biệt quan trọng trong các ứng dụng yêu cầu mô hình khoanh vùng lỗi rõ ràng, như kiểm tra bề mặt công nghiệp hoặc y tế hình ảnh.

• Kết quả quá trình huấn luyện

Trong quá trình huấn luyện trên 10 loại sản phẩm khác nhau thuộc bộ dữ liệu MVTec AD. Nhìn chung, mô hình mô hình đạt được kết quả rất khả quan với các chỉ số đánh giá bao gồm AUROC-image, AUROC-pixel và AUPRO-pixel. Điều này cho thấy khả năng phân biệt khuyết tật bề mặt rất tốt trên cả cấp độ ảnh và pixel, với hiệu suất trung bình cao và ổn định trên nhiều loại bề mặt và vật thể. Kết quả chi tiết các chỉ số đánh giá và thời gian huấn luyện cho từng danh mục được thể hi

Tập khảo sát	AUROC-image	AUROC-pixel	AUPRO-pixel	Thời gian huấn luyện
Bottle	1.000	0.9885	0.9859	47 phút 25 giây
Capsule	0.9454	0.9560	0.9768	76 phút 41 giây
Pill	0.9836	0.9889	0.9801	59 phút 46 giây
Metal nut	0.9971	0.9301	0.9651	66 phút 43 giây
Hazelnut	1.000	0.9735	0.9895	73 phút 24 giây
Carpet	0.9823	0.9809	0.9717	77 phút 57 giây
Leather	1.000	0.9931	0.9961	65 phút 47 giây
Tile	0.9996	0.9952	0.9924	66 phút 40 giây
Wood	0.9960	0.9751	0.9718	65 phút 52 giây
Grid	1.0000	0.9831	0.9823	53 phút 32 giây
Trung Bình	0.9904	0.9764	0.9812	65 phút 23 giây

Bảng 1: Tổng hợp kết quả huấn luyện

Chỉ số AUROC-image đạt trung bình **0.9904**, cho thấy mô hình gần như phân biệt hoàn toàn giữa ảnh bình thường và ảnh dị thường. Đặc biệt, một số tập dữ liệu như *Bottle*, *Hazelnut*, *Leather* và *Grid* đạt điểm tuyệt đối hoặc gần tuyệt đối (1.0000), chứng minh khả năng nhận diện dị thường và phân loại rất rõ ràng ở cấp

độ tổng thể của ảnh. Tuy nhiên, ở một số tệp dữ liệu như *Capsule* và *Metal nut* có giá trị thấp hơn (0.9454 và 0.9971), điều này cho thấy với những tệp có những loại dị thường nhỏ hoặc tinh vi mô hình vẫn còn gặp đôi chút khó khăn ở bước phân loại ảnh tổng thể.

Ở cấp độ pixel, chỉ số AUROC-pixel trung bình là **0.9764**, cho thấy mô hình không chỉ có khả năng phát hiện dị thường ở cấp độ ảnh mà còn có thể thực hiện ở cấp độ pixel. Các tệp như *Tile* và *Leather* đạt chỉ số AUROC-pixel lên đến trên 0.99, các tệp còn lại cũng có kết quả tốt trong khoảng từ 0.97-0.99. Tuy nhiên, cũng có tệp có giá trị thấp hơn như là *Metal nut* (0.9301) do đặc thù của tệp này bao gồm chi tiết nhỏ, phức tạp.

Chỉ số AUPRO-pixel đạt giá trị trung bình **0.9812**, thể hiện khả năng khoanh vùng chính xác các vùng dị thường của mô hình trên các ảnh có lỗi. Khác với AUROC-pixel vốn xét toàn bộ pixel trong ảnh, AUPRO chỉ tính toán trên các vùng dị thường thực sự, đồng thời đánh giá độ trùng khớp giữa vùng dự đoán và vùng ground truth thông qua nhiều ngưỡng xác suất. Nhờ đó, AUPRO phản ánh rõ ràng hơn độ chính xác hình học trong việc phát hiện lỗi. Các tệp như *Leather*, *Tile*, *Grid* đạt AUPRO trên 0.99, cho thấy mô hình không chỉ phát hiện đúng mà còn khoanh vùng sát với vùng lỗi thực tế. Trong khi đó, một số lớp như *Metal nut* (0.9651) và *Wood* (0.9718) có AUPRO thấp hơn, cho thấy mô hình đôi lúc bỏ sót hoặc lan rộng ra ngoài vùng dị thường, đặc biệt khi dị thường có kích thước nhỏ, rời rạc hoặc có kết cấu phức tạp.

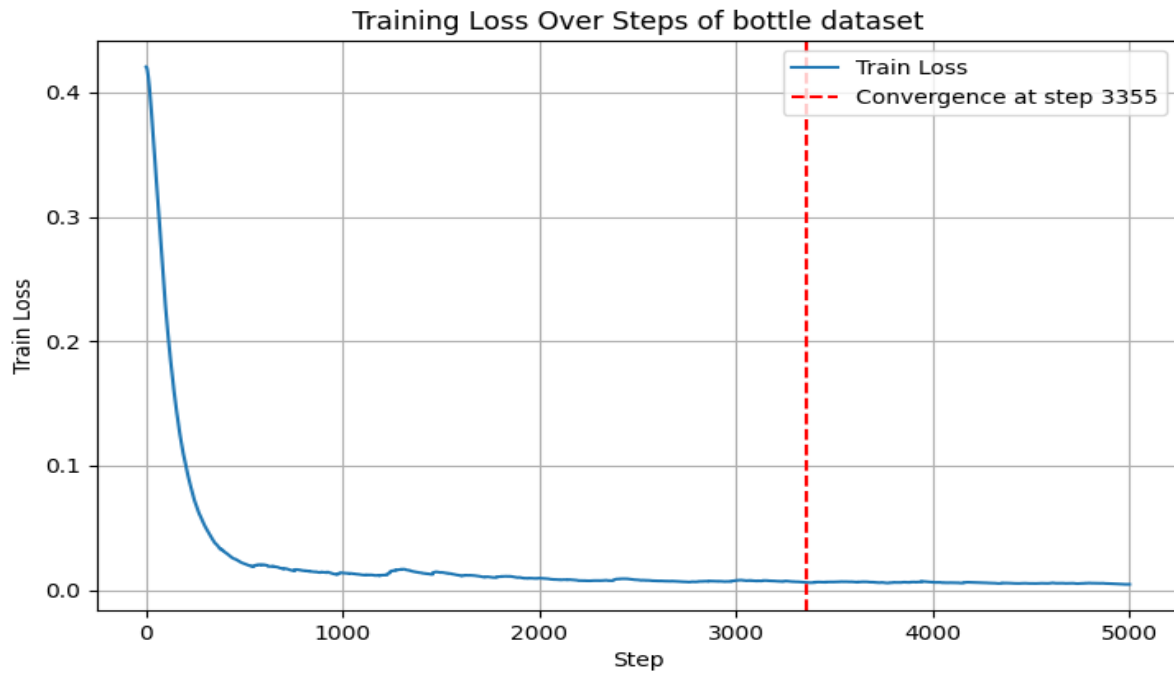
Về thời gian huấn luyện, thời gian trung bình để huấn luyện một mô hình trên từng lớp là **65 phút 23 giây**, nằm trong phạm vi chấp nhận được khi triển khai thực tế. Các lớp như *Capsule* và *Carpet* có thời gian huấn luyện dài hơn (trên 76 phút), là do độ phức tạp cũng như sự đa dạng hình học trong ảnh làm quá trình hội tụ kéo dài hơn.

Tổng thể, kết quả thực nghiệm chứng minh rằng mô hình MemSeg kết hợp với backbone ResNet18 cho thấy khả năng học nhanh, độ chính xác cao và thời gian

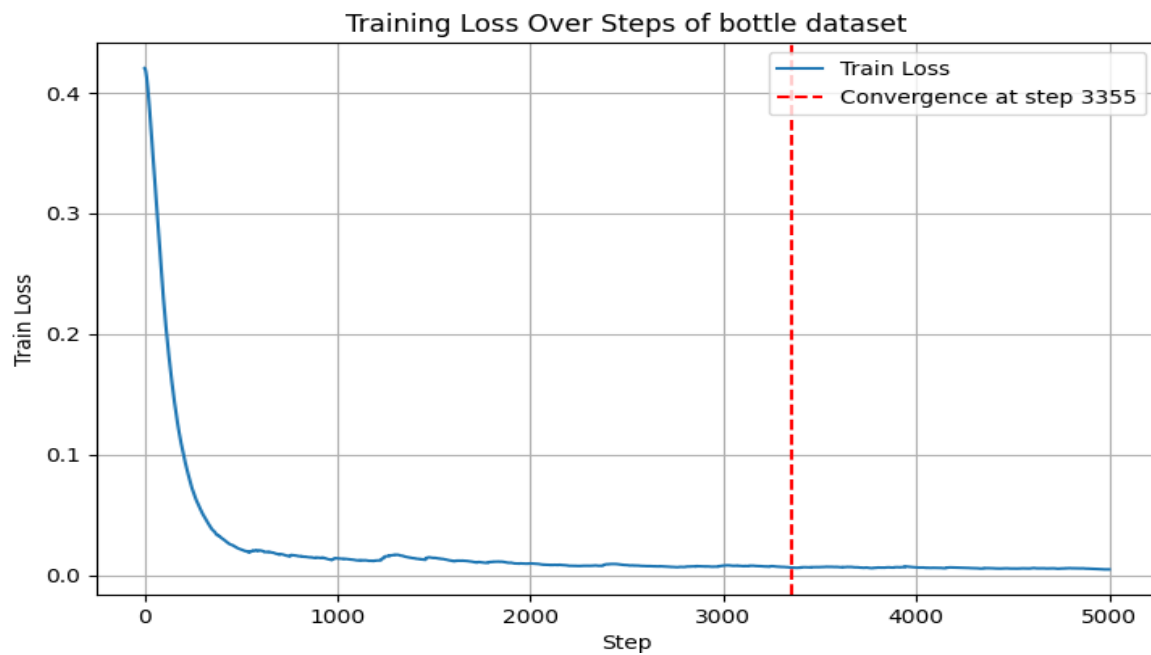
huấn luyện hiệu quả trong việc phát hiện và khoanh vùng lỗi bề mặt trên đa dạng đối tượng trong bộ dữ liệu MVTec.

- **Biểu đồ chi tiết huấn luyện của các tệp data khác nhau**

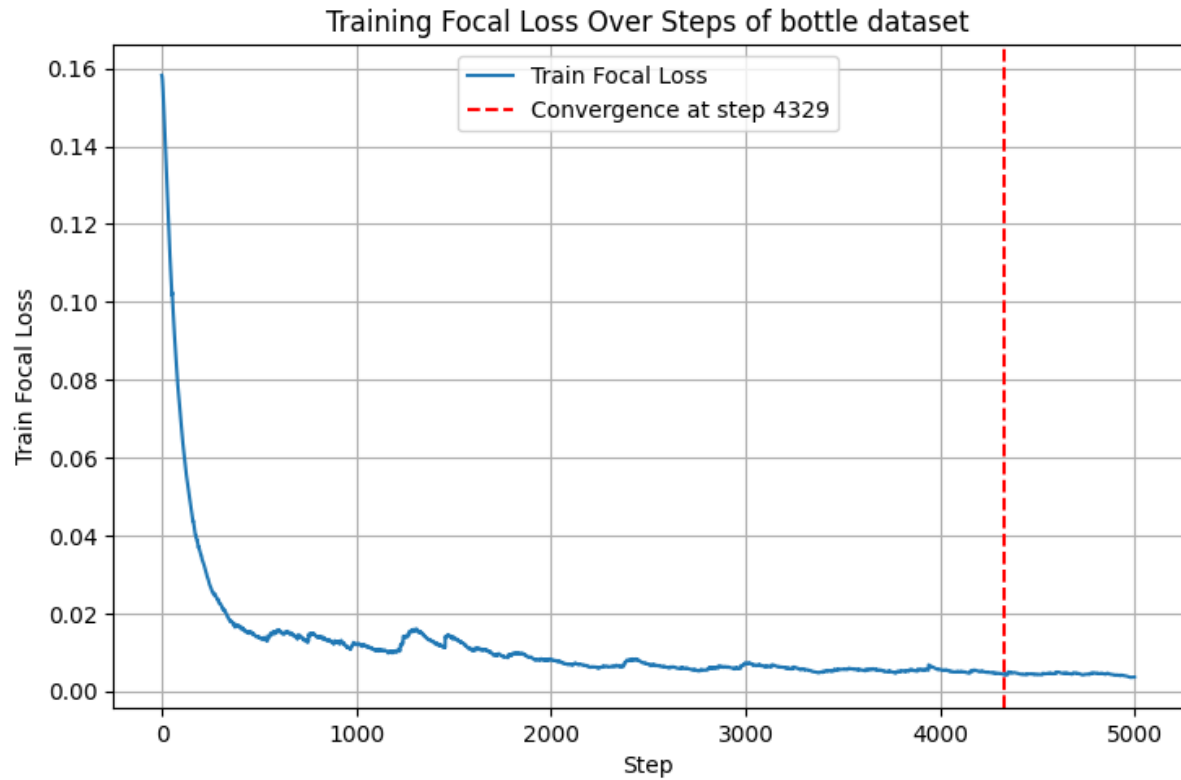
- 1. Bottle**



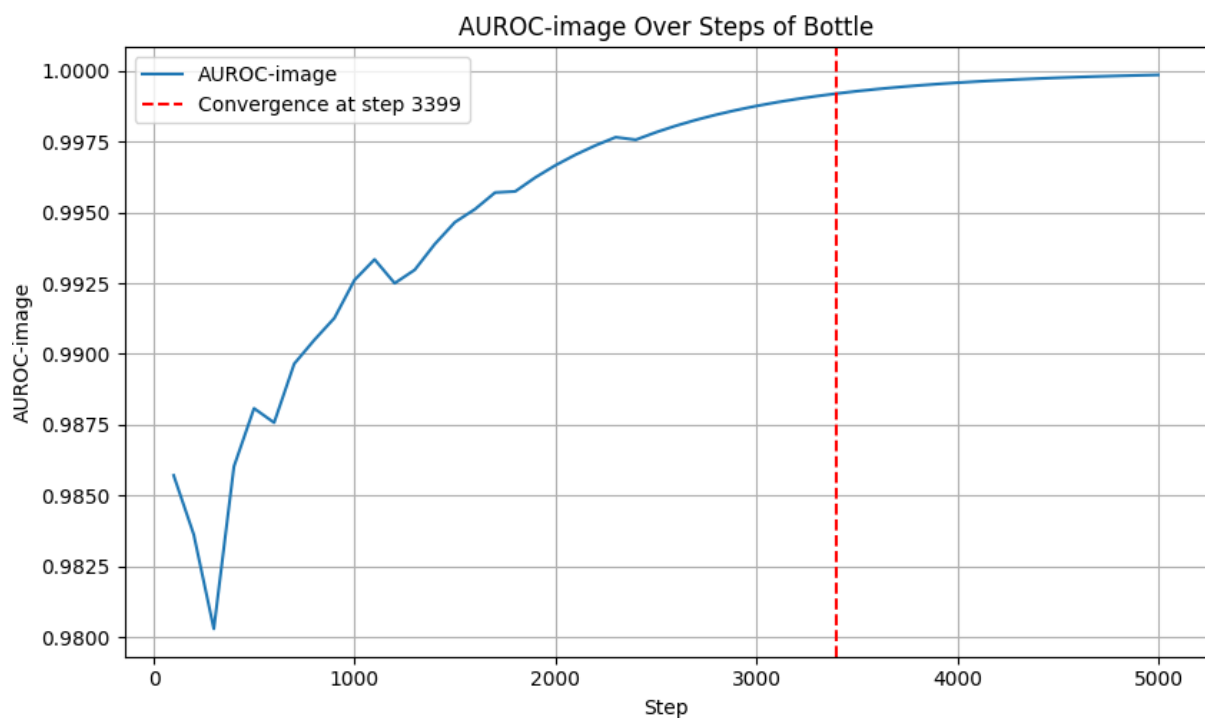
Hình 5.5: Biểu đồ hàm train loss của bottle



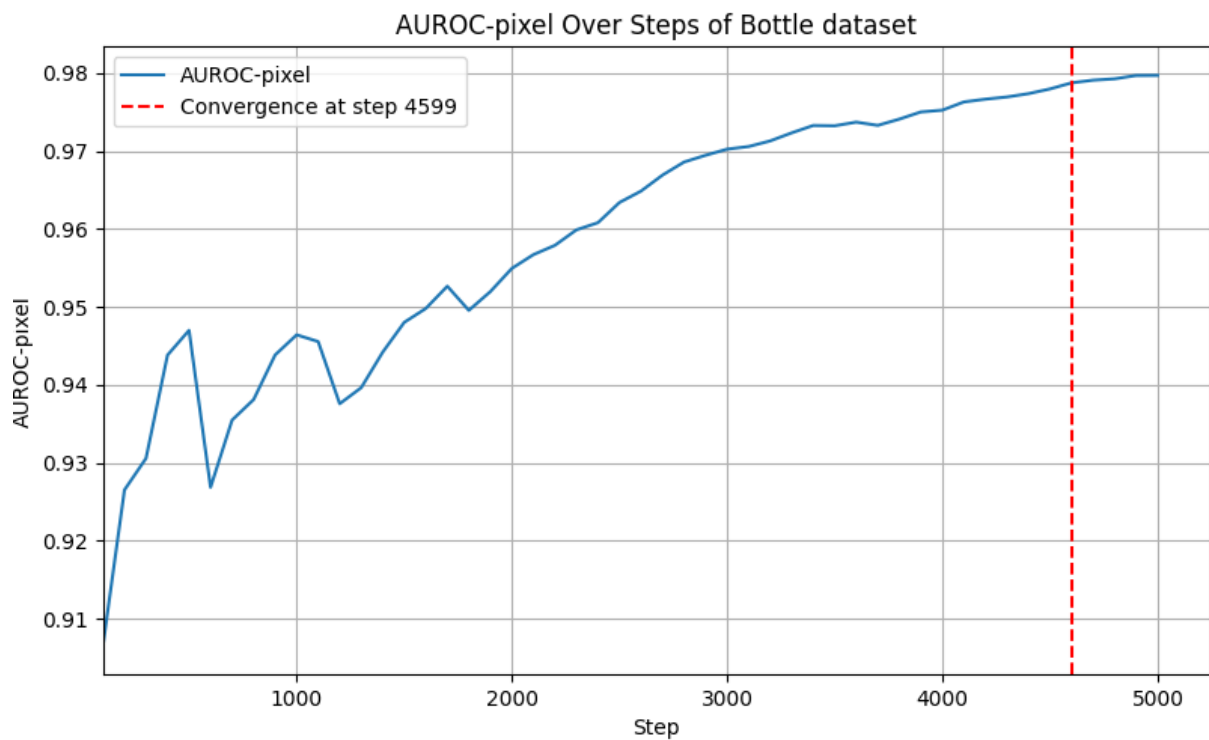
Hình 5.6: Biểu đồ hàm train L1 loss của bottle



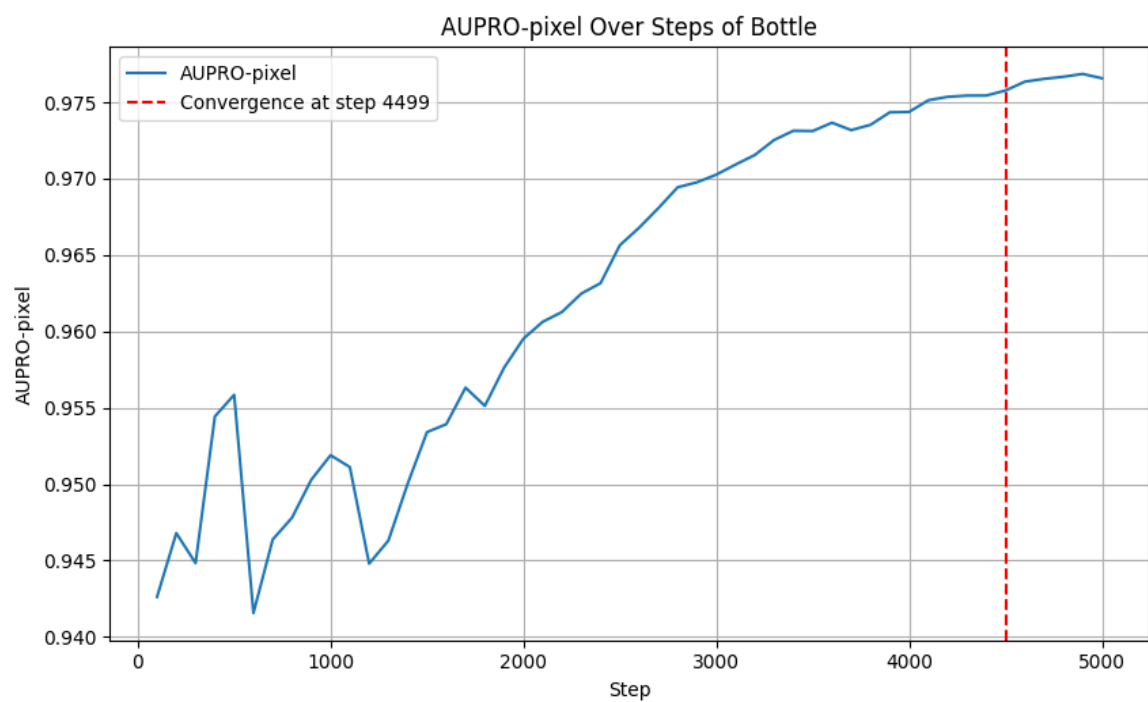
Hình 5.7: Biểu đồ hàm train focal loss của bottle



Hình 5.8: Biểu đồ chỉ số AUROC-image của bottle

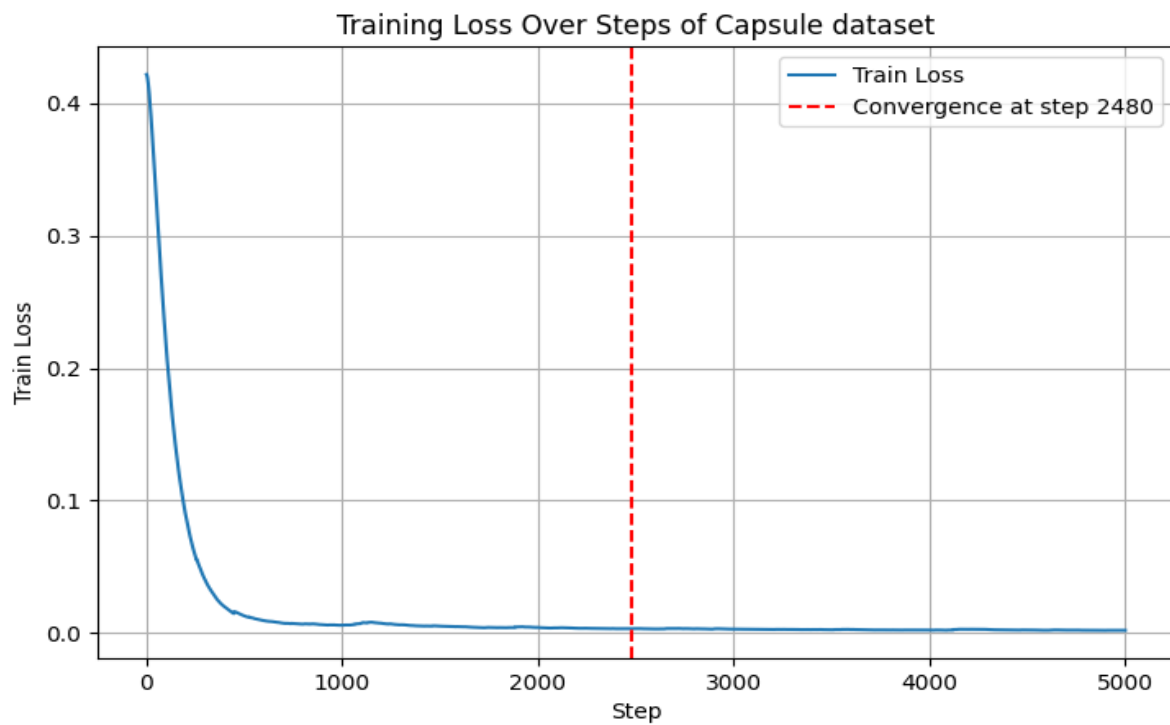


Hình 5.9: Biểu đồ AUROC-pixel của bottle

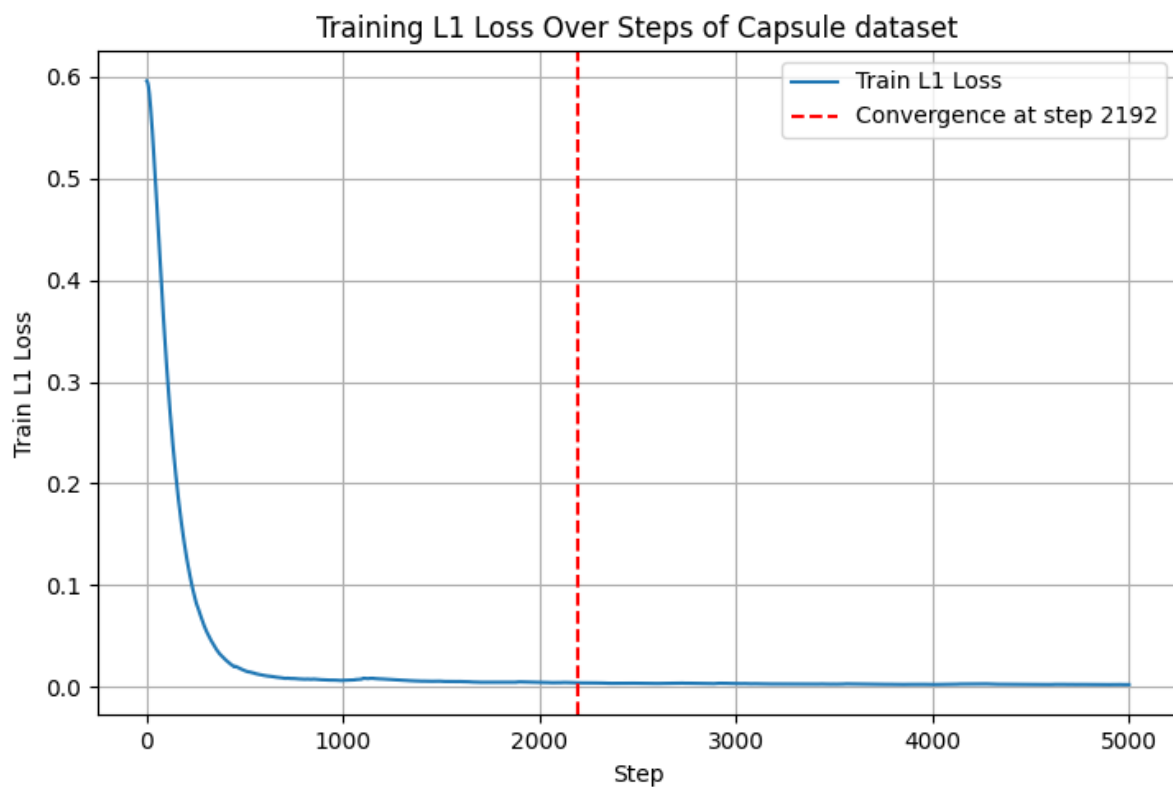


Hình 5.10: Biểu đồ AUPRO-pixel của bottle

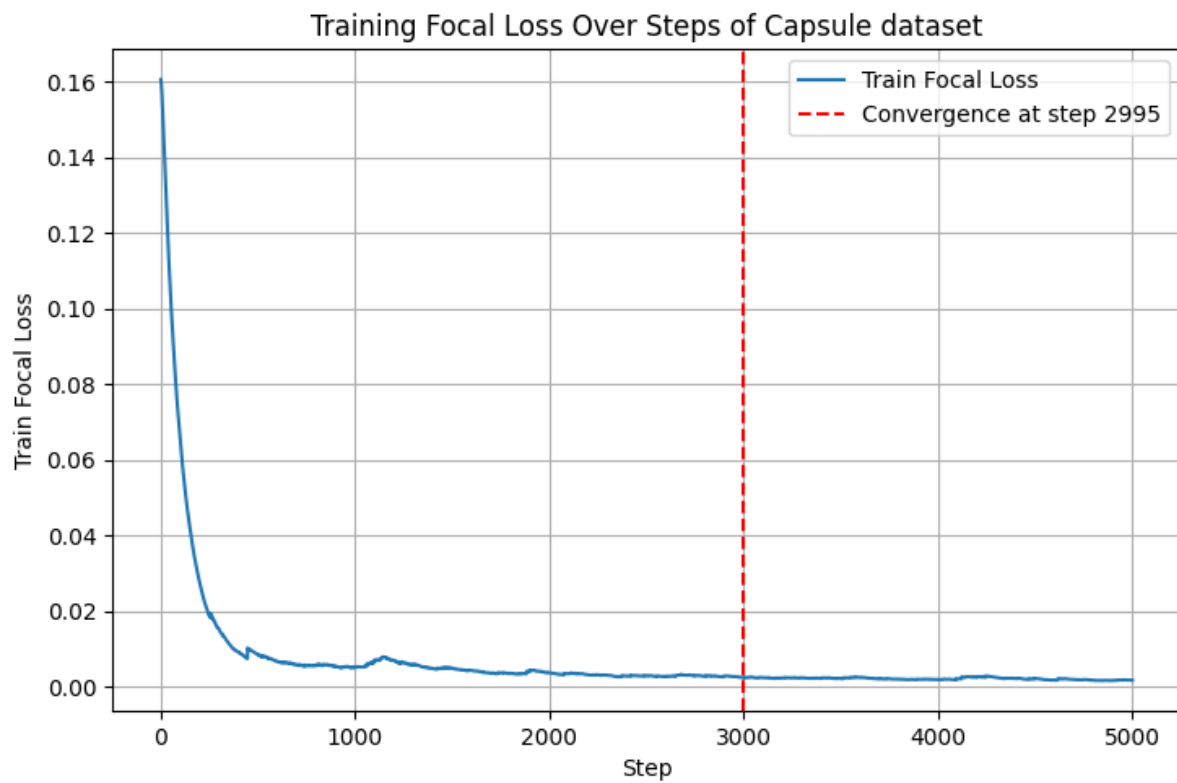
2. Capsule



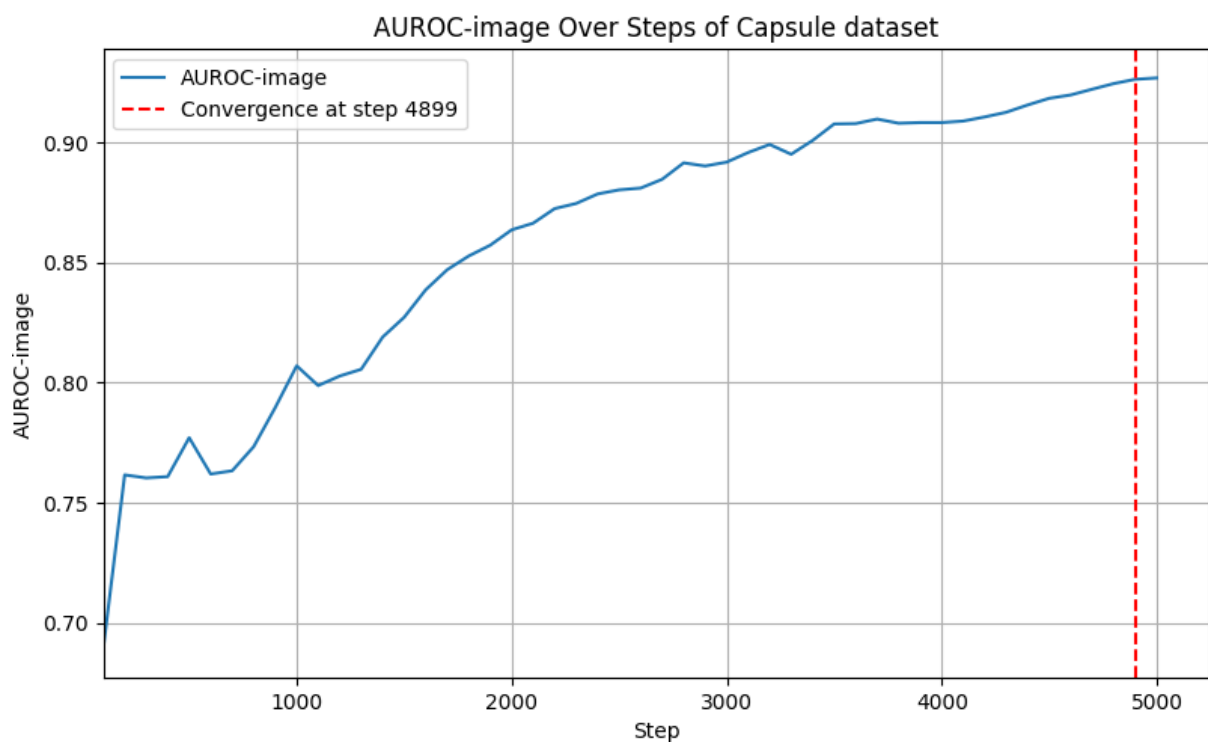
Hình 5.11: Biểu đồ hàm train loss của capsule



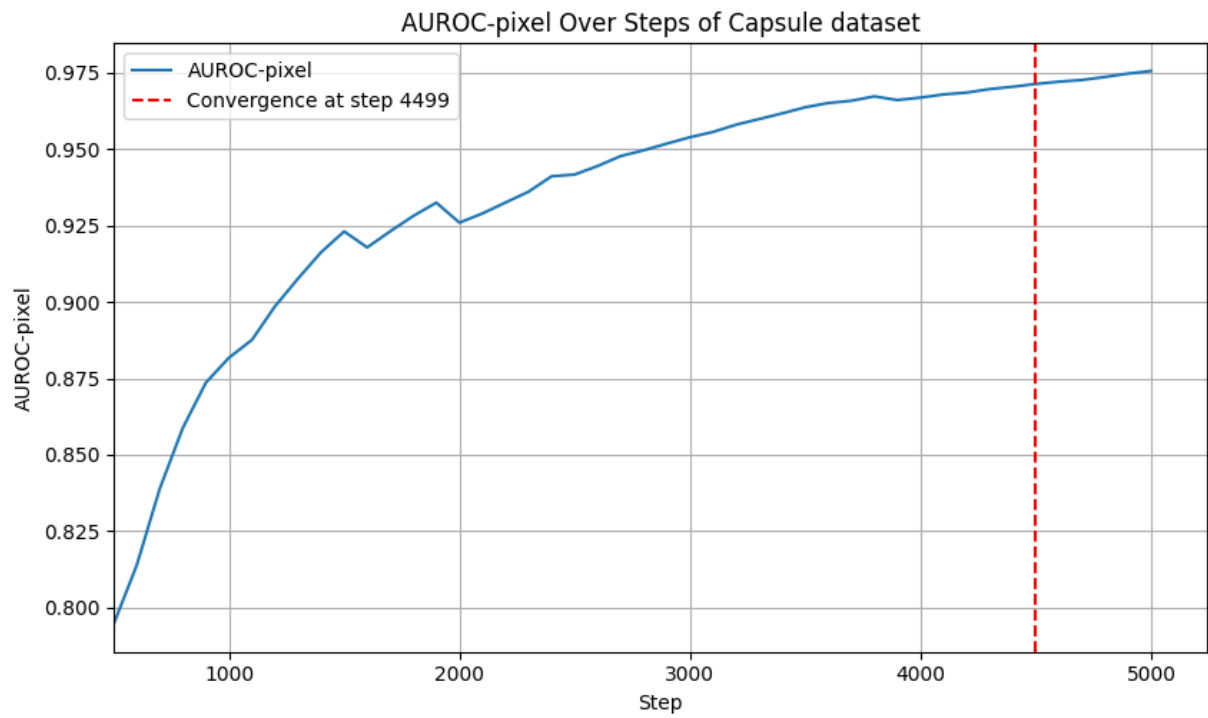
Hình 5.12: Biểu đồ hàm train L1 loss của capsule



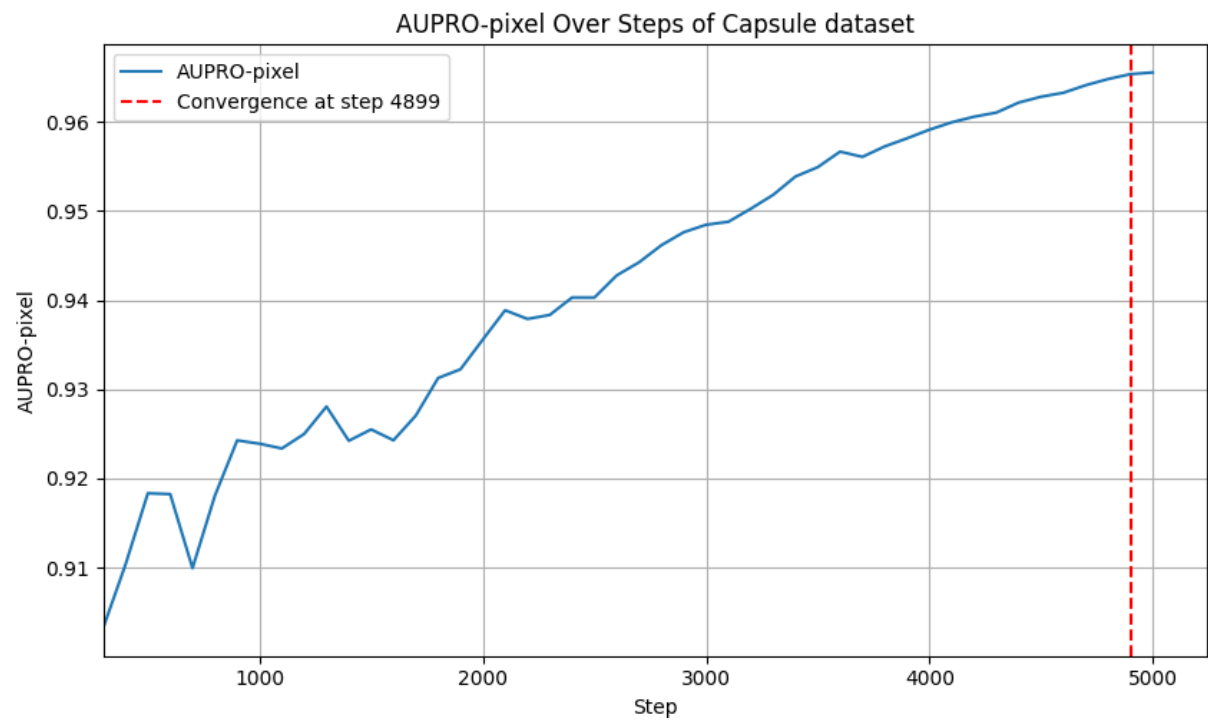
Hình 5.13: Biểu đồ hàm train focal loss của capsule



Hình 5.14: Biểu đồ AUROC-image của Capsule

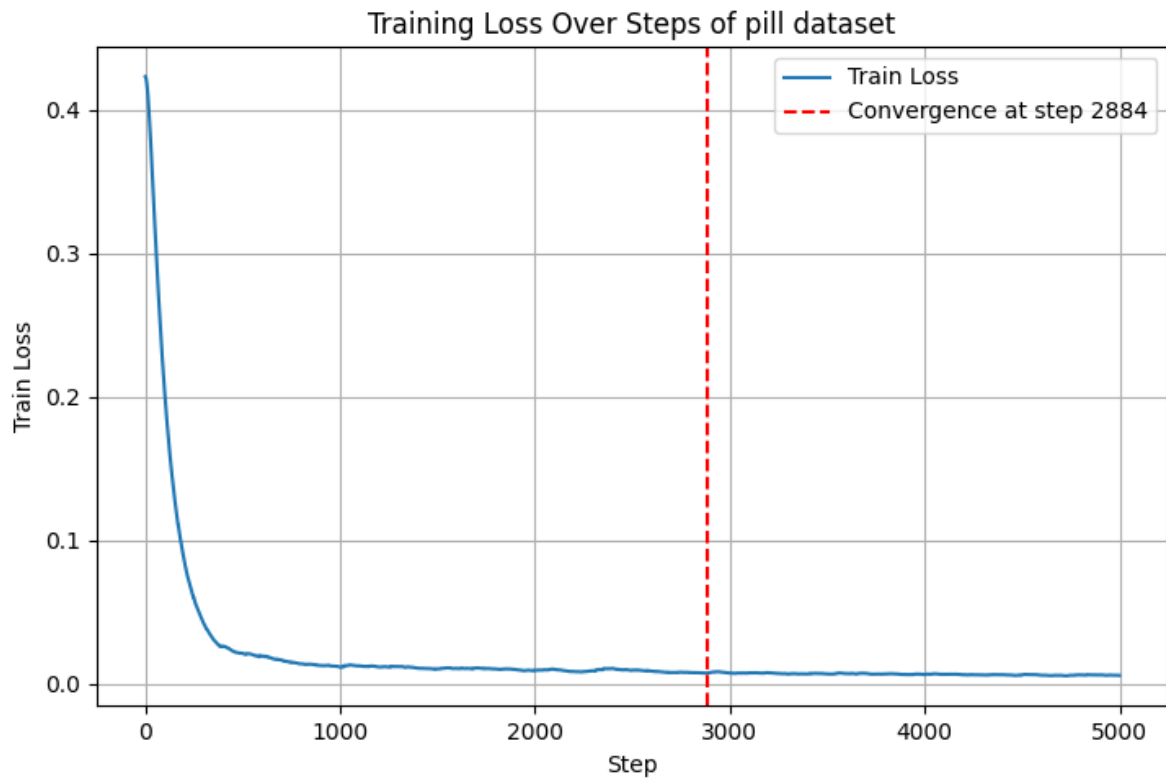


Hình 5.15: Biểu đồ AUROC-pixel của Capsule

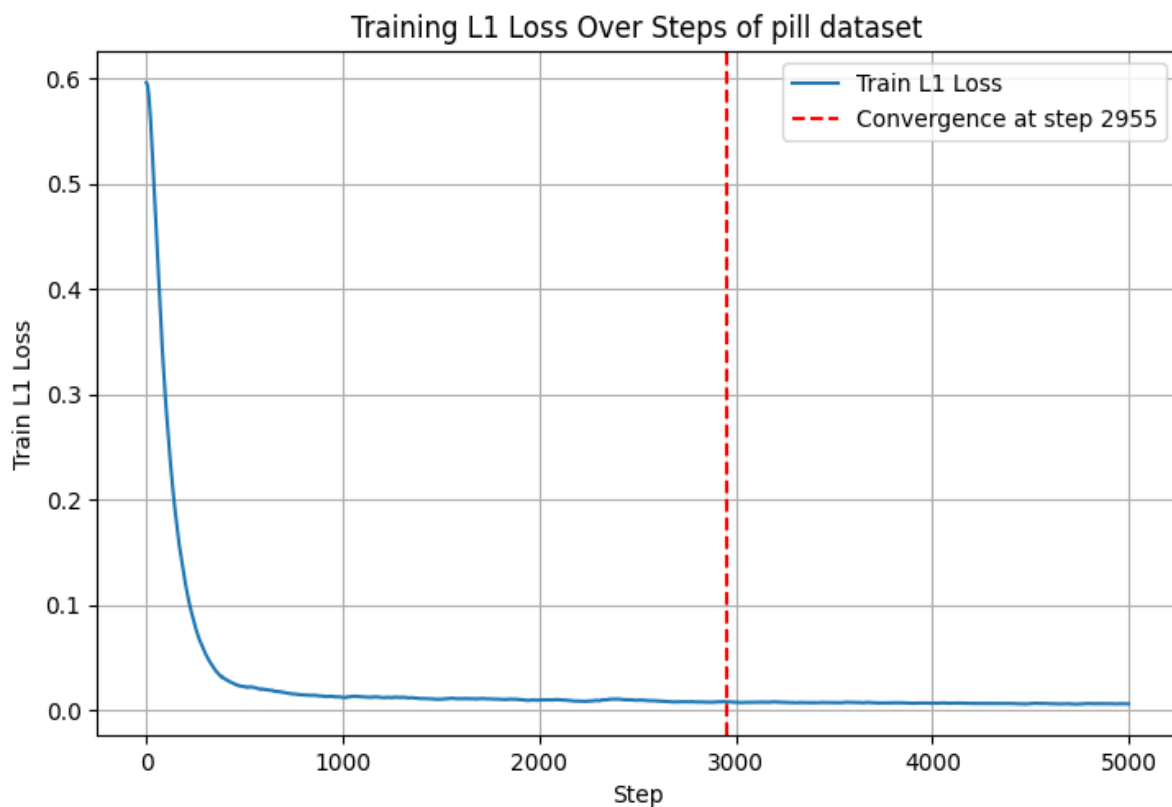


Hình 5.16: Biểu đồ AUPRO-pixel của tập Capsule

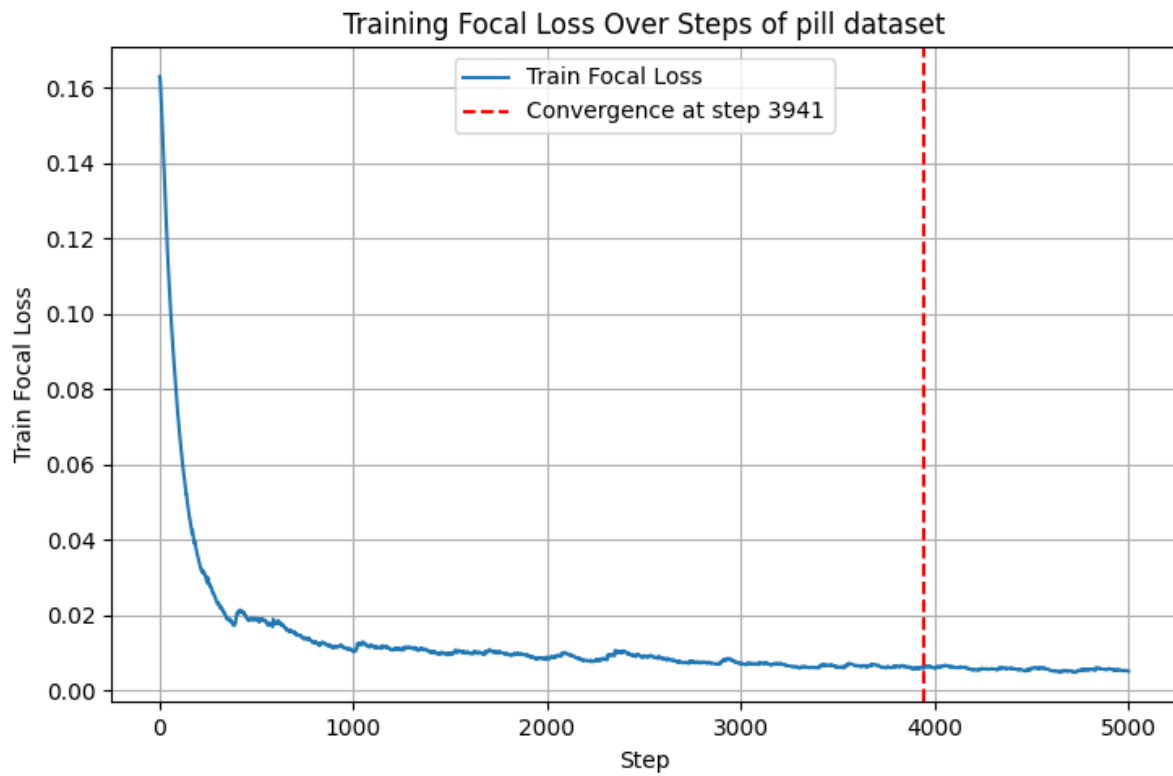
3. Pill



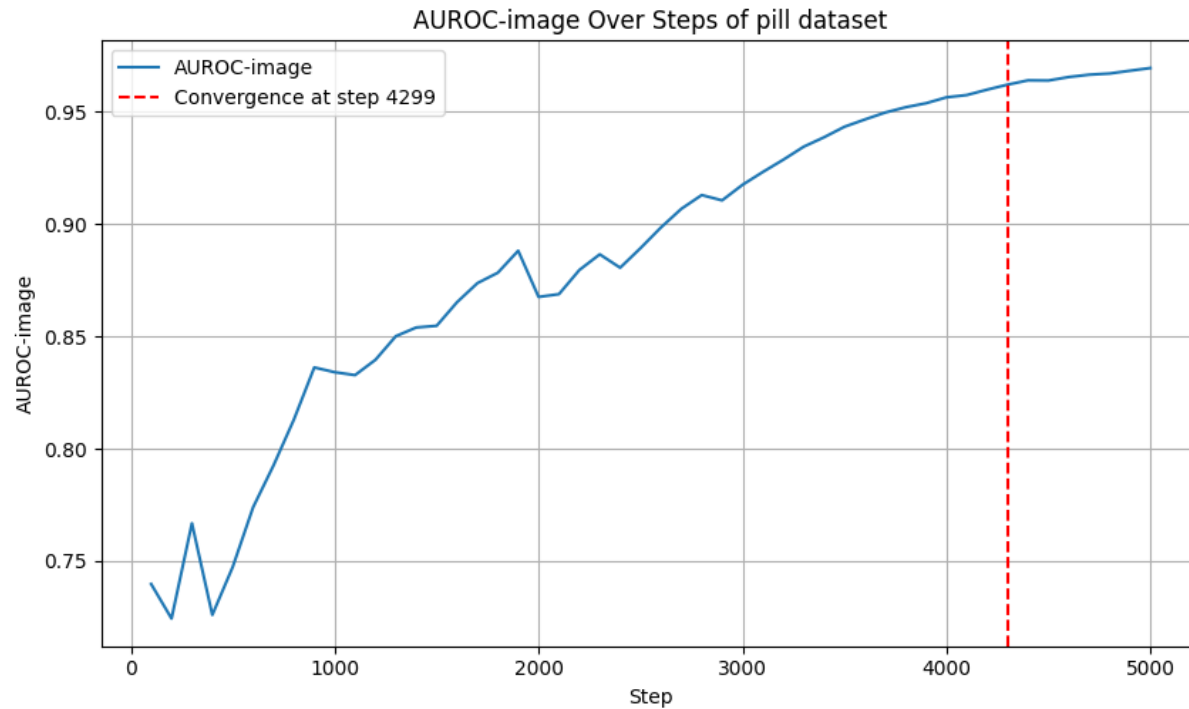
Hình 5.17: Biểu đồ hàm train loss của Pill



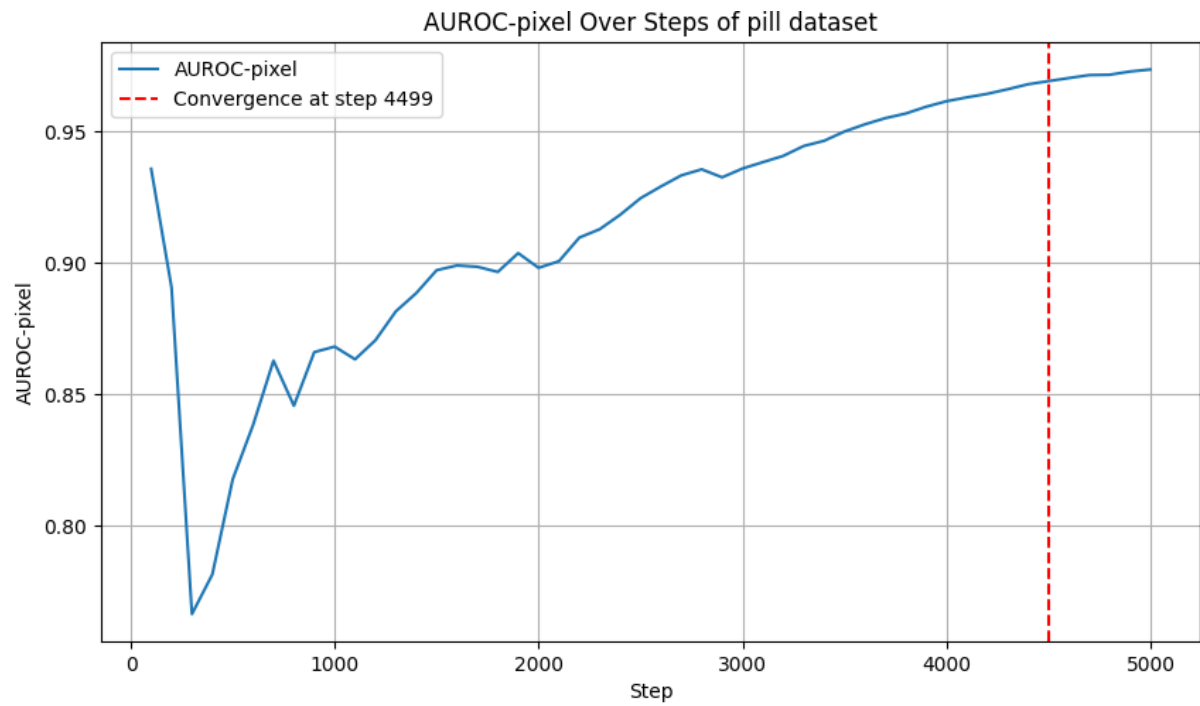
Hình 5.18: Biểu đồ hàm train L1 loss của Pill



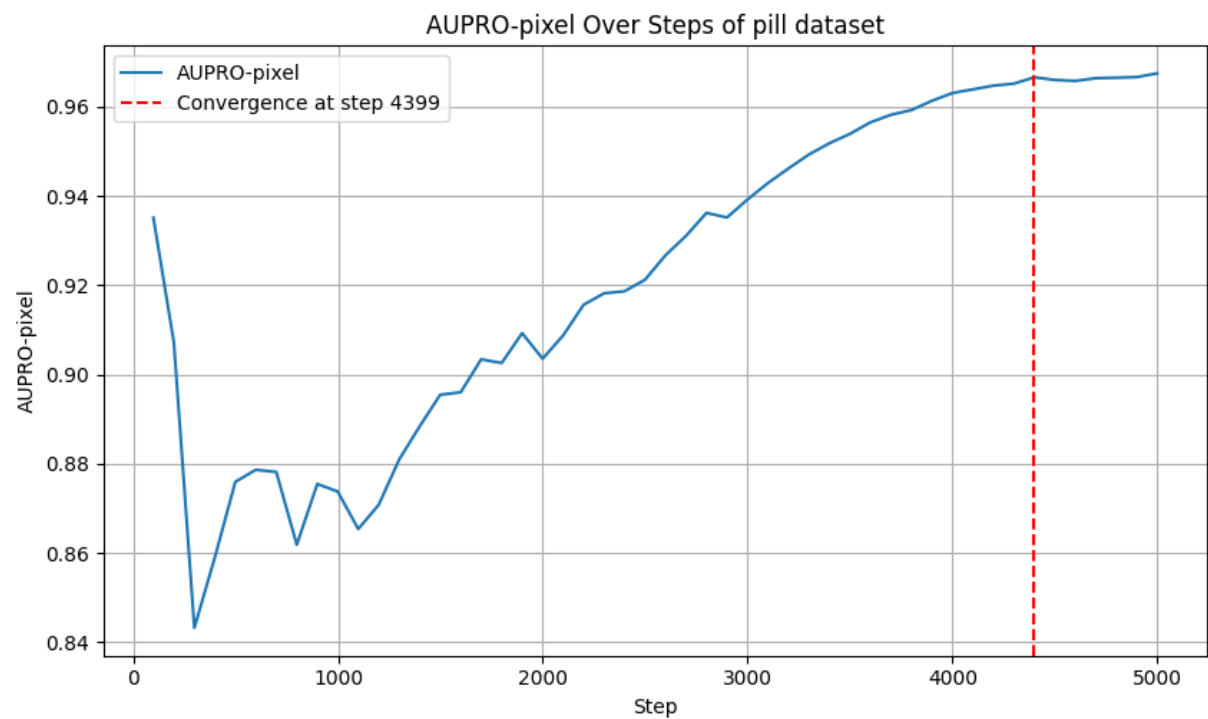
Hình 5.19: Biểu đồ hàm train focal loss của Pill



Hình 5.20: Biểu đồ AUROC-image của Pill

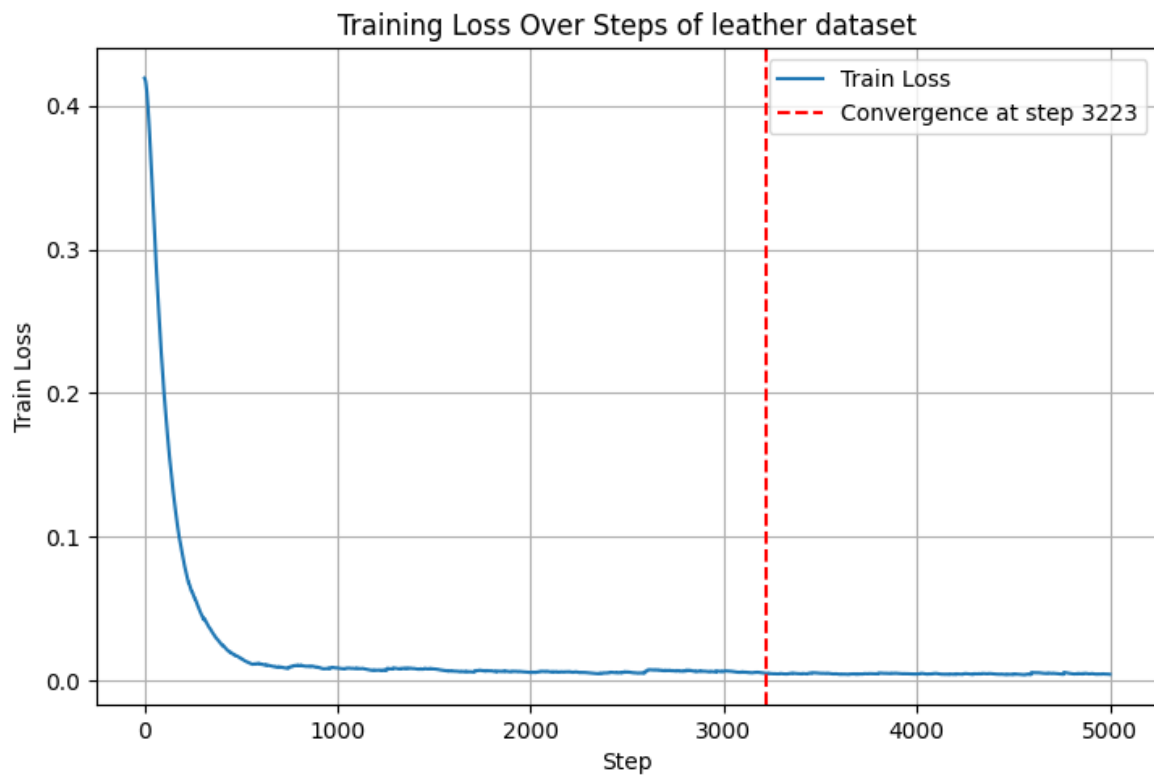


Hình 5.21: Biểu đồ AUROC-pixel của Pill

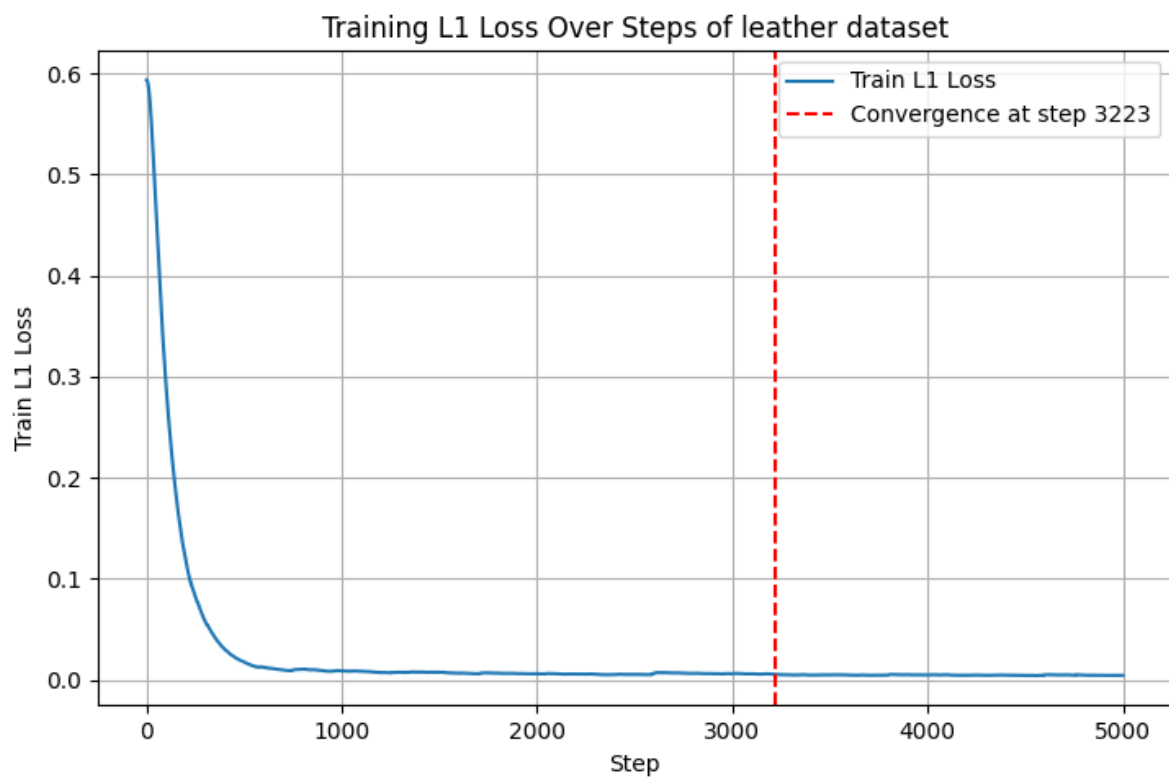


Hình 5.22: Biểu đồ AUPRO-pixel của Pill

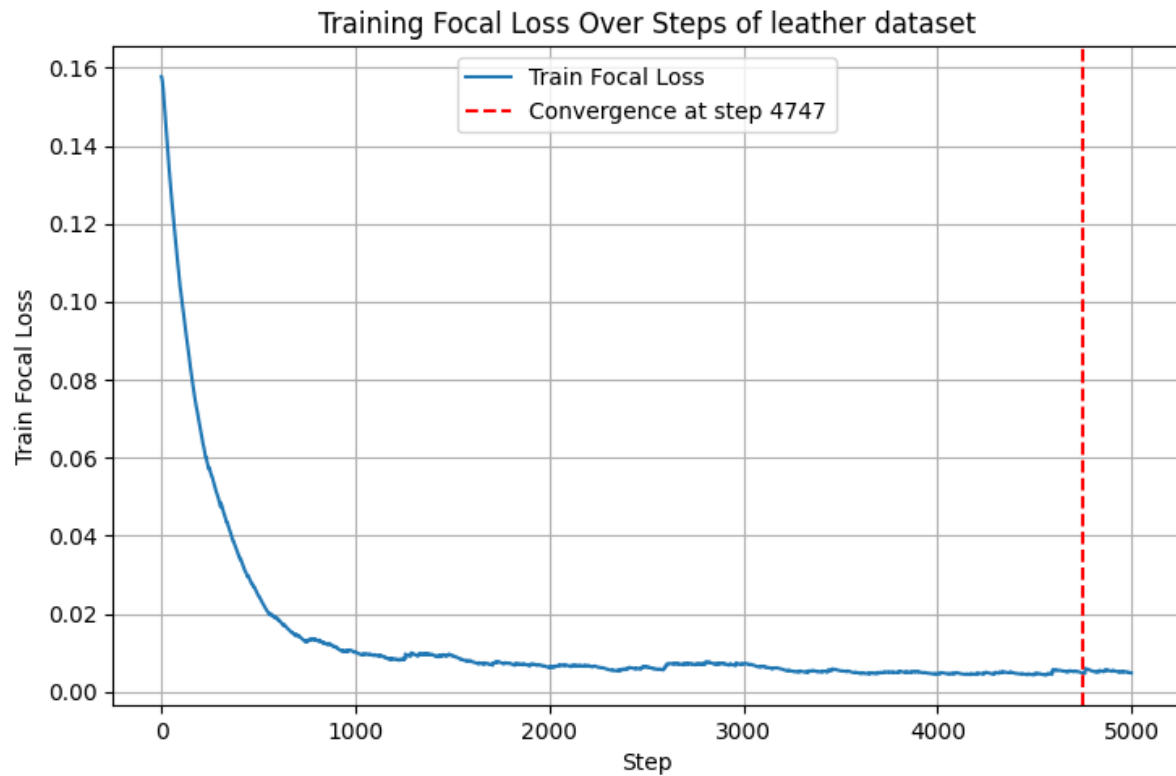
4. Leather



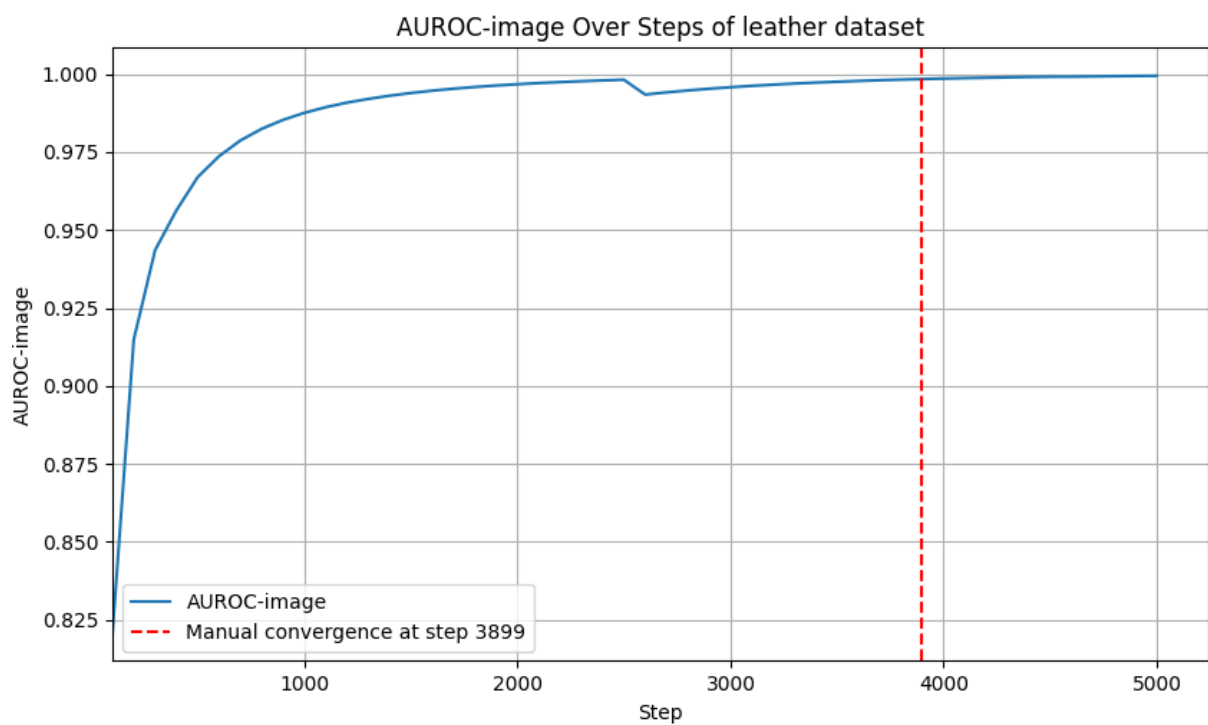
Hình 5.23: Biểu đồ hàm train loss của Leather



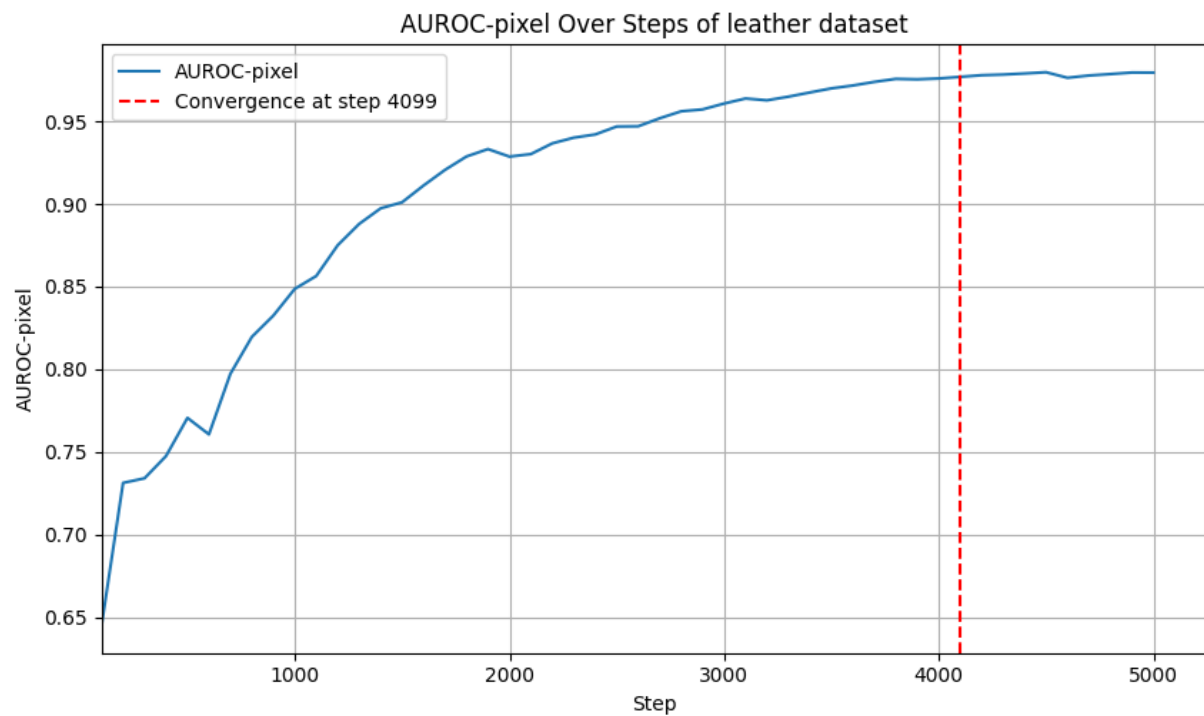
Hình 5.24: Biểu đồ hàm train L1 loss của tập Leather



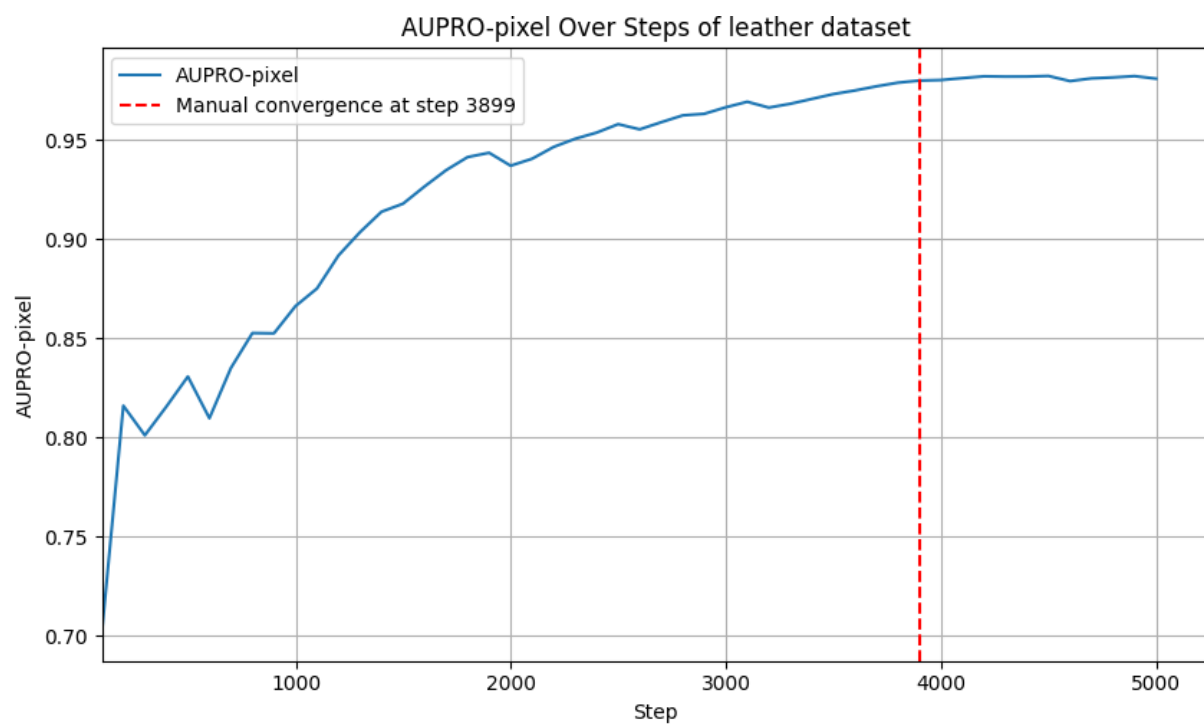
Hình 5.25: Biểu đồ hàm train focal loss của Leather



Hình 5.26: Biểu đồ AUROC-image của Leather

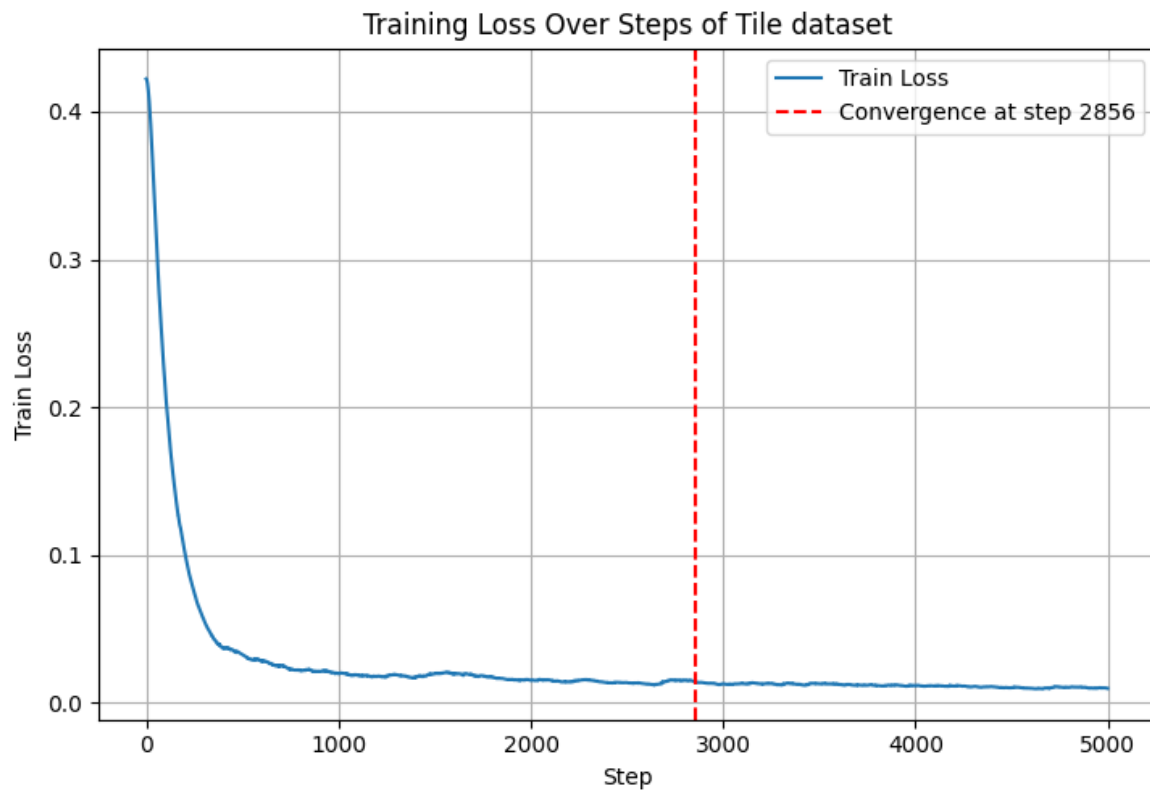


Hình 5.27: Biểu đồ AUROC-pixel của tập Leather

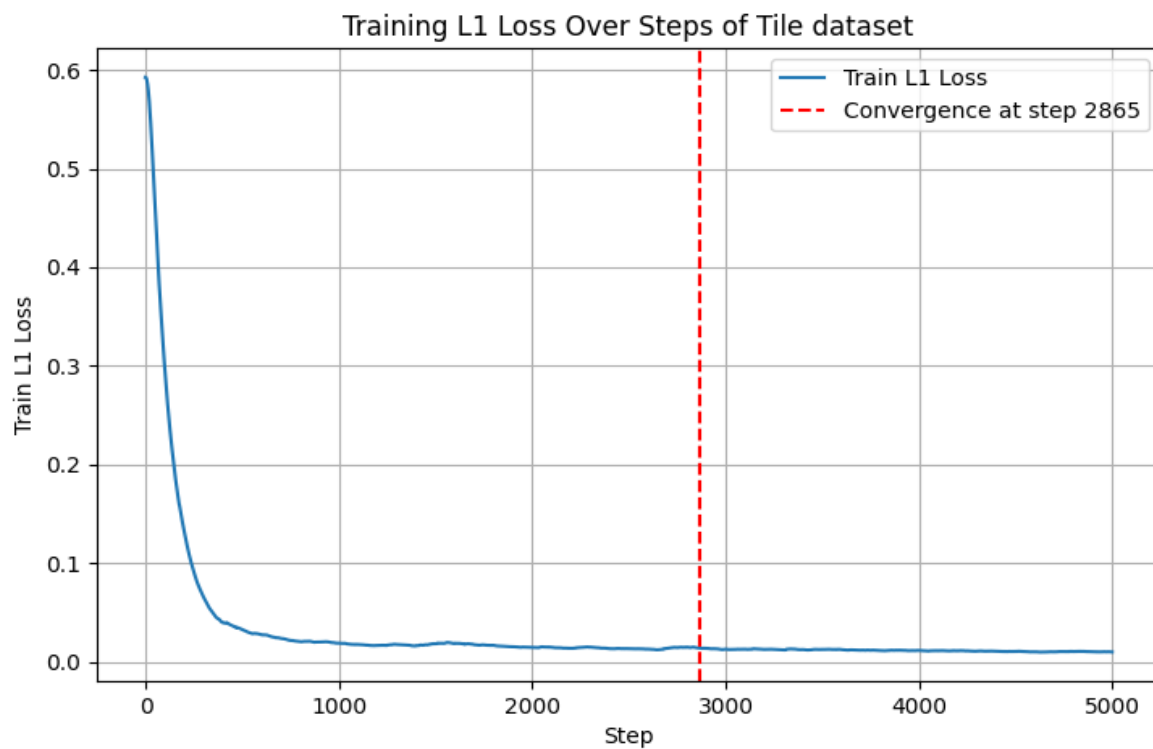


Hình 5.28: Biểu đồ AUPRO-pixel của tập Leather

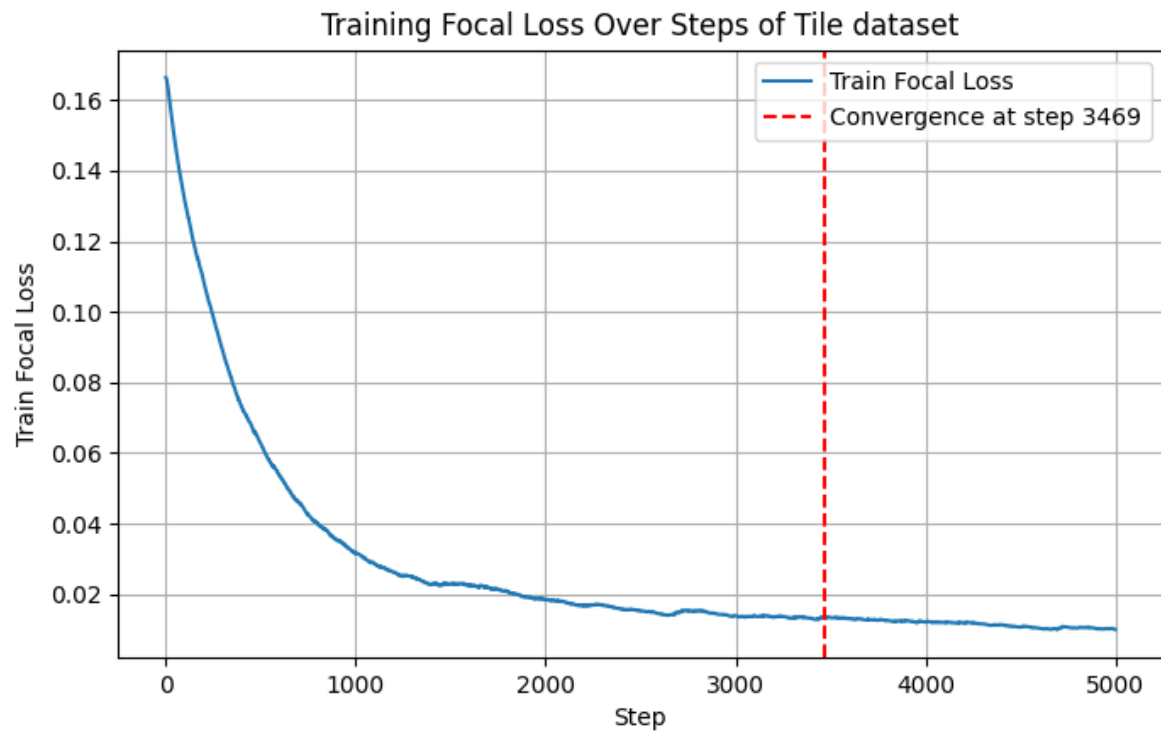
5. Tile



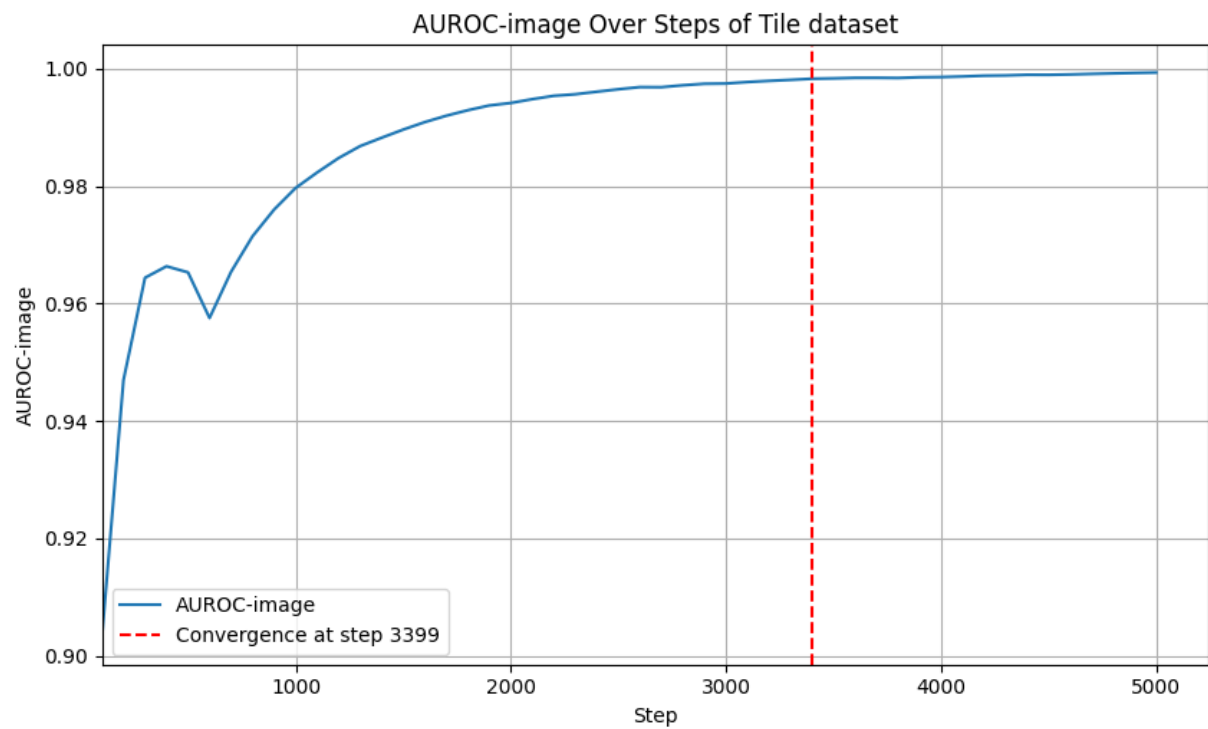
Hình 5.29: Biểu đồ hàm train loss của Tile



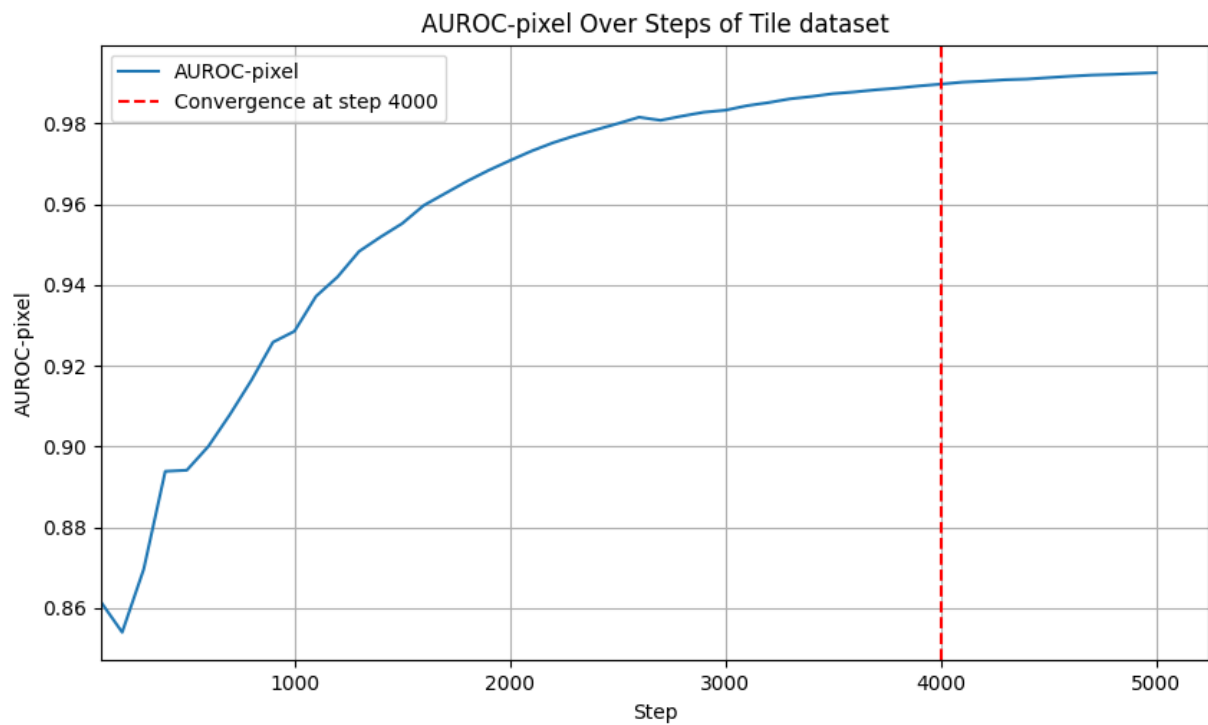
Hình 5.30: Biểu đồ hàm train L1 loss của Tile



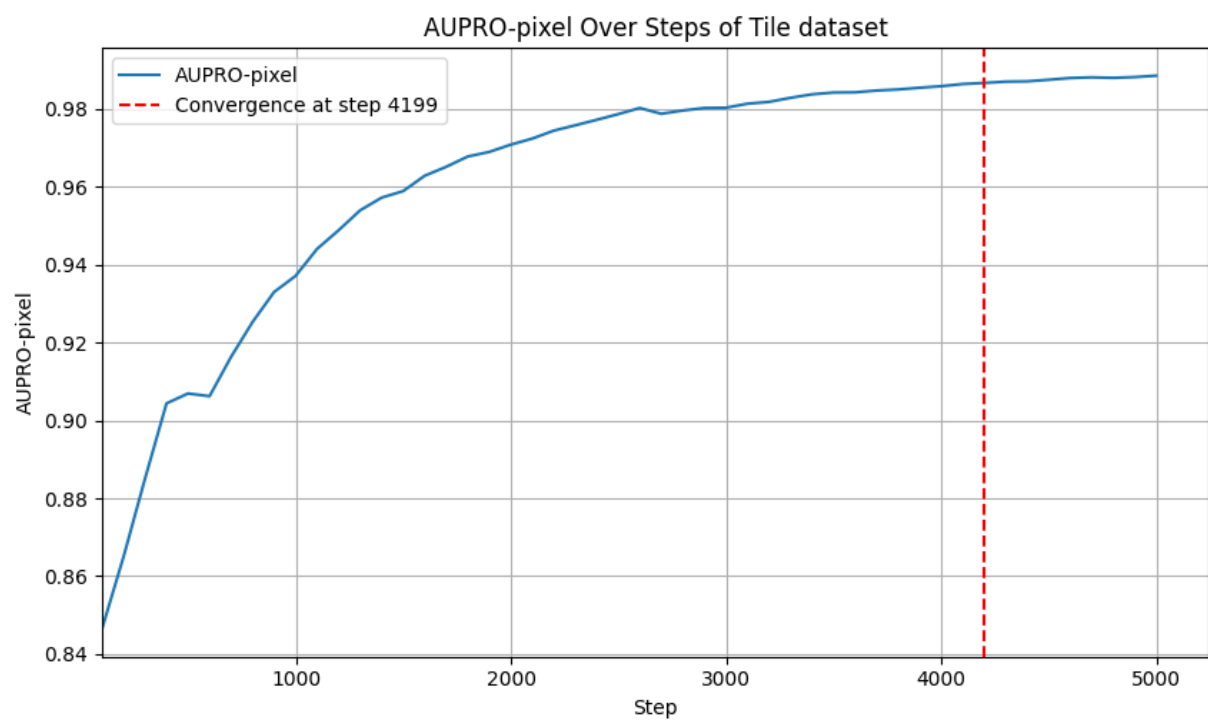
Hình 5.31: Biểu đồ hàm train focal loss của Tile



Hình 5.32: Biểu đồ AUROC-image của Tile

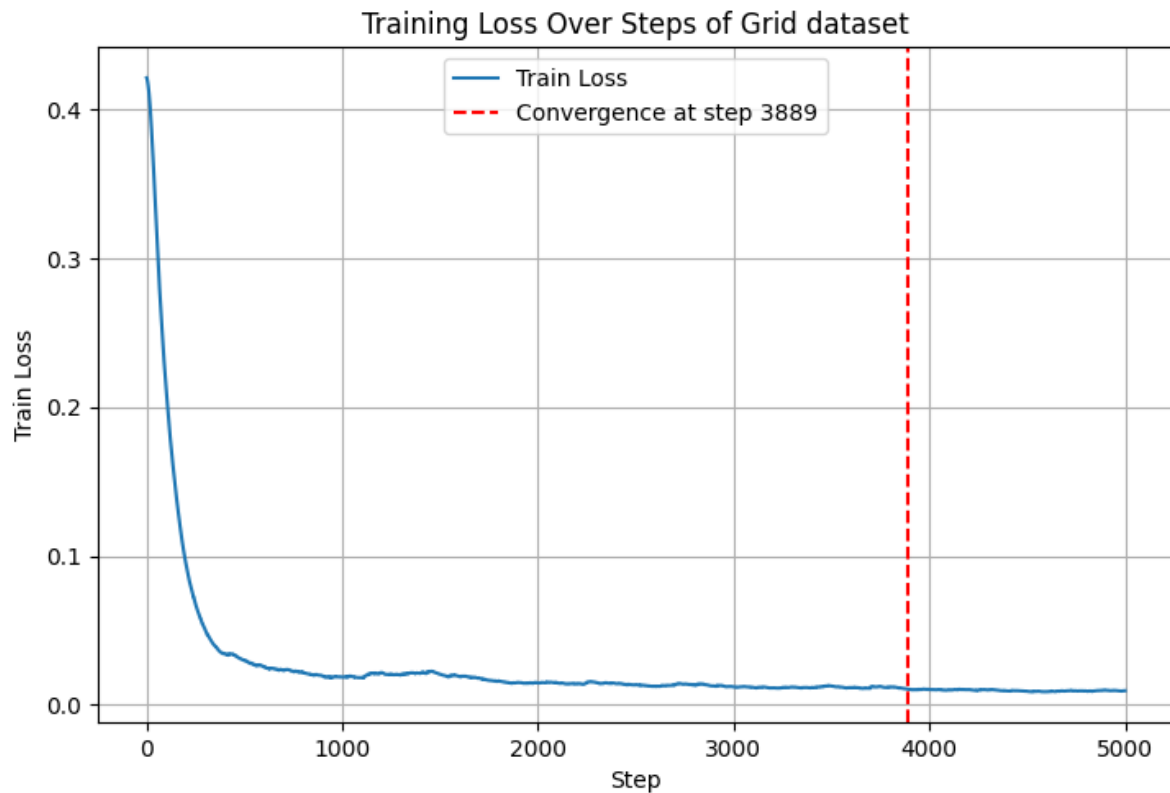


Hình 5.33: Biểu đồ AUROC-pixel của Tile

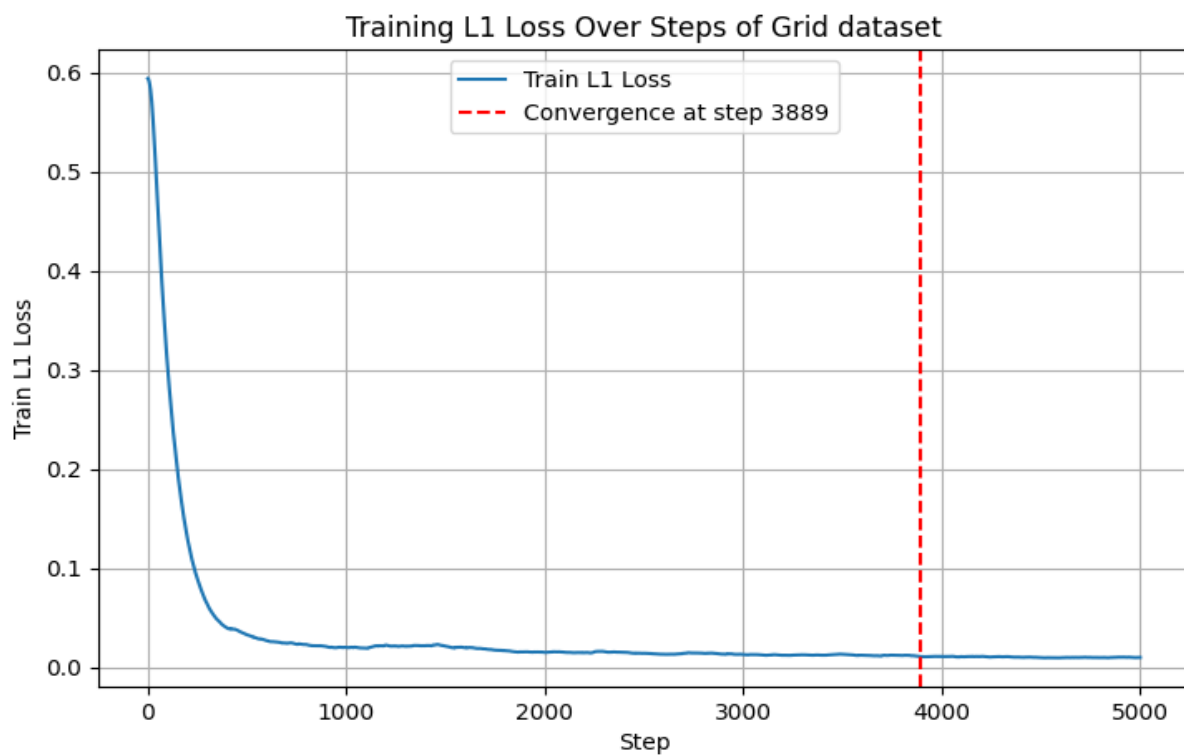


Hình 5.34: Biểu đồ AUPRO-pixel của Tile

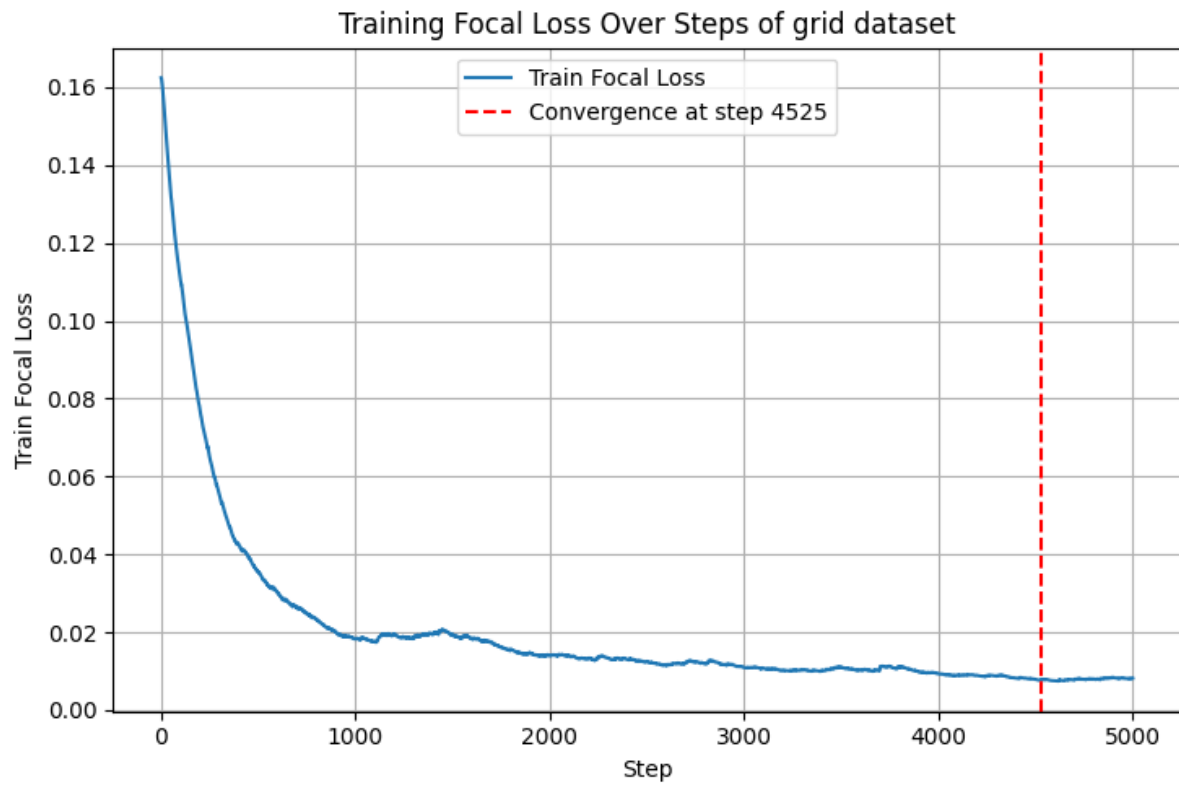
6. Grid



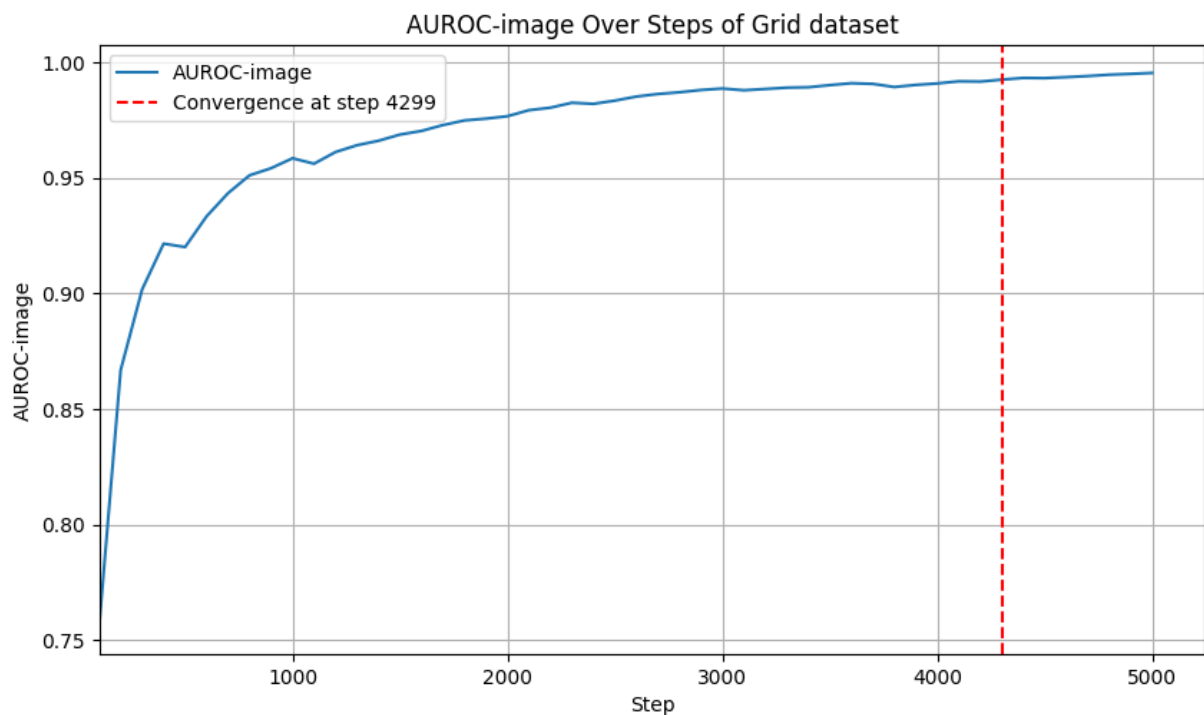
Hình 5.35: Biểu đồ hàm train loss của Grid



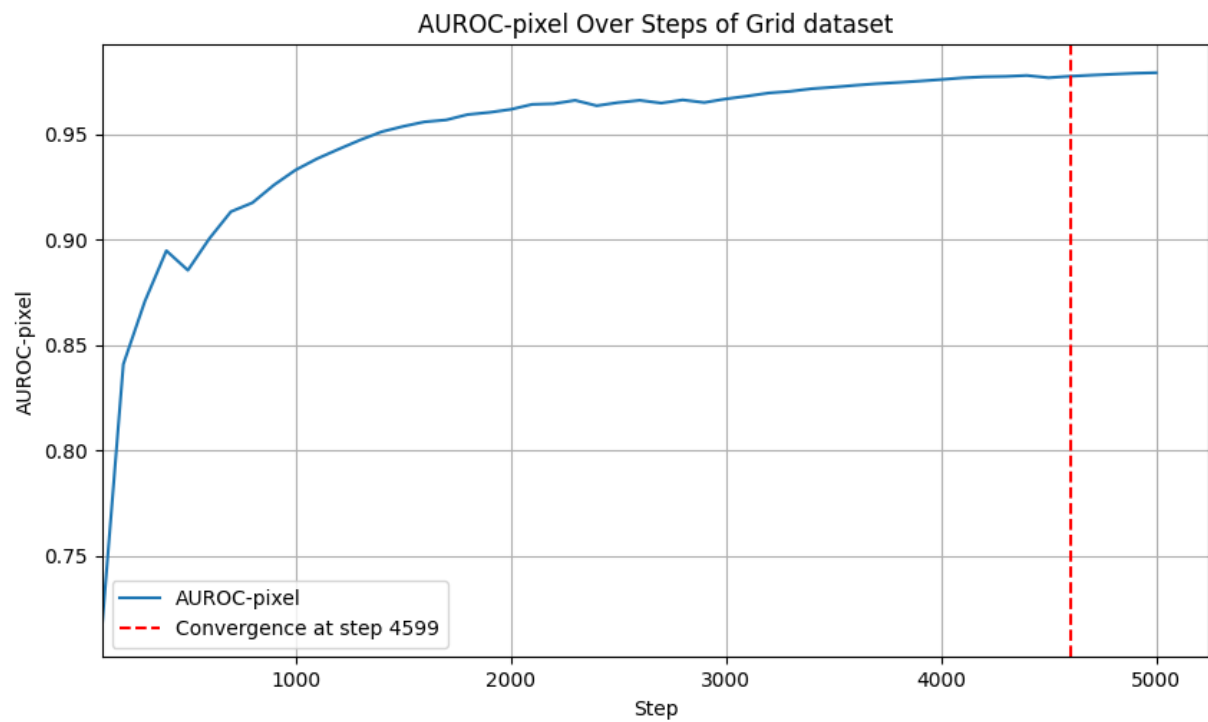
Hình 5.36: Biểu đồ hàm train L1 loss của Grid



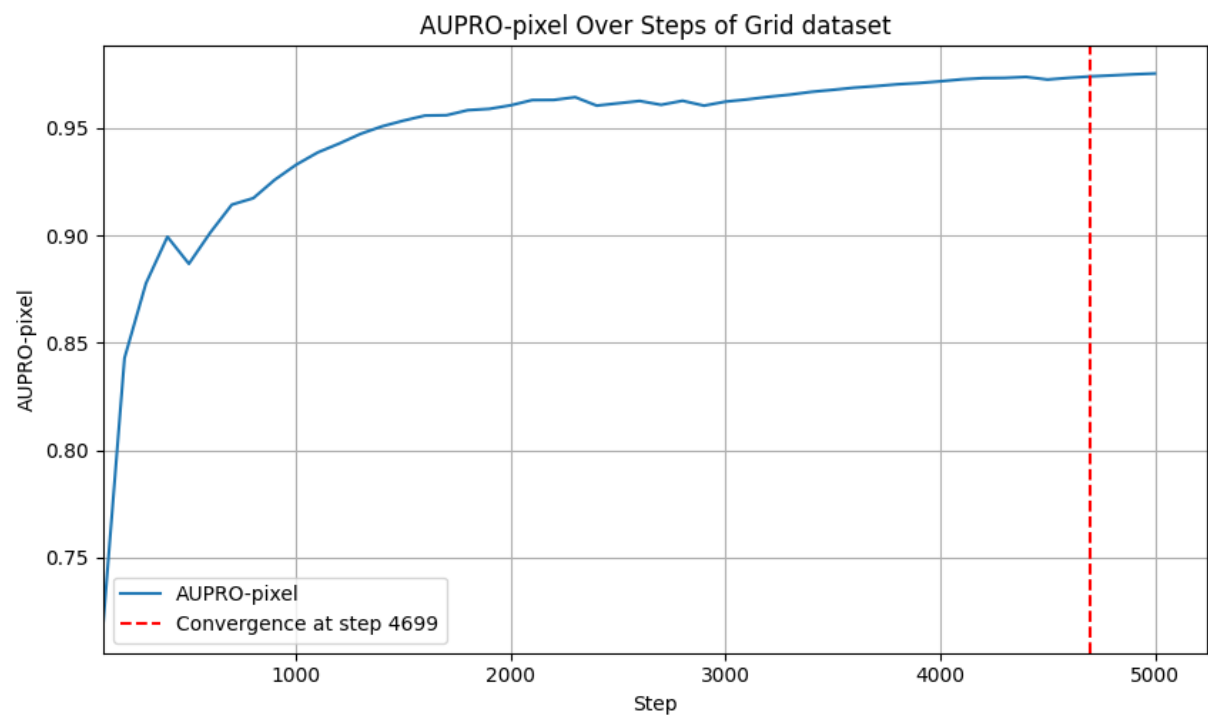
Hình 5.37: Biểu đồ hàm train focal loss của Grid



Hình 5.38: Biểu đồ AUROC-image của Grid

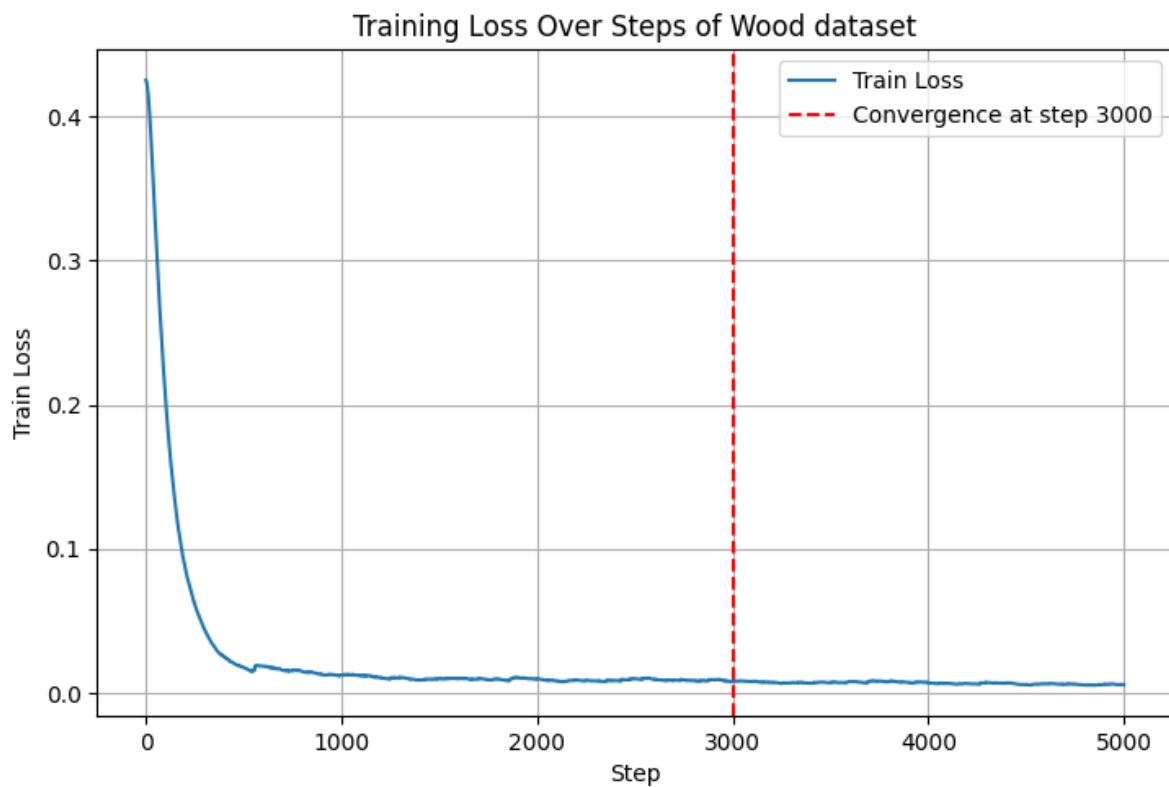


Hình 5.39: Biểu đồ AUROC-pixel của Grid

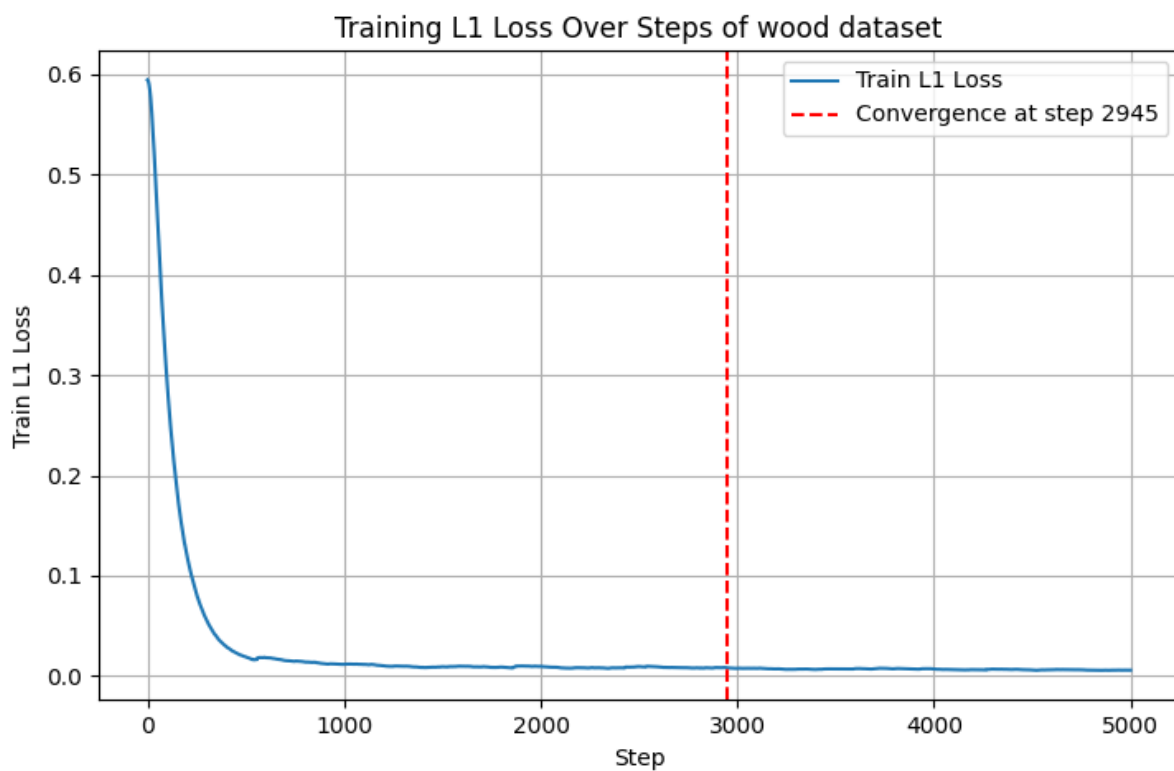


Hình 5.40: Biểu đồ AUPRO-pixel của Grid

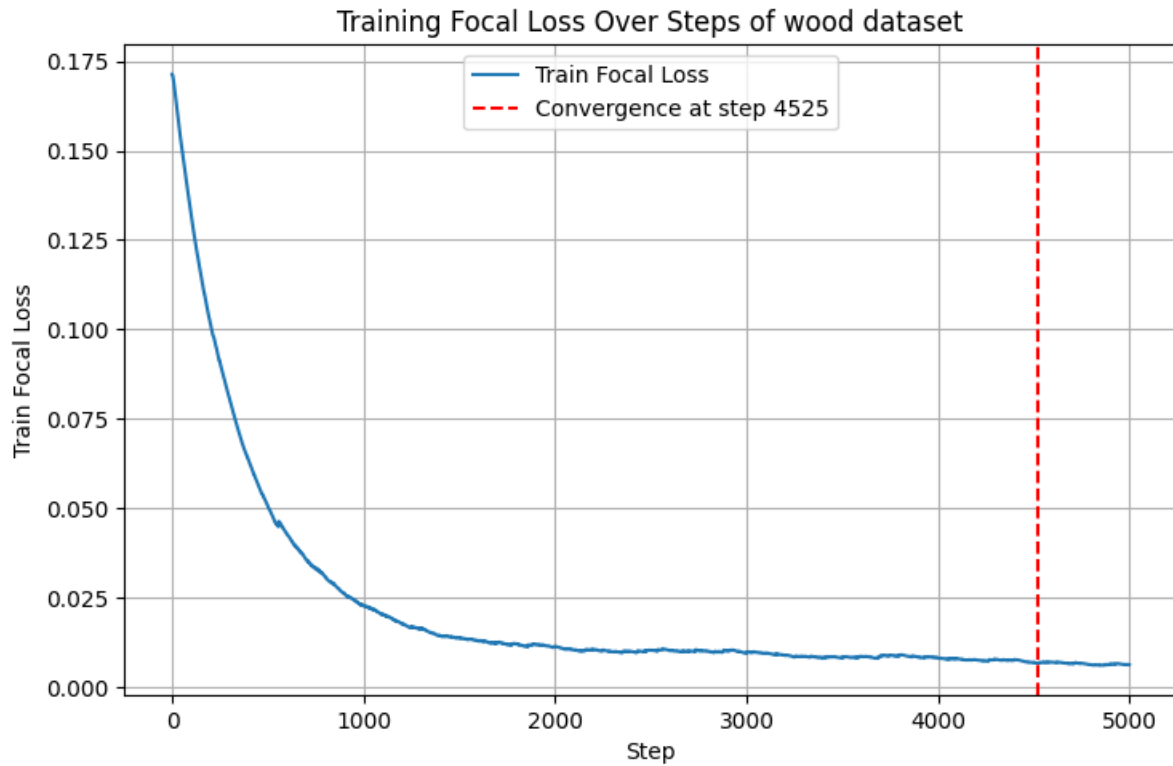
7. Wood



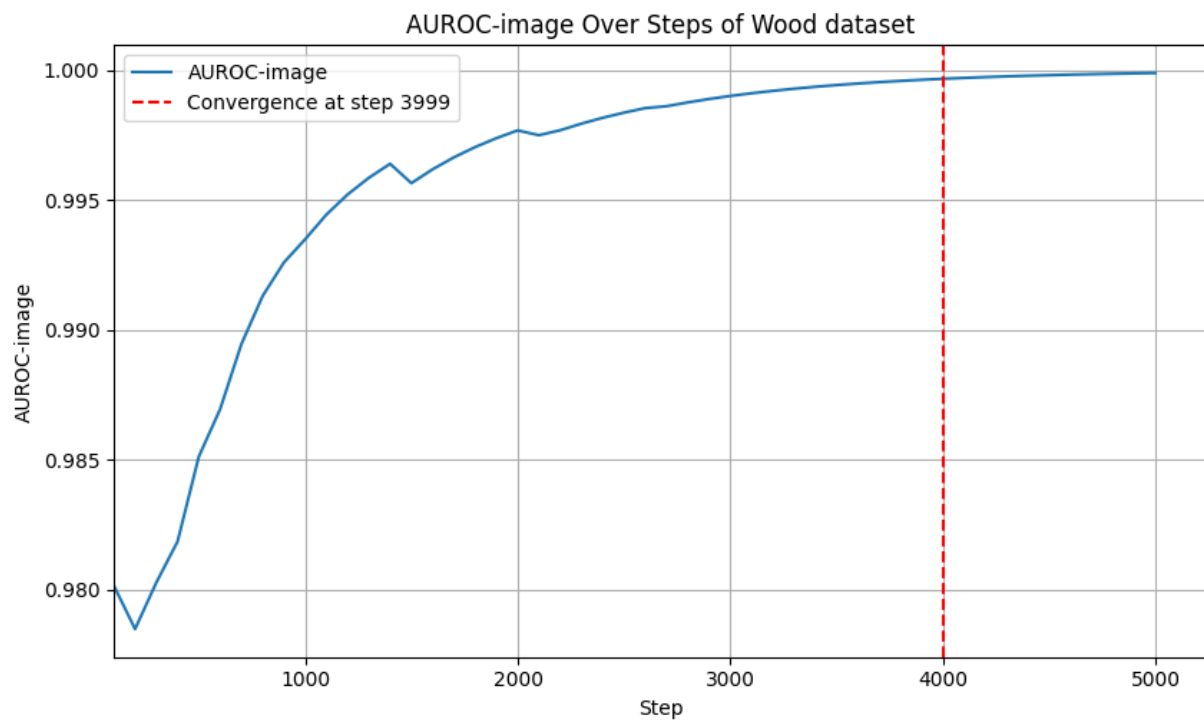
Hình 5.41: Biểu đồ hàm train loss của Wood



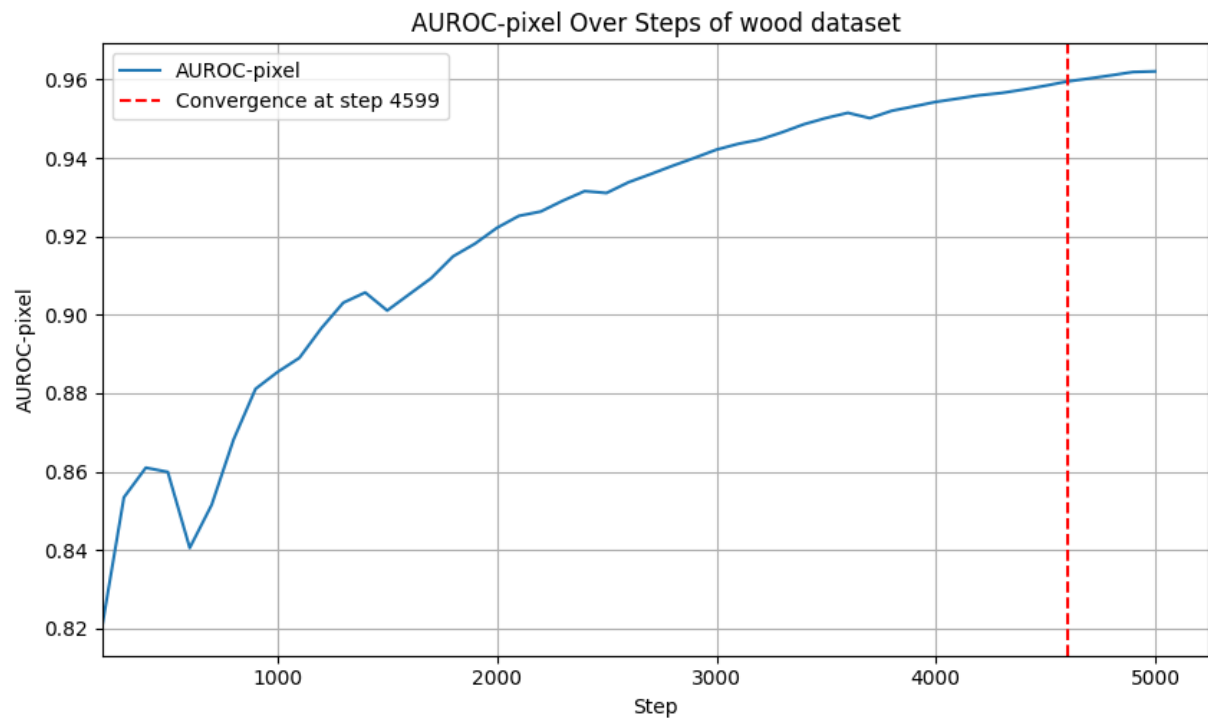
Hình 5.42: Biểu đồ hàm train L1 loss của Wood



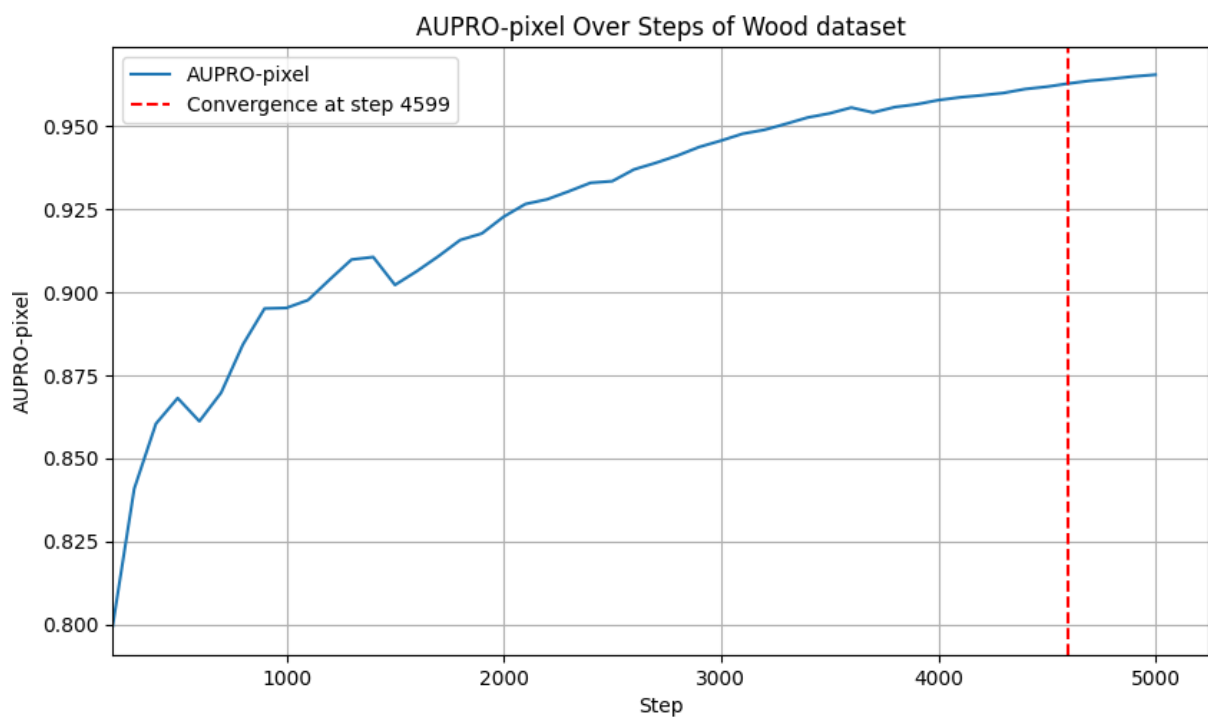
Hình 5.43: Biểu đồ hàm train focal loss của Wood



Hình 5.44: Biểu đồ AUROC-image của Wood

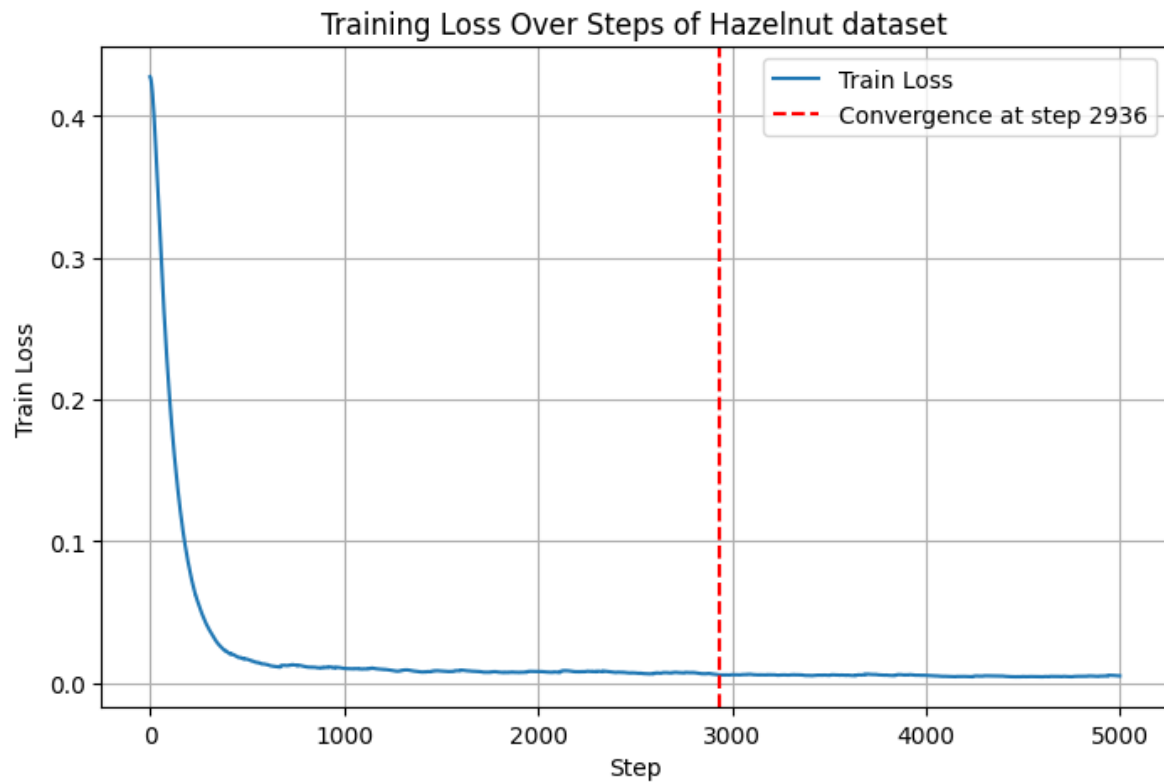


Hình 5.45: Biểu đồ AUROC-pixel của Wood

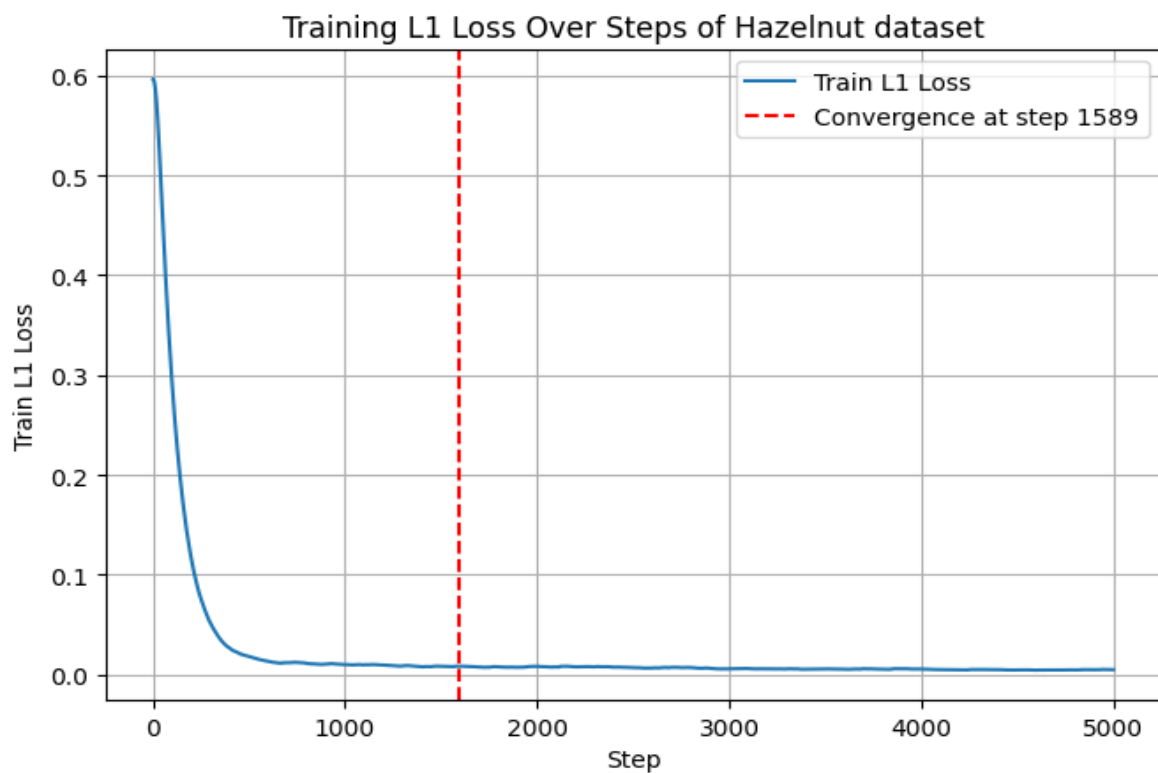


Hình 5.46: Biểu đồ AUPRO-pixel của Wood

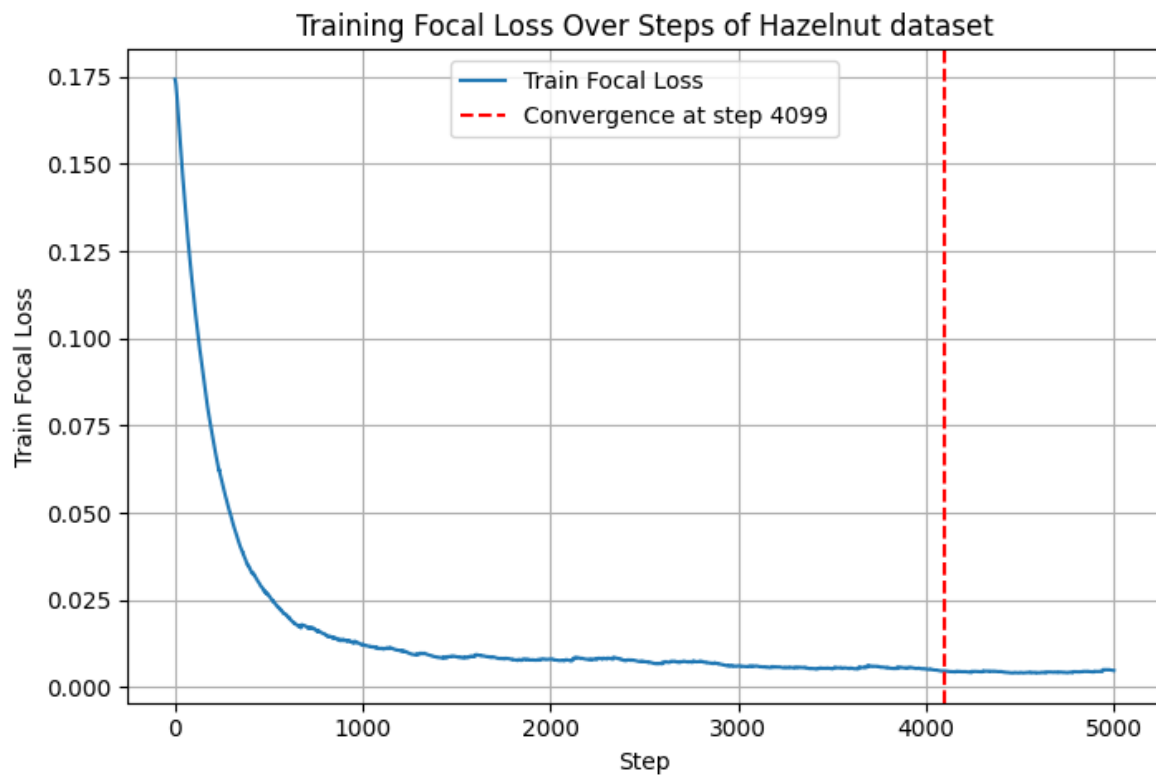
8. Hạt phỉ (hazelnut)



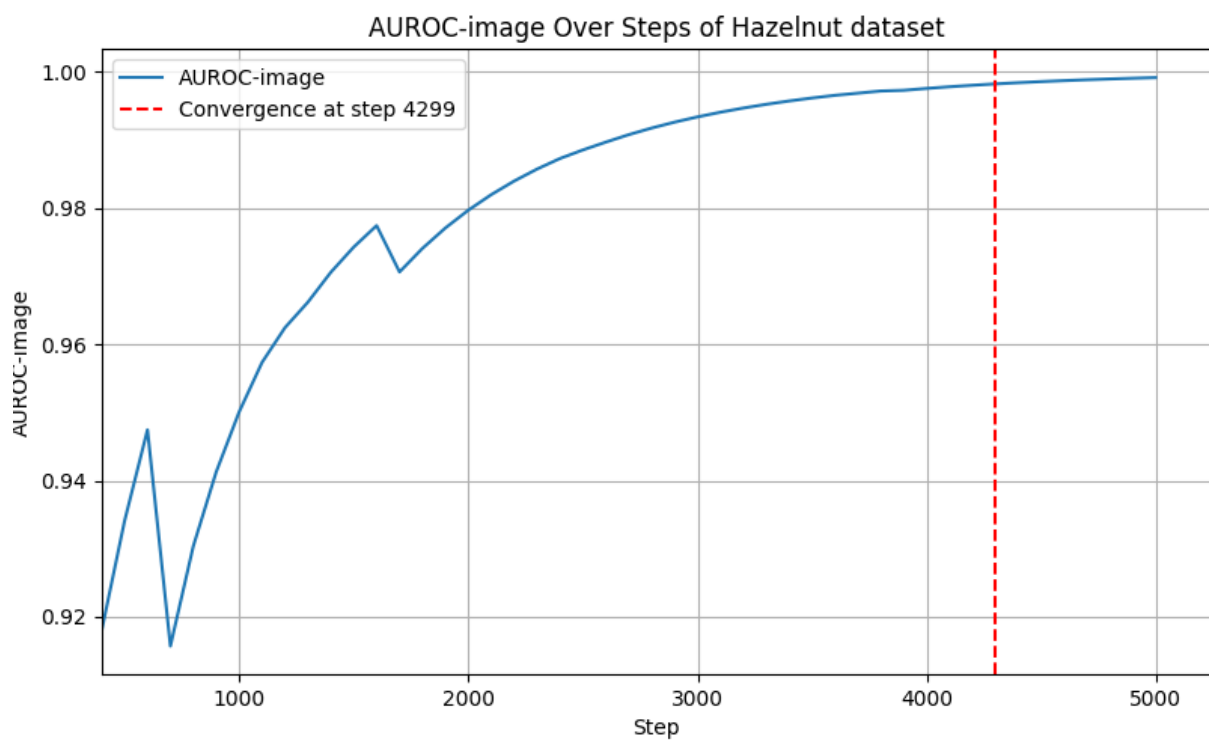
Hình 5.47: Biểu đồ hàm train loss của hazelnut



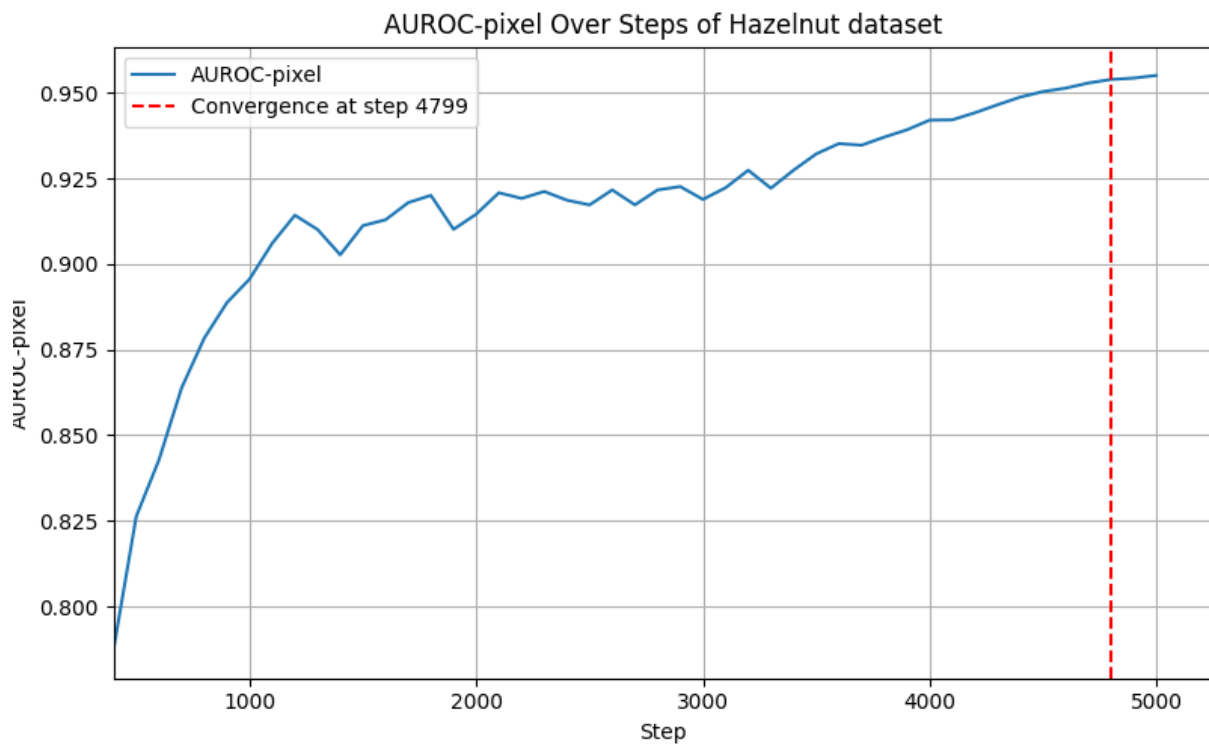
Hình 5.48: Biểu đồ hàm train L1 loss của hazelnut



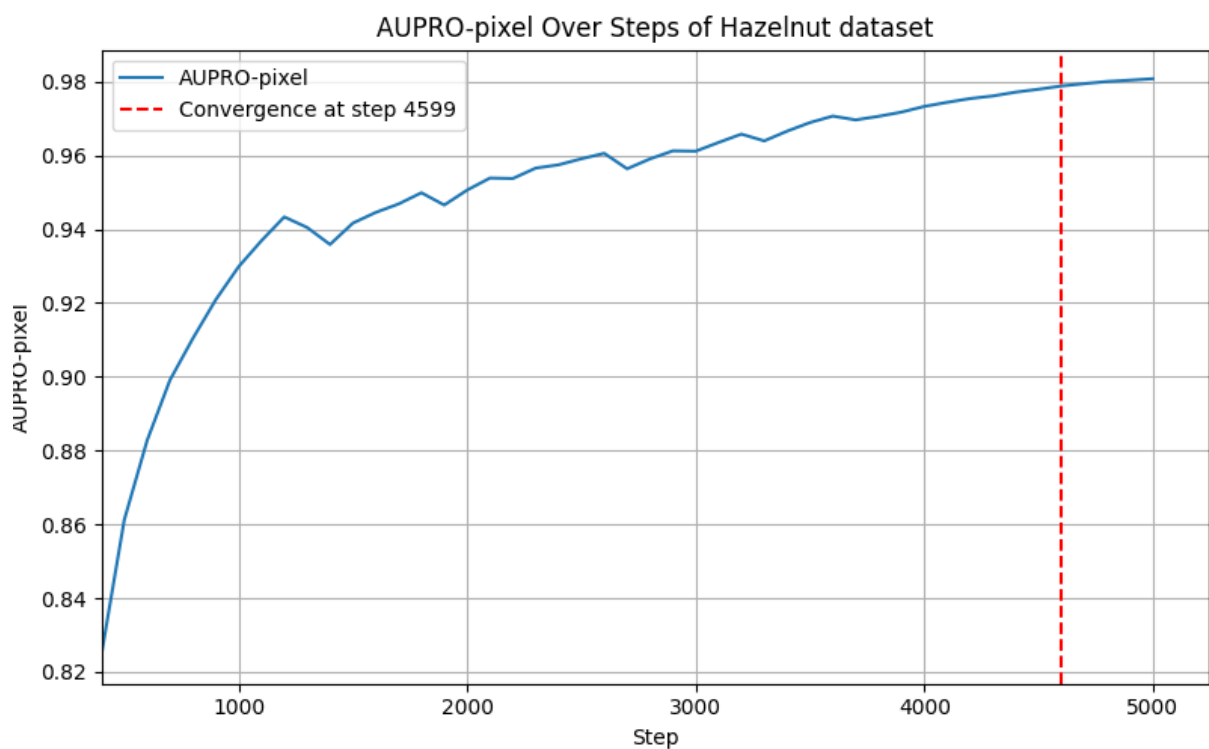
Hình 5.49: Biểu đồ hàm train focal loss của hazelnut



Hình 5.50: Biểu đồ AUROC-image của hazelnut

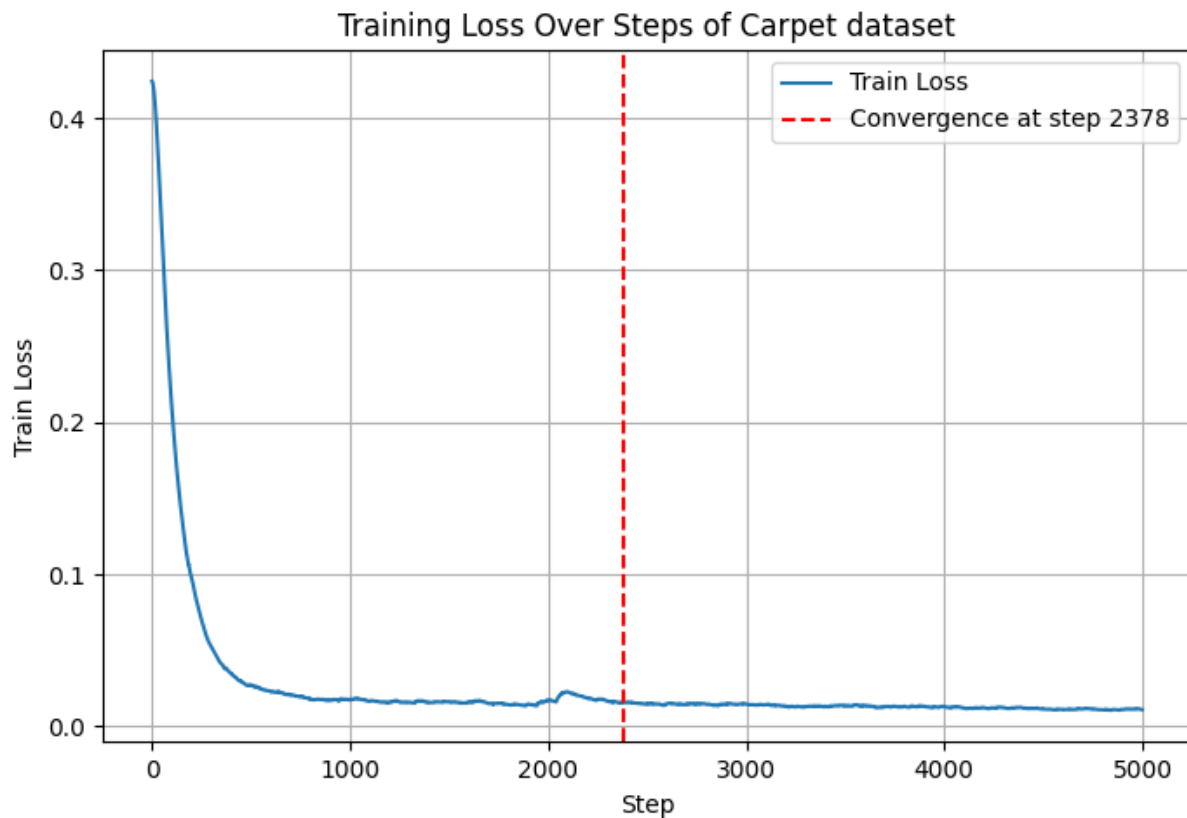


Hình 5.51: Biểu đồ AUROC-pixel của hazelnut

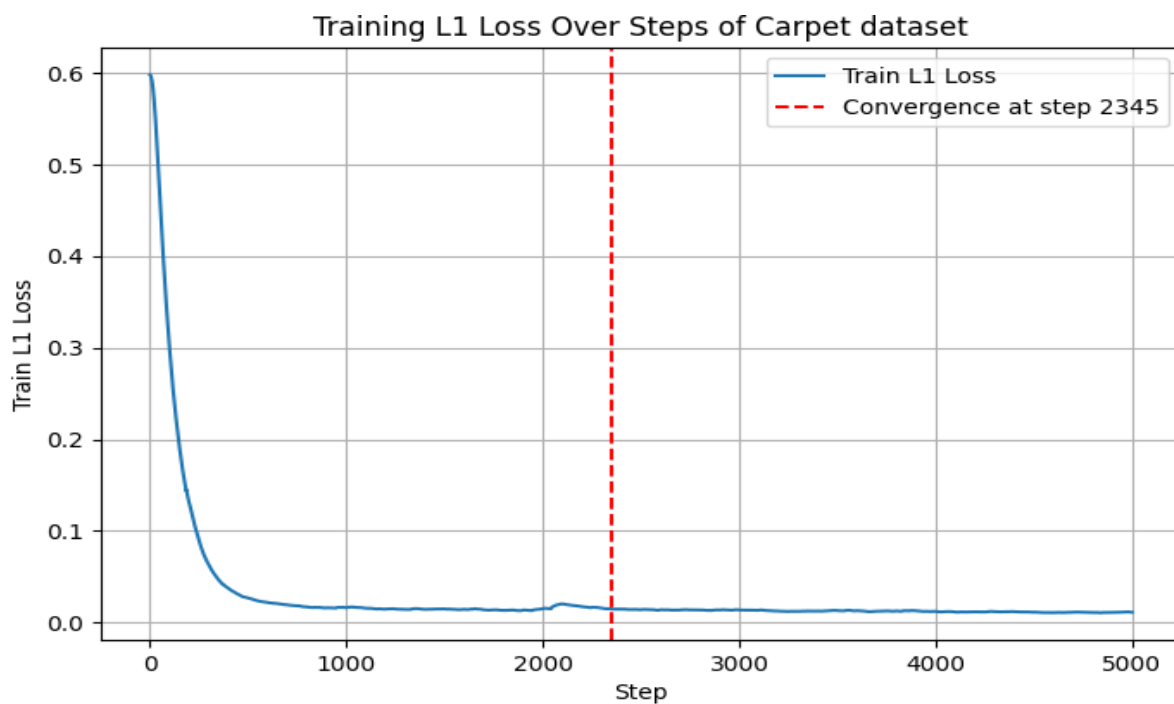


Hình 5.52: Biểu đồ AUPRO-pixel của tập hazelnut

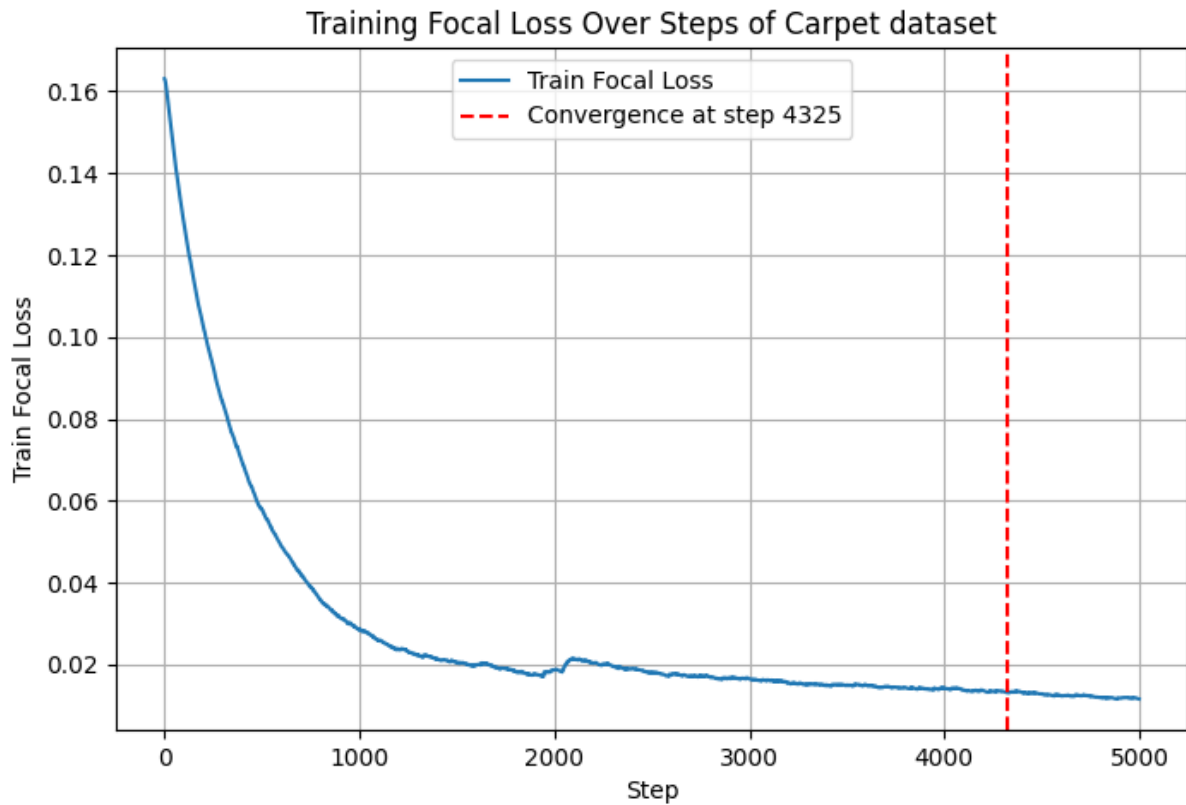
9. Thảm (carpet)



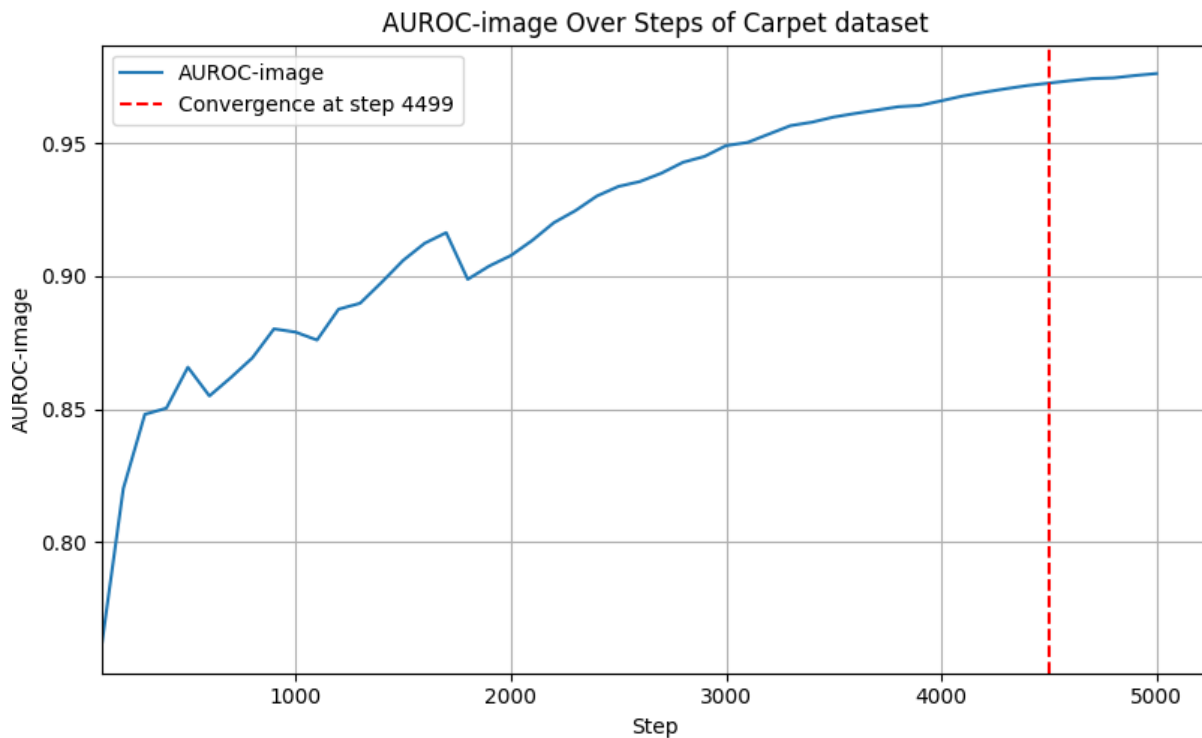
Hình 5.53: Biểu đồ hàm train loss của carpet



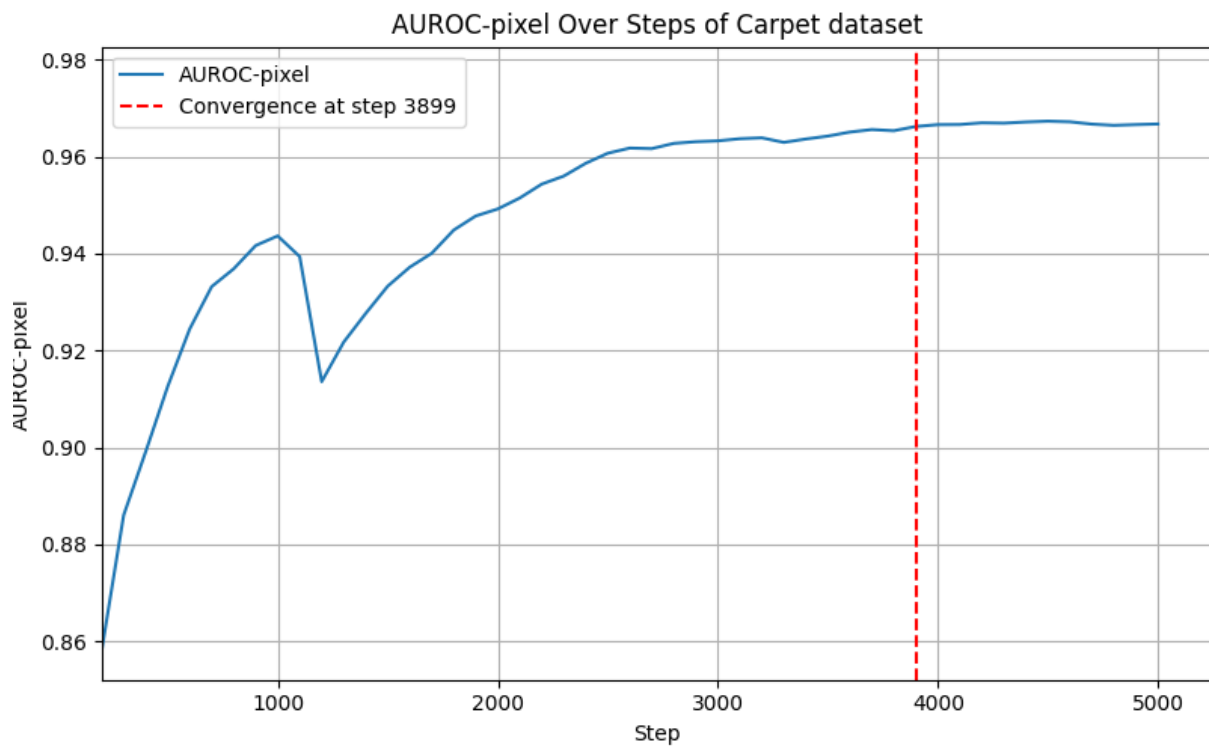
Hình 5.54: Biểu đồ hàm train L1 loss của carpet



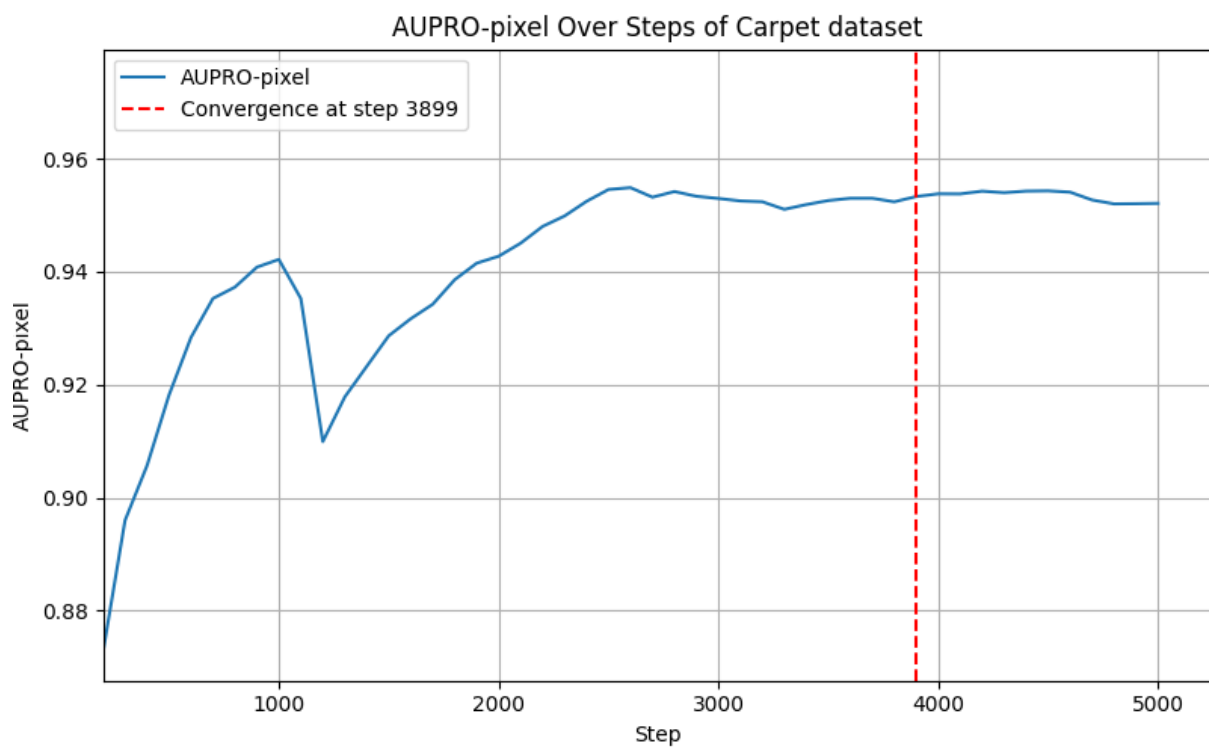
Hình 5.55: Biểu đồ hàm train focal loss của carpet



Hình 5.56: Biểu đồ AUROC-image của carpet

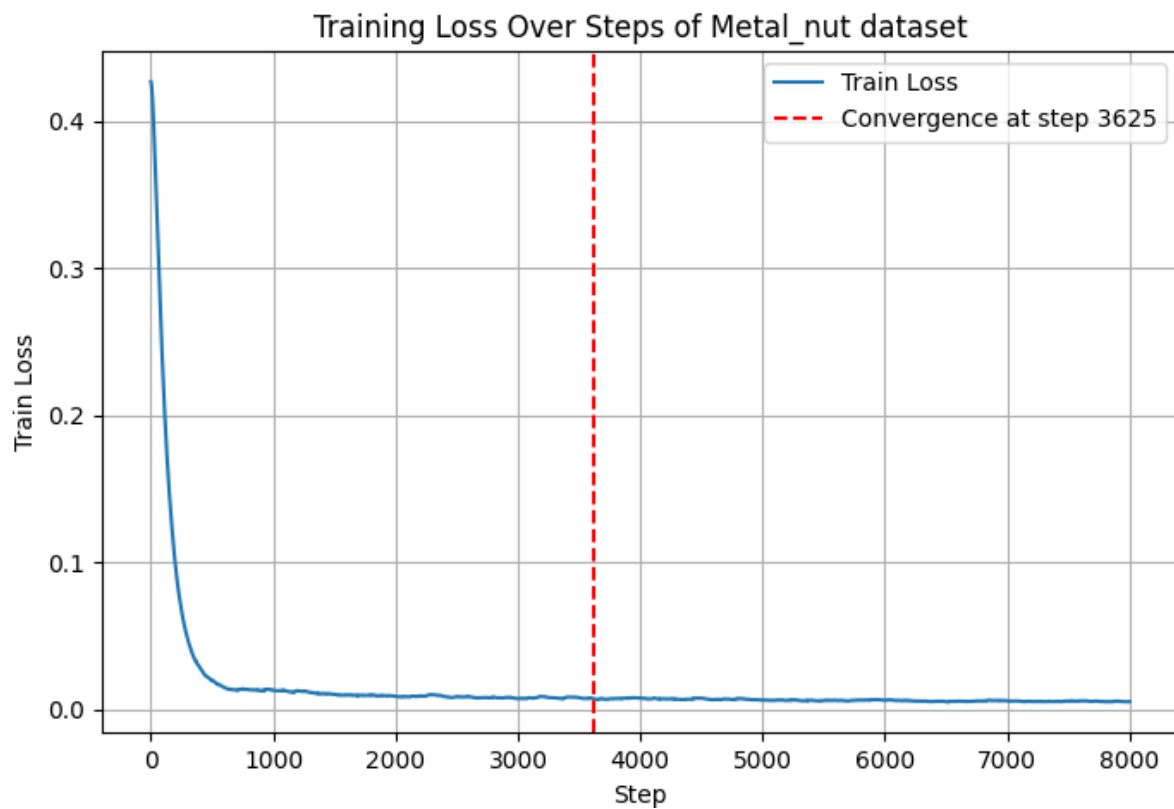


Hình 5.57: Biểu đồ AUROC-pixel của carpet

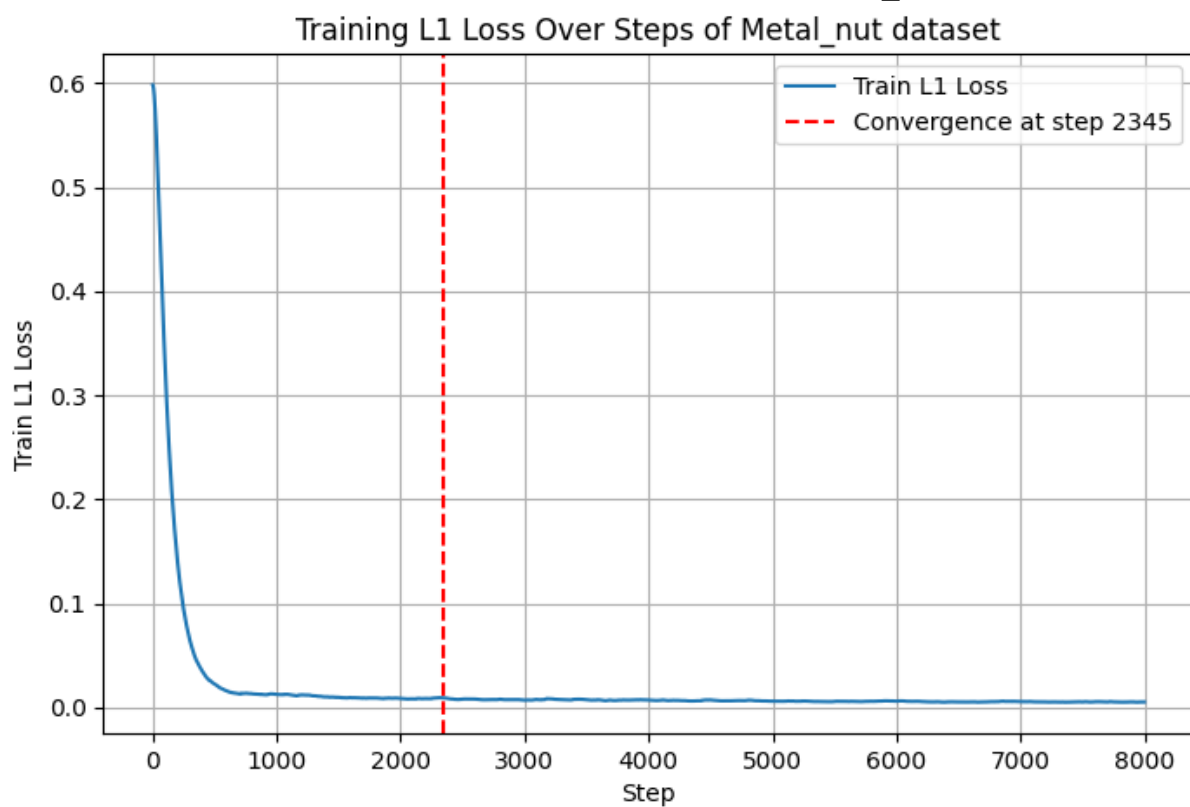


Hình 5.58: Biểu đồ AUPRO-pixel của carpet

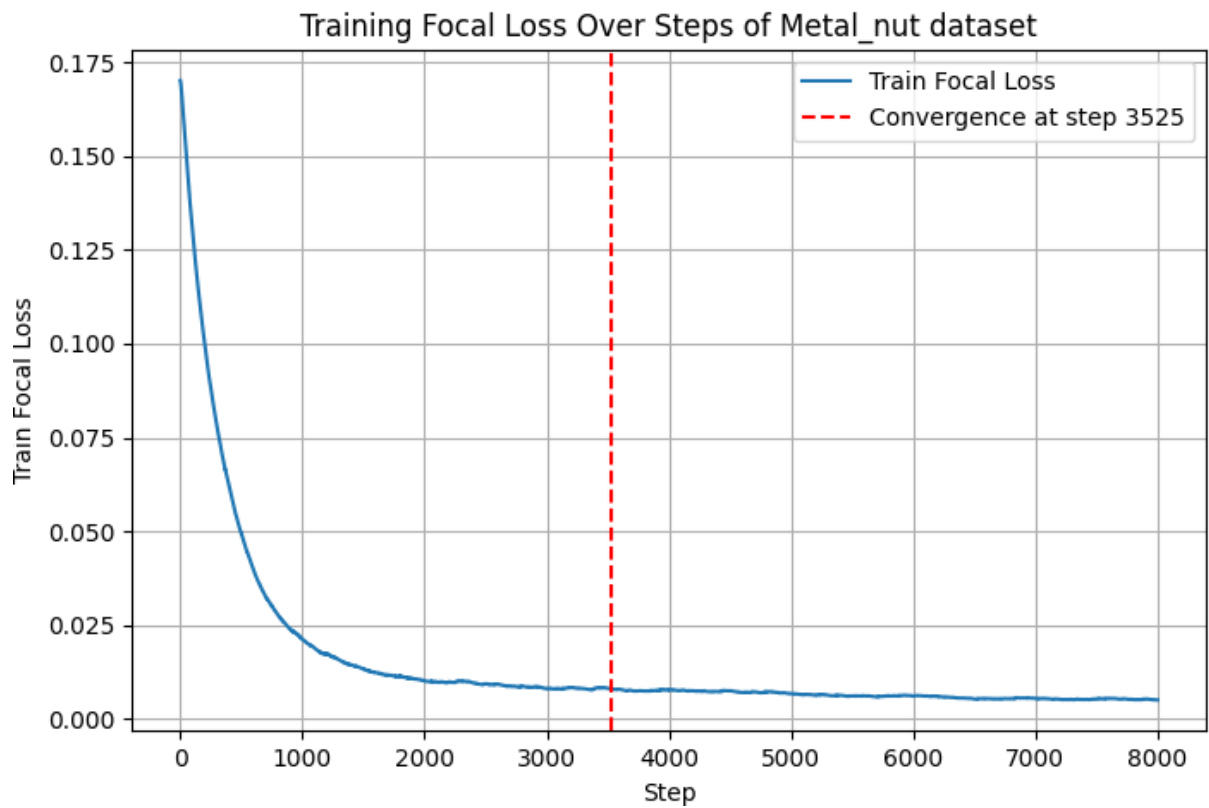
10. Metal_nut



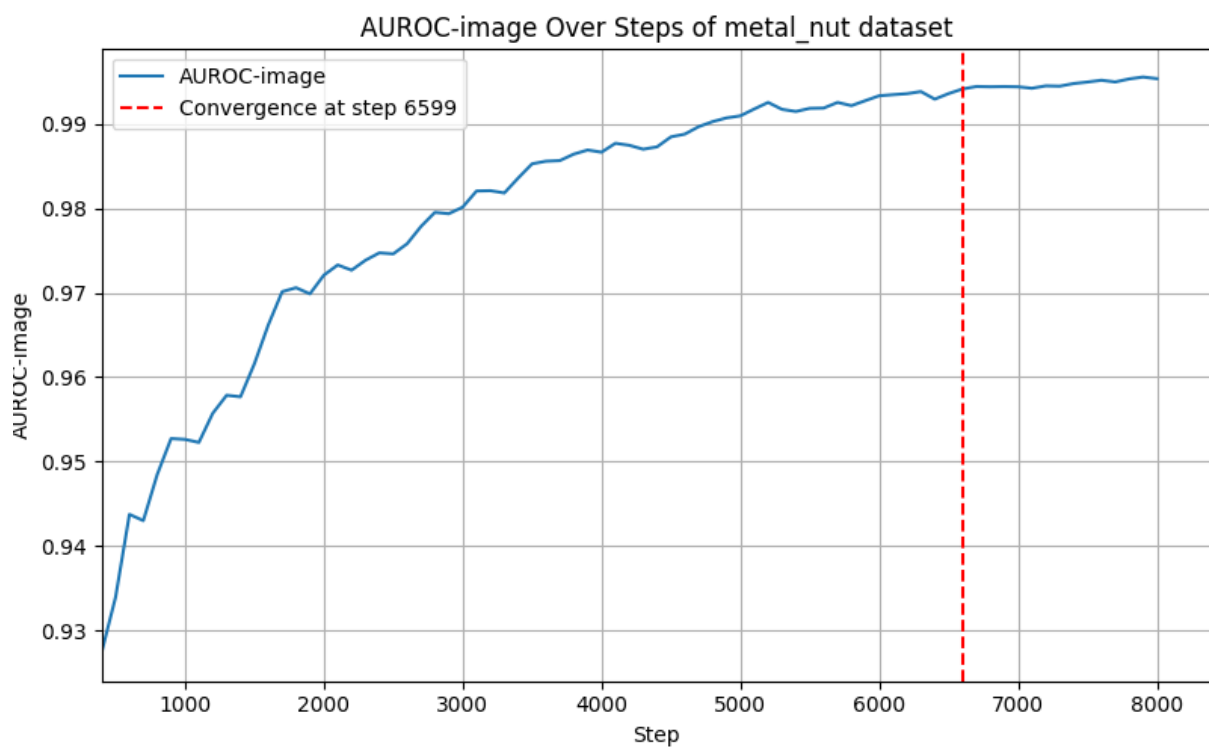
Hình 5.59: Biểu đồ hàm train loss của metal_nut



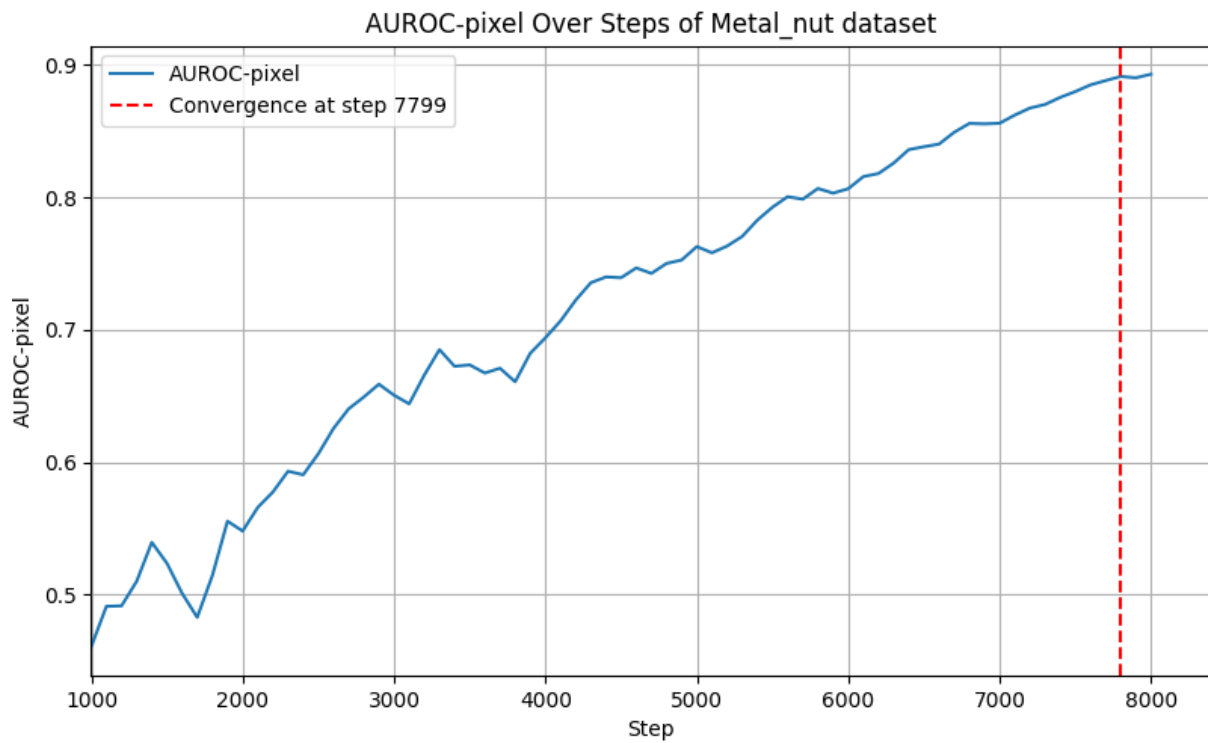
Hình 5.60: Biểu đồ hàm train L1 loss của metal_nut



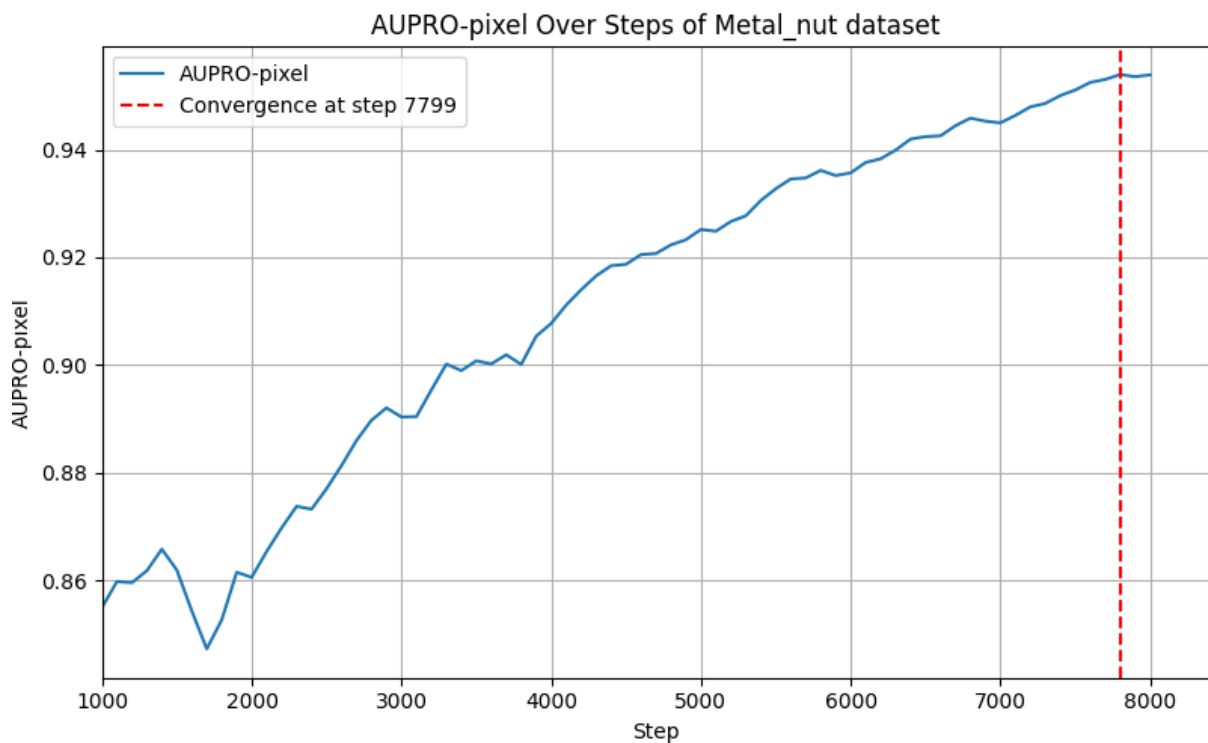
Hình 5.61: Biểu đồ train focal loss của metal_nut



Hình 5.62: Biểu đồ AUROC-image của metal_nut



Hình 5.63: Biểu đồ AUROC-image của metal_nut



Hình 5.64: Biểu đồ AUPRO-pixel của metal_nut

Sau khi phân tích các biểu đồ huấn luyện của 10 danh mục thuộc tập dữ liệu MVTEC AD, có thể nhận thấy rằng mô hình MemSeg nhìn chung thể hiện khả năng hội tụ tốt và ổn định trên đa dạng loại đối tượng và kết cấu bề mặt. Đối với các danh mục có hoa văn lặp lại hoặc bề mặt kết cấu đồng nhất như tile, grid, leather, wood, mô hình dễ dàng học được đặc trưng bình thường và nhanh chóng phát hiện dị thường, thể hiện rõ qua train loss và L1 loss giảm nhanh chỉ sau vài trăm bước huấn luyện. Quá trình huấn luyện diễn ra mượt mà, ít dao động, cho thấy mô hình nhanh chóng học được cách phát hiện được các khuyết tật bề mặt. Trong khi đó, các danh mục có khuyết tật kích thước nhỏ hoặc độ tương phản thấp như pill, capsule, hazelnut lại cho thấy quá trình tối ưu mất nhiều thời gian hơn, với hiện tượng plateau kéo dài và dao động loss mạnh hơn ở giai đoạn đầu. Điều này phản ánh rằng mô hình gặp nhiều khó khăn hơn khi phải phân biệt các khuyết tật mờ và nhỏ. Sau khoảng 2000 bước huấn luyện, các đường loss dần ổn định và hội tụ, cho thấy rằng mô hình vẫn có khả năng thích nghi và phát hiện được các khuyết tật tinh vi nếu được huấn luyện đủ lâu và với cơ chế điều chỉnh tốc độ học hợp lý.

Xét riêng về các chỉ số đánh giá, mô hình đạt được chỉ số AUROC ở cấp độ ảnh (AUROC-image) cao trên hầu hết các danh mục, thường vượt mức 98%, cho thấy khả năng phân biệt hiệu quả giữa ảnh bình thường và ảnh bất thường ở mức tổng thể. Chỉ số AUROC ở cấp độ pixel (AUROC-pixel) cũng đạt mức cao, phản ánh năng lực định vị chính xác các vùng dị thường tại từng điểm ảnh, đặc biệt rõ nét ở những danh mục có lỗi nổi bật về cấu trúc hoặc độ tương phản. Đáng chú ý, chỉ số AUPRO-pixel (Area Under the Per-Region Overlap) đạt giá trị rất cao trên nhiều danh mục, cho thấy rằng không chỉ nhận diện được vị trí vùng lỗi, mô hình còn định hình đúng biên dạng và cấu trúc hình học của vùng bất thường, bám sát ground truth một cách hiệu quả. Sự đồng bộ giữa các chỉ số đánh giá cho thấy tính ổn định toàn diện của mô hình trong các khía cạnh phát hiện và phân đoạn.

5.5. Quá trình phát hiện khuyết tật trên Jetson Nano

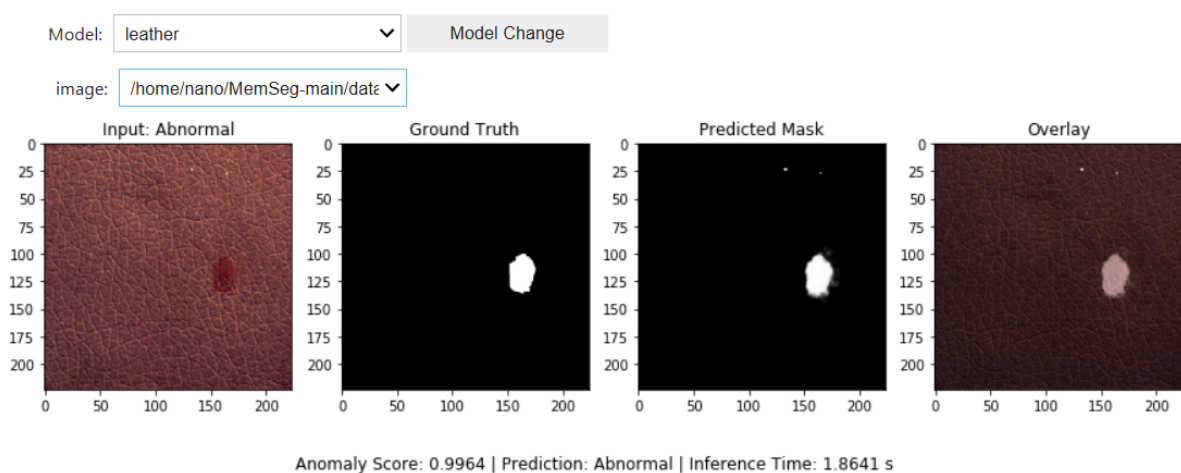
Sau khi thực hiện huấn luyện, ta sẽ có được tập trọng số của mô hình, từ đó ta sẽ bắt đầu thực hiện phát hiện khuyết tật dựa trên tập ảnh test. Quá trình này được thực hiện trên Kit Jetson nano, kết quả được theo dõi bằng trình duyệt web trên PC với giao diện người dùng như sau.

- **Lựa chọn mô hình (Model):** Người dùng có thể chọn danh mục đối tượng, trong ví dụ này là "leather".
- **Lựa chọn ảnh (image):** Cho phép chọn ảnh từ thư mục dữ liệu để thực hiện phát hiện khuyết tật bề mặt.

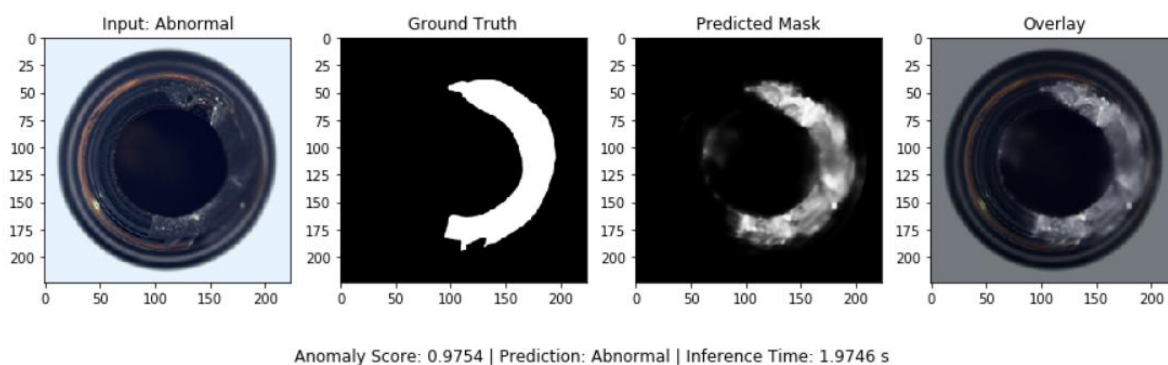
Sau khi chạy mô hình, hệ thống hiển thị 4 ảnh phản hồi chính:

- **Input:** Ảnh đầu vào có thể là bình thường hoặc bất thường
- **Ground Truth** - Mặt nạ bất thường thực tế, biểu thị vùng lỗi trên bề mặt.
- **Predicted Mask** - Mặt nạ bất thường được mô hình dự đoán
- **Overlay** - Ảnh chồng kết quả dự đoán lên ảnh gốc, giúp dễ dàng quan sát và đối chiếu vị trí bất thường.
- **Anomaly Score:** Điểm số bất thường
- **Prediction:** Abnormal - Dự đoán của mô hình
- **Inference time:** Thời gian phát hiện khuyết tật của mô hình.

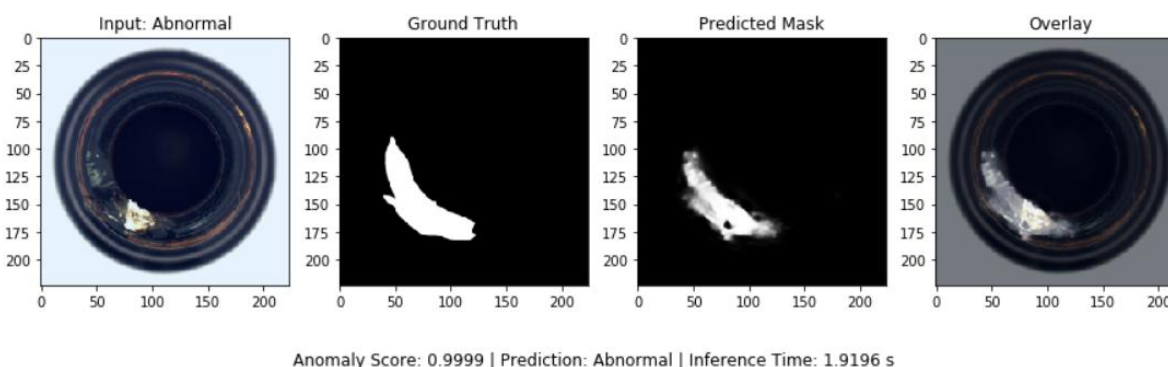
Model Select



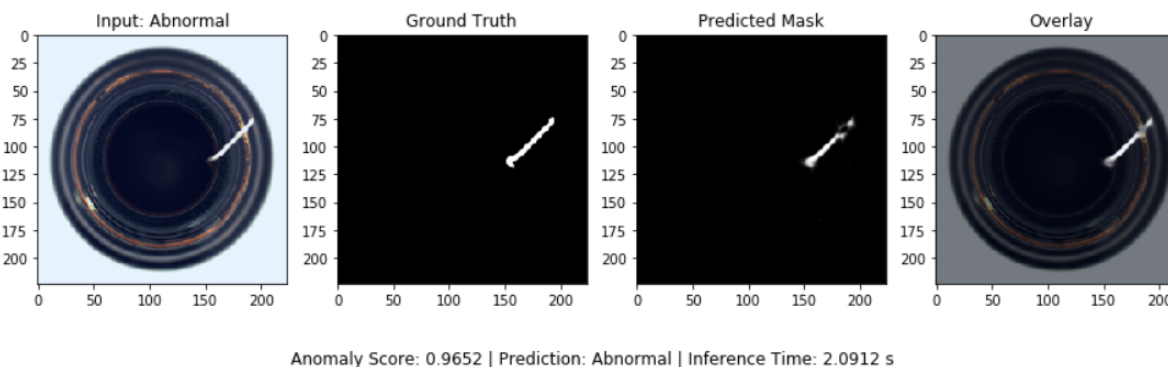
Hình 5.65: Giao diện kết quả phát hiện của hệ thống



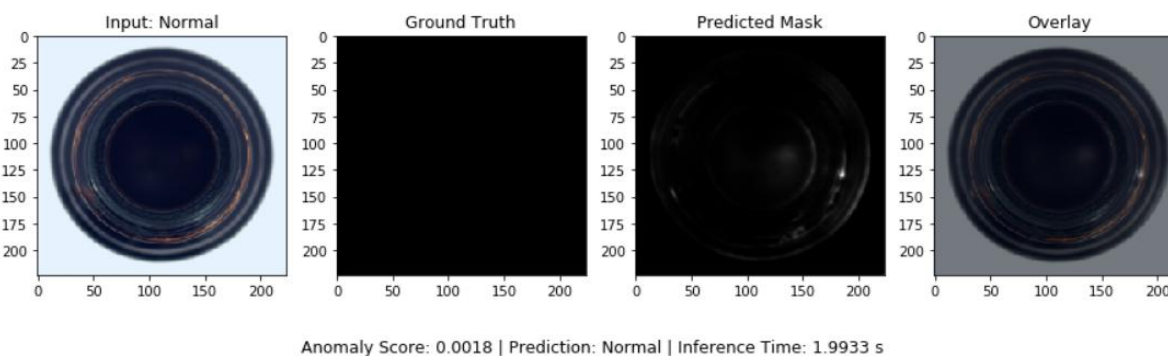
Hình 5.66: Phát hiện cho mẫu chai (bottle) bị vỡ lớn



Hình 5.67: Phát hiện cho mẫu chai (bottle) bị vỡ nhỏ

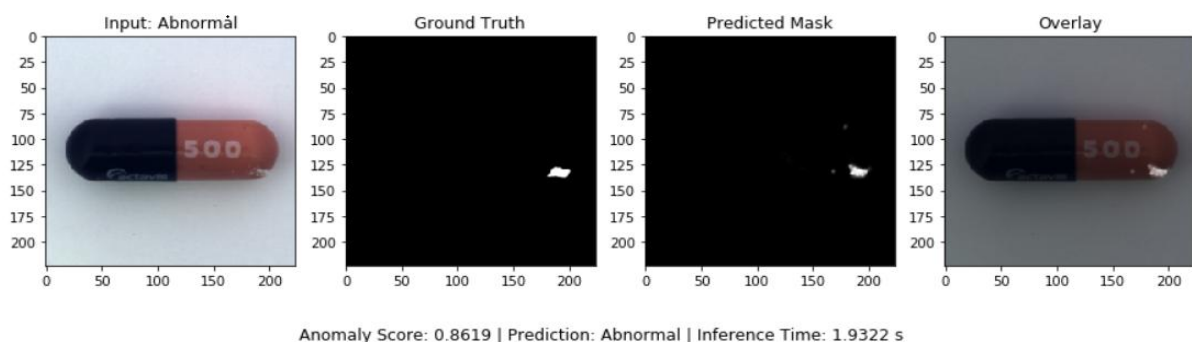


Hình 5.68: Phát hiện cho mẫu chai (bottle) bị nhiễm bẩn

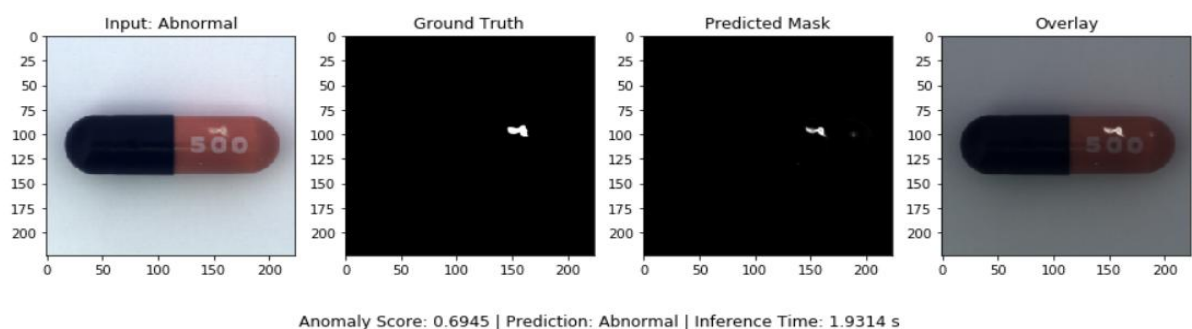


Hình 5.69: Phát hiện cho mẫu chai (bottle) bình thường

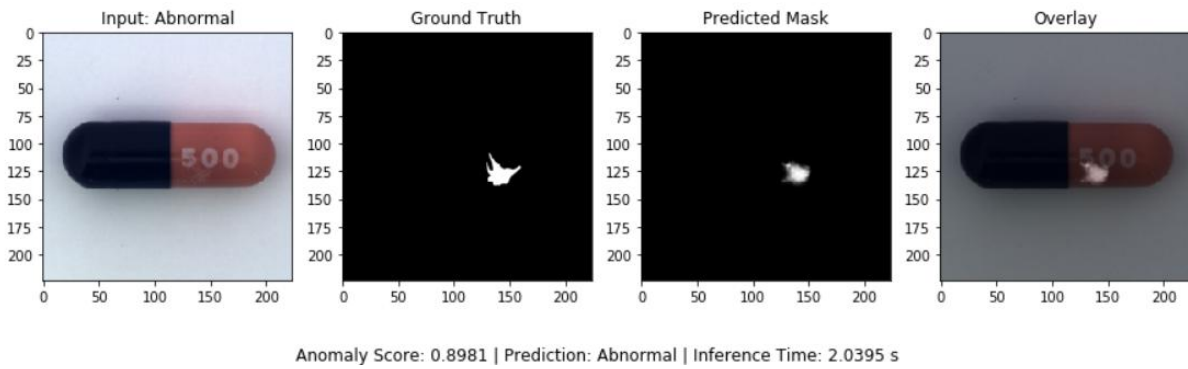
Đầu tiên, đối với danh mục chai (bottle), kết quả cho thấy mô hình MemSeg thể hiện khả năng nhận diện bất thường chính xác với đối với nhiều dạng khuyết tật khác nhau, bao gồm nứt vỡ và nhiễm bẩn bề mặt. Đối với mẫu bị lỗi vỡ lớn, mô hình dự đoán chính xác và đạt được giá trị anomaly score rất cao là 0.9754, trong đó, ta thấy rằng mặt nạ dự đoán (predicted mask) xác định vùng nứt theo đúng hình dạng vòng cung, thể hiện khả năng xử lý tốt với các bất thường biên dạng phức tạp. Trên mẫu vỡ nhỏ, anomaly score tăng lên 0.9999, mặt nạ dự đoán tuy có vùng biên hơi mở rộng nhưng vẫn bám đúng khu vực nứt, cho thấy độ nhạy cao của mô hình với các bất thường kích thước nhỏ. Với trường hợp nhiễm bẩn, mô hình ghi nhận score 0.9652 và mặt nạ dự đoán xác định chính xác vị trí vết bẩn có độ tương phản thấp, khẳng định khả năng phát hiện tốt trong điều kiện tín hiệu yếu hoặc mức nhiễu cao. Đặc biệt, ở mẫu chai bình thường, anomaly score chỉ ở mức rất thấp 0.0018, và mặt nạ dự đoán gần như không kích hoạt bất kỳ vùng nào, khẳng định tính ổn định và khả năng hạn chế false positive của mô hình. Thời gian suy luận trung bình trên các mẫu dao động từ 1.91 đến 2.09 giây, hoàn toàn đáp ứng được yêu cầu kiểm tra tự động theo thời gian thực trong môi trường công nghiệp.



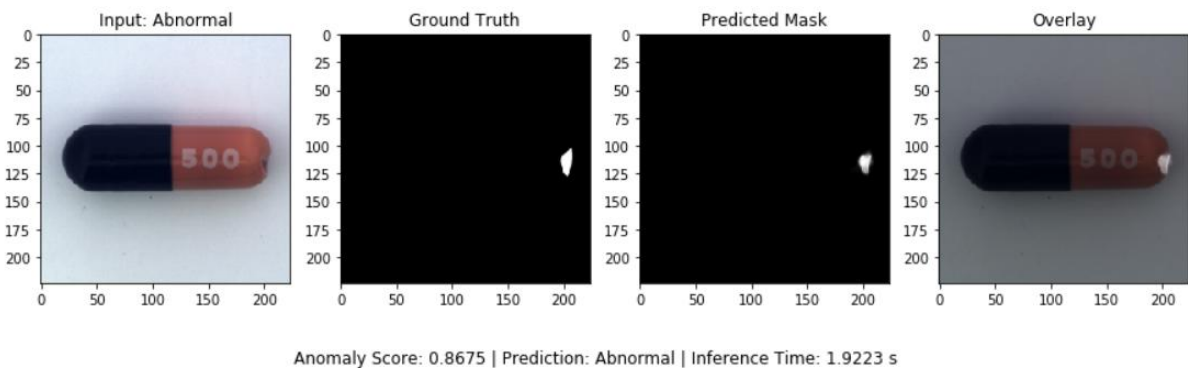
Hình 5.70: Phát hiện cho mẫu viên nang (capsule) bị nứt vỡ



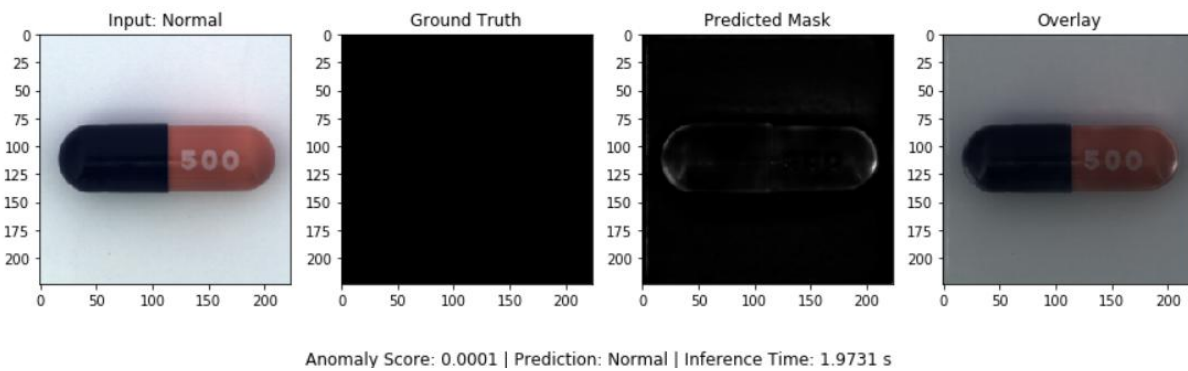
Hình 5.71: Phát hiện cho mẫu viên nang (capsule) bị thủng



Hình 5.72: Phát hiện cho mẫu viên nang (capsule) bị trầy xước



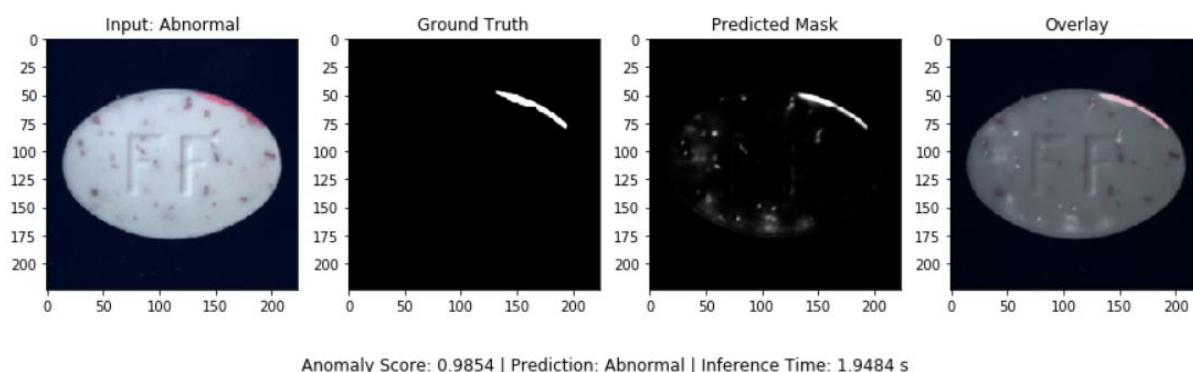
Hình 5.73: Phát hiện cho mẫu viên nang (capsule) bị móp méo



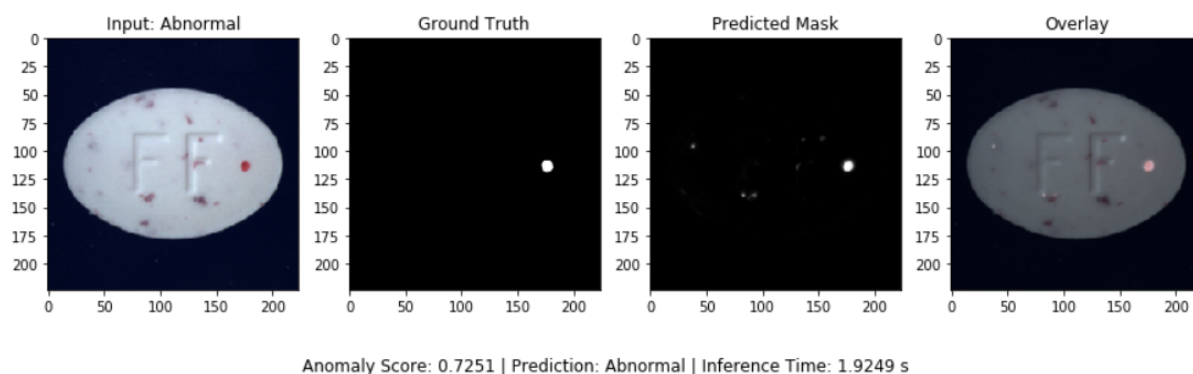
Hình 5.74: Phát hiện cho mẫu viên nang (capsule) bình thường

Ở danh mục viên nang (capsule), mô hình MemSeg tiếp tục cho thấy độ chính xác cao trong việc phát hiện và phân đoạn các khuyết tật có kích thước nhỏ và hình dạng đa dạng trên bề mặt vật thể. Đầu tiên, với mẫu viên nang có khuyết tật nứt vỡ, Phát hiện chính xác với điểm số bất thường (anomaly score) khá cao đạt 0.8619, mặt nạ dự đoán khớp tương đối tốt với Ground Truth, bám đúng khu vực rạn nứt nhỏ ở thân viên nang, dù có một chút nhiễu nhẹ ngoài vùng nhưng vẫn giữ được hình dạng lỗi rõ nét cho thấy được khả năng phân loại chính xác của mô hình ở cấp độ pixel

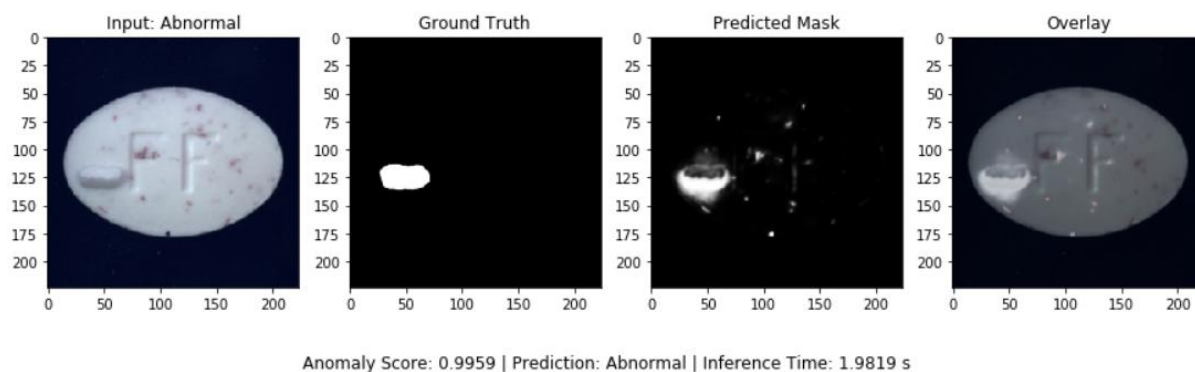
cũng như mô hình đã xác định tốt vùng xuất hiện khuyết tật. Với mẫu viên nang xuất hiện khuyết tật thủng vỏ, mô hình vẫn có khả năng dự đoán chính xác, tuy nhiên điểm số bất thường chỉ đạt 0.6945, mặt nạ dự đoán cũng khá chính xác nhưng vùng biên chưa được sắc nét khi so sánh với GT, điều này là hợp lý do lỗ thủng rất nhỏ và có độ tương phản thấp với nền. Trong trường hợp viên nang trầy xước, MemSeg đạt anomaly score 0.8981, và mặt nạ dự đoán bám khá sát vùng lỗi thực tế, thể hiện khả năng xử lý tốt các bất thường có tính lan mờ. Đối với mẫu bị biến dạng móp méo, mô hình vẫn có hiệu suất dự đoán rất chính xác với anomaly score đạt giá trị 0.8675, kết quả của mặt nạ dự đoán cho thấy mô hình vẫn đảm bảo phát hiện chính xác vùng biến dạng dù không có biên sắc nét rõ ràng, đây là tín hiệu rất tích cực về khả năng nhận diện các lỗi hình học có đặc điểm là đường biên không rõ ràng. Cuối cùng, ở mẫu viên nang bình thường, mô hình cho giá trị anomaly score rất thấp là 0.0001, mặt nạ dự đoán không phát hiện vùng bất thường nào, xác nhận độ ổn định và tỷ lệ false positive cực thấp. Thời gian suy luận cho mọi trường hợp viên nang ổn định quanh 1.92 – 2.04 giây.



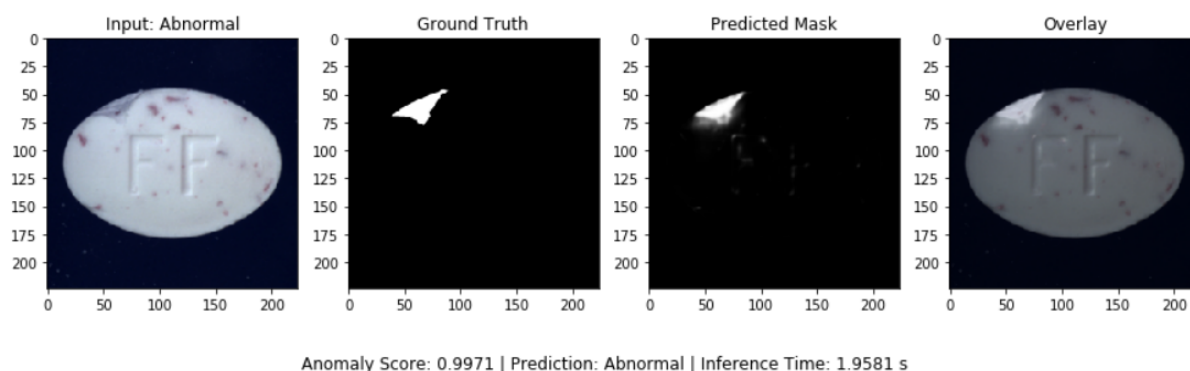
Hình 5.75: Phát hiện cho mẫu viên thuốc (pill) bị dính màu



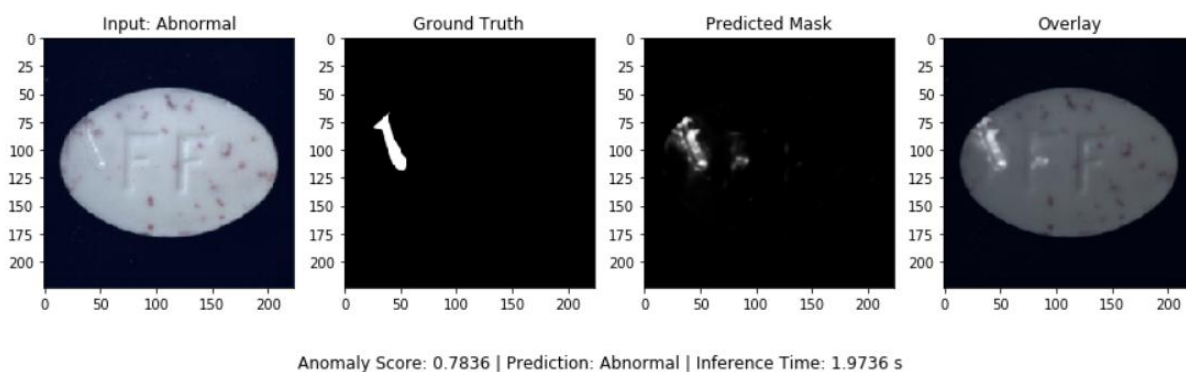
Hình 5.76: Phát hiện cho mẫu viên thuốc (pill) bị dính màu



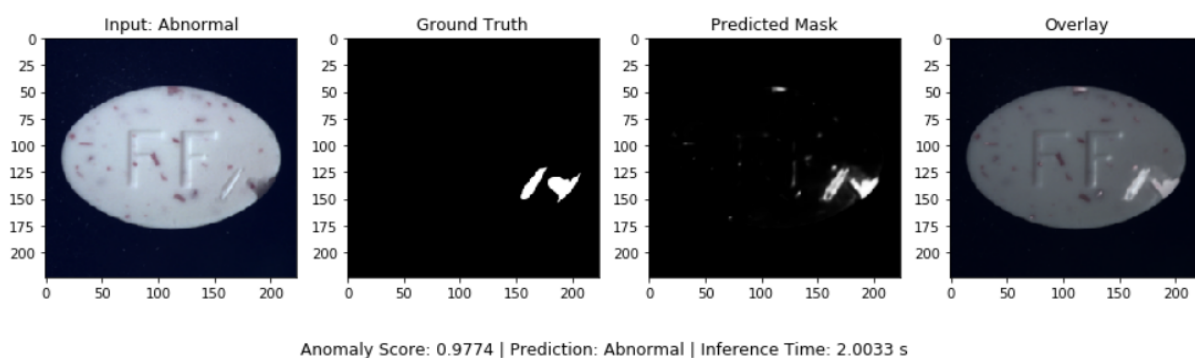
Hình 5.77: Phát hiện cho mẫu viên thuốc (pill) nhiễm bẩn bề mặt



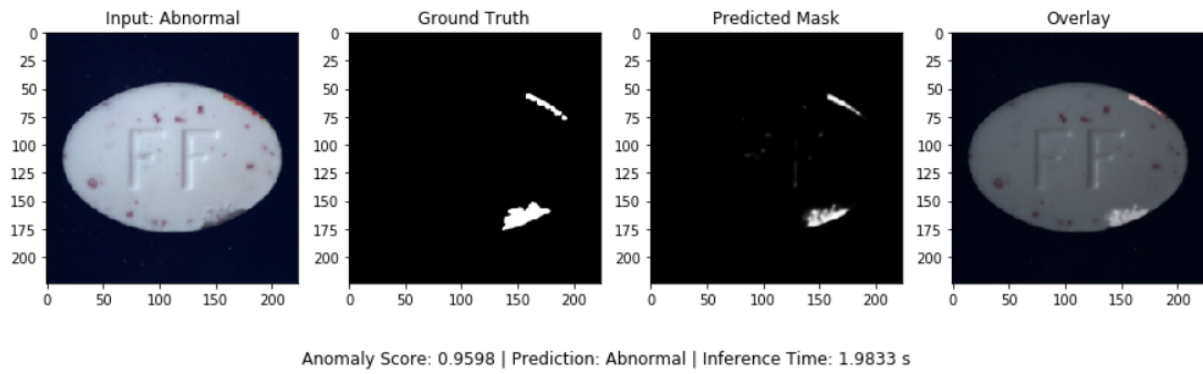
Hình 5.78: Phát hiện cho mẫu viên thuốc (pill) bị nứt vỡ cạnh



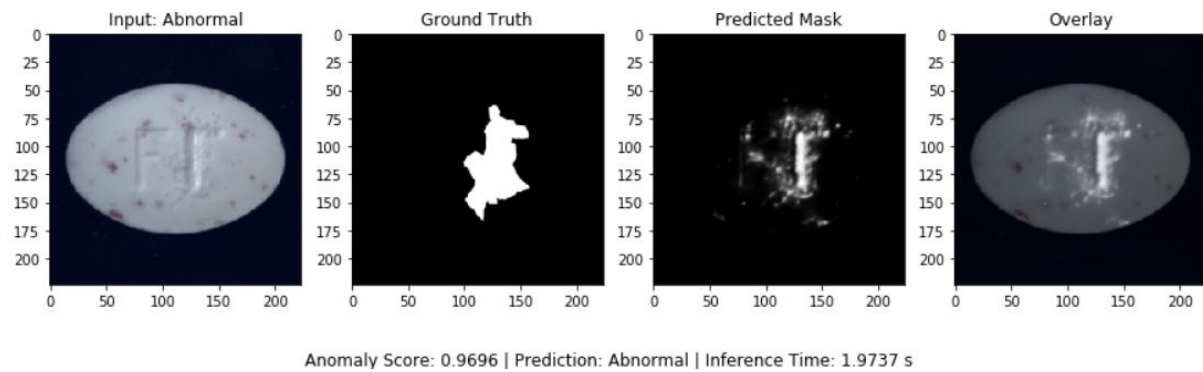
Hình 5.79: Phát hiện cho mẫu viên thuốc (pill) bị trầy xước bề mặt



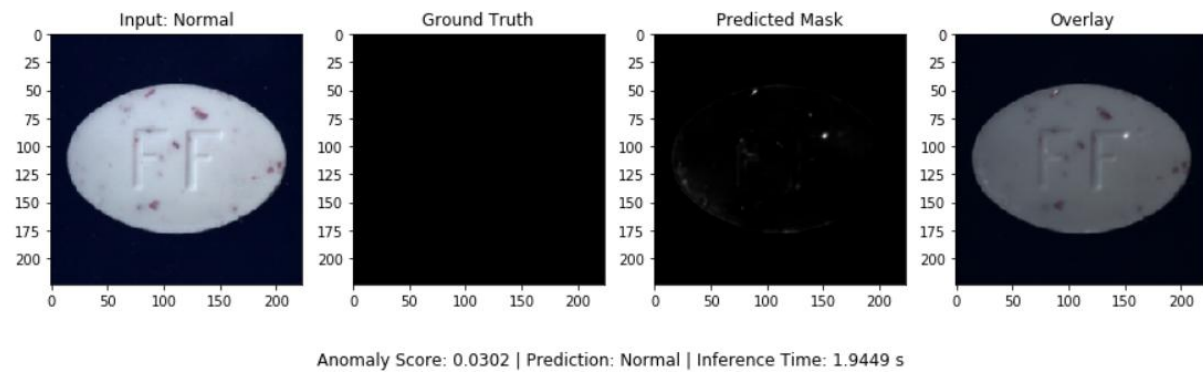
Hình 5.80: Phát hiện cho mẫu viên thuốc (pill) bị nứt vỡ cạnh và trầy xước



Hình 5.81: Phát hiện cho mẫu viên thuốc (pill) bị nứt vỡ 2 cạnh



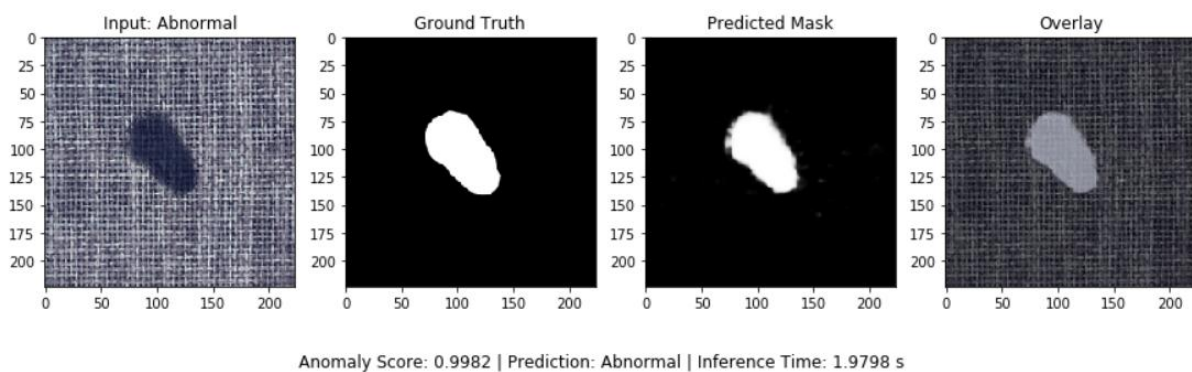
Hình 5.82: Phát hiện cho mẫu viên thuốc (pill) bị in sai ký hiệu



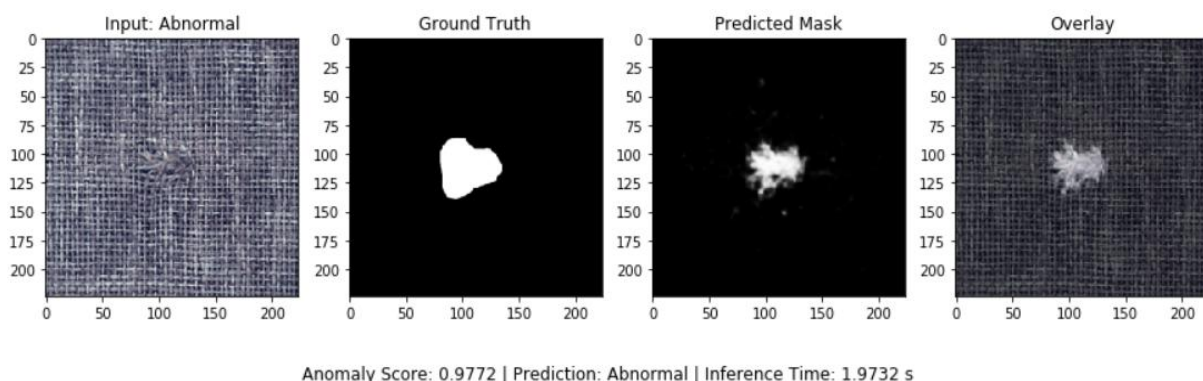
Hình 5.83: Phát hiện cho mẫu viên thuốc (pill) bình thường

Tiếp theo, ta tiến hành đánh giá cho danh mục thuốc viên (pill), giống với viên nang (capsule) đây cũng là một dược phẩm với kích thước khá nhỏ. Ở mẫu viên thuốc (pill) xuất hiện khuyết tật dính màu, MemSeg dự đoán chính xác với anomaly score là 0.9854 và 0.7251 cho hai trường hợp vết dính khác nhau về vị trí và kích thước, predicted mask đã xác định chính xác vị trí vết màu. Với mẫu thuốc bị nhiễm bẩn bề mặt, mô hình cũng dự đoán chính xác với anomaly score là 0.9959 và mặt nạ dự đoán bao phủ chính xác lên vùng bẩn, chứng tỏ khả năng nhạy bén với các dạng

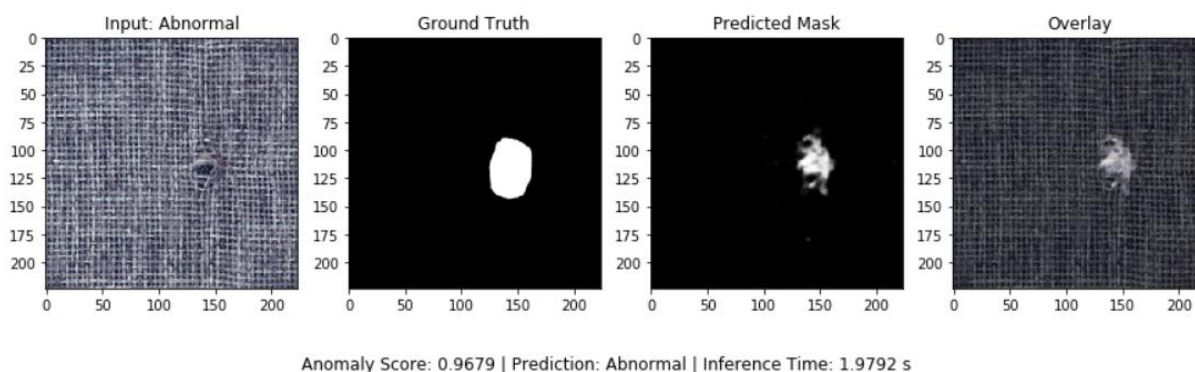
bất thường lan tỏa. Ở trường hợp khuyết tật nứt vỡ cạnh, MemSeg đạt anomaly score là 0.9971, Predicted Mask định vị gần trọn rìa vỡ, trong khi với vết trầy xước mờ, anomaly score giảm còn 0.7836 dù vẫn phân loại chính xác và biên mask dự đoán có phần thiếu sắc nét, điều này hợp lý vì khuyết tật quá nhỏ, độ tương phản thấp. Trên mẫu lỗi phức tạp kết hợp cả nứt và trầy xước, mô hình vẫn phân loại đúng và giữ được anomaly score lên đến 0.9774 với mặt nạ dự đoán phát hiện rõ rệt hai vùng lỗi riêng biệt. Tương tự, mẫu thuốc lỗi với vết nứt vỡ hai cạnh ghi nhận anomaly score đạt 0.9598. Đáng chú ý, ở trường hợp lỗi in sai ký hiệu trên bề mặt viên thuốc, mô hình vẫn định vị được vùng in bị lỗi với điểm số là 0.9696, cho thấy khả năng tổng quát hóa cao đối với các dạng bất thường phức tạp. Cuối cùng, trên ảnh bình thường, anomaly score chỉ là 0.0302 và predicted mask hầu như trống, cho thấy tỷ lệ false positive rất thấp. Thời gian phát hiện khuyết tật nằm trong khoảng 1.92–2.04 giây mỗi ảnh.



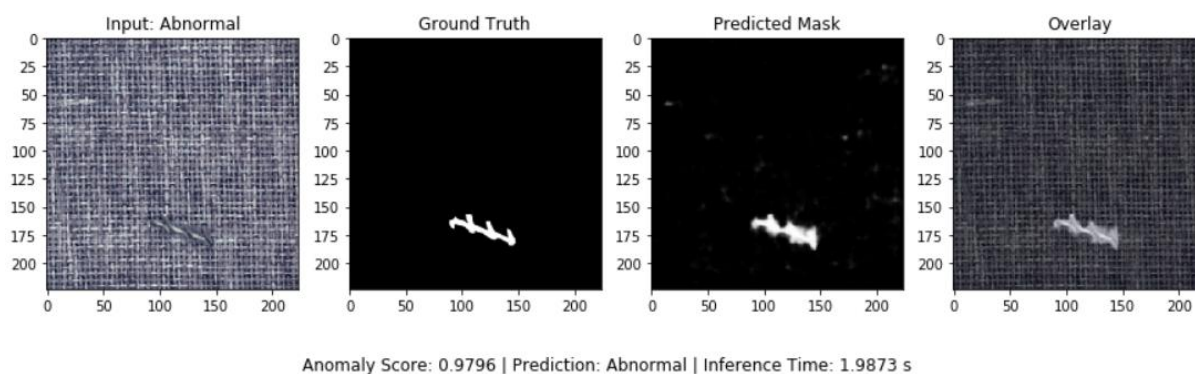
Hình 5.84: Phát hiện cho mẫu thảm (carpet) bị dính màu



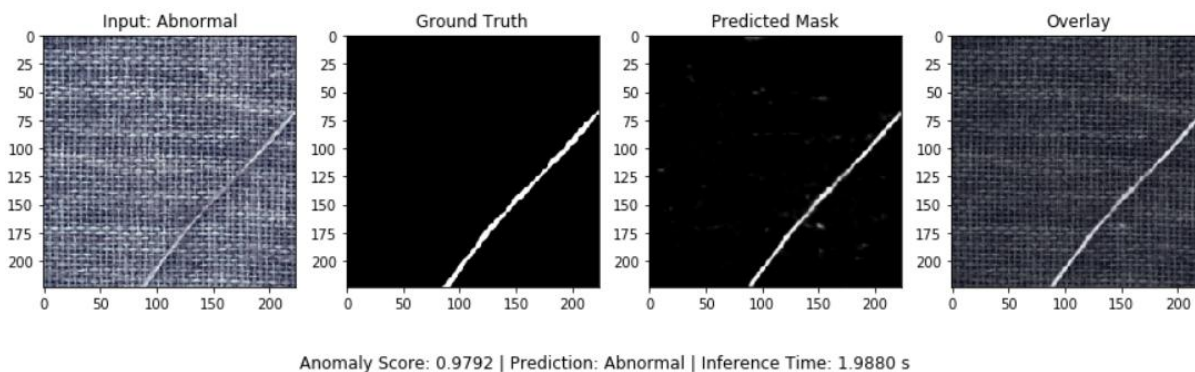
Hình 5.85: Phát hiện cho mẫu thảm (carpet) bị rách



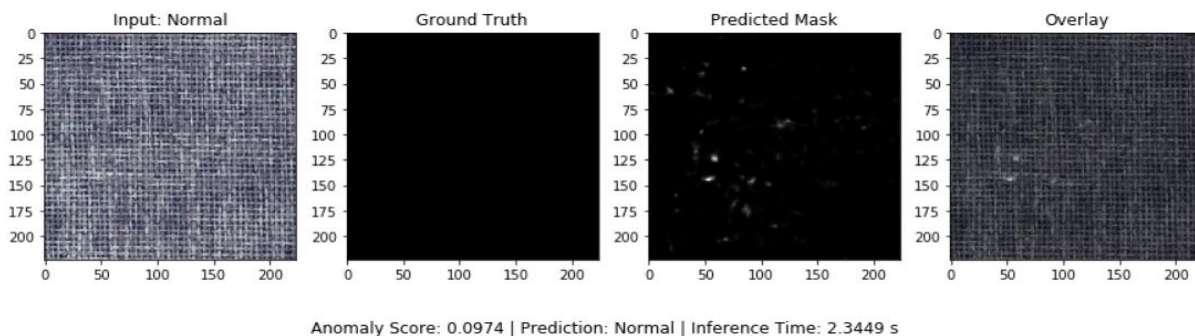
Hình 5.86: Phát hiện cho mẫu thảm (carpet) bị thủng lỗ



Hình 5.87: Phát hiện cho mẫu thảm (carpet) bị dị vật kim loại bề mặt

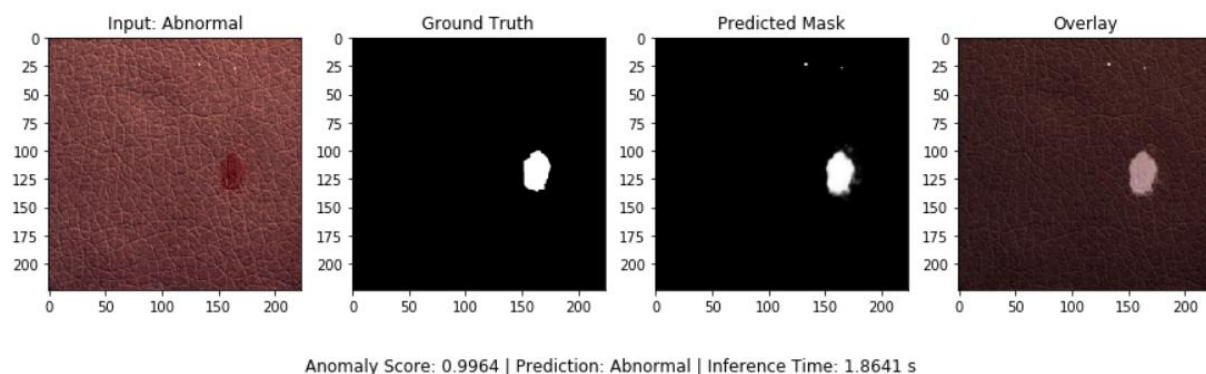


Hình 5.88: Phát hiện cho mẫu thảm (carpet) bị lỗi sợi chỉ

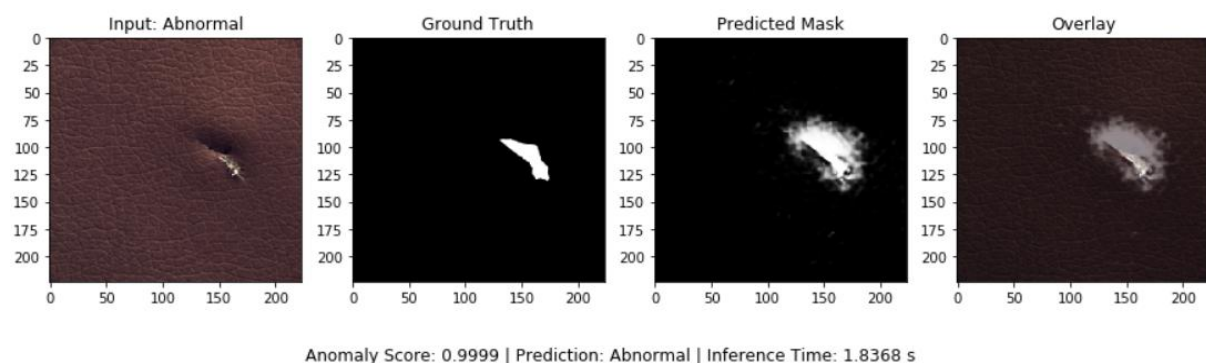


Hình 5.89: Phát hiện cho mẫu thảm (carpet) bình thường

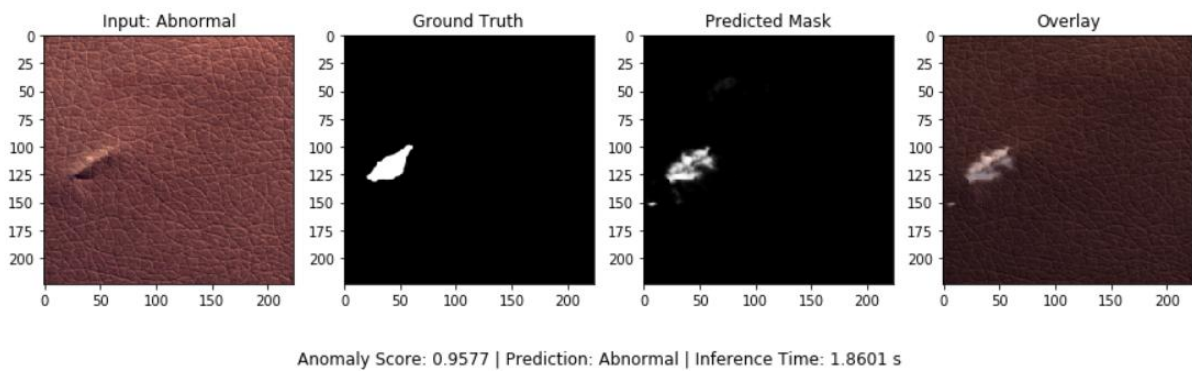
Đối với danh mục thảm (carpet) xuất hiện khuyết tật dính màu, MemSeg dự đoán chính xác với anomaly score lên đến 0.9982, mặt nạ dự đoán xác định gần trọn vẹn vùng dính màu, cho thấy khả năng phân vùng tốt đối với khuyết tật nổi bật về màu sắc. Với mẫu thảm bị rách, anomaly score đạt 0.9772, mô hình phát hiện được vị trí lỗi, tuy mask có phần loang nhẹ, song vẫn giữ được tính phân biệt. Trên mẫu thảm bị thủng lỗ, mô hình ghi nhận anomaly score là 0.9679, mặt nạ dự đoán phát hiện vùng lỗi dù chưa thật sắc nét, phù hợp với tính chất nhiễu và biên dạng không rõ ràng của khuyết tật này. Đối với mẫu có dị vật kim loại bề mặt, anomaly score đạt 0.9796, mặt nạ dự đoán bám sát vùng dị vật hẹp, cho thấy mô hình vẫn nhạy với vật thể có độ phản xạ cao. Với lỗi sợi chỉ trên bề mặt thảm, anomaly score là 0.9792, mô hình vẫn phát hiện đúng đường lỗi kéo dài, dù độ sắc nét ở mask dự đoán bị ảnh hưởng bởi hình dạng mảnh và đều. Đáng chú ý, trên mẫu bình thường, anomaly score chỉ là 0.0974, predicted mask hầu như không đánh dấu vùng bất thường, chứng tỏ tỷ lệ false positive thấp. Thời gian suy luận cho các mẫu thảm dao động từ 1.97 đến 2.34 giây mỗi ảnh, cho thấy hiệu suất khả dụng trong thực tế.



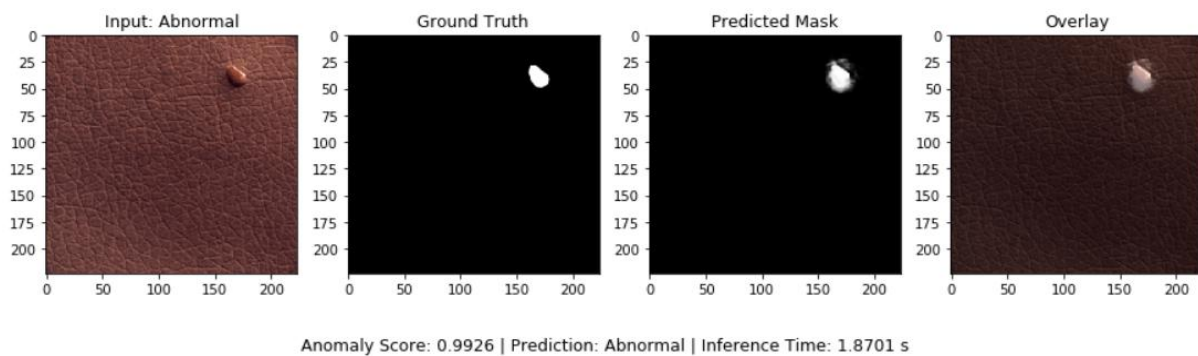
Hình 5.90: Phát hiện cho mẫu da (leather) dính màu



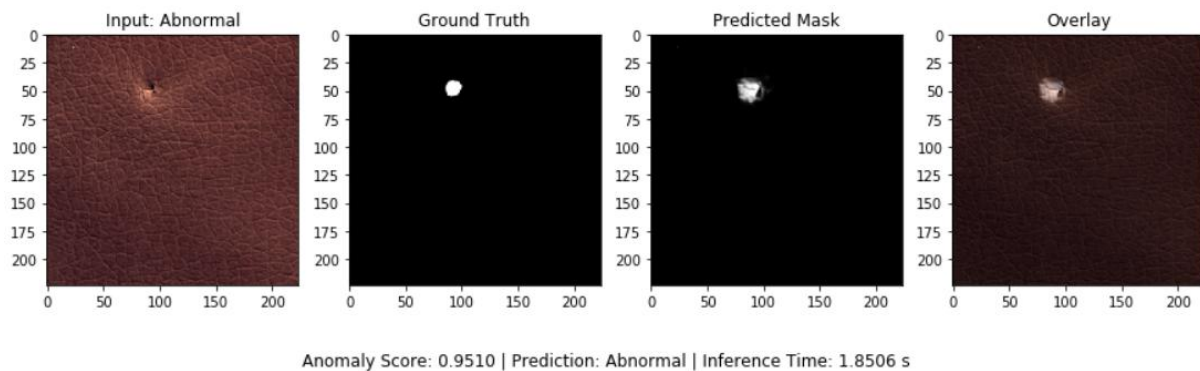
Hình 5.91: Phát hiện cho mẫu da (leather) bị rách



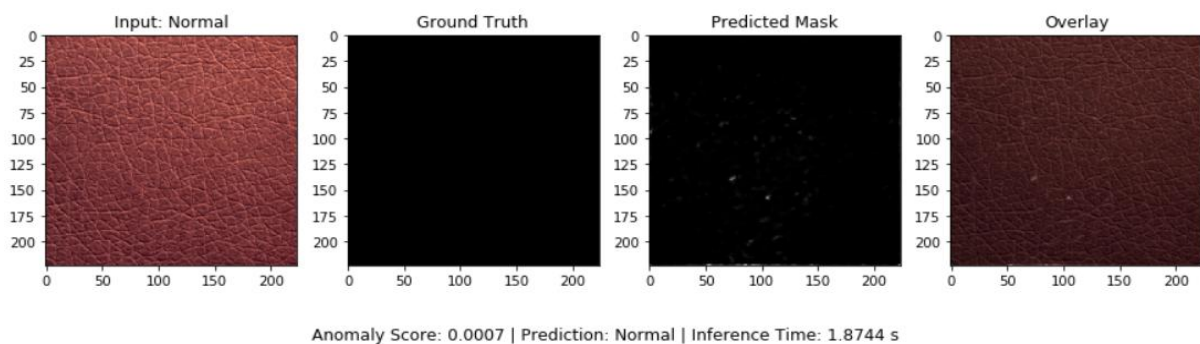
Hình 5.92: Phát hiện cho mẫu da (leather) bị nứt



Hình 5.93: Phát hiện cho mẫu da (leather) bị dấn keo

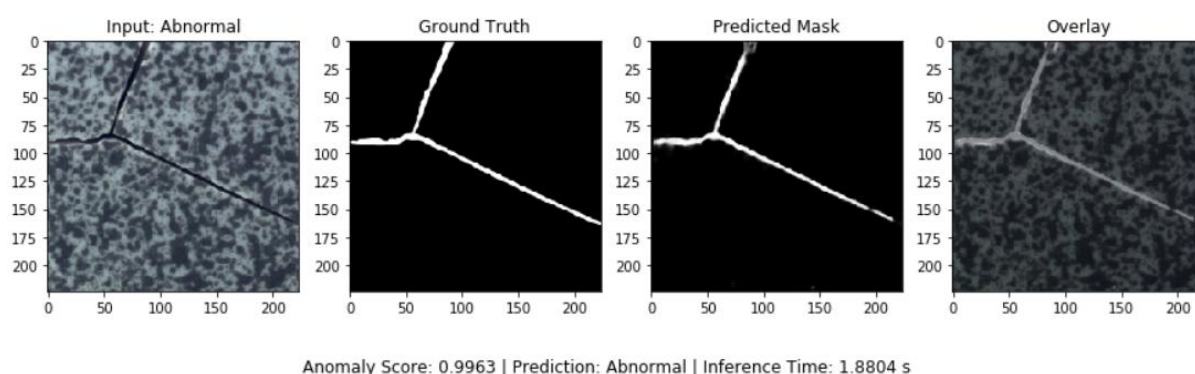


Hình 5.94: Phát hiện cho mẫu da (leather) bị thủng lỗ

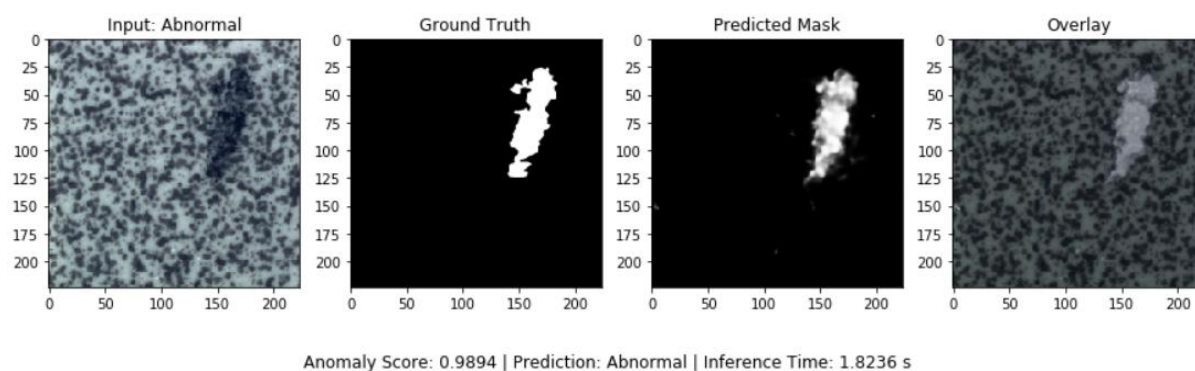


Hình 5.95: Phát hiện cho mẫu da (leather) bình thường

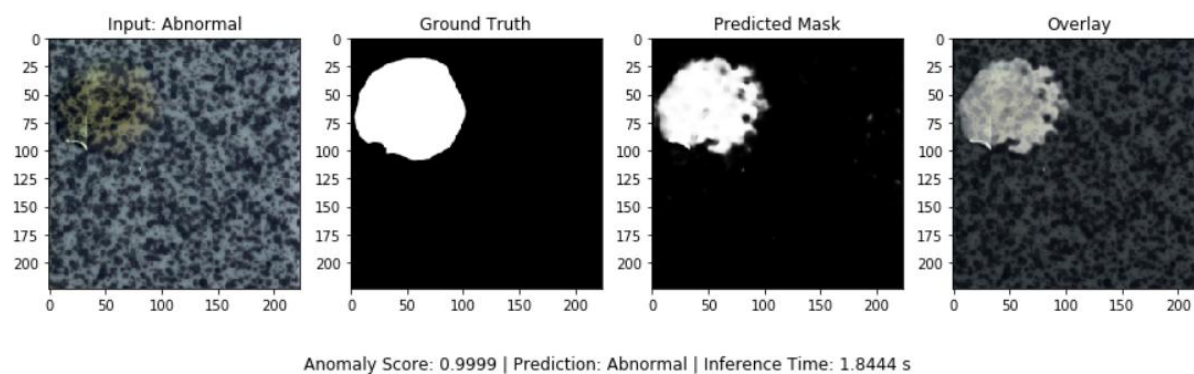
Ở danh mục da (leather) bị lỗi dính màu trên bề mặt, mô hình dự đoán chính xác với anomaly score lên đến 0.9964, mặt nạ dự đoán bao phủ gần như trọn vẹn vùng dính màu, cho thấy khả năng phân vùng tốt đối với khuyết tật có độ tương phản cao. Với mẫu da bị lỗi rách, anomaly score đạt 0.9999, mô hình phát hiện đúng vị trí lỗi nhưng mask dự đoán có xu hướng loang rộng, phản ánh độ nhạy khá cao. Trên mẫu da bị nhăn, anomaly score là 0.9577, mô hình vẫn xác định được khu vực bất thường, tuy mask chưa thật sự sắc nét do biên dạng lỗi không rõ ràng. Với trường hợp bị lỗi dính keo, anomaly score đạt 0.9926, mặt nạ dự đoán bám sát vùng bị dính keo, thể hiện khả năng nhận diện tốt của mô hình đối với các lỗi nhỏ, tập trung. Tiếp theo, với mẫu da bị thủng lỗ, anomaly score ghi nhận là 0.9510, mô hình vẫn phát hiện đúng vị trí lỗ thủng nhưng mask chưa thực sự rõ ràng do kích thước khuyết tật nhỏ và ít nổi bật. Cuối cùng, trên mẫu bình thường, anomaly score chỉ là 0.0007, predicted mask hầu như không đánh dấu bất kỳ vùng bất thường nào, chứng tỏ mô hình cũng có tỷ lệ false positive rất thấp. Thời gian suy luận cho các mẫu da dao động từ 1.85 đến 1.87 giây mỗi ảnh.



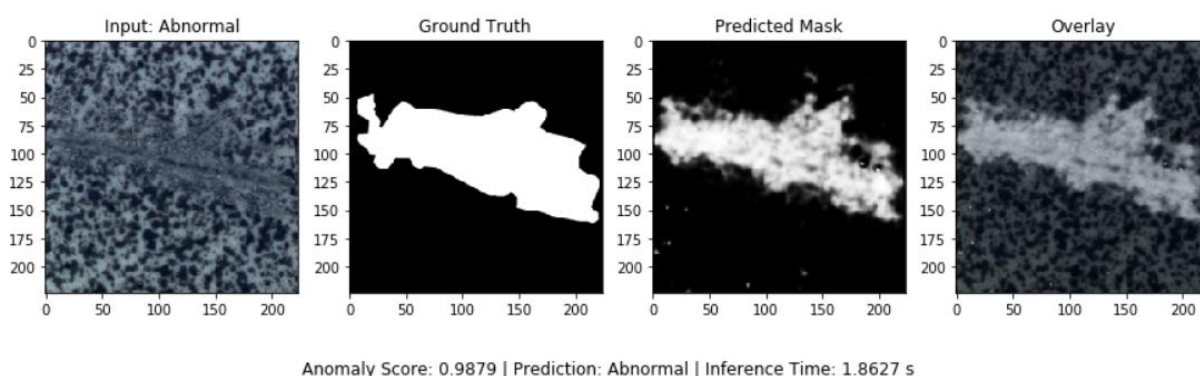
Hình 5.96: Phát hiện cho mẫu gạch lát (tile) bị nứt vỡ



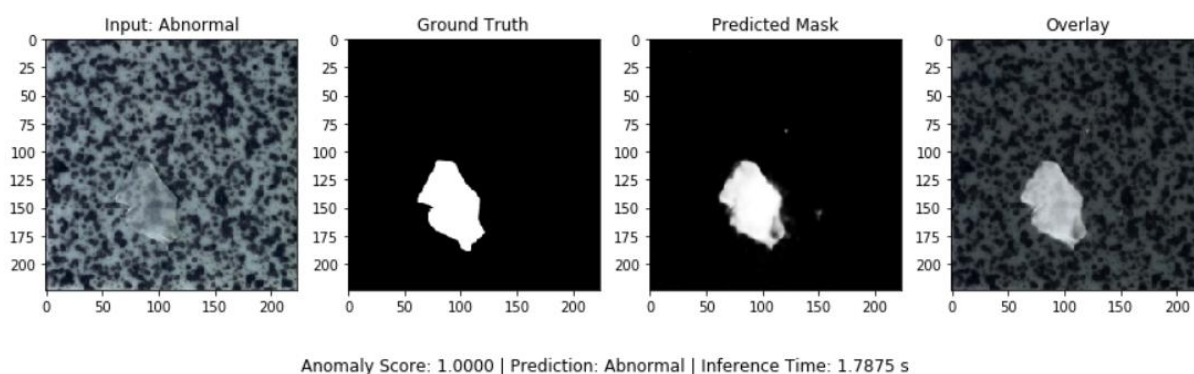
Hình 5.97: Phát hiện cho mẫu gạch lát (tile) có vết xám bất thường



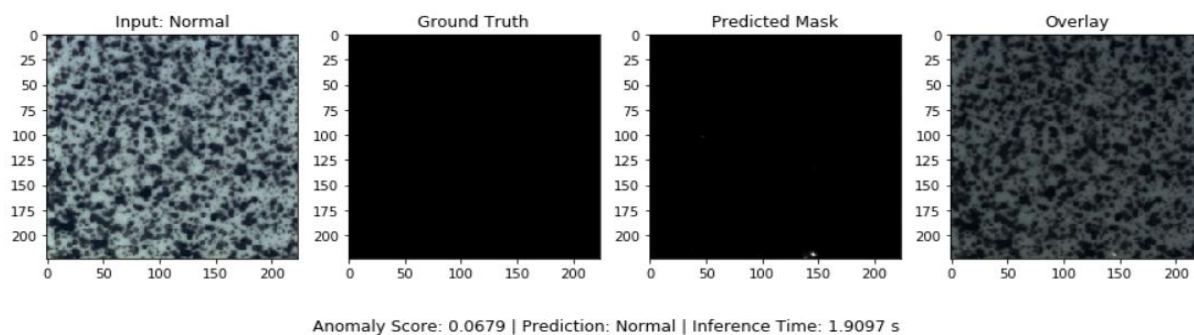
Hình 5.98: Phát hiện cho mẫu gạch lát (tile) bị dính dầu



Hình 5.99: Phát hiện cho mẫu gạch lát (tile) bị trầy xước

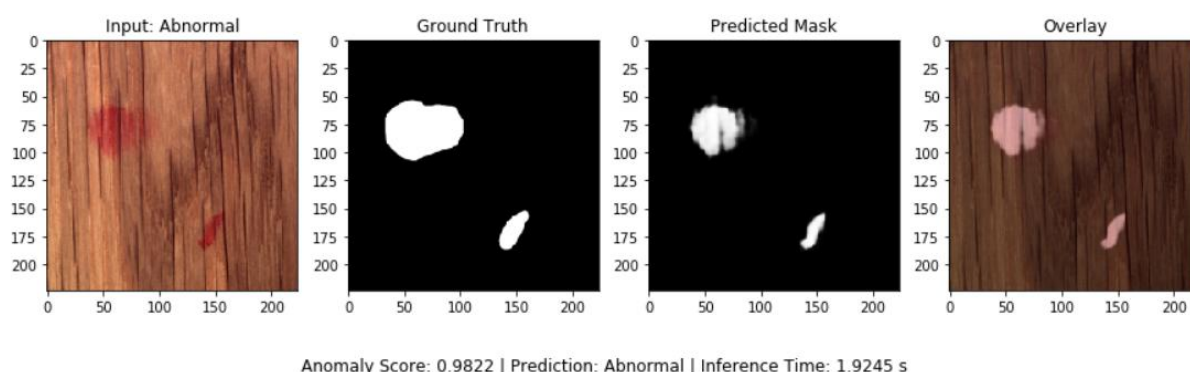


Hình 5.100: Phát hiện cho mẫu gạch lát (tile) bị vệt keo

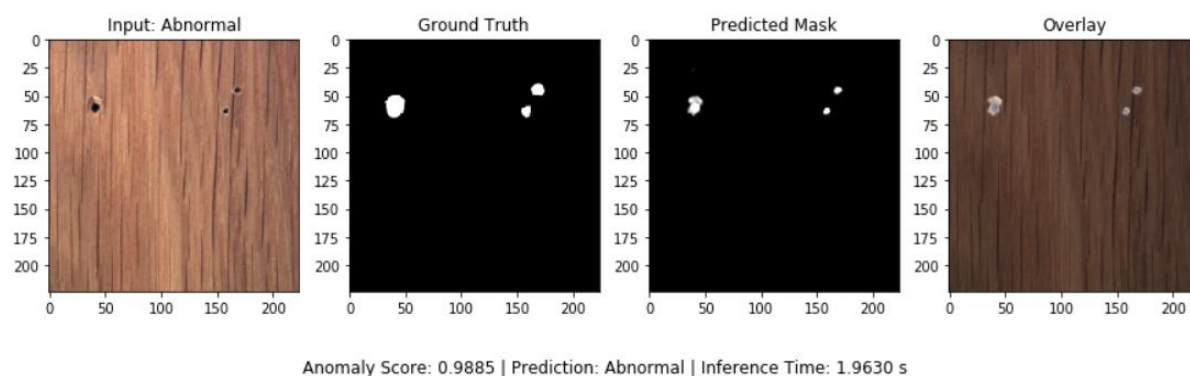


Hình 5.101: Phát hiện cho mẫu gạch lát (tile) bình thường

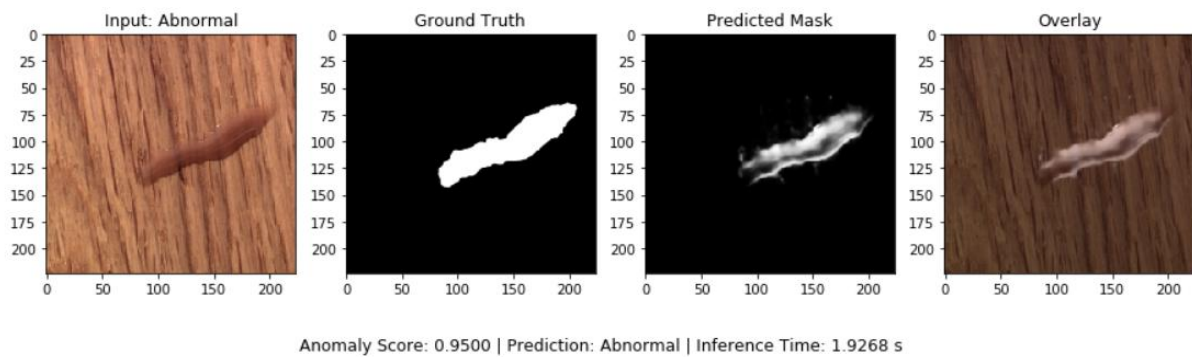
Ở mẫu gạch lát (tile) bị nứt vỡ, MemSeg dự đoán chính xác với anomaly score 0.9963, mặt nạ dự đoán xác định chính xác theo hình dạng đường nứt, cho thấy khả năng nhận diện tốt với khuyết tật dạng biên kéo dài. Với mẫu gạch có vết xám bất thường bề mặt, anomaly score đạt 0.9894, mask bao phủ gần toàn bộ vùng bất thường dù có một chút loang nhẹ ở rìa, thể hiện độ nhạy tốt đối với thay đổi màu sắc lạ. Đối với mẫu gặp lỗi dính dầu, anomaly score lên đến 0.9999, mặt nạ dự đoán trùng khớp gần như hoàn toàn với ground truth, cho thấy khả năng phát hiện hiệu quả với các mảng bám rõ rệt. Đối với mẫu bị trầy xước, anomaly score đạt 0.9879, mask tuy có vùng mờ lan rộng hơn so với ground truth nhưng vẫn bám đúng vùng lỗi chính, phù hợp với tính chất lan rộng của vết trầy. Trên mẫu bị vệt keo, anomaly score đạt mức tối đa 1.0000, mask dự đoán sắc nét, bám sát khuyết tật hình khối rõ, cho thấy khả năng phân biệt mạnh mẽ. Đặc biệt, với mẫu gạch lát bình thường, anomaly score chỉ ở mức 0.0679 và predicted mask không đánh dấu vùng bất thường, chứng tỏ mô hình có độ chính xác cao và tỷ lệ false positive rất thấp. Thời gian suy luận cho mỗi ảnh gạch lát dao động từ 1.78 đến 1.91 giây.



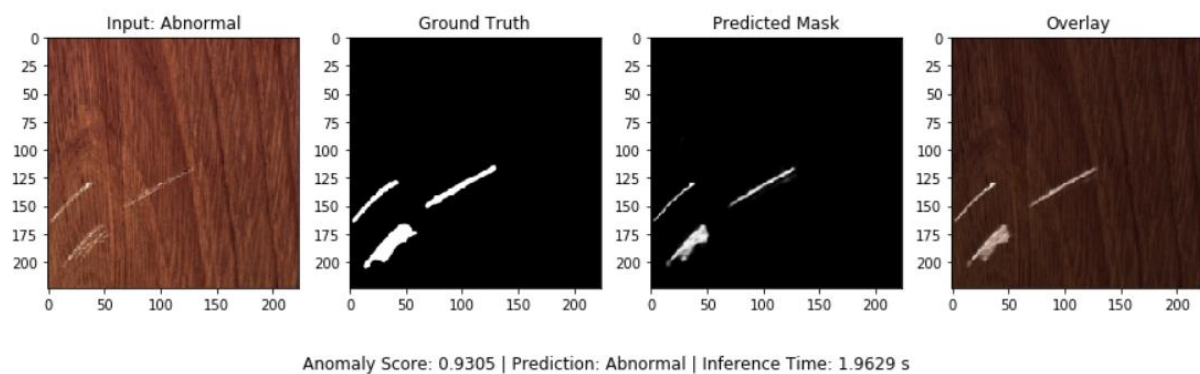
Hình 5.102: Phát hiện cho mẫu gỗ (wood) bị dính màu



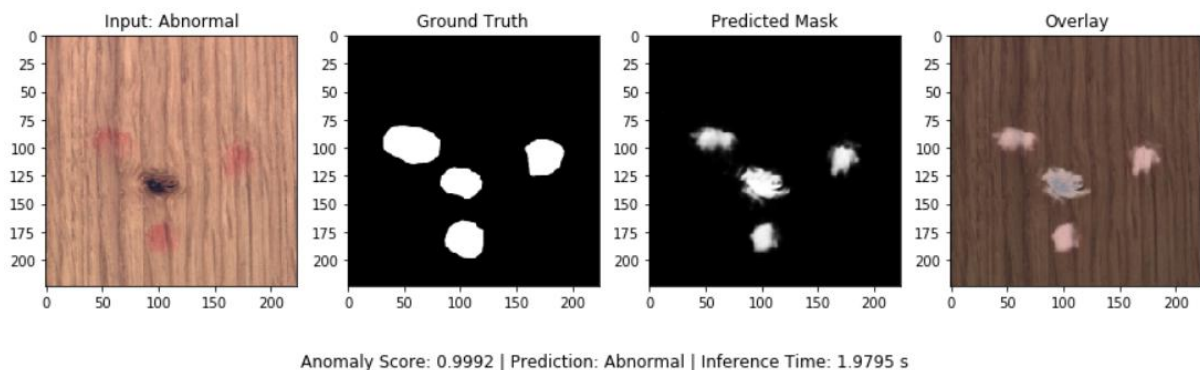
Hình 5.103: Phát hiện cho mẫu gỗ (wood) bị thủng lỗ



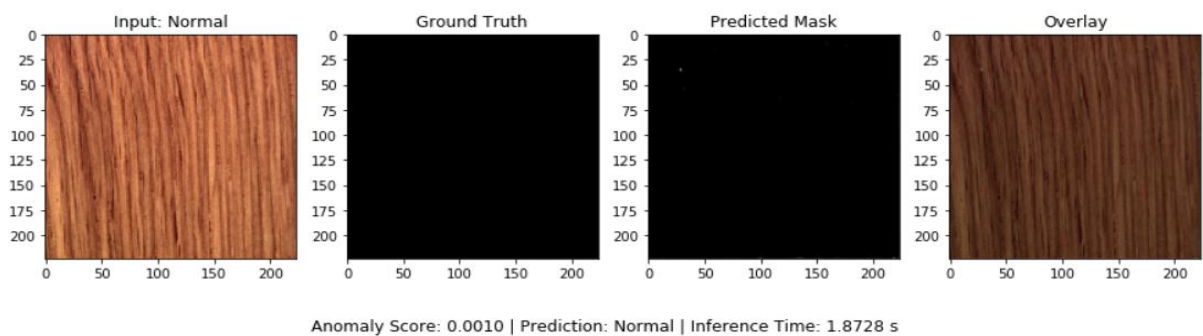
Hình 5.104: Phát hiện cho mẫu gỗ (wood) có vết loang, thấm nước



Hình 5.105: Phát hiện cho mẫu gỗ (wood) bị trầy xước

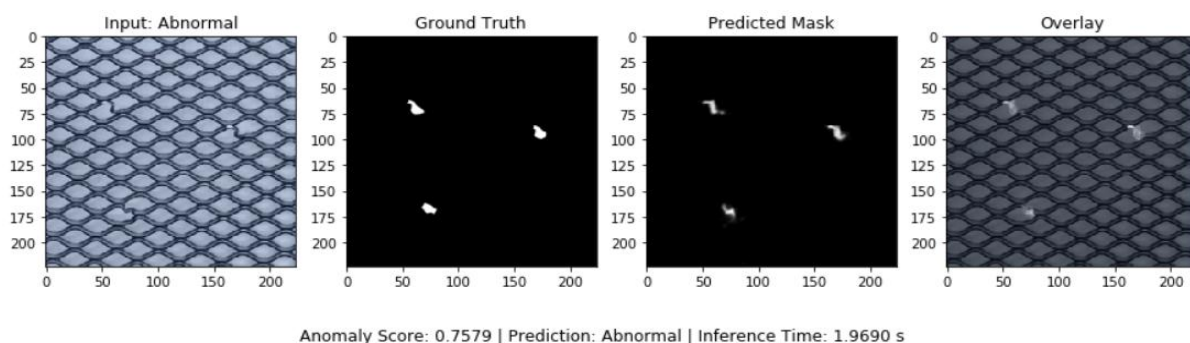


Hình 5.106: Phát hiện cho mẫu gỗ (wood) bị dính màu và thùng lỗ

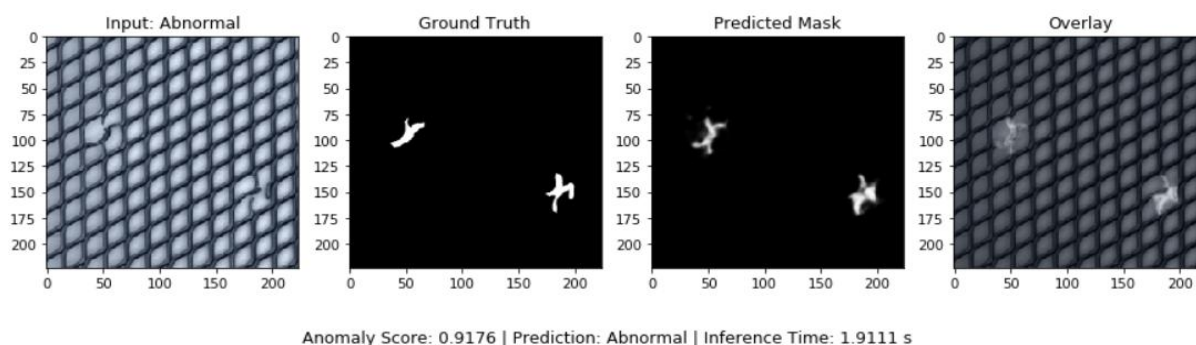


Hình 5.107: Phát hiện cho mẫu gỗ (wood) bình thường

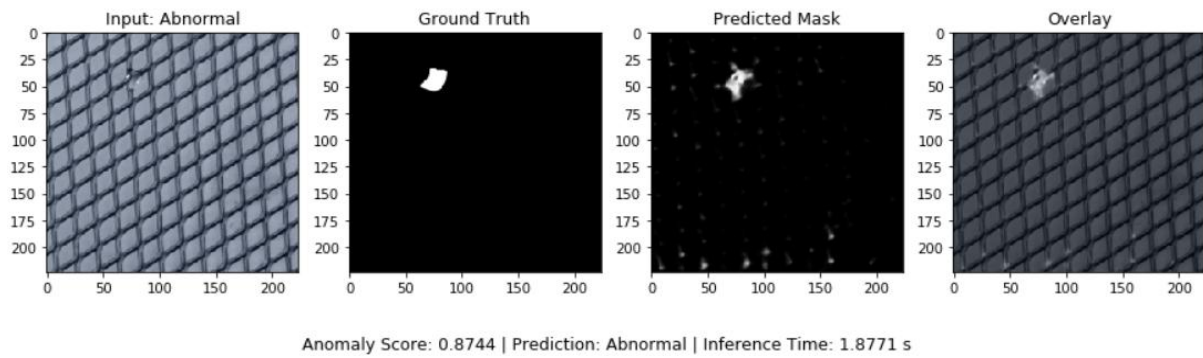
Đầu tiên, đối với mẫu gỗ bị dính màu, mô hình phát hiện tốt với anomaly score đạt 0.9822, mặt nạ dự đoán xác định chính xác hai vùng lỗi dính màu nổi bật trên bề mặt gỗ, cho thấy khả năng phát hiện lỗi về màu sắc tốt. Ở mẫu gỗ bị thùng lỗ, anomaly score đạt 0.9885, các lỗ nhỏ được phát hiện rõ ràng với mặt nạ dự đoán khớp vị trí, thể hiện độ nhạy cao với lỗi dạng lỗ nhỏ phân tán. Với mẫu bị vết loang, thâm nước, anomaly score 0.9500, mask dự đoán bám sát vết dài mờ, tuy biên không sắc nét nhưng mô hình vẫn phân đoạn được vùng chính xác. Mẫu bị trầy xước có score 0.9305, mask dự đoán theo đúng đường xước dài mảnh, cho thấy hiệu quả trong việc phát hiện các lỗi hẹp, kéo dài. Với mẫu gỗ có khuyết tật phức tạp kết hợp giữa việc bị dính màu bề mặt và thùng lỗ, anomaly score đạt được cũng rất cao là 0.9992, điều này cho thấy rằng mô hình có khả năng nhận diện được đồng thời nhiều loại lỗi trên bề mặt gỗ, chứng tỏ tính tổng quát và ổn định trong các trường hợp đa khuyết tật. Cuối cùng, ở mẫu gỗ bình thường, anomaly score rất thấp chỉ 0.0010, không có vùng bất thường nào xuất hiện trên mask, khẳng định khả năng phân biệt rõ ràng giữa mẫu lỗi và mẫu tốt. Thời gian suy luận dao động quanh 1.87 đến 1.98 giây, vẫn đảm bảo hiệu quả cho hệ thống phát hiện lỗi trong thực tế.



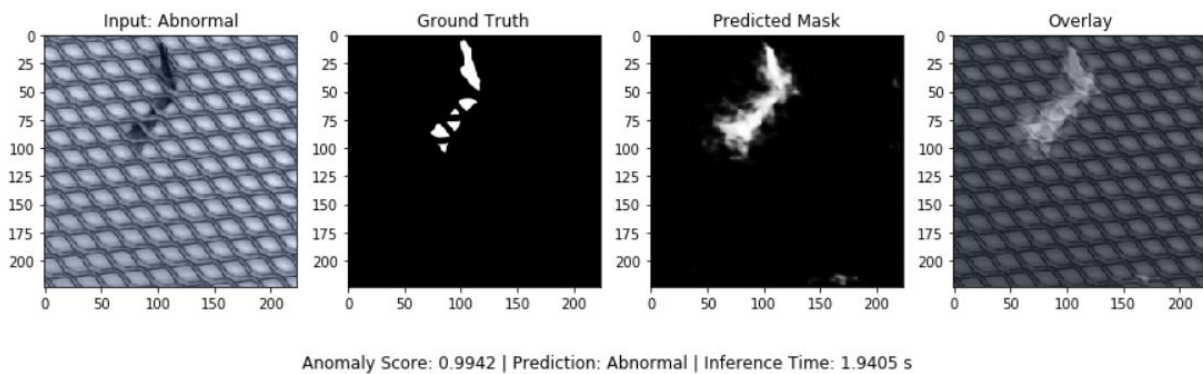
Hình 5.108: Phát hiện cho mẫu lưới (grid) bị cong



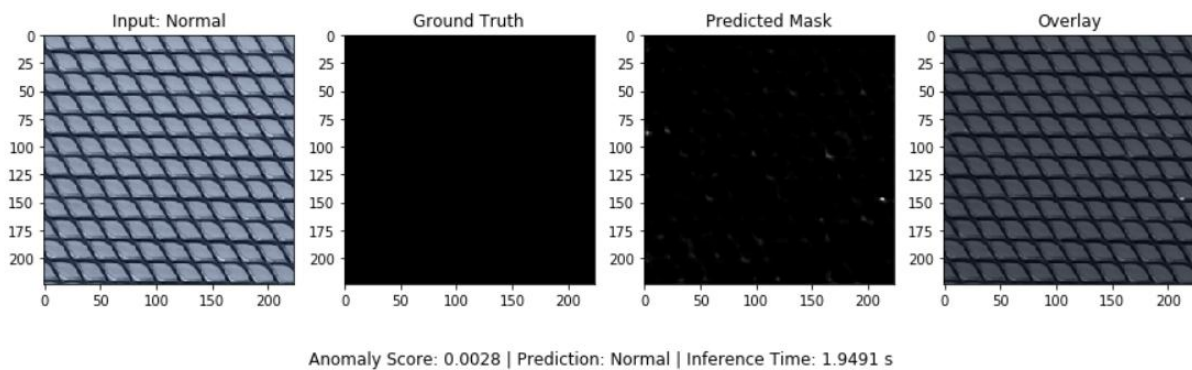
Hình 5.109: Phát hiện cho mẫu lưới (grid) bị gãy



Hình 5.110: Phát hiện cho mẫu lưới (grid) bị dính keo



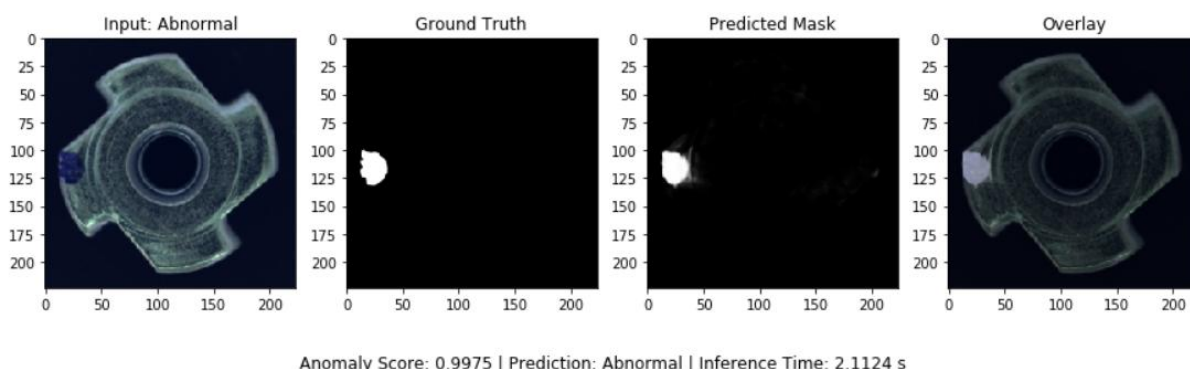
Hình 5.111: Phát hiện cho mẫu lưới (grid) có dị vật



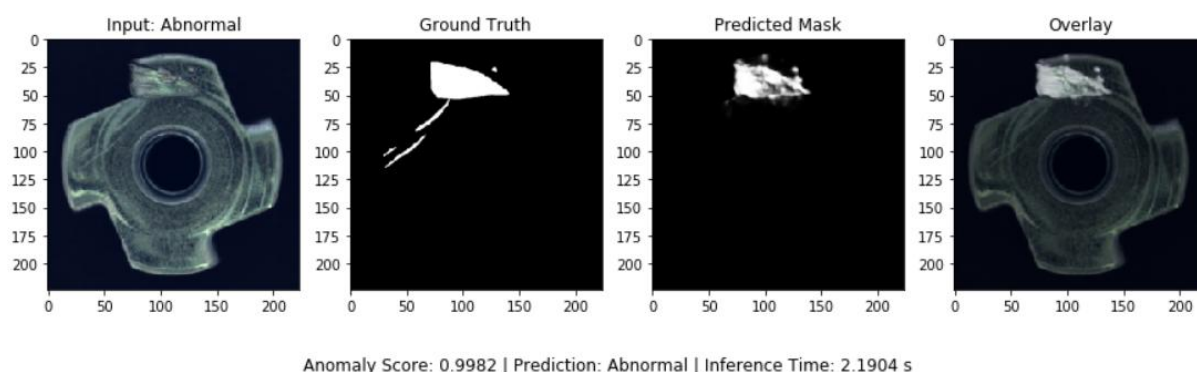
Hình 5.112: Phát hiện cho mẫu lưới (grid) bình thường

Với mẫu lưới (grid) có bề mặt bị cong, mô hình đạt anomaly score 0.7579, mặt nạ dự đoán đánh dấu được các vùng cong lệch tuy tương đối nhỏ, cho thấy mô hình có khả năng phát hiện các biến dạng hình học không đều. Ở mẫu lưới bị gãy, anomaly score đạt 0.9176, mặt nạ dự đoán khớp tốt với các đoạn lưới bị đứt, cho thấy độ nhạy cao với lỗi dạng kết cấu đứt gãy. Với mẫu bị dính keo, anomaly score là 0.8744, vùng lỗi được xác định khá chính xác dù đây là hình dạng lỗi không rõ

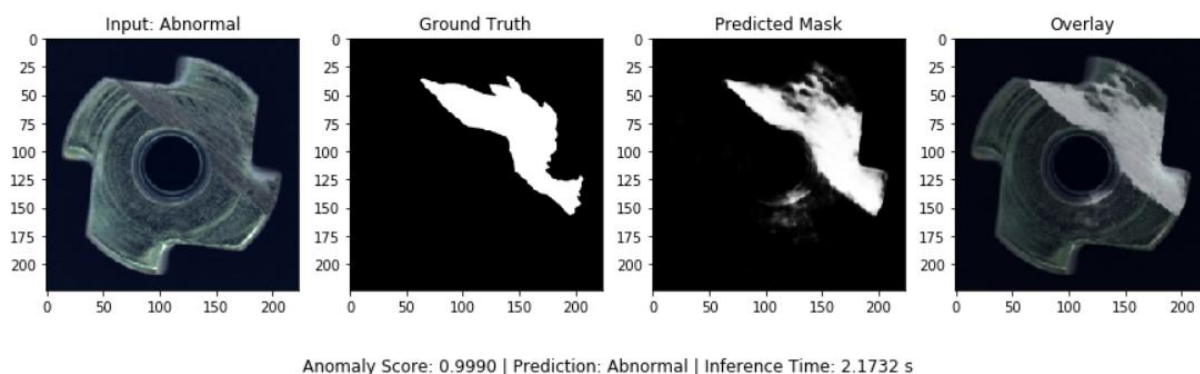
ràng, chúng tỏ mô hình vẫn xử lý tốt các khuyết tật lan rộng và khó phân ranh giới. Mẫu có dị vật đạt score rất cao 0.9942, mô hình tái hiện rõ dị vật lớn trên bề mặt lưới, phản ánh khả năng nhận diện các vật thể lạ có độ tương phản cao. Cuối cùng, ở mẫu lưới bình thường, anomaly score chỉ 0.0028, không có vùng bất thường nào được đánh dấu, khẳng định khả năng phân biệt rõ giữa mẫu lỗi và mẫu tốt. Thời gian phát hiện khuyết tật dao động quanh 1.87 đến 1.95 giây.



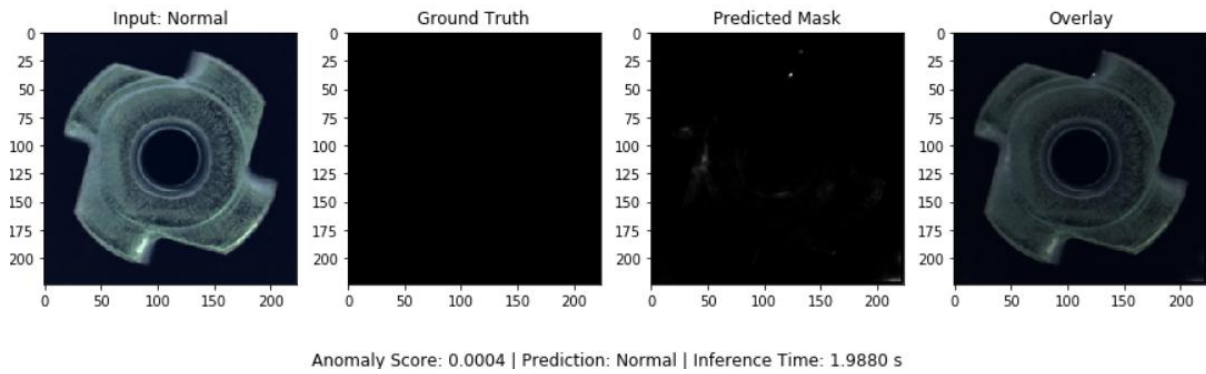
Hình 5.113: Phát hiện cho mẫu đai ốc (metal nut) dính mạt



Hình 5.114: Phát hiện cho mẫu đai ốc (metal nut) bị cong

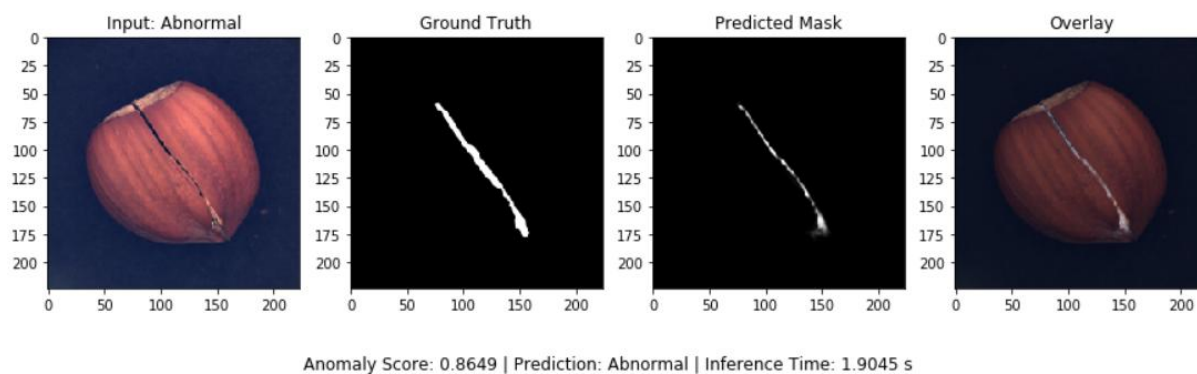


Hình 5.115: Phát hiện cho mẫu đai ốc (metal nut) bị trầy xước

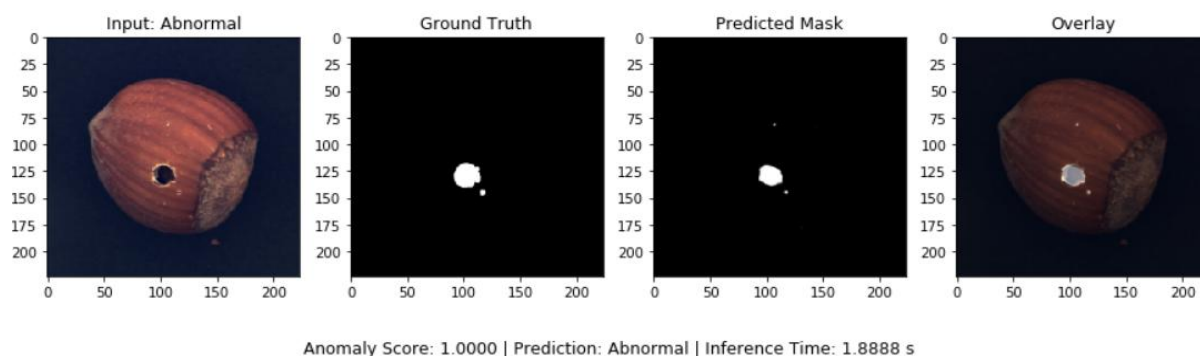


Hình 5.116: Phát hiện cho mẫu đai ốc (metal nut) bình thường

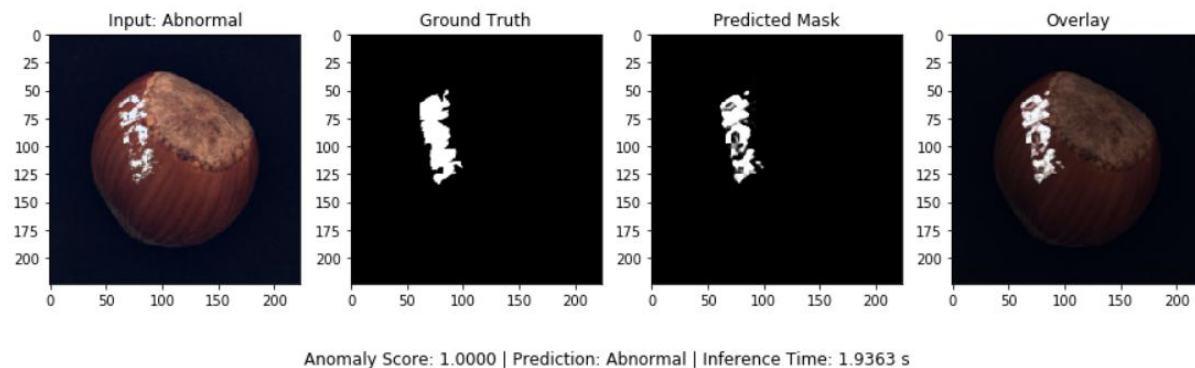
Đối với nhóm mẫu đai ốc kim loại (metal nut), MemSeg thể hiện khả năng phát hiện bất thường vượt trội, đặc biệt với các lỗi hình học và bề mặt đặc trưng. Cụ thể, với mẫu đai ốc bị dính mực, mô hình đạt Anomaly Score 0.9975, mặt nạ dự đoán bám sát khu vực mực loang và chỉ hơi lan nhẹ ở viền ngoài, cho thấy khả năng nhận diện tốt với các khuyết tật màu tối cục bộ. Với mẫu bị cong, anomaly score đạt 0.9982, mặt nạ dự đoán bao phủ tương đối chính xác phần biến dạng hình học, đặc biệt là ở cạnh rìa, phản ánh khả năng phát hiện bất thường hình dạng rõ rệt. Trên mẫu bị trầy xước, mô hình ghi nhận anomaly score lên đến 0.9990, mặt nạ dự đoán trùng khớp gần như hoàn toàn với ground truth, chỉ có một vài vùng biên mờ nhẹ do texture lặp lại trong vật liệu kim loại, đây là một kết quả rất khả quan cho phân đoạn pixel-level trên nền bề mặt có cấu trúc phức tạp. Đáng chú ý, trên mẫu đai ốc bình thường, mô hình dự đoán rất chính xác với anomaly score chỉ 0.0004, mặt nạ dự đoán không kích hoạt vùng nào đáng kể, khẳng định độ tin cậy và tỷ lệ false positive cực thấp của mô hình. Tương tự, thời gian phát hiện vẫn dao động từ 1.98 đến 2.17 giây.



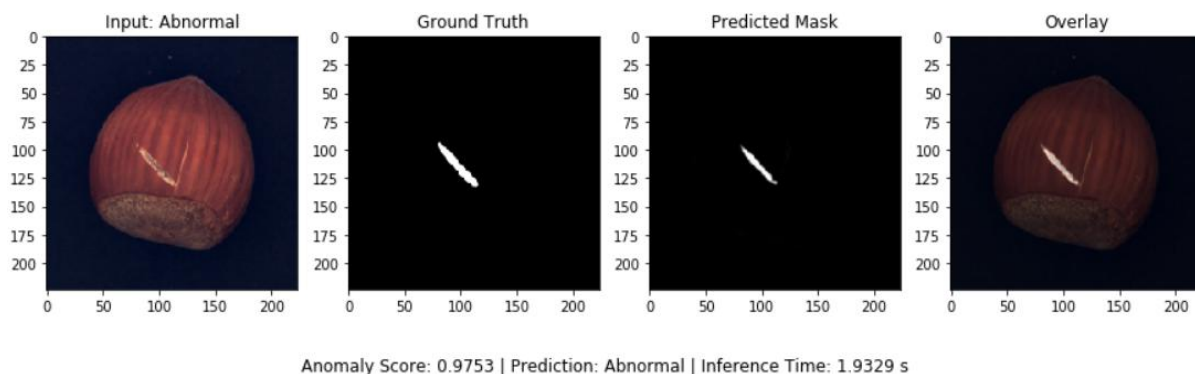
Hình 5.117: Phát hiện cho mẫu hạt phỉ (hazelnut) bị nứt



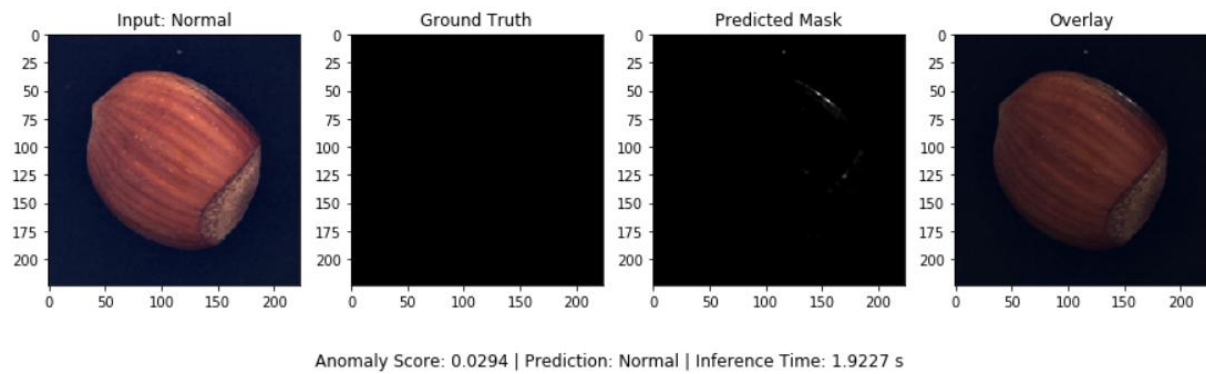
Hình 5.118: Phát hiện cho mẫu hạt phỉ (hazelnut) bị thủng lỗ



Hình 5.119: Phát hiện cho mẫu hạt phỉ (hazelnut) bị lỗi in ấn



Hình 5.120: Phát hiện cho mẫu hạt phỉ (hazelnut) bị trầy xước



Hình 5.121: Phát hiện cho mẫu hạt phỉ (hazelnut) bình thường

Ở danh mục hazelnut (hạt phỉ), mô hình MemSeg tiếp tục thể hiện hiệu năng rất tốt trong việc phát hiện và định vị các dạng khuyết tật với độ chính xác cao ở cả cấp độ hình ảnh và điểm ảnh (pixel-level). Cụ thể, với mẫu bị lỗi nứt vỏ, anomaly score đạt 0.8649, mặt nạ dự đoán định vị đúng dọc theo đường nứt kéo dài và khá mảnh, cho thấy mô hình có khả năng phát hiện các khuyết tật dạng tuyến tính, mảnh và có độ tương phản thấp. Trên mẫu hạt bị thủng lỗ, MemSeg đạt mức anomaly score tuyệt đối là 1.0000, mặt nạ dự đoán trùng khớp hoàn hảo với Ground Truth, cho thấy khả năng phát hiện rất mạnh với các bất thường dạng điểm có ranh giới rõ rệt. Đối với lỗi in ấn sai, anomaly score cũng đạt giá trị 1.0000, và mặt nạ dự đoán bám sát vùng ký tự in lỗi, đây là một dạng bất thường phức tạp do sự thay đổi texture lặp không đều trên nền tự nhiên. Với mẫu bị trầy xước, mô hình ghi nhận score 0.9753, mặt nạ dự đoán xác định đúng theo chiều dài vết trầy, mặc dù có một chút lan mờ ở biên, đây là hiện tượng thường gặp trong trường hợp texture nền tương đối đồng đều. Cuối cùng, với mẫu bình thường, anomaly score rất thấp chỉ ở mức 0.0294, mặt nạ dự đoán hầu như không kích hoạt, cho thấy tỷ lệ false positive rất thấp và độ ổn định cao. Thời gian phát hiện cho các mẫu hazelnut dao động trong khoảng 1.88 đến 1.94 giây mỗi ảnh.

❖ Nhận xét chung

Qua quá trình đánh giá trên 10 danh mục thuộc bộ dữ liệu MVTec AD được trình bày ở trên, ta có thể nhận thấy rằng mô hình MemSeg cho ra một hiệu suất rất cao và ổn định trong việc phát hiện và phân đoạn các khuyết tật bề mặt công nghiệp, cả về cấp độ hình ảnh (image-level) lẫn cấp độ điểm ảnh (pixel-level). Trong hầu hết các trường hợp, mô hình đều dự đoán chính xác, đạt anomaly score rất cao (thường > 0.95) đối với các ảnh chứa bất thường, và score rất thấp ($\approx 0.0001-0.03$) đối với các ảnh bình thường, đồng thời xác định được đường biên phân tách rõ ràng giữa hai lớp và tỷ lệ false positive rất thấp.

Một điểm nổi bật của mô hình MemSeg là khả năng xử lý tốt với đa dạng các loại khuyết tật, bao gồm:

- **Bất thường hình học** như móp méo, cong, vỡ, thủng, gãy, biến dạng kết cấu.
- **Bất thường màu sắc** như dính màu, sai ký hiệu, vết xám hoặc dị vật.
- **Bất thường lan mờ hoặc tuyến tính** như trầy xước, vết loang, nứt dài.

Mô hình cũng cho thấy khả năng tổng quát hóa cao, khi vẫn giữ được độ chính xác tốt với các lỗi phức tạp hoặc kết hợp nhiều dạng bất thường trên cùng một mẫu, đặc biệt trong các danh mục như gỗ (wood), thuốc viên (pill), gạch lát (tile) và hạt phỉ (hazelnut). Trong các tình huống mà mẫu thử xuất hiện khuyết tật nhỏ, mờ, hoặc có độ tương phản thấp, mô hình vẫn xác định được tương đối chính xác vùng lỗi, dù đôi khi đường biên của mặt nạ dự đoán có thể không thật sự sắc nét, điều này là dễ hiểu và chấp nhận được trong thực tế công nghiệp. Ngoài ra, thời gian suy luận của mô hình trên từng ảnh dao động từ 1.87 đến 2.17 giây, tốc độ suy luận này hoàn toàn phù hợp để có thể tích hợp vào hệ thống kiểm tra chất lượng thời gian thực trong môi trường sản xuất công nghiệp.

6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1. Kết luận

Trong luận văn này, mô hình MemSeg đã được đã tìm hiểu, phân tích và áp dụng vào việc phát hiện khuyết tật bề mặt trên bộ dữ liệu MVTec AD, cho kết quả phát hiện và phân đoạn khuyết tật với độ chính xác cao. Quá trình huấn luyện cho thấy mô hình hội tụ tốt và ổn định trên nhiều dạng dữ liệu khác nhau. Đồng thời ta thành công thiết kế và thực hiện một hệ thống phần cứng bao gồm Kit NVIDIA Jetson Nano và camera CSI để áp dụng mô hình nêu trên để thực hiện khảo sát quá trình phát hiện khuyết tật bề mặt. Kết quả quá trình phát hiện khuyết tật cho thấy mô hình không chỉ hiệu quả về độ chính xác mà còn khả thi về mặt thời gian xử lý, đây là một yếu tố rất quan trọng trong môi trường hệ thống nhúng nhúng.

Việc triển khai mô hình trên phần cứng thực tế cho thấy tiềm năng ứng dụng cao của mô hình MemSeg trong các hệ thống kiểm tra chất lượng tự động, có yêu cầu thời gian thực. Những kết quả đạt được là cơ sở quan trọng để tiếp tục mở rộng mô hình cho các loại sản phẩm công nghiệp khác trong tương lai.

6.2. Hướng phát triển

Trong tương lai, mô hình có thể được mở rộng để phát hiện các loại khuyết tật phức tạp hơn hoặc áp dụng cho các loại sản phẩm công nghiệp đa dạng hơn ngoài tập dữ liệu MVTec AD. Việc cải tiến kiến trúc mô hình, tích hợp thêm cơ chế attention nâng cao hoặc thuật toán tạo bất thường mô phỏng có thể giúp nâng cao độ chính xác trong các tình huống khó hơn. Hơn nữa, tối ưu hóa mô hình để giảm thời gian phát hiện khuyết tật bề mặt để áp dụng trên các thiết bị nhúng có cấu hình và hiệu năng hạn chế cũng là một hướng đi tiềm năng. Ngoài ra, xây dựng giao diện người dùng trực quan, hệ thống báo lỗi tự động kết hợp cùng hệ thống phân loại bằng cơ khí sẽ giúp mô hình dễ dàng tích hợp vào các dây chuyền sản xuất thực tế.

TÀI LIỆU THAM KHẢO

- [1] M. Yang, P. Wu, and H. Feng, “MemSeg: A semi-supervised method for image surface defect detection using differences and commonalities,” *Eng. Appl. Artif. Intell.*, vol. 119, p. 105835, Mar. 2023.
- [2] P. Bergmann, M. Fauser, D. Sattlegger, and C. Steger, “MVTec AD-a comprehensive real-world dataset for unsupervised anomaly detection,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2019, pp. 9592-9600
- [3] P. Bergmann, S. Löwe, M. Fauser, D. Sattlegger, and C. Steger, “Improving unsupervised defect segmentation by applying structural similarity to autoencoders,” *arXiv preprint arXiv:1807.02011*, 2018.
- [4] T. W. Tang, W. H. Kuo, J. H. Lan, C. F. Ding, H. Hsu, and H. T. Young, “Anomaly detection neural network with dual auto-encoders GAN and its industrial inspection applications,” *Sensors*, vol. 20, no. 12, pp. 3336, 2020.
- [5] T. Schlegl, P. Seeböck, S. M. Waldstein, U. Schmidt-Erfurth, and G. Langs, “Unsupervised anomaly detection with generative adversarial networks to guide marker discovery,” in *Proc. Int. Conf. Inf. Process. Med. Imaging (IPMI)*, 2017, pp. 146-157.
- [6] O. Ronneberger, P. Fischer, and T. Brox, “U-Net: Convolutional networks for biomedical image segmentation,” in *Proc. Int. Conf. Med. Image Comput. Comput.-Assist. Interv. (MICCAI)*, 2015, pp. 234-241.
- [7] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 770-778.
- [8] J. Deng, W. Dong, R. Socher, L. J. Li, K. Li, and L. Fei-Fei, “ImageNet: a large-scale hierarchical image database,” in *Proc. 2009 IEEE Conf. Computer Vision and Pattern Recognition*, pp. 248-255, 2009.
- [9] Zavrtanik, Vitjan, M. Kristan, and D. Skocaj, “DRAEM: a discriminatively trained reconstruction embedding for surface anomaly detection,” in *Proc. IEEE/CVF Int. Conf. Computer Vision*, pp. 8330-8339, 2021.

- [10] K. Perlin, “An image synthesizer,” *ACM SIGGRAPH Comput. Graph.*, vol. 19, no. 3, pp. 287-296, 1985.
- [11] Q. Hou, D. Zhou, and J. Feng, “Coordinate attention for efficient mobile network design,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2021, pp. 13713–13722.
- [12] M. Cimpoi, S. Maji, I. Kokkinos, S. Mohamed, and A. Vedaldi, “Describing textures in the wild,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2014, pp. 3606-3613
- [13] T. Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, “Focal loss for dense object detection,” in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, 2017, pp. 2980-2988.
- [14] C. L. Li, K. Sohn, J. Yoon, and T. Pfister, "CutPaste: Self-supervised learning for anomaly detection and localization," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2021, pp. 9664–9674.
- [15] J. Song, K. Kong, Y. I. Park, S. G. Kim, and S. J. Kang, "AnoSeg: Anomaly segmentation network using self-supervised learning," *arXiv preprint arXiv:2110.03396*, 2021.
- [16] N. Cohen and Y. Hoshen, “Sub-image anomaly detection with deep pyramid correspondences,” *arXiv preprint arXiv:2005.02357*, 2020.
- [17] T. Defard, A. Setkov, A. Loesch, and R. Audigier, “Padim: a patch distribution modeling framework for anomaly detection and localization,” *In: International Conference on Pattern Recognition*, pp. 475-489, 2021.